

The Pennsylvania State University
J. Jeffrey and Ann Marie Fox Graduate School

**APPLICATIONS OF RANDOM MATRIX THEORY
TO DEEP LEARNING**

A Dissertation in
Mathematics

by

Yitzchak Shmalo

© 2025 Yitzchak Shmalo

Submitted in Partial Fulfillment
of the Requirements
for the Degree of

Doctor of Philosophy

August 2025

The dissertation of Yitzchak Shmalo was reviewed and approved by the following:

Leonid Berlyand

Professor of Mathematics

Dissertation Advisor

Chair of Committee

Jairo Bochi

Professor of Mathematics

Bing Li

Verne M. Willaman Professor of Statistics

Pierre-Emmanuel Jabin

Professor of Mathematics

Director of Graduate Studies

Abstract

In this thesis, we explore the applications of Random Matrix Theory (RMT) in the training and optimization of Deep Neural Networks (DNNs). RMT provides a framework for analyzing the statistical properties of large matrices. In the context of DNNs, we can use RMT to study the properties of weight layer matrices. We primarily focus on pruning DNN parameters (weights). This pruning is guided by RMT and results in a drastic reduction in DNN parameters while preserving or even improving DNN accuracy.

We begin by establishing rigorous mathematical results for fully connected DNNs using RMT. We prove that training reduces the randomness in the weight matrices as the weights converge toward local or global minima of the loss function. Furthermore, we show that removing randomness from the weight matrices does not decrease accuracy. We also show that removing randomness in the weight layer matrices decreases the loss function when it includes L2 regularization. We prove the RMT-based Pruning Theorem, which demonstrates how RMT-guided pruning of singular values can drastically reduce the number of parameters in the DNN while preserving its accuracy. These results are further generalized to encompass a wider range of DNN architectures, such as CNNs, ResNets, and ViTs.

We present numerical experiments that validate these mathematical findings. For Vision Transformers (ViTs) trained on ImageNet, we demonstrate that our approach can reduce the number of DNN parameters by 50% with less than a 1% loss in accuracy, representing the state-of-the-art in pruning for these ViT models ¹. We also present numerical experiments that lead to some open questions. For example, for fully connected DNNs, RMT-based pruning not only simplifies the loss landscape but also increases accuracy and decreases variance across random initialization. We do not yet have a rigorous theoretical understanding of this behavior.

Finally, we address the stability of accuracy during DNN training. In this context, the training of a DNN is performed via the minimization of a cross-entropy loss function, and the performance metric is accuracy (the proportion of objects that are classified correctly). While training results in a decrease in loss, the accuracy does not necessarily increase during training and may sometimes even decrease. The goal of achieving stability of accuracy is to establish sufficient conditions on the data that ensure that if accuracy is high at some initial time, it remains high throughout training. A recent result [8] introduces a doubling condition on the training data, which ensures the stability of accuracy during training. In this work, we remove several restrictions of the doubling condition, provide a more practical condition, and show both mathematically and numerically that it leads to stability of accuracy results. Our work also offers RMT-based insights into the conditions that lead to stable training. The results of this dissertation are published in the following papers: [9, 53, 10].

¹Code for this can be found at https://github.com/yspennstate/RMT_pruning_ViT. More code for this work can be found at https://github.com/yspennstate/RMT_pruning_2/blob/main/rmt_pruning_2.ipynb.

Contents

List of Figures	vii
List of Tables	xi
Acknowledgements	xii
Chapter One: Introduction	1
1 Preliminaries and introducing the main results	1
1.1 Introduction to Deep Neural Networks	1
1.2 Marchenko-Pastur distribution of eigenvalues of random matrices	3
1.3 Marchenko-Pastur distribution applied to Deep Learning	4
1.4 RMT-based model of DNN training	6
1.4.1 RMT-based model of DNN training	6
1.4.2 Stability of Accuracy	6
2 Overview of the analytical and numerical results	6
2.1 A theorem on how training reduces randomness in DNNs	9
2.2 A theorem on how pruning randomness in DNNs reduces loss but keeps accuracy the same	9
2.3 A theorem of stability of accuracy	13
Chapter Two: Pruning in Deep Learning using Random Matrix Theory	15
3 Analytical results on RMT methods in Deep Learning	15
3.1 Assumptions for theorems in Chapter 2	15
3.2 Analytical results for RMT-based decrease in loss	18
3.3 Key Lemmas: how removing randomness from DNN weight layers affects the output, loss, and accuracy of a DNN	20
3.4 How pruning affects classification confidence (for low-rank weight layer matrices)	21
3.5 Key technical Lemma: removing random weights for DNN with an arbitrary many layers does not affect classification confidence	23
3.6 Pruning Theorem for DNNs: how pruning random weights using PM distribution affects the classification confidence	24
3.7 Simple example of DNN with one hidden layer	25
3.8 Pruning Theorem for accuracy: how pruning affects accuracy	26
4 Numerical results for pruning Visual Transformers	26
4.1 Pruning strategy based on Random Matrix Theory	27
4.1.1 Singular vector pruning	28
4.1.2 Direct coefficients pruning	29
4.1.3 Regularization	29
5 MP-based pruning of fully connected DNNs	32
5.1 Regularization and MP-based pruning for fully connected DNNs	32
Chapter Three: Stability of Accuracy for the Training of DNNs	38
6 Stability of accuracy for the training of DNNs Via the Uniform Doubling Condition	38
6.1 Introduction to Stability of Accuracy	38

7	Motivation and some numerical examples	43
7.1	Overfitting: the connection between the accuracy on the training set and accuracy on the test set	46
8	Rigorous mathematical results for stability of accuracy and numerical examples	46
	Bibliography	54
	Appendix	60
A	More examples of MP-based pruning together with regularization and sparsification: fully connected DNNs and CNNs	61
A.0.1	Training of fully connected DNNs on MNIST: simplifying the loss landscape . . .	62
A.0.2	MP-based pruning of CNNs on MNIST and Fashion MNIST	68
A.0.3	Numerics for training DNNs on CIFAR-10: reducing parameters via MP-based pruning	73
B	A simple illustration of the relationship between RMT-based pruning and regularization	74
B.1	Regularization in a simple regression setting	74
B.2	Proof of Lemmas 3.10 and 3.13.	78
B.3	Training of a DNN with three weight layers using L1 and L2 regularization with varying layer widths	82
B.3.1	Analysis of $b(N)$	83
B.4	Proof of Lemma B.5	85
C	Proof of Theorems 2.4, 2.5, 3.7 and 3.8.	86
C.1	Asymptotic behavior of singular values and vectors of the deformed matrix model given in Subsection 2	86
C.2	Proof of Theorems 2.4, 2.5, 3.7 and 3.8.	87
D	ViT model architecture	90
D.1	Image Representation and Embedding	90
D.2	Positional Encoding	90
D.3	Self-Attention Mechanism	90
D.4	Calculation of Attention	90
D.4.1	Output of the Attention Layer	91
D.5	Position-Wise Feed-Forward Networks	91
E	Some known results on perturbation of matrices	91
E.1	Asymptotics of singular values and singular vectors of deformation matrix	91
E.2	Gershgorin's Circle Theorem	93
F	An Approximation Lemma – pruned matrix W' approximates the low-rank matrix S	93
F.1	Numerics for Example 3.15	101
F.2	Details for Example 3.16	101
G	Proof for Pruning Theorem 2.7	101
H	Other algorithms required for implementing RMT-SVD based pruning of DNN	103
H.1	BEMA algorithm for finding λ_+	103

H.2	The role of singular value decomposition in deep learning	105
H.3	Eliminating singular values while preserving accuracy	106
H.4	MP fit Criteria: Checking if the ESD of X fits a MP distribution	107
H.5	The MP and spike metrics of a DNN model	108
I	Some of the proofs and numerics	110
I.1	Proof of Lemma 3.11	110
I.2	Effectiveness of MP-based pruning for different initialization methods	111
I.3	A Regression Problem: MP-based pruning in regression	113
I.4	Numerical example used to calculate δX	116
I.5	Hyperparameters for Subsection A.0.1	116
I.6	Hyperparameters for Subsection A.0.1	117
I.7	Hyperparamters for Subsection A.0.2	117
I.8	CNN architecture description	117
I.8.1	Pooling and regularization details	118
I.9	The hyperparamters for Subsection A.0.3	118
J	Some algorithms	118
K	Numerical results for stability of accuracy	121
K.1	Numerics for a different classification problem	125
K.2	An application of the SUDC	128
L	Previous results for stability of accuracy.	129
L.1	Stability of accuracy via uniform doubling condtion on $\delta X(T, \alpha)$	131
L.2	Doubling condition on the data set	132
L.3	Why truncations in the uniform doubling condition might not be important	133
M	More motivation for introducing the uniform doubling condition	134
N	Proof of main result: stability theorem via uniform doubling condition on T	135
N.1	Proof of Proposition L.4	140
N.2	Proof of Theorem 8.6	140
O	Other Activation Functions	141
P	Proof of Lemma N.2	144
P.1	Proof of Lemma N.2.	144
P.2	Proof of Lemma N.5: Upper bound on the loss	144

List of Figures

Figure 1	Histogram of the pruned matrix W'_2 (left) and the difference $W_2 - W'_2$ (right). .	5
Figure 2	Accuracy of DNN before pruning (red); Lower theoretical bound on the accuracy of pruned DNN (blue). The difference between red and blue values decreases from 4% for $N = 500$ to 0.6% for $N = 10,000$	13
Figure 3	Histogram of δX of the test set for the final epoch of training. On the x-axis, we have the size of δX	22
Figure 4	Graph of f_W . On the x-axis, we have σ_i	26
Figure 5	The norm $\ S - W'\ _2$ is shown for 10 random matrices with elements i.i.ds taken from $N(0, \frac{1}{N})$ and S a diagonal matrix with elements on the diagonal given by 30, 40, 50, 60, 70. We see that the norm is very close to $f_W \approx 1$	27
Figure 6	A graphical comparison between f_W and 2σ as functions of σ . The x-axis represents the variable σ ranging from -25 to 25. The y-axis provides the computed values for both f_W and 2σ . We see that $f_W \approx \sigma $, which is smaller than 2σ for positive σ . Note, to obtain this numerical result, we must ensure that $\sigma_r \geq \sigma$. .	28
Figure 7	The top 1 accuracy and top-5 accuracy of ViT-base vs. percentage of parameters kept for pruning through RMT-based sparsification and no fine-tuning. The accuracy of the DNN is given as a percentage above the data points.	31
Figure 8	The top 1 accuracy and top-5 accuracy of ViT-large model vs. percentage of parameters kept for pruning through RMT-based sparsification and no fine-tuning. The accuracy of the DNN is given as a percentage above the data points. .	31
Figure 9	Comparison of pruning results between our method and CP-ViT [55]	32
Figure 10	DNN performance vs. sparsification	36
Figure 11	Accuracy and loss as we add noise to the weight layers of the DNN.	37
Figure 12	Despite the progress in training and reduction in loss, the accuracy has experienced a significant decline, falling from 48% at epoch 0 to 38% at epoch 25. . .	41
Figure 13	Histogram of δX at epoch 1 and 27	42
Figure 14	Decision boundary plot for a two-class classification problem generated by a random polynomial function of degree 6, with added noise. The blue and red points represent the two classes, and the blue line shows the original polynomial. .	44
Figure 15	The graph of the training and test accuracy of our DNN. The x-axis represents the epochs of the DNN, and the y-axis represents its accuracy.	45
Figure 16	Avoiding small clusters is a sufficient but not necessary condition for stability of accuracy.	45
Figure 17	left: slab in $\mathbb{R}^2$. Width is κ , the center is a dashed line, and normal vector $\vec{u}$ is shown. right: slab in $\mathbb{R}^3$	47
Figure 18	left: truncated slab S_ϵ . The truncation is a solid line (in black) and the slab is a dashed line (in red). right: S_ϵ is shown in dashed lines (red) and its scale $S_{\kappa\epsilon}$ is shown as the dotted lines (in blue).	47
Figure 19	Truncated slab S_ϵ . The truncation is a solid line (black), and the original slab is a dashed line (in red). We also see the scaled slab $S_{\kappa\epsilon}$ as a dotted line (in blue).	48
Figure 20	Mapping $f_1 : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. The set B_1 is shown in green.	49
Figure 21	What are the F_i maps?	49
Figure 22	C_2 , represented on the y -axis, for different values of b , represented on the x -axis, with fixed $\xi = .4$. We see that C_2 decreases from .06 when $b = 2$ to .03 when $b = 5$. This is a significant change in terms of stability of accuracy.	51
Figure 23	C_2 , represented on the y -axis, for different values of ξ , represented on the x -axis. .	53

Figure 24	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test set.	63
Figure 25	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the training set. The sub-figures correspond to the different initial topologies: (a) [784, 2000, 2000, 1000, 500, 10], (b) [784, 1500, 3000, 1500, 500, 10]. The other examples in Fig. 24 have similar-looking accuracies on their training set.	64
Figure 26	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test set. The sub-figures correspond to the different initial topologies: (a) [784, 2000, 4000, 2000, 500, 10], (b) [784, 2000, 2000, 2000, 2000, 1000, 500, 10], (c) [784, 3000, 4000, 3000, 500, 10].	65
Figure 27	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the training set. The sub-figures correspond to the different initial topologies: (a) [784, 3000, 4000, 3000, 500, 10], (b) [784, 2000, 2000, 2000, 2000, 1000, 500, 10].	67
Figure 28	Analysis of DNN Training and Pruning	67
Figure 29	Accuracy vs. number of parameters kept. The percentage of parameters kept is also shown (above each point on the graph).	69
Figure 30	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach, on the test and training sets.	70
Figure 31	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the Fashion MNIST test and training sets.	71
Figure 32	Accuracy vs. number of parameters kept for CNNs trained on Fashion MNIST with and without MP-based pruning.	72
Figure 33	Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test and training sets.	73
Figure 34	Accuracy vs. number of parameters kept for CNNs trained on CIFAR-10 with and without MP-based pruning.	75
Figure 35	Example of data generated from the problem statement on one axis. The blue line is the function y_0 , and the blue dots are the data points.	76
Figure 36	Cumulative distribution of singular values of the optimal weight matrix	76
Figure 37	Cumulative distribution of singular values of the $L2$ regularised weight matrix	77
Figure 38	Cumulative distribution of singular values of the $L1$ regularised weight matrix	77
Figure 39	Cumulative distribution of singular values of the weight matrix prior to pruning	77
Figure 40	Plot of $a(N)$ vs. the width N of the middle layer $\mathbf{W}_2$. This plot shows how the complexity-related metric $a(N)$ behaves as the width N is increased, reflecting changes in the norms of the weight matrices and their dependence on N	83
Figure 41	Plot of test accuracy vs. the width N of the middle layer $\mathbf{W}_2$. This plot demonstrates how the performance of the DNN changes with increasing N , showing the relationship between network width and generalization ability on the Fashion MNIST dataset.	84
Figure 42	Plot of $b(N)$ vs. the width N of the middle layer $\mathbf{W}_2$. This plot shows how the metric $b(N)$ behaves as the size of the middle layer increases, incorporating a stronger dependence on N^2	84
Figure 43	Plot of test accuracy vs. the width N of the middle layer $\mathbf{W}_2$. This plot demonstrates how the network's accuracy changes as N increases, with weight matrices initialized using $1/N^2$	85

Figure 44	Entry-wise distribution of a <u>bulk</u> left singular vector after the normalisation $u_i \mapsto \sqrt{N} u_i$. In both panels the bars show the empirical histogram, while the solid curve is the standard normal density. Panel (a) confirms that for a pure Gaussian matrix R the bulk singular vectors are isotropic. Panel (b) shows that the same delocalisation persists when a rank- r super-critical spike S is added, i.e. for $X = R + S$	96
Figure 45	Left: Dot product of the left singular vectors. Right: Dot product of the right singular vectors.	101
Figure 47	In blue, we have the ESD of X ; in red, the Marchenko-Pastur distribution, which best fits the ESD based on the BEMA algorithm.	105
Figure 48	The ESD of X_l and its best fit MP distribution	106
Figure 49	Two hidden layers from the original, in the right figure, have 10 nodes (total 100 parameters). Layers are transformed into three hidden layers, in the left figure, in pruned DNN. The first layer has 10 nodes, the second layer has 2 nodes (keeping only two singular values in the SVD), and the third layer has 10 nodes, resulting in a total of 20 parameters.	107
Figure 50	Number of eigenvalues kept is shown on the x -axis while the accuracy is shown on the y -axis.	107
Figure 51	Best fit MP distribution (on the left) for $\alpha = .25$ and $\beta = .8$ for a layer of the base ViT model. Here the MP fit error is .74 while the percentage of singular values smaller than $\sqrt{\lambda_+}$ is 73%. For more on the right figure, see Section B.	109
Figure 52	Best fit MP distribution (on the left) for $\alpha = .25$ and $\beta = .8$ for a layer of the base ViT model. Here the MP fit error is .01 while the percentage of singular values smaller than $\sqrt{\lambda_+}$ is 99.73%. For more on the right figure, see Section B.	109
Figure 53	In the left figure, we have α vs LRM and on the right we have α vs MP fit metric. This is for the base ViT model studied in 7.	109
Figure 54	Accuracy vs Epoch for He and Xavier Initializations	113
Figure 55	Illustration of the regression problem.	114
Figure 56	Training loss comparison between normal and pruned DNN.	115
Figure 57	Test loss comparison between normal and pruned DNN.	115
Figure 58	Illustration of a decision boundary for a binary classification challenge created using a random polynomial function, supplemented with noise. The distinct blue and red points signify the two classes.	116
Figure 59	SUDC for data set with two classes, red and blue. We start with a single slab of small width and continue to double it (shown in different colors) until the doubling condition is not satisfied.	122
Figure 60	DNN training loss and training accuracy vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.	122
Figure 61	C_2 , represented on the y -axis, vs η , represented on the x -axis.	123
Figure 62	DNN training loss and training accuracy vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.	123
Figure 63	DNN loss on training set and accuracy on test set vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.	124
Figure 64	Number of objects in data set vs $\bar{\beta}$	125
Figure 65	DNN loss on test set and accuracy on test set vs $\bar{\beta}$	126
Figure 66	DNN loss on test set and accuracy on test set vs $\bar{\beta}$	127
Figure 67	Illustrative example of the generated data. The black curve showcases the function $g_1(x)$, the red points indicate the first class, and the blue points signify the second class.	128

Figure 68	DNN loss on test set and accuracy on test set vs $\bar{\beta}$	129
Figure 69	An example of a histogram of $\delta X(T)$ with slabs (in this case intervals) κI and I not satisfying the doubling condition.	130
Figure 70	δX for final epoch of DNN.	132
Figure 71	The curve H is shown as a squiggly curve (in purple). Here $\mu(S_\beta) = \mu(S_{\kappa\beta})$, and so would not satisfy the doubling condition.	135
Figure 72	Slabs in $\mathbb{R}^2$. Red dashed lines are the original slab, and black solid lines are two truncations. $\mu(S_{\kappa\beta}) = \mu(S_\beta)$ (slab bounded by truncations), thus the doubling condition is not satisfied.	137
Figure 73	Slab in $\mathbb{R}^2$. Red dashed lines are the original slab (without the truncation), and black solid lines are two truncations. S^0 is the center slab after truncation. Even though truncations are not parallel, still $\mu(S_{\kappa\beta}^0) = \mu(S_\beta^0)$, thus the doubling condition is not satisfied.	138
Figure 74	Classification confidence for the toy DNN for Example 7.1 at different stages of training.	145

List of Tables

Table 1	Comparison with different ViT pruning methods on the ImageNet dataset. The accuracy without finetuning was not mentioned in VTP, PoWER, and HVT but were replicated in [55]. We fine-tune for only 7 epochs on the ImageNet training set. We see that the larger the DNN, the better our method performs, which makes sense from an RMT perspective. This is also reassuring, given that modern AI models are often much larger than these models.	33
Table 2	Performance of fully connected DNN at final epoch for various training strategies.	34
Table 3	Performance of fully connected DNN at final epoch for various training strategies.	34
Table 4	Hyperparameter values	35
Table 5	Performance of normal and pruned DNNs for different initial topologies (on test set).	62
Table 6	DNN topology, initial and final parameters for the pruned DNNs, with percentage reductions.	63
Table 7	Comparison of training and test losses between normal and pruned DNNs for different topologies.	64
Table 8	Performance of normal and pruned DNNs for different initial topologies.	65
Table 9	DNN topology, initial and final parameters for the pruned DNNs, with percentage reductions.	66
Table 10	Comparison of training and test losses between normal and modified DNNs for different topologies.	66
Table 11	Mean Squared Errors of Different Models for the simple regression task	78
Table 12	Training hyperparameters for the DNN.	115

Acknowledgements

I would like to express my gratitude to my advisor, Leonid Berlyand, whose support and guidance have been the cornerstone of my Ph.D. journey. The hours upon hours of discussions we’ve shared have not only sharpened my mathematical thinking but also opened my eyes to the captivating topics that fill this dissertation. His encouragement, keen insights, and genuine passion for discovery have shaped me as a researcher and as a person. I owe so much of where I am today to his mentorship, and I feel incredibly lucky to have worked under his guidance.

I am also deeply thankful to Marian Gidea and Emil Prodan, both of whom inspired me to pursue a Ph.D. Their passion and encouragement gave me my drive to embark on this academic journey, and I am profoundly grateful for their influence.

I’d like to thank the members of my committee, Jairo Bochi, Pierre-Emmanuel Jabin, and Bing Li, for their time and insights. I owe a lot to all of you, and I very much appreciate everything that you have done for me and for this dissertation.

Furthermore, I’m deeply grateful to the mathematicians from Penn State, Caltech, JPL, and beyond who’ve enriched my growth as a scholar. Their generosity—whether through joint papers, inspiring talks, or patient explanations—has been invaluable. I’d like to especially thank: Ievgenii Afanasiev, Alexander Gavrikov, Dmitry Golovaty, Anna Lytova, Houman Owhadi, Etienne Sandier, Clarke Saffin, Viktor Slavin, Sasha Sodin, Jouni Susiluoto, and Lei Zhang. Each has left a unique mark on my work, and I’m better for it.

Moreover, I owe a huge debt to my friends and colleagues, many of whom teamed up with me on papers that paved the way for this dissertation. Their collaboration, sharp feedback, and shared excitement for the work have made all the difference, and I’m so thankful for each of them. Special gratitude goes to: Theo Bourdais, Hai Chi, Robby Creese, Spencer Dang, Jon Jenkins, Mariia Kiyashko, and Oleksii Krupchytskyi. Their presence has turned a challenging journey into one filled with camaraderie and joy.

I also want to extend my thanks to NASA for supporting me through the AIST program (Kernel Flows: Emulating Complex Models for Massive Data Sets). Likewise, I’m appreciative of Jouni Susiluoto and Houman Owhadi for hosting me during my JPL internship. This work was partially supported by the NSF grant OISE-2401227. The findings and conclusions herein do not necessarily reflect the views of the funding agency.

Chapter One

Introduction

1 Preliminaries and introducing the main results

1.1 Introduction to Deep Neural Networks

In many real-world applications, a **classification task** involves assigning each element of a set S to one of K distinct classes via a function called a *classifier*. The nature of the elements in S can vary widely depending on the problem at hand. Here are a few practical examples:

- **Feature Vectors in $\mathbb{R}^n$:** Each element $s \in S$ could be represented as a vector of features in $\mathbb{R}^n$, where each component corresponds to a measurable attribute. For example, in a medical diagnosis task, s might represent patient data such as age, blood pressure, cholesterol level, etc. The classifier then predicts the disease category based on these features.
- **Images as Matrices:** In image classification, each element $s \in S$ might be a matrix (or tensor) representing an image. The entries of the matrix correspond to pixel intensities (or color channel values). For instance, in a task to classify images as either containing a cat or a dog, the classifier learns to map these matrices to the correct class (dog or cat).
- **Text Documents:** In natural language processing, elements of S can be text documents or sentences. These documents are often transformed into numerical vectors using techniques such as bag-of-words, TF-IDF, or word embeddings. The goal then is to classify the documents into categories like spam vs. non-spam emails, or different topics.
- **Other Data Structures:** Elements in S might also include more complex structures such as graphs (e.g., social networks, molecules in chemistry) or time series data (e.g., stock market trends, sensor readings). Each type of data requires specific feature extraction methods to convert raw data into a format amenable to classification.

Formally, assume there exists a true classification function

$$\hat{\phi} : S \rightarrow \{1, 2, \dots, K\},$$

where K is the number of classes. Given a labeled training set $T \subset S$, such that we know the classes for elements $s \in T$ (that is we know $\hat{\phi}$ on T), our goal is to develop an **approximate classifier**

$$\phi : S \rightarrow \{1, 2, \dots, K\},$$

which predicts the class labels for new, unseen elements of S (elements in $S \setminus T$) as accurately as possible.

To do so, we often use a Deep Neural Network (DNN) as a parameter-dependent classifier $\phi(s, \alpha)$, where $\alpha \in \mathbb{R}^p$ are its parameters. The DNN outputs a probability vector $(p_1(s), \dots, p_K(s))$. We can think of $p_i(s)$ as the probability of s belonging to the class i , and while we know the correct classes of

the objects $s \in T$, the goal is to extend $\hat{\phi}$ from T to all of S via $\phi(s, \alpha)$. Thus, we wish to obtain a ϕ which maps s to the same class as $\hat{\phi}$ when $s \in T$, with the hope that ϕ will also map elements $s \in S$ to their correct class. We take $i(s)$ to be the correct class of s , so that $p_{i(s)}$ gives us the probability that the DNN classifies s in its correct class.

For the most part, we consider fully connected DNNs which are a composition $\rho \circ X(\cdot, \alpha)$, with ρ the softmax (given in (2)) and $X(\cdot, \alpha)$ defined as follows. In the case of fully connected DNNs, we take

$$X(\cdot, \alpha) = \lambda \circ M_L(\cdot, \alpha) \circ \cdots \circ \lambda \circ M_1(\cdot, \alpha) \quad (1)$$

with:

- $M_k(\cdot, \alpha)$ an affine function $\mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ parameterized by a weight matrix $W_k \in \mathbb{R}^{N_k \times N_{k-1}}$ and bias vector $\beta_k \in \mathbb{R}^{N_k}$ such that $M_k(x) = W_k \cdot x + \beta_k$.
- $\lambda : \mathbb{R}^m \mapsto \mathbb{R}^m$ is a nonlinear activation function. In this case, we assume λ is the absolute value activation function, such that $\lambda : x \in \mathbb{R}^m \mapsto (|x_1|, \dots, |x_m|) \in \mathbb{R}^m$.
- The softmax function ρ normalizes the outputs of $X(\cdot, \alpha)$ into a probability vector over $\{1, \dots, K\}$. The individual components of ρ are computed as:

$$\rho(v)_i = \frac{\exp(v_i)}{\sum_{i=1}^K \exp(v_i)} \text{ for } v \in \mathbb{R}^K \quad (2)$$

Given the neural network architecture above, we train our classifier $\phi(\cdot, \alpha)$ by minimizing the cross-entropy loss. This loss quantifies the accuracy of our classifier over the training set, and is defined as

$$L(\alpha) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha)). \quad (3)$$

DNNs have become a dominant tool for tackling classification tasks, where objects within a set $S \subset \mathbb{R}^n$ are categorized. DNNs are trained on labeled datasets $T \subset \mathbb{R}^n$ by minimizing the loss function, such as the cross-entropy loss function in (3), with the goal of maximizing classification accuracy. Through this training process, DNNs have achieved state-of-the-art results on many real-world classification challenges, including handwriting recognition [36], image classification [33], speech recognition [28], and natural language processing [60]. My research focuses on enhancing the efficiency and accuracy of DNNs through insights from Random Matrix Theory (RMT), with specific applications in pruning and regularization. My research also involves studying the stability of accuracy of DNNs during training, that is, the relationship between the minimization of loss and the increase in DNN accuracy.

RMT has been increasingly applied in deep learning to understand the spectral properties of weight matrices, offering insights into the training dynamics and generalization capabilities of neural networks [43, 40]. Recent studies have used RMT to develop stopping criteria [45], regularization methods [67], and noise filtering [57]. Additionally, RMT has been utilized to analyze the spectrum of weight layers [62] and study the input-output Jacobian matrix [49, 50]. Investigations into RMT-based initializations for DNNs have also been conducted [51]. Research has demonstrated that the spectral characteristics of DNNs can provide insightful information about the network's performance, even in the absence of test data. In particular, [44, 42] demonstrate that the spectrum of the weight layers of a DNN can be indicative of its accuracy on unseen test sets.

In this work, we show that analyzing these spectral properties allows for the pruning of weights in a DNN, preserving performance by identifying and removing weights that minimally contribute to

predictive accuracy. For fully connected DNNs, this pruning can also increase accuracy by up to 30%. Vision Transformers (ViTs) [18] are a class of models that process image data by dividing images into patches and applying self-attention mechanisms, allowing them to capture long-range dependencies in images (for details on ViT see Appendix D). My work shows that applying RMT-based techniques for pruning, pre-trained ViTs (trained on the ImageNet dataset) can be used to prune their parameters by 50% with less than a 1% drop in accuracy. For pruning ViTs without afterward fine-tuning them, we can remove 25% of the parameters with less than a 1% drop in accuracy. To the best of my knowledge, this is state-of-the-art (specifically for pruning of the models ViT-B16/224, ViT-L16/224 and the DeiT series pre-trained on ImageNet).

Specifically, my research includes:

- Developing RMT-based pruning methods for DNNs, particularly ViTs, leading to significant reductions in model size with minimal loss in accuracy (see Section 4). For fully connected DNNs, RMT-based pruning leads to an increase in accuracy (see Section 5).
- Analytical results for DNNs (see Sections 2, 3 and 8):
 - Showing that randomness in weight matrices diminishes during training, contributing to a deeper understanding of training dynamics.
 - Developing an RMT-based understanding of how regularization works. This shows that pruning DNNs leads to a reduction in loss if L2 regularization is used.
 - Providing bounds on how RMT-based pruning of DNNs will affect their accuracy.
 - Studying the stability of accuracy of DNNs during training.

1.2 Marchenko-Pastur distribution of eigenvalues of random matrices

The Marchenko-Pastur (MP) distribution [41], a cornerstone of RMT, describes the asymptotic behavior of singular values of large random matrices. To begin, we introduce the empirical spectral distribution (ESD) of an $N \times M$ matrix G as follows:

Definition 1.1. *The ESD of an $N \times M$ matrix G is given by:*

$$\mu_{G_M} = \frac{1}{M} \sum_{i=1}^M \delta_{\sigma_i}, \quad (4)$$

where σ_i denotes the i th non-zero singular values of G , and δ represents the Dirac measure.

Theorem 1.2 (Marchenko and Pastur (1967) [41]). *Let W be an $N \times M$ random matrix with $M \leq N$. The entries $W_{i,j}$ are independent and identically distributed random variables with mean 0 and variance $\sigma^2 < \infty$. Define $X = \frac{1}{N} W^T W$. Assuming that $N \rightarrow \infty$ and $\frac{M}{N} \rightarrow c \in (0, +\infty)$, the ESD of X , denoted by μ_{X_M} , converges weakly in distribution to the Marchenko-Pastur probability distribution:*

$$\frac{1}{2\pi\sigma^2} \frac{\sqrt{(\lambda_+ - x)(x - \lambda_-)}}{cx} \mathbf{1}_{[\lambda_-, \lambda_+]} dx \quad (5)$$

with

$$\lambda_{\pm} = \sigma^2(1 \pm \sqrt{c})^2. \quad (6)$$

This theorem asserts that the eigenvalue distribution of a random matrix converges to the Marchenko-Pastur distribution as its dimensions increase. The MP distribution is given by a deterministic function, dependent on two parameters: the variance of the random variables in the initial matrix, σ^2 , and the ratio of the number of columns to the number of rows, c .

1.3 Marchenko-Pastur distribution applied to Deep Learning

In the context of neural networks, the MP distribution can be used to distinguish between signal and noise within the weight matrices W_l . Singular values that fit the MP distribution correspond to randomness within the matrix, while those that are larger and deviate from the distribution indicate meaningful information.

Because training involves minimizing the loss function (i.e., (3)), random noise and information from the dataset get encoded in the weight matrices W_l during the training process. Before training, these matrices are often initialized with i.i.d. entries according to a normal distribution $\mathcal{N}(0, \frac{1}{N})$, with N the number of rows or columns of the matrices (or some other probability distribution). Given the large size of these matrices, the Empirical Spectral Distribution (ESD) of $X_l = \frac{1}{N}W_l^T W_l$ is close to the MP distribution before training. Previous studies have used the deformed model (or spiked model) in random matrices to examine W_l , see [43, 57]. That is, they have assumed that $W_l = R_l + S_l$, with R_l a random matrix (noise) and S_l a highly-correlated matrix (information). This is based on the observation that the ESD of X_l after training exhibits eigenvalues that diverge from the Marchenko-Pastur limit λ_+ (are bigger than λ_+). This observation suggests that the weight matrices contain signals obtained from the data after training.

With random initialization and training through Gradient Descent (GD), and for τ a step size, the sequence of parameters during training $\alpha(t)$ satisfies

$$\alpha(0) \text{ is i.i.d. Gaussian} \quad (7)$$

$$\alpha(t+1) = \alpha(t) - \tau \nabla L(\alpha(t)). \quad (8)$$

While $\nabla L(\alpha(t))$ contains information derived from the training data (signal), it also includes noise due to the data itself possibly being noisy. Thus, each update of α contains both noise and signal. To account for this observation, we model the training process using the following suppositions:

Supposition 1 After t steps of training, the weight matrix at layer l can be decomposed as

$$W_l(t) = R_l(t) + S_l(t) \quad (9)$$

where $S_l(t)$ represents the signal (structured information learned from the data) and $R_l(t)$ is an independent random noise perturbation.

As training progresses, we expect the magnitude of the signal $\|S_l(t)\|_F$ to increase while the magnitude of the noise $\|R_l(t)\|_F$ decreases, with $\|\cdot\|_F$ the Frobenius norm of a matrix. This work will explore this model in detail. We also suppose that the matrix S_l is of low rank, that is, the information in the data can be encoded in a low-rank matrix. That is:

Supposition 2 $S_l(t)$ is a low rank matrix, with r nonzero singular values and r not depending on the size of the matrix N .

Given this decomposition $W_l = R_l + S_l$, we will use the MP distribution to separate the signal from the noise. To do so, we sometimes assume that the signal in W_l is strong enough to be distinguishable from the noise. Since the singular values of R_l are determined by the MP distribution, with maximal value $\sqrt{\lambda_+}$, this supposition can be reformulated as:

Supposition 3 The r largest singular values of W_l are larger than $\sqrt{\lambda_+}$, for λ_+ the largest eigenvalue of $R_l^T R_l$.

Example 1.3. We consider a fully connected DNN with the architecture

$$[784, 4000, 4000, 10],$$

trained on the Fashion MNIST dataset for 100 epochs to obtain an accuracy of approximately 89% (on the test set). After training, we perform a pruning procedure on the weight matrix

$$W_2 \in \mathbb{R}^{4000 \times 4000},$$

in two steps:

1. **Singular Value Thresholding:** Set to zero every singular value of W_2 that is smaller than $\sqrt{\lambda_+}$.
2. **Parameter Sparsification:** Apply a thresholding operation by setting all parameters with absolute values smaller than 0.01 to zero.

Let W'_2 denote the pruned version of W_2 . Remarkably, replacing W_2 with W'_2 in the network leaves the overall accuracy unchanged. Replacing W_2 with $W_2 - W'_2$ gives us an accuracy of 10%, an indication that $W_2 - W'_2$ is random (we have 10 classes) and the information in the weight layer matrix W_2 is contained in W'_2 .

Figure 1 illustrates the outcome of the pruning. The left histogram shows the distribution of entries of W'_2 , while the right histogram displays the distribution of the entries of $W_2 - W'_2$. Notice that $W_2 - W'_2$ resembles a random matrix (analogous to R_2), whereas W'_2 is low-rank (with rank 52) and its nonzero entries are highly correlated (for example, none of them are below .01). Thus, we can model $W_2 = S_2 + R_2$, with $S_2 = W'_2$ and $R_2 = W_2 - W'_2$.

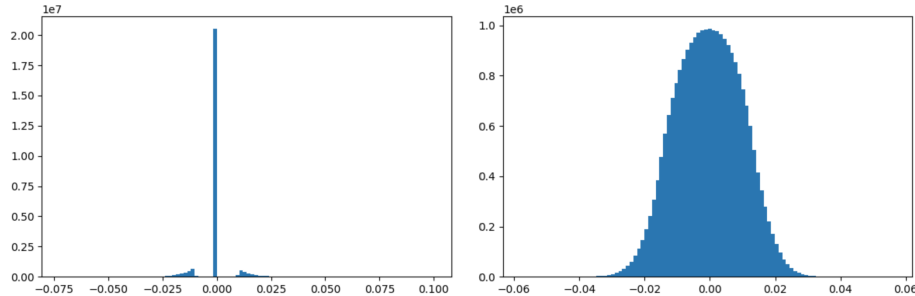


Figure 1: Histogram of the pruned matrix W'_2 (left) and the difference $W_2 - W'_2$ (right).

In Section 4, we will use Supposition 2 to design a pruning strategy and apply it to ViT and fully connected models. In Section 3, we formalize these assumptions and prove that removing $R_l(t)$ does not decrease accuracy (see Theorem 3.8). The theory, through Theorem 2.4, provides insights into how the training reduces the noise in weight matrices. We first present our result for a simplified case where the DNN has only three weight layer matrices and no bias vectors.

Additionally, we study how the inclusion of L2 regularization, commonly implemented as weight decay in deep learning [34], affects the random matrix R_l and the evolution of the weights as training progresses. The L2 regularization term is given by

$$L_{L2}(\alpha) = \mu \sum_n \|W_n\|_F^2,$$

where μ is a regularization parameter. This term helps control overfitting by constraining the magnitude of weights. It also ensures that weights have a preference for decaying to zero during training, effectively

removing noise from the weights. The random matrix R_l contains only noise and no information, so we can expect that adding $L2$ loss to the loss function will help ensure that, during training, the random matrix R_l gets smaller and even vanishes. Thus, we will show how this regularization term is closely connected with the random matrix R_l and how it behaves during training.

1.4 RMT-based model of DNN training

The theoretical side of my work is underpinned by rigorous mathematical analysis. As mentioned, a central theme is the study of the spectral behavior of large random matrices and its implications for deep learning. Key contributions include:

1.4.1 RMT-based model of DNN training

We model the weight matrices in DNNs as a sum of a random component R_l and a low-rank component S_l , so that

$$W_l = R_l + S_l.$$

Through rigorous mathematical analysis, we show that (see Sections 2 and 3):

- Under training (with $L2$ regularization), the Frobenius of the random component R_l decays, meaning that the network progressively emphasizes the informative part S_l .
- Removing R_l via pruning reduces the $L2$ loss while maintaining accuracy, with quantitative bounds established through rigorous analysis.

1.4.2 Stability of Accuracy

A further rigorous mathematical analysis in my work examines the stability of DNN accuracy during training (see Section 8). By imposing conditions (such as the uniform doubling condition on the training data), we have proven that high accuracy at an initial time propagates to later training stages. We will also show that DNNs with small singular values in their weight layer matrices require a stricter condition on the training set they are trained on to achieve stability of accuracy. These small singular values, from an RMT framework, would correspond to randomness in the weight layer matrices, which might explain why they are indicative of the DNNs' stability during training.

2 Overview of the analytical and numerical results

Pruning Neural Networks using RMT

My research proposes a novel method for pruning pre-trained DNNs, with a particular emphasis on Vision Transformers. By analyzing the spectral properties of weight matrices, we identify and remove weights associated with eigenvalues corresponding to random noise as predicted by the MP distribution. Conversely, weights corresponding to significant eigenvalues are preserved, as they are critical to the network's function. Applying this RMT-based pruning method to ViTs trained on the ImageNet dataset, we achieved a reduction of 50% in model parameters with less than a 1% loss in accuracy (see Section 4). For pruning without afterward fine-tuning on the ImageNet training set, we can remove 25% of the parameters with less than a 1% drop in accuracy. I believe this represents a state-of-the-art result in pruning for these ViT models. Notably, this method does not require access to validation or test sets during pruning, highlighting its practical applicability.

Furthermore, using RMT-based pruning has shown to **increase** accuracy in fully connected DNNs, in some cases by up to 30% (Section 5). In future work, I plan to study why this increase happens

theoretically; numerically, it seems to occur because of the simplification of the loss landscape. Understanding this phenomenon could lead to more effective pruning strategies that not only reduce model size but also enhance performance.

The rigorous mathematical results

A significant aspect of my work involves developing a theoretical mathematical framework that explains the interplay between randomness in weight matrices, training dynamics, and pruning in deep neural networks. For example, we proved that as DNN weights approach a local minimum in the loss during training, the randomness in the weights vanishes (see Theorem 3.7). Specifically, for DNNs trained with regularization techniques like $L2$ regularization, the random components within the weight matrices decrease as the training progresses. This means that if we model the weight layer matrices W_k with the deformed matrix $W_k = R_k + S_k$, then $\|R_k\|_F \rightarrow 0$ as training progresses (if the DNN weights are approaching a minimum in the loss).

Assumptions for Main Theorems: Gaussian case for a 3-Layer DNN

We have grouped the assumptions for our theorems into 3 main assumptions, which we detail here. Consider a DNN ϕ with three weight matrices W_1 , W_2 , and W_3 , and the absolute value or ReLU activation.

Recall the induced (operator) ∞ -norm of a matrix W with entries w_{ij} :

$$\|W\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |w_{ij}|.$$

Assumption 2.1. Architecture of the DNN: We consider the following DNN ϕ :

$$X(\alpha, s) = \lambda \circ W_3 \circ \lambda \circ W_2 \circ \lambda \circ W_1 s, \quad s \in T, \quad (10)$$

$$\phi(\alpha, s) = \rho \circ X(\alpha, s) \quad (11)$$

where W_1, W_2, W_3 are the weight layer matrices and λ is the absolute value or ReLU activation function, T is the training set, X is the output of the DNN before softmax, and ρ is the softmax. We also denote $\{W_l(t)\}_{l=1}^3$ the weight layer matrices to highlight the dependence on training time t .

We wish to study the behavior of the inner layer W_2 for large N , as training progresses. Thus, we assume the outer layers satisfy:

$\forall s \in T \cup T'$, the union of training and test sets, we have

$$a(N, s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{3/8}} \rightarrow 0 \quad (12)$$

as $N \rightarrow \infty$ (see Subsection B.3 for more details).

Assumption 2.2. Deformed matrix: The $N \times N$ matrix $W_2(t)$, is such that:

$$W_2(t) = R_2(t) + S_2(t), \quad (\text{deformed matrix}) \quad (13)$$

where $R_2(t)$ is a matrix with i.i.d. random normal entries that have mean zero and variance less than $\frac{1}{N}$, i.e. such that $R_2(t)_{i,j} \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \frac{g(t)}{N})$, where $g(t) \leq 1$. While the value of $g(t)$ is unimportant and taken to be less than 1 for simplicity, it is necessary that $\text{Var}[R_2(t)_{i,j}] = \mathcal{O}(\frac{1}{N})$ so that $R_2(t)$ has a bounded spectral norm as $N \rightarrow \infty$.

Remark 2.1. Similar results will apply with a rectangular matrix $W_2(t) \in \mathbb{R}^{N \times M}$ such that

$$\frac{M}{N} \rightarrow c \in (0, +\infty) \quad (14)$$

Assumption 2.3. Low-rank nature of $S_2(t)$: Let λ_+ be the rightmost edge of the Marchenko-Pastur distribution for the empirical spectral distribution of the Wishart matrix $R_2^T(t)R_2(t)$ as $N \rightarrow \infty$. Writing $S_2(t)$ in its singular value decomposition (SVD) form with singular values $\sigma_i(t)$ and singular vectors $u_i(t), v_i(t) \in \mathbb{R}^N$:

$$S_2(t) = \sum_{i=1}^N \sigma_i(t) u_i(t) v_i^T(t) = \sum_{i=1}^r \sigma_i(t) u_i(t) v_i^T(t), \quad (15)$$

the singular values $\sigma_i(t)$ must satisfy:

$$\sigma_i(t) = 0 \quad (N \geq i > r) \quad \text{indicating low-rank nature of } S_b(t) \quad (16)$$

$$\sigma_i(t) > \frac{\sqrt{\lambda_+}}{2} + \epsilon_0 \quad (1 \leq i \leq r) \quad \text{to ensure eigenvalues extend beyond the MP limit.} \quad (17)$$

Here $\epsilon_0 < 1$ and ensures that the r -th singular value of W_2 , denoted $\sigma_r(W_2)$, will be bigger than $\sqrt{\lambda_+}$ as $N \rightarrow \infty$. We assume that as $N \rightarrow \infty$, r remains constant, ensuring the low rank of $S_2(t)$.

Remark 2.2. Important Remark: It is important to note that condition (17) is often not required in our results, and for some results the only requirement is that S_2 be a low-rank matrix with r nonzero singular values (and r independent of N). For example, Theorems 2.4, 2.5 and Corollary 2.6 do not require condition (17). Theorems 2.7 and 2.9 require that S_2 be low ranked and that condition 17 is satisfied.

In deep learning, these assumptions on the weight matrix are based on the behavior of signal and noise in the weights as training progresses. As training unfolds, the weight matrix acquires structure from the data. The assumption that singular values of S_2 (the low-rank part of the weight matrix decomposition) exceed a certain constant reflects the premise that the data signal dominates over random noise. The low-rank nature of S_2 is consistent with the observation that often the structure in the data is low-dimensional. These assumptions can be relaxed, for example, R_2 can be any i.i.d. matrix with some basic assumptions, or a matrix with i.i.d. rows (see Assumption 3.2). For numerical simulations relevant to these assumptions, see [62].

Loss Function

The loss function with Frobenius norm regularization is given by:

$$L(\alpha) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s)) + \mu \sum_{i=1}^3 \|W_i\|_F^2, \mu > 0, \quad (18)$$

where $\|W_i\|_F$ denotes the Frobenius norm of the matrix W_i , μ is the regularization parameter, and $i(s)$ denotes the correct class of the object $s \in T$.

We define the accuracy of the DNN on a set A as the number of objects in A classified correctly by the DNN, divided by the number of total objects in the set. That is:

$$acc_\alpha := acc(\phi(\alpha(t))) := \frac{|\{s \in A : X_{i(s)}(s, \alpha) > \max_{j \neq i(s)} X_j(s, \alpha)\}|}{|A|}, \quad (19)$$

where $|\cdot|$ is the number of elements in a set.

2.1 A theorem on how training reduces randomness in DNNs

Theorem 2.4. *Suppose Assumptions 2.1, 2.2 and 2.3 hold. If the parameters of the DNN α converge to a local minimum of the loss (18), denoted $\alpha^*(N)$, as $t \rightarrow \infty$, then $\forall \epsilon > 0$:*

$$\lim_{N \rightarrow \infty} \lim_{t \rightarrow \infty} \mathbb{P}(\|R_2\|_F \leq \epsilon) \rightarrow 1. \quad (20)$$

The proof of this theorem can be found in Appendix C. As mentioned, for this theorem condition (17) is not required and the only requirement is that S_2 be low ranked.

Explanation: The theorem states that the random part of the weight matrix W_2 vanishes as the network is trained and converges towards a local minimum ($\alpha(t) \rightarrow \alpha^*$). As a consequence of the theorem, it is impossible to decompose W_2 into $R_2 + S_2$, where R_2 satisfies Assumption 2.2 and S_2 satisfies Assumption 2.3. If such a decomposition were possible, the ESD of W_2 would exhibit an MP bulk and spikes (see [3, 7, 14] and Appendices C.2 and H.1). However, at the local minimum, as we will show, W_2 's ESD does not have an MP bulk, ruling out any such decomposition.

It should be noted that for a matrix R with i.i.d, centered Gaussian entries,

$$\frac{1}{N} \|R\|_2^2 \stackrel{a.s.}{\underset{N \rightarrow \infty}{\sim}} 4 \text{Var}[R_{ij}] \quad (21)$$

$$\|R\|_F^2 \stackrel{a.s.}{\underset{N \rightarrow \infty}{\sim}} N^2 \text{Var}[R_{ij}]. \quad (22)$$

Through Assumption 2.2, we assume i.i.d Gaussianity with mean zero, and $\text{Var}[R_{ij}] = \mathcal{O}(\frac{1}{N})$. However, we may see from (22) that this does not straightforwardly imply $\|R\|_F \rightarrow 0$, as the theorem proves. Hence, we prove that the random fluctuations of the weight matrices have to be much smaller than originally assumed.

One might assume that a local minimum of the loss is deterministic, and thus the random part of the weights would have to vanish. While intuitive, this reasoning is not accurate. First, the theorem states that for **any** decomposition of W_2 into $R_2 + S_2$ satisfying the above assumptions we would have that $\lim_{N \rightarrow \infty} \lim_{t \rightarrow \infty} \mathbb{P}(\|R_2\|_F \leq \epsilon) \rightarrow 1$. This means that at a local minimum of the loss if one takes a nonzero random matrix R_2 and then sets $S_2 = W_2 - R_2$, we would have that S_2 will **not** satisfy Assumption 2.3 (with some probability depending on N). Otherwise, the theorem would be violated. Furthermore, the training data T itself might have noise, and so the loss landscape, which depends on the training data, might have some noise/randomness. That is, the local minima might be somewhat random (meaning that the parameters α at the local min might be random). However, when we train with regularization, so long as we can decompose W_2 into $R_2 + S_2$ satisfying Assumptions 2.2 and 2.3, we have that $\lim_{N \rightarrow \infty} \lim_{t \rightarrow \infty} \mathbb{P}(\|R_2\|_F \leq \epsilon) \rightarrow 1$.

It is interesting to note that this phenomenon does not necessarily occur when there is no regularization. In fact, without regularization, one can replace the random matrix R_2 with a different random matrix (or add any random matrix to W_2), and the magnitude of the loss will not change. Thus, around every local minima, there would be a flat region of minima, all corresponding with parameters $W_2 + R_2^*$, with R_2^* being some random matrix satisfying Assumption 2.2.

2.2 A theorem on how pruning randomness in DNNs reduces loss but keeps accuracy the same

We can also show that reducing the randomness in the parameters reduces the loss but preserves accuracy on either the training or test set.

Theorem 2.5. *Suppose Assumptions 2.1, 2.2 and 2.3 hold. Let R_2 be removed so that the second*

weight layer matrix is S_2 alone. Then for any fixed training time t , and for the training or test set, we have

$$\lim_{N \rightarrow \infty} \mathbb{P}(L(\alpha_{S_2}) \leq (L(\alpha_{W_2}) - \mu \|R_2\|_F^2)) \rightarrow 1. \quad (23)$$

Furthermore, suppose only Assumptions 2.1 and 2.2 hold. Then $\exists G(N) \rightarrow 0$ as $N \rightarrow \infty$ s.t.

$$\lim_{N \rightarrow \infty} \mathbb{P}\left(|acc_{\alpha_{W_2}}(t) - acc_{\alpha_{S_2}}(t)| \leq G(N)\right) \rightarrow 1, \quad (24)$$

where α_{S_2} and α_{W_2} represent the DNNs with weight layer matrices S_2 and W_2 respectively, L is the loss function in (18) and μ is the regularization hyperparameter.

The proof of this theorem can be found in Appendix C. As mentioned, for this theorem condition (17) is not required and the only requirement is that S_2 be low ranked.

Explanation: This theorem is useful in that it shows that pruning the random weights in a DNN can speed up training by reducing the loss. Theorem 2.4 states that if training approaches a local minimum, then the randomness vanishes. The question that comes to mind is, can we converge faster to the minimum by removing the randomness of the weights? Theorem 2.5 provides a partial answer to that question, stating that, at the very least, removing the random matrix leads to faster training (in the sense that the loss decreases via pruning).

As I will show in future work, these phenomena are also related to concepts in simple linear regression, where exact solutions exist if the loss has a regularization term.

For a numerical simulation that shows that removing (or adding) a random matrix from the DNN reduces (or increases) its loss, see Example 5.3. Furthermore, the example shows that accuracy does not change when a random matrix is added to the weight layer matrices. The intuition behind this result lies in the perturbations caused by R_2 : the cross-entropy term is not affected by adding or removing R_2 , while the regularization term is influenced by its Frobenius norm and decreases as we remove R_2 . The reason the cross-entropy term is not affected, is because the components of the output vector don't change when we add or remove a random matrix to W_2 , and so the accuracy and cross-entropy terms are also not affected. Theorem 2.5 formalizes this, showing that removing the random part R_2 of the weight matrix W_2 in the DNN $\phi(\alpha, s)$ reduces the loss by an amount proportional to $\|R_2\|_F^2$ while keeping the accuracy the same. Specifically, when $W_2 = S_2$ (i.e., R_2 is removed), the loss satisfies:

$$L(\alpha_{S_2}, s) + \mu \|R_2\|_F^2 \leq L(\alpha_{W_2}, s), \quad (25)$$

with high probability. This implies that the loss decreases by approximately $\mu \|R_2\|_F^2$ when R_2 is removed, as the regularization term involving R_2 vanishes.

This effect can be extended by introducing a scaling factor $\gamma \in [0, 1]$ to R_2 , defining the modified weight matrix as:

$$W_2(\gamma) = \gamma R_2 + S_2. \quad (26)$$

As γ decreases from 1 to 0, the randomness in W_2 is reduced, and the corresponding loss decreases proportionally by $\gamma \mu \|R_2\|_F^2$.

Thus, we can obtain the following corollary:

Corollary 2.6. Suppose Assumptions 2.1, 2.2 and 2.3 hold. For any $\gamma \in [0, 1]$, define the modified second-layer weight matrix by

$$W_2(\gamma) = \gamma R_2 + S_2,$$

and let $\alpha_{W_2(\gamma)}$ denote the corresponding DNN. Then for any fixed training time t , and for the training or test set, we have

$$\lim_{N \rightarrow \infty} \mathbb{P}\left(L(\alpha_{W_2(\gamma)}) \leq \left(L(\alpha_{W_2}) - \gamma \mu \|R_2\|_F^2\right)\right) \rightarrow 1.$$

The proof of this corollary can be found in Appendix C. As mentioned, for this corollary condition (17) is not required, and the only requirement is that S_2 be low ranked. One can also prove the following corollary:

Corollary 2.1. *Suppose Assumptions 2.1, 2.2, and 2.3 hold. Moreover, assume that the weight matrix of the second layer is given solely by the random component, i.e.,*

$$W_2 = R_2 \quad (\text{so that } S_2 \equiv 0).$$

Then the pre-softmax output X_i satisfies, $\forall \epsilon > 0$ and for s an element of the training or test set, we have

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\max_{1 \leq i \leq n} \|X_i\|_\infty < \epsilon \right) = 1. \quad (27)$$

Consequently, since $\text{softmax}(0) = (\frac{1}{K}, \frac{1}{K}, \dots, \frac{1}{K})$, the probability assigned uniformly to all K classes implies that $\forall \epsilon > 0$ the accuracy of the DNN converges in probability to

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\text{acc}_{\alpha_{W_2}}(t) \leq \frac{1}{K} + \epsilon \right) = 1. \quad (28)$$

The pruning theorem and accuracy bounds

Another main theoretical contribution is the development of the *Pruning Theorem*, which provides specific bounds on how much the accuracy of a DNN will drop when we prune the network using MP-based pruning. This pruning is different then the one given in Theorem 2.5, in that we don't remove all of R_2 (which in practice would be hard to do). Rather, we use the MP threshold $\sqrt{\lambda_+}$ to prune the singular values of the DNN, leading to a reducing in rank of W_2 . We also use this reduction in rank to remove parameters from the DNN, and derived conditions under which pruning certain weights—those associated with noise as predicted by the MP distribution—will result in minimal loss of accuracy.

We now introduce the *classification confidence*. Take $X(s, \alpha)$ to be the output of the final layer in our DNN before softmax, see (10). The classification confidence is defined as follows:

$$\delta X(s, \alpha) := X_{i(s)}(s, \alpha) - \max_{j \neq i(s)} X_j(s, \alpha). \quad (29)$$

In other words,

- $\delta X(s, \alpha(t)) > 0 \Rightarrow s$ is well-classified by ϕ .
- $\delta X(s, \alpha(t)) < 0 \Rightarrow s$ is misclassified by ϕ

Theorem 2.7 (The Pruning Theorem for a single object). *Take ϕ to be a 3-layer DNN, see (10), with weight layer matrices satisfying Assumptions 2.2 and 2.3 from above and $s \in T$ or $s \in T'$, with T the training and T' the **test** set. Construct the truncated matrix W'_2 by pruning the singular values of W_2 smaller or equal to $\sqrt{\lambda_+} + \frac{\epsilon_0^2}{2}$, with λ_+ the MP rightmost edge of the ESD of $R_2^T R_2$ and ϵ_0 coming from Assumption 2.3. Then, for the classification confidence threshold of the non-pruned DNN:*

$$E_N(s) := \sqrt{2} \|W_1 s\|_2 \|W_3\|_\infty + 2a(N, s) \quad (30)$$

we have the conditional probability

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\delta X(s, \alpha_{W'_2}) > 0 \mid \delta X(s, \alpha_{W_2}) > E_N(s) \right) \rightarrow 1, \quad (31)$$

with $a(N, s)$ coming from (12). α_{W_2} are the parameters of the DNN that has the weight matrix W_2 and $\alpha_{W'_2}$ are the parameters of the DNN with the weight layer matrix W'_2 .

See Section G for the proof of this theorem.

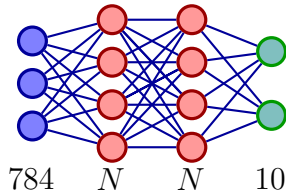
Remark 2.3. Several previous studies, such as [70, 69, 11, 2, 68], have leveraged SVD to prune small singular values from DNN weight matrices, focusing on empirical techniques like energy thresholds and validation set error monitoring. In this way, these works have employed singular value pruning to obtain low-rank approximations of DNN weight layer matrices. This approach is distinguished by the use of RMT to rigorously separate signal from noise. In contrast to heuristic or empirical thresholding, Theorem 2.7 shows that by pruning the singular values of W_2 less than or equal to $\sqrt{\lambda_+} + \frac{\epsilon_0^2}{2}$ —with λ_+ being the rightmost edge of the empirical spectral distribution of $R_2^T R_2$ —one systematically removes the low-rank components corresponding to noise. This RMT-based strategy not only provides a principled criterion for pruning but also guarantees that the resulting low-rank approximation retains the essential information for the network’s predictive performance.

The pruning threshold $\sqrt{\lambda_+} + \frac{\epsilon_0^2}{2}$ ensures that singular values of the signal matrix $S_2(t)$ are preserved while removing those of the noise matrix R_2 . By Assumption 2.3, the singular values of $S_2(t)$, $\sigma_i(t)$, satisfy $\sigma_i(t) > \frac{\sqrt{\lambda_+}}{2} + \epsilon_0$ for $1 \leq i \leq r$, ensuring they exceed $\sqrt{\lambda_+}$. As $N \rightarrow \infty$, Theorem E.4 states that, when, for example, $g(t) = 1$ (see Assumption 2.2) the r largest singular values of W_2 , $\sigma'_i(W)$, converge almost surely to $\frac{1+\sigma_i^2}{\sigma_i}$ (for $\sigma_i > 1$) or 2 (for $\sigma_i < 1$). Given that for $1 \leq i \leq r$ we assume $\sigma_i > \frac{\sqrt{\lambda_+}}{2} + \epsilon_0 > 2$, we have $\sigma'_i(W_2) \rightarrow \frac{1+\sigma_i^2}{\sigma_i}$ which is larger than $\sqrt{\lambda_+} + \frac{\epsilon_0^2}{2}$ (for $\epsilon_0 < 1$) (see Lemma F.4). Meanwhile, the singular values of R_2 are bounded by $\sqrt{\lambda_+}$ almost surely, as $\sigma_{\max}(R_2) \xrightarrow{a.s.} \sqrt{\lambda_+}$. The buffer $\frac{\epsilon_0^2}{2}$ accounts for finite- N Tracy-Widom fluctuations, ensuring that $\sigma'_i(W)$ from $S_2(t)$ exceed the threshold, while all singular values of R_2 are pruned as $N \rightarrow \infty$.

In the numerical part of the paper, the value of λ_+ was determined using the BEMA algorithm (see Subsection H.1.) This algorithm approximates λ_+ by incorporating the Tracy-Widom distribution to account for the finite-size effects and the fluctuations of the largest eigenvalue of $R^T R$. By using this algorithm, we obtain a more accurate estimation of λ_+ for practical, finite-dimensional settings, which is crucial for effectively applying the pruning theorem in real-world scenarios.

These theoretical bounds have been confirmed numerically, demonstrating that MP-based pruning is highly effective in reducing model complexity without significantly impacting accuracy, see Example 2.8. Furthermore, one can expand the theorem to one about DNN accuracy: if γ proportion of the objects $s \in T'$ are such that their classification confidence satisfies $\delta X(s, \alpha_{W_2}) \geq E_N(s)$, then after pruning the DNN accuracy will be bigger than γ (with some probability depending on N), see Theorem 2.9. It is important to note that $E_N(s)$ depends on the variance in R_2 . The smaller the variance of the components of R_2 , the smaller $E_N(s)$ will be.

Example 2.8. Numerically, we trained a three-layer DNN on Fashion MNIST, iterating the width N of the middle layer matrix W_2 (it is a square matrix) from 500 to 10,000, and the initialization of matrices having entries i.i.d. from $\mathcal{N}(0, \frac{1}{N^2})$ (this is considered a small variance for DNN initialization). The DNN is trained on Fashion MNIST for 20 epochs. An illustration of this DNN is given below:



When $N = 10,000$, we obtained a 89.5% accuracy on the test set, and 89.1% had their classification confidence was bigger than $E_N(s)$. That is, based on Theorem 2.7, only .4% of the DNN’s accuracy would be lost if we pruned the DNN’s singular values using the RMT threshold λ_+ . After pruning, the

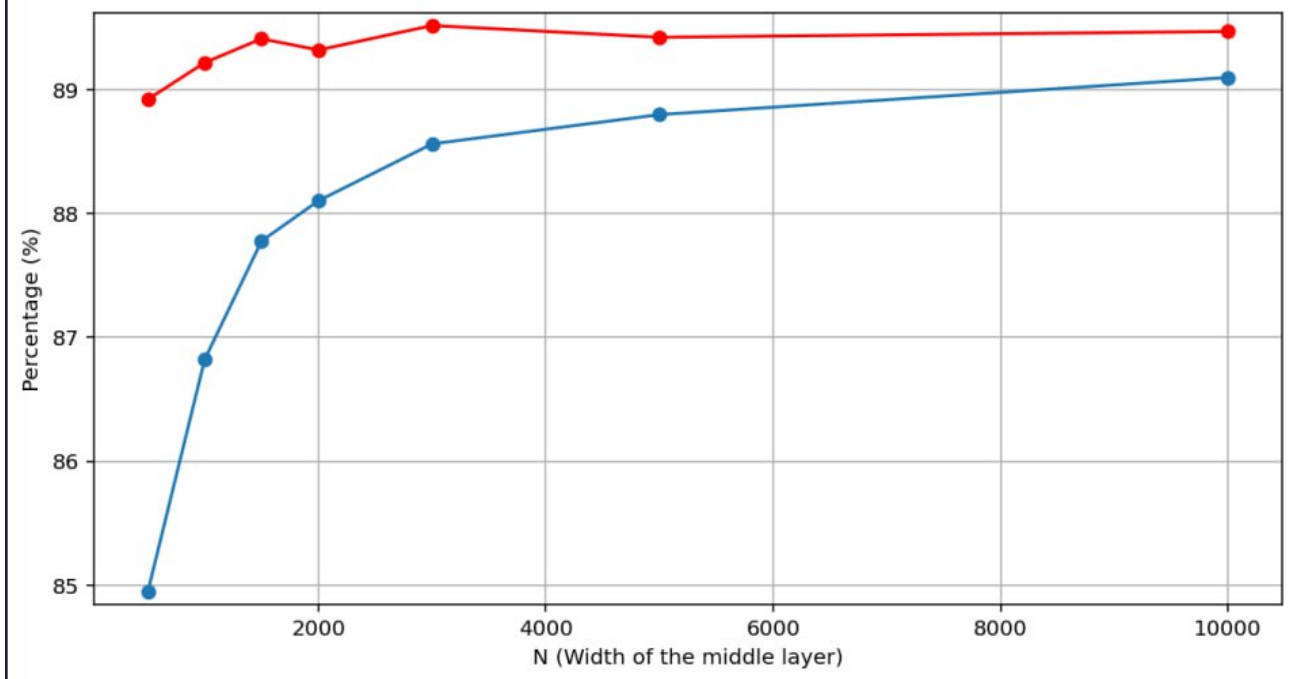


Figure 2: Accuracy of DNN before pruning (red); Lower theoretical bound on the accuracy of pruned DNN (blue). The difference between red and blue values decreases from 4% for $N = 500$ to 0.6% for $N = 10,000$.

accuracy indeed only dropped by .3%, see Fig. 2. The Pruning Theorem explains why RMT is particularly well-suited for pruning DNNs. By identifying and removing singular values (or DNN weights) associated with random noise, we simplify the network while preserving the essential signal necessary for accurate predictions.

This theorem can also be extended to the accuracy of the whole DNN. First, define, for a set T' :

$$\text{acc}_{\alpha,0}(t) = \frac{\#\{s \in T' \text{ s.t. } \delta X(s, \alpha) > 0\}}{\#T'} \quad (32)$$

$$\text{acc}_{\alpha,E_N(s)}(t) = \frac{\#\{s \in T' \text{ s.t. } \delta X(s, \alpha) > E_N(s)\}}{\#T'} \quad (33)$$

Then, we can state the following theorem:

Theorem 2.9 (The Pruning Theorem for accuracy). *Under the conditions of Theorem 2.7, we have:*

$$\mathbb{P} \left(\text{acc}_{\alpha_{W'_2},0}(t) \geq \text{acc}_{\alpha_{W_2},E_N(s)}(t) \right) \rightarrow 1 \text{ as } N \rightarrow \infty. \quad (34)$$

Proof. By the proof of Theorem 2.7, after pruning, the classification confidence of each object s is reduced by at most $E_N(s)$, with probability going to 1 as N goes to infinity. Theorem 2.9 then follows, given that the elements of the training or test sets are finite. $\square$

2.3 A theorem of stability of accuracy

We have also studied the stability of accuracy during the training of DNNs. In this context, the training of a DNN is performed via the minimization of a cross-entropy loss function, and the performance metric is accuracy (the proportion of objects that are classified correctly). While training results in a decrease

in loss, the accuracy does not necessarily increase during the process and may sometimes even decrease. The goal of achieving stability of accuracy is to ensure that if accuracy is high at some initial time, it remains high throughout training. In the third chapter of this dissertation, I show that for a training set T that satisfies the *uniform doubling condition*, there exists a family of DNNs such that a DNN from this family with high accuracy on the training set at some training time t_0 will have high accuracy for all time $t > t_0$.

The uniform doubling condition is essentially a condition on the data that ensures that the training data is more or less uniformly distributed. Here the training set $T \subset \mathbb{R}^n$ and the uniform doubling condition is essentially satisfied if a ball in $\mathbb{R}^n$ of radius 2 has twice as many data points as a ball with the same center having radius 1 (this is a simplification of the condition).

I show that high accuracy at some initial time implies high accuracy for future time. That is, for some time t_0 during training, we would like that:

$$\text{high acc}(t_0) \rightarrow \text{high acc}(t), \forall t > t_0. \quad (35)$$

This is achieved with the following steps:

$$\text{high acc}(t_0) \rightarrow \text{small loss}(t_0) \rightarrow \text{small loss}(t) \rightarrow \text{high acc}(t). \quad (36)$$

The step $\text{high acc}(t_0) \rightarrow \text{small loss}(t_0)$ is the most difficult part and is achieved by introducing the uniform doubling condition on the training set. The step $\text{small loss}(t_0) \rightarrow \text{small loss}(t)$ is fundamental to training because training is essentially the minimization of loss, and the step $\text{small loss}(t) \rightarrow \text{high acc}(t)$ is achieved using the inequality:

$$1 - \text{acc}(t) \leq \frac{\text{loss}(t)}{\log(2)}. \quad (37)$$

The idea of stability of accuracy is, therefore, the following statement: **Stability of accuracy:** if accuracy is high at t_0 and for some $t > t_0$ we have that $\text{loss}(t) \leq \text{loss}(t_0)$ then accuracy at t will be high. I also confirm numerically that having training data that is uniform in $\mathbb{R}^n$ leads to a stronger correlation between the increase in accuracy and minimization of loss (and vice versa).

Chapter Two

Pruning in Deep Learning using Random Matrix Theory

3 Analytical results on RMT methods in Deep Learning

In this section, we generalize the previous RMT results for DNNs with different architectures and for the case where the random matrix R is not necessarily i.i.d. Gaussian.

3.1 Assumptions for theorems in Chapter 2

Assumption 3.1. Consider a DNN denoted by ϕ , and assume that it can be written as:

$$\phi = \rho \circ \psi_2 \circ (R + S) \circ \psi_1, \quad (38)$$

where ψ_1 and ψ_2 are arbitrary functions and ρ is softmax. Take $W = R + S$ and assume that the matrices R and S satisfying the following assumptions:

Assumptions on the matrices R and S

We considered a class of admissible matrices W , where $W := R + S$ and W , R , and S satisfy the following three assumptions. The first assumption is a condition on R :

Assumption 3.2. Assume R is a $N \times M$ matrix such that for any vector v we have:

$$\mathbb{P}(\|Rv\|_\infty > d_1(N)J(v)) \leq d_2(N), \quad (39)$$

with $d_1(N), d_2(N) \rightarrow 0$ as $N \rightarrow \infty$ and $J(v)$ some function that depends on v alone (note, v depends on M). Further, as $N \rightarrow \infty$, we have that $\sigma_{\max}(R) \rightarrow \sqrt{\lambda_+}$ a.s.

Example 3.3. For example, when R is i.i.d. Gaussian, one can show, using the Borell-TIS inequality (see Theorem B.2), that:

$$\mathbb{P}\left(\|Rv\|_\infty > \left(\sqrt{\frac{2 \log N}{N}} + \frac{1}{N^{\frac{3}{8}}}\right)\|v\|_2\right) \leq 2 \exp\left(-\frac{N^{\frac{1}{4}}}{2}\right)$$

See Subsection B.2 for a proof.

We then assume the following for the matrix S :

Assumption 3.4. Assume S is a matrix with $S = \sum_{i=1}^r \sigma_i u_i v_i^T = U \Sigma V^T$, with σ_i the singular values and u_i, v_i^T column and row vectors of U and V . Thus, S has r non-zero singular values corresponding to the diagonal entries of Σ , and all other singular values of S are zero.

Finally, we assume for $W = R + S$:

Assumption 3.5. Take σ_i to be the singular values of S , with corresponding left and right singular vectors u_i and v_i^T , and σ'_i to be the singular values of $W = R + S$, with corresponding left and right singular vectors u'_i and $v_i'^T$. First we assume that $\frac{N}{M} \rightarrow c \in (0, +\infty)$ as $N \rightarrow \infty$. Second, assume also that we know explicit functions $g_{\sigma_i, R}$, $g_{v_i, R}$ and $g_{u_i, R}$ such that as $N \rightarrow \infty$:

$$\sigma'_i(W) \xrightarrow{a.s.} \begin{cases} g_{\sigma_i, R} & \sigma_i > \bar{\theta}(\lambda_+) \\ \sqrt{\lambda_+} & \sigma_i < \bar{\theta}(\lambda_+), \end{cases} \quad (40)$$

$$|\langle u'_i, u_i \rangle|^2 \xrightarrow{a.s.} \begin{cases} g_{u_i, R} & \sigma_i > \bar{\theta}(\lambda_+) \\ 0 & \sigma_i < \bar{\theta}(\lambda_+), \end{cases} \quad (41)$$

and

$$|\langle v'_i, v_i \rangle|^2 \xrightarrow{a.s.} \begin{cases} g_{v_i, R} & \sigma_i > \bar{\theta}(\lambda_+) \\ 0 & \sigma_i < \bar{\theta}(\lambda_+). \end{cases} \quad (42)$$

Third, also assume that for $i \neq j$:

$$|\langle v'_i, v_j \rangle|^2 \xrightarrow{a.s.} 0 \quad (43)$$

and

$$|\langle u'_i, u_j \rangle|^2 \xrightarrow{a.s.} 0. \quad (44)$$

Here we take $\bar{\theta}(\lambda_+)$ to be a known explicit function depending on λ_+ , for example see (153). We assume that $\sigma_i > \bar{\theta}(\lambda_+)$. For the function $g_{\sigma_i, R}$ we assume the following: if $\sigma_i > \bar{\theta}(\lambda_+)$ then $\sigma'_i(W) > \lambda_+$ a.s., furthermore, $\sigma'_i(W) > \sigma_i$ a.s. For γ a variable between 0 and 1, take $W(\gamma) = \gamma R + S$. Then, all the singular values of $W(\gamma)$ decrease monotonically a.s. as $N \rightarrow \infty, \gamma \rightarrow 0$ (see Theorem C.1 as an example). For example, for R and S satisfying Assumptions 2.2-2.3, we have that this assumption is also satisfied.

Remark 3.1. Important Remark: It is important to note that Assumption 3.5 is often not required in our results, and the only requirement is that for γ a variable between 0 and 1, take $W(\gamma) = \gamma R + S$. Then, all the singular values of $W(\gamma)$ decrease monotonically a.s. as $N \rightarrow \infty, \gamma \rightarrow 0$. For example, Theorems 3.7, 3.8, and Corollary 3.9 do not require Assumption (3.5) (and only require the simple assumption mentioned). Theorems 3.14 and 3.17 require Assumption 3.5.

In the Pruning Theorem (3.14), we assume that a weight matrix W of the DNN satisfies Assumptions 3.2-3.5, that is $W = R + S$, with R a random matrix, S a low-rank matrix, and that the non-zero singular values of S are bigger than some threshold $\bar{\theta}(\lambda_+)$. Empirically, it has been observed that these assumptions are reasonable for weight matrices of a DNN; see [62, 57]. There are various spiked models in which Assumptions 3.2-3.5 hold, for more on the subject see [17, 3, 7, 16, 14, 5, 47, 1, 12, 4, 37, 72, 16, 46, 35]. Also, a number of works in RMT address the connection between a random matrix R and the singular values and singular vectors of the deformed matrix $W = R + S$, see [6, 7].

For example, one can show that the following two simpler properties on the matrices R and S are sufficient to ensure that R and S satisfy the above Assumptions 3.2-3.5. Recall that a bi-unitary invariant random matrix R is a matrix with components taken from i.i.d.s such that for any two unitary matrices U and V^T , the components of the matrix URV^T have the same distribution as the components of R . We then assume the following properties:

Property 1 (statistical isotropy): Assume R to be a bi-unitary invariant random $N \times M$ matrix with components taken from i.i.d.s with zero mean and variance $\frac{1}{N}$.

We then assume the following property for S :

Property 2 (low rank of matrix): Assume S is a low-rank matrix with $S = \sum_{i=1}^r \sigma_i u_i v_i^T = U \Sigma V^T$, with σ_i the singular values and u_i, v_i^T column and row vectors of U and V . Thus, S has r non-zero singular values contained on the diagonal entries of Σ , and all other singular values are zero. We also assume that these r singular values of S have multiplicity 1. Finally, we assume that $\frac{N}{M} \rightarrow c \in (0, +\infty)$ as $N \rightarrow \infty$.

An explicit relationship between Properties 1-2 and Assumptions 3.2-3.5 can be found in [7]. The Property 1 is indeed strong, as it implies that the random matrix R is random in every direction. In other words, for any unitary matrices U and V^* , the matrix URV^* has the same distribution as R . Random matrices with complex Gaussian entries, also known as Ginibre matrices, are a class of random matrices that are bi-unitary invariant [32].

Assumptions 3.4 and Property 2 are related to the low-rank property of the matrix S , see [62, 57] for how this assumption is related to DNNs. Often, we consider the case where we initialize the weight layer of a DNN using a Gaussian random matrix (see Example C.1) and, after training, we obtain that $W = R + S$ with R still a Gaussian random matrix and S having low rank. The Pruning Theorem 3.14 can then be employed to determine that removing the small singular values of W will not affect much the accuracy of the DNN. This is because the deformed model $W = R + S$ satisfies Assumptions 3.1-3.5, see Example C.1. This insight can be used to reduce the number of parameters in the DNN without sacrificing its performance, as will be further discussed in Subsection H.3.

For a DNN ϕ satisfying Assumption 3.1, we start by defining,

$$g_\phi(s) := \|\psi_1 s\|_2 \|\psi_2\|_{2 \rightarrow 2}, \quad (45)$$

where $\|\cdot\|_{2 \rightarrow 2}$ is the induced 2 operator norm (or global Lipschitz constant with respect to the ℓ_2 norm)

$$\|A\|_{2 \rightarrow 2} := \sup_{v \neq w} \frac{\|A(v) - A(w)\|_2}{\|v - w\|_2} \quad (46)$$

(for the case $\|\psi_2\|_{2 \rightarrow 2}$). Note that $\psi_1 s$ is simply a vector. We also define,

$$h_\phi(s) := J(\psi_1 s) \|\psi_2\|_{\max \rightarrow \max}, \quad (47)$$

where $\|\cdot\|_{\max \rightarrow \max}$ is the induced operator norm (or global Lipschitz constant with respect to the $\ell_{\max}$ norm)

$$\|A\|_{\max \rightarrow \max} := \sup_{v \neq w} \frac{\|A(v) - A(w)\|_{\max}}{\|v - w\|_{\max}} \quad (48)$$

and $J(\cdot)$ is given in (39).

Example 3.6. For example, take ϕ to be a fully connected DNN with $W_1, W_2, \dots, W_L$ and $\beta_1, \beta_2, \dots, \beta_L$ the weight matrices and bias vectors of the DNN. We assume the use of the absolute value or ReLU activation functions. Assume that Assumption 3.1-3.5 holds for this DNN, that is that $\exists b$ such that for the layer matrix W_b we have $W_b = R_b + S_b$, with R_b and S_b satisfying Assumptions 3.2-3.5. For a vector s , define:

$$(W_n + \beta_n)s := (W_n s + \beta_n). \quad (49)$$

Then

$$g_\phi(s) \leq \|(W_{b-1} + \beta_{b-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1)s\|_2 \|(W_{b+1} + \beta_{b+1}) \circ \dots \circ (W_L + \beta_L)\|_{2 \rightarrow 2}, \quad (50)$$

and

$$h_\phi(s) \leq \|(W_{b-1} + \beta_{b-1}) \circ \cdots \circ \lambda \circ (W_1 + \beta_1)s\|_2 \|(W_{b+1} + \beta_{b+1}) \circ \cdots \circ (W_L + \beta_L)\|_{\max \rightarrow \max}, \quad (51)$$

where s is an element of the test or training set.

We now formulate theoretical results that provide a rigorous relationship between pruning and accuracy.

3.2 Analytical results for RMT-based decrease in loss

In the context of analyzing DNNs, we consider the cross-entropy loss function with L2 regularization for a DNN $\phi(\cdot, \alpha(t))$, expressed as follows:

$$L(\alpha(t)) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(t))) + \mu \sum_{i=1}^L \|W_i(t)\|_F^2, \quad (52)$$

where W_i are the weight layer matrices of the DNN, and $i(s)$ denotes the correct class of the object s . The term μ acts as a regularization constant, which is employed to prevent overfitting in the DNN.

Theorem 3.7. *Let ϕ be a DNN satisfying Assumptions 3.1-3.5. Assume the following:*

1. $\forall s \in T$, the quantity $a_\phi(N, s) := h_\phi(s)d_1(N) \rightarrow 0$ as $N \rightarrow \infty$ (with $h_\phi(s)$ defined in (47) and $d_1(N)$ coming from (39) see Subsection B.3 for more details).
2. The loss function (52) attains a local minimum at $\alpha^*(N)$, where N is the size of the matrix W .
3. As training time $t \rightarrow \infty$, the parameter $\alpha(t, N)$ converges to $\alpha^*(N)$.

Then, $\forall \epsilon > 0$:

$$\lim_{N \rightarrow \infty} \lim_{t \rightarrow \infty} \mathbb{P}(\|R\|_F \leq \epsilon) \rightarrow 1. \quad (53)$$

The proof of this theorem can be found in Appendix C.

For a simplified version of this theorem, when ϕ only has three layer matrices, see Theorem 2.4. In such a case, we have $W = W_2$. This theorem states that the probability that the norm of the random matrix R exceeds a threshold ϵ goes to zero. This theorem implies that, for W , it is impossible to find any matrix R, S satisfying Assumptions 3.1-3.5 such that $W = R + S$. For simplicity, assume that R is a random matrix with i.i.d. components such that $\sqrt{N}R$ satisfies the assumptions of Theorem 1.2. If such a decomposition were possible, the ESD of $W = R + S$ would exhibit an MP bulk and spikes, see [3, 7, 14] and Appendices C.2 and H.1. However, as we will prove, at the local minimum, the ESD of W does not display an MP bulk and spikes, which implies that such a decomposition is not possible under the given assumptions.

This theorem examines the behavior of weight layer matrices in a deep neural network as it approaches a local minimum during training. Specifically, it focuses on the evolution of the matrix $W = R + S$.

The theorem states that as the training progresses (i.e., as t approaches infinity), if the parameters of the DNN converge towards a local minimum ($\alpha(t) \rightarrow \alpha^*$), the influence of the random component $R(t)$ in the weight matrix W diminishes. This is quantified by showing that the probability of the norm of R being greater than a small threshold ϵ becomes increasingly small. In other words, the random part of the weight matrix W effectively vanishes as the network is trained, leading to a more deterministic behavior of the weight matrix at the local minimum.

The result leverages RMT to explain the behavior of the DNN during training. The cross-entropy term in the loss function, as defined in Equation (52), is responsible for learning patterns from the training data. However, when the data exhibits randomness, the DNN may inadvertently learn this randomness, causing the weights in the network to reflect the random nature of the data. For more on this behavior, see [57].

To mitigate this randomness in the parameters, the L2 regularization term plays a crucial role. By penalizing large weights, the L2 regularization helps the network reduce the magnitude of weights corresponding to noisy or random patterns in the data. This is achieved by minimizing the regularization term in the loss. Assuming that this minimization in L2 of the weights does not increase the cross entropy term in the loss by μ (L2 reduction), the regularization term will "remove" the random parameters in the DNN. As training progresses, this regularization term forces the weights associated with randomness to approach zero, effectively eliminating the random component of the DNN's weight matrices. The result is a more deterministic network, where the DNN focuses on the true patterns in the data while diminishing the influence of noise. It is important to note that Assumption 3.4, which ensures that the matrix $S(t)$ has no small nonzero singular values, essentially is a statement about the data T . It says that any noise from the data cannot be (in some sense) "much larger" than the deterministic part of the data (relative also to the hyperparameter μ).

We can also show that reducing the randomness in the parameters reduces the loss:

Theorem 3.8. *Assume the setup and conditions of Theorem 3.7, particularly regarding the DNN ϕ satisfying Assumptions 3.1-3.5. Assume that $\forall s \in T \cup T'$, the quantity $a_\phi(N, s) := h_\phi(s)d_1(N) \rightarrow 0$ as $N \rightarrow \infty$.*

Let R be removed such that the second weight layer matrix is S alone. Then for any fixed time t and for the training and test sets, we have

$$\lim_{N \rightarrow \infty} \mathbb{P}(L(\alpha_S) \leq (L(\alpha_W) - \mu \|R\|_F^2)) \rightarrow 1 \quad (54)$$

Furthermore, if Assumptions 3.1-3.2 hold (and $a_\phi(N, s) := h_\phi(s)d_1(N) \rightarrow 0$ as $N \rightarrow \infty$), then $\exists G(N) \rightarrow 0$ as $N \rightarrow \infty$ s.t.

$$\lim_{N \rightarrow \infty} \mathbb{P}\left(|acc_{\alpha_W}(t) - acc_{\alpha_S}(t)| \leq G(N)\right) \rightarrow 1, \quad (55)$$

where α_S and α_W are the parameters of the DNNs with weight layer matrices S and W respectively, L is the loss function in (52) and μ is the regularization hyperparameter.

Remark 3.2. While this theorem focuses on pruning only one weight layer matrix, in practice, we might prune many layer matrices one after the other. Nevertheless, one can apply this theorem multiple times, to multiple layers, to obtain similar results.

For a simplified version of this theorem in the case where the DNN is fully connected with 3 weight layer matrices, see Theorem 2.5. For a numerical simulation that shows that removing (or adding) a random matrix from the DNN reduces its loss, see Example 5.3. One can also generalize Corollary 2.1 for the situation given under Assumptions 3.1-3.5.

This effect can also be extended by introducing a scaling factor $\gamma \in [0, 1]$ to R , defining the modified weight matrix as:

$$W(\gamma) = \gamma R + S. \quad (56)$$

As γ decreases from 1 to 0, the randomness in W is reduced, and the corresponding loss decreases proportionally by $\gamma\mu\|R\|_F^2$.

Thus, we can obtain the following corollary:

Corollary 3.9. *Suppose Assumptions 3.1-3.2 hold (and $a_\phi(N, s) := h_\phi(s)d_1(N) \rightarrow 0$ as $N \rightarrow \infty$). For any $\gamma \in [0, 1]$, define the modified second-layer weight matrix by*

$$W(\gamma) = \gamma R + S,$$

and let $\alpha_{W(\gamma)}$ denote the corresponding DNN. Then, for any fixed training time t , and for the training and test sets, we have

$$\lim_{N \rightarrow \infty} \mathbb{P}\left(L(\alpha_{W(\gamma)}) \leq \left(L(\alpha_W) - \gamma \mu \|R\|_F^2\right)\right) \rightarrow 1.$$

3.3 Key Lemmas: how removing randomness from DNN weight layers affects the output, loss, and accuracy of a DNN

In order to prove the main result, we will consider a few critical lemmas. These lemmas describe how the output, accuracy, and loss of a DNN remain largely unaffected when we replace the matrix W with S , its low-rank (or highly correlated) part. In other words, the random component of W does not significantly affect the loss, accuracy, and output of the DNN. This outcome should be expected, as the random part should act more like noise, without causing substantial changes in the loss and accuracy. The following lemmas illustrate this concept.

Recall: We define the final output of the DNN before softmax as:

$$X = \psi_2 \circ (R + S) \circ \psi_1(s). \quad (57)$$

Lemma 3.10. *Given (57), let $W = R + S$ be a $N \times M$ matrix such that $W(t) = R(t) + S(t)$, with $R(t)$ a random matrix satisfying Assumption 3.2.*

Suppose we replace the weight layer matrix W with the matrix S . Then

$$\mathbb{P}\left(|X_i(s, \alpha_S) - X_i(s, \alpha_W)| \leq d_1(N)h_\phi(s)\right) \geq 1 - d_2(N). \quad (58)$$

See (39) for the details on $d_1(N)$ and $d_2(N)$, and (47) for the details of $h_\phi(s)$. Here, α_S are the parameters of the DNN, which has the weight matrix S , and α_W are the parameters of the DNN with the weight layer matrix W .

For a proof of the lemma, see Subsection B.2.

Essentially, this lemma discusses the impact of removing a random matrix component from a weight matrix. For explanation purposes, we assume that R is i.i.d. Gaussian.

- **Lemma's Claim:** It states that under this replacement, the difference in outputs (represented by X_i) of the DNN with original weight matrix W and the DNN with S is small with high probability. Specifically, the probability that the output difference exceeds $d_1(N)h_\phi(s) = a(N, s)$ is less than $d_2(N) = 2 \exp(-\frac{N^{1/4}}{2})$.

- **Intuitive Explanation:**

- *Role of Randomness:* In DNNs, especially wide ones, random values in weights will have a diminishing effect on the network's behavior due to averaging effects.
- *Dominance of highly correlated Part:* The low-rank matrix $S(t)$ likely encapsulates the essential structure the network has learned, making the random component $R(t)$ less critical, particularly in large networks.
- *Small Difference in Output:* Thus, it makes sense that removing the random matrix will not affect the output of the DNN.

In summary, the lemma states that in a DNN with a large number of neurons, the specific random values in the weight matrix do not play a major role in the network's function.

The Pruning Theorem: how MP-based pruning affects DNN accuracy

In this section, we introduce the Pruning Theorem, which provides the relationship between the accuracy of a DNN before and after being pruned. The central idea in these results can be described as follows. Suppose a weight layer W of the DNN satisfies the above Assumptions 3.1-3.5. Then, the removal of small singular values of W , smaller than the MP-based threshold $\sqrt{\lambda_+}$, does not change the classification confidence of an object $s \in \mathbb{R}^n$ by a "large amount" (see (30)). That is, the classification confidence before and after pruning is essentially the same for sufficiently large matrix W .

In essence, these results suggest that it is possible to maintain the performance of the DNN while reducing the number of parameters by eliminating the small singular values, which are considered less influential in terms of the network's overall accuracy. This insight can be used to create more efficient DNN architectures, leading to reduced computational complexity and memory requirements without sacrificing model performance, see Example 5.1.

3.4 How pruning affects classification confidence (for low-rank weight layer matrices)

Now, we state a theoretical result that shows how pruning the singular values of the weight layers of a DNN impacts the DNN accuracy. This result is not based on RMT but will help understand the results that follow. In the following lemma, we assume that we are given a threshold $\sqrt{\lambda_+}$, which we use to prune the singular values of the layers of the DNN. In general, this threshold is given by the MP distribution and numerically can be found using the BEMA algorithm, see Subsection H.1.

Lemma 3.11. *Let ϕ be a DNN satisfying Assumptions 3.1-3.5. Assume we prune the layer matrix W to obtain W' by removing singular values of W smaller than $\sqrt{\lambda_+}$. For any input s (either from the training set T or the test set T'), denote the change in classification confidence due to pruning as:*

$$\Delta(\delta X) = |\delta X(s, \alpha_W) - \delta X(s, \alpha_{W'})|. \quad (59)$$

Here $X(s, \alpha_W)$ and $X(s, \alpha_{W'})$ are the outputs of the final layer before softmax of the DNN with weight layer matrices W and W' respectively.

Then

$$\Delta(\delta X) \leq \sqrt{2\lambda_+} \|\psi_2\|_{2 \rightarrow 2} \|\psi_1 s\|_2, \quad (60)$$

See Subsection I.1 for a proof of this Lemma.

Remark 3.3. Let $W_1, W_2, \dots, W_L$ and $\beta_1, \beta_2, \dots, \beta_L$ be the weight matrices and bias vectors of a DNN with the absolute value activation function. For the simplified case when the bias vectors are zero, this lemma says that the change in classification confidence δX after pruning is bounded by:

$$\Delta(\delta X) \leq \sqrt{2\lambda_+} \|\lambda \circ W_{b-1} \circ \dots \circ \lambda \circ W_1 s\|_{2\sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L)}. \quad (61)$$

This means that if elements were well classified before the pruning and $\sqrt{2\lambda_+} \|\lambda \circ W_{b-1} \circ \dots \circ \lambda \circ W_1 s\|_{2\sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L)}$ is small relative to $\delta X(s, \alpha_W)$, then after pruning s will stay accurately classified. When the basis vectors are not zero, we have:

$$\Delta(\delta X) \leq \sqrt{2\lambda_+} \|\lambda \circ (W_{b-1} + \beta_{b-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1) s\|_{2\sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L)}. \quad (62)$$

A crucial observation from the lemma is the product of the maximum singular values, denoted as $\sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L)$. These singular values can be considerably large, implying that their product can amplify the magnitude of the bound (61), thereby making it substantial.

At first glance, this might seem concerning as it suggests that pruning might lead to a large drop in the network's accuracy. However, this is not necessarily a grave issue. Subsequent sections will introduce

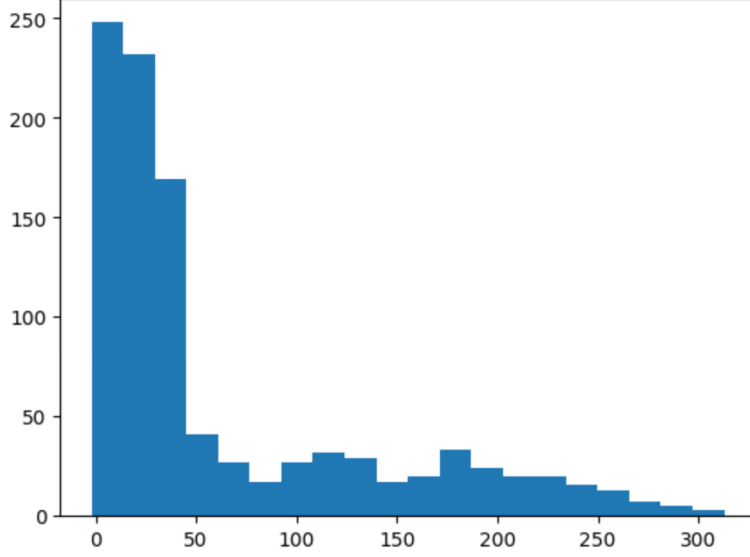


Figure 3: Histogram of δX of the test set for the final epoch of training. On the x-axis, we have the size of δX .

two more theoretical results based on RMT, which will elucidate why, in practice, this potential drop in classification confidence due to pruning does not occur.

Furthermore, it is crucial to note that the lemma provides a worst-case scenario. In real-world scenarios, the actual impacts of pruning are expected to be much milder than what the lemma indicates. This is a common theme in theoretical computer science and machine learning: the worst case doesn't always reflect the average or typical case.

Furthermore, the product $\sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L)$ is used as a naive bound on the Lipschitz constant of the function $W_L \circ \lambda \circ \dots \circ \lambda \circ W_{b+1}$. In practice, this value can be substantially smaller. There exist other methodologies for estimating the Lipschitz constant of this function, which might yield a more conservative estimate. See [20] for more on the numerical estimation of the Lipschitz constant in deep learning.

Another practical takeaway from this theorem is the preference to prune the final layers of the DNN rather than the earlier layers. The reasoning is simple: pruning the latter stages has a lesser effect on the overall accuracy, making it a safer bet in terms of maintaining the network's performance. However, this also depends on $\|\lambda \circ (W_{b-1} + \beta_{b-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1)s\|_2$ which depends on the earlier layers in the network.

In essence, while the theorem paints a potentially alarming picture of pruning's effects, practical simulations and further theoretical results can assuage these concerns. The nuanced understanding provided by the theorem can guide efficient pruning strategies, ensuring minimal loss in accuracy.

Example 3.12. *The following example shows the histogram of δX of a trained DNN for the problem given in Subsection I.4. We train a DNN with one hidden layer. The weight layer matrices W_1, W_2 were initialized with components taken from i.i.d.s, normally distributed with zero mean and variance $\frac{1}{N_i}$. We obtained a 98% accuracy on the training set, which had 1000 objects. δX of the test set, after the 600th epoch of training, is given in Fig. 3.*

For the most part, we have $\|s\|_2 \leq \sqrt{2}$ and if we were to prune the first layer of the DNN, we would obtain $\sqrt{2\lambda_+} \sigma_{\max}(W_2) \|s\|_2 \leq 6.5$. However we see that for many objects s , δX can be much larger than 6.5.

Next, we would like to obtain a better result than Lemma 3.11 using the properties of random

matrices.

3.5 Key technical Lemma: removing random weights for DNN with an arbitrary many layers does not affect classification confidence

The following lemma describes how the classification confidence changes when the weight layer matrix $W = R + S$ is changed with the weight layer matrix S – the ultimate pruning.

Lemma 3.13. *Take ϕ to be a DNN satisfying Assumptions 3.1-3.5 and fix object s from the test set T' (or training set T). Suppose we replace the weight layer matrix W with the low-rank matrix S . Then we have that there $\exists D(N), a(N)$ such that for the classification confidence threshold of the non-pruned DNN:*

$$E' := 2d_1(N)h_\phi(s) \quad (63)$$

we have the conditional probability

$$\mathbb{P}\left(\delta X(s, \alpha_S) > 0 \mid \delta X(s, \alpha_W) > E'\right) \geq 1 - D(N), \quad (64)$$

with $D(N), d_1(N) \rightarrow 0$ as $N \rightarrow \infty$ and $h_\phi(s)$ coming from (47). Here, α_S are the parameters of the DNN, which has the weight matrix S , and α_W are the parameters of the DNN with the weight layer matrix W .

Here, when R has components i.i.d. from $N(0, \frac{1}{N})$ then $d_1(N) = \frac{1}{N^{\frac{3}{8}}} + \sqrt{\frac{2 \log N^2}{N}}$ and we have that $D(N) = 2 \exp\left(-\frac{N^{1/4}}{2}\right)$. For a proof of the Lemma, see Subsection B.2.

Remark 3.4. Here, we take s from the test set T' ; however, the result also holds if we take s from the training set T . Furthermore, using a proof similar to the one given in Subsection B.2, one can show a more general result. That is, taking

$$\Delta(\delta X) := |\delta X(s, \alpha_{S_b}) - \delta X(s, \alpha_{W_b})|, \quad (65)$$

we have that

$$\mathbb{P}\left(\Delta(\delta X) \leq E'\right) \geq 1 - D(N). \quad (66)$$

Remark 3.5. Lemma 3.13 addresses the removal of parameters while preserving accuracy in a more general context than MP-based pruning. In particular, it also explains the numerics of parameter removal via sparsification (see Subsection A.0.1). Indeed, in Lemma 3.13 when the random matrix R has entries taken from i.i.d.s with zero mean and variance $\frac{1}{N}$ as $N \rightarrow \infty$ the entries of R are small w.r.t. sparsification threshold $\xi(N)$. If, in addition, we assume that the entries of S are large (c.f. Assumption 3.4), then large entries of $W = R + S$ are entries of S with high probability. Therefore, sparsifying W by removing the entries smaller than the $\xi(N)$ amounts to replacing the weight layer matrix W with the matrix S . Therefore, Lemma 3.13 implies that sparsification does not change the DNN's outputs and preserves accuracy in the sense of (58).

Remark 3.6. Imagine you are trying to predict the weather. Initially, your prediction is based on both the randomized patterns you have observed over time (noise in the input layer) and your initial random weights of the DNN (matrix R) and the deterministic factors you are sure of (matrix S). Now, if you decide to base your prediction just on the deterministic factors (that is, totally remove the random part of the weight layer matrix). Then, how much would your confidence in the prediction change? This Lemma provides a bound on that change.

The Lemma states that there exists a function $D(N) \rightarrow 0$ as $N \rightarrow \infty$ (the size of the matrix) increases. The magnitude of this change in classification confidence (how much our "confidence" drops when we remove the random part) is given by E' .

Most importantly, the conditional probability states that if our initial confidence (with the random matrix) was bounded away by $E' > 0$, then after removing the randomness, our confidence would most likely be at least 0. And as the size N of the matrix increases, the probability that our classification confidence would be bigger than 0 becomes closer to 1.

In essence, even if we remove the randomness from our prediction model (in this case, the DNN), we can still be quite confident about our predictions, especially as our layer widths grow.

Remark 3.7. We want to understand what happens to the classification confidence threshold E' in (64) as $N \rightarrow \infty$. Assuming that

$$h_\phi(s) \leq C \quad (67)$$

for all N , then $E' \rightarrow 0$ as $N \rightarrow \infty$. This is because $d_1(N)$ goes to zero as N increases. Consequently, the contribution from $d_1(N)$ becomes negligible, implying that the effect of dropping the random matrix and only keeping the highly correlated matrix becomes inconsequential.

3.6 Pruning Theorem for DNNs: how pruning random weights using PM distribution affects the classification confidence

Theorem 3.14 (The Pruning Theorem for a single object). *Take ϕ to be a DNN satisfying Assumptions 3.1 and $W = R + S$ satisfying Assumptions 3.1-3.5 and fix object s from the test set T' . Construct the truncated matrix W' by pruning the singular values of W smaller or equal to $\sqrt{\lambda_+} + \epsilon_0(r)$, where λ_+ is the rightmost edge of the MP distribution of the ESD of $R^T R$ (see Assumption 3.2) and $\epsilon_0(r) < \sigma_r(W) - \sqrt{\lambda_+}$. Then we have that there exists an explicit function $f_W > 0$ such that for the classification confidence threshold of the non-pruned DNN:*

$$E_N(s) := (\sqrt{2} \min\{f_W, \sqrt{\lambda_+}\} g_\phi(s) + d_1(N) h_\phi(s)), \quad (68)$$

we have the conditional probability

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\delta X(s, \alpha_{W'}) > 0 \mid \delta X(s, \alpha_W) > E_N(s) \right) \rightarrow 1, \quad (69)$$

with $a(N) \rightarrow 0$ as $N \rightarrow \infty$, $g_\phi(s)$ coming from (45) and $h_\phi(s)$ coming from (47). f_W is given in Lemma F.6. Also, α_W are the parameters of the DNN that has the weight matrix W , and $\alpha_{W'}$ are the parameters of the DNN with the weight layer matrix W' .

See Subsection G for a proof of this theorem.

Remark 3.8. Here we take s from the test set T' ; however, the result also holds if we take s from the training set T . Furthermore, using the proof given in Subsection G, one can show a more general result. That is, taking

$$\Delta(\delta X) := |\delta X(s, \alpha_{W'}) - \delta X(s, \alpha_W)|, \quad (70)$$

we have that

$$\lim_{N \rightarrow \infty} \mathbb{P} \left(\Delta(\delta X) \leq E_N(s) \right) \rightarrow 1. \quad (71)$$

Remark 3.9. The truncation of the matrix W is done by pruning the singular values smaller or equal to $\sqrt{\lambda_+} + \epsilon_0(r)$. This choice is made because, as $N \rightarrow \infty$, the largest singular value $\sigma_{\max}(R)$ of the random matrix R converges to $\sqrt{\lambda_+}$ a.s. However, for finite N , there can be fluctuations around λ_+ due to the inherent randomness of the matrix. These fluctuations are described by the Tracy-Widom

distribution. In the numerical part of the paper, the value of λ_+ was determined using the BEMA algorithm (see Subsection H.1.) This algorithm approximates λ_+ by incorporating the Tracy-Widom distribution to account for the finite-size effects and the fluctuations of the largest eigenvalue of $R^T R$. By using this algorithm, we obtain a more accurate estimation of λ_+ for practical, finite-dimensional settings, which is crucial for effectively applying the pruning theorem in real-world scenarios.

Pruning Theorem 3.14 shows that if we replace the matrix W with a truncated matrix W' , then for any given object $s \in T'$, we have that if the classification confidence $\delta X(s, \alpha)$ is positive enough for matrix W , it stays positive for the truncated matrix W' with high probability. In other words, almost all well-classified objects remain well-classified after replacing W with W' . We also show numerically that it is easier to prevent overfitting using matrix W' instead of the larger matrix W . We verified that removing small singular values based on the MP-based threshold $\sqrt{\lambda_+}$ for the case when the weight matrices W were initialized with $N(0, \frac{1}{N})$ does not reduce the accuracy of the DNN; see Example H.3.

3.7 Simple example of DNN with one hidden layer

The following is a simple example of the Pruning Theorem:

Example 3.15. Take ϕ to be a DNN with three weight layer matrices W_1, W_2, W_3 and the absolute value activation function, and take $s \in T'$. This is:

$$\phi(s, \alpha) = \rho \circ \lambda \circ W_3 \circ \lambda \circ W_2 \circ \lambda \circ W_1 s, \quad s \in \mathbb{R}^n. \quad (72)$$

Assume $W_2 = S_2 + R_2$ satisfies Assumptions 3.1-3.5 and W_1, W_3 are arbitrary. More specifically, assume R_2 to be a random matrix with i.d.ds taken from the distribution $N(0, \frac{1}{N})$ and S_2 to be a $N \times N$ low-rank matrix with non-zero singular values bigger than 1.1.

Take W' to be the same as W but with all the singular values of W smaller than 2.000001 set to zero. Then for f_W the positive function given in (74) we have that :

$$\mathbb{P}\left(\delta X(s, \alpha_{W'}) > 0 \mid \delta X(s, \alpha_W) > \sqrt{2}(d_1(N)\|W_1 s\|_2\|W_3\|_\infty + f_{W_2}\|W_1 s\|_2\sigma_{\max}(W_3))\right) \rightarrow 1, \quad (73)$$

as $N \rightarrow \infty$.

Here we have that for σ_r the smallest non-zero singular value of S_2 :

$$f_W = \sqrt{\left(\sigma_i^2 + \left(\frac{1 + \sigma_i^2}{\sigma_i}\right)^2\right) - 2(1 + \sigma_i^2)\sqrt{\left(1 - \frac{1}{\sigma_i^2}\right)}}. \quad (74)$$

Note that as $\sigma_r \rightarrow \infty$ we have $f_W \rightarrow 1$. In fact, we show numerically that already for $\sigma_r \geq 5$ we have $|f_W - 1| \leq .045$, see Fig 4.

Note that this estimate is given in terms of the singular values of S (which are σ_i). However, the singular values of S might not be known. Nevertheless, by Theorem C.1, we have that as $N \rightarrow \infty$ the singular values of S can be obtained directly from the singular values of W via $\sigma'_i = \frac{1 + \sigma_i^2}{\sigma_1}$. Thus, as $N \rightarrow \infty$ this estimate can be obtained in terms of singular values of W only, which is why we use the notation f_W and not f_S . For simplicity, we keep using the current notation.

We numerically checked that for R a 3000×3000 random matrix initialized with the above Gaussian distribution, and for S a diagonal matrix with 5 non-zero singular values given by 30, 40, 50, 60, 70, we have $\|S - W'\|_2 \approx f_W \approx 1$, see Fig. 5. Thus, in this example $f_W < \sqrt{\lambda_+}$, given that $\sqrt{\lambda_+} = 2$, and so in (68) we would have $\min\{f_W, \sqrt{\lambda_+}\} \approx 1$. Thus, Theorem 3.14 provides a better result than Lemma 3.11. For more on this example, see Subsection F.1.

It is an important question: under what conditions of R and S would we have that $f_W = c$ such that $c < \sqrt{\lambda_+}$.

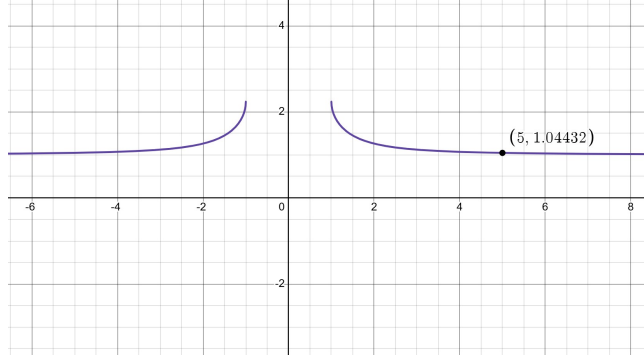


Figure 4: Graph of f_W . On the x-axis, we have σ_i .

Example 3.16. Consider R to be an $n \times n$ symmetric (or Hermitian) matrix with independent, zero mean, normally distributed entries. The variance of the entries is $\frac{\sigma^2}{n}$ on the diagonal and $\frac{\sigma^2}{2n}$ on the off-diagonal.

In the setting where $S = \sum_{i=1}^r \sigma_i u_i u_i^T$, let u'_i be the unit eigenvectors of $W = R + S$ associated with its r largest eigenvalues. Assuming that for all $1 \leq i \leq r$ we have $\sigma_i > \sigma$, then:

$$f_W = \max_{1 \leq i \leq r} \sqrt{\left(\sigma_i^2 + \left(\sigma_i + \frac{\sigma^2}{\sigma_i} \right)^2 \right) - 2\sigma_i \left(\left(\sigma_i + \frac{\sigma^2}{\sigma_i} \right) \sqrt{\left(1 - \frac{\sigma^2}{\sigma_i^2} \right)} \right)}. \quad (75)$$

In this example, $\sqrt{\lambda_+} = 2\sigma$. Thus, in Fig. 6 we compare f_W vs 2σ . As mentioned, it would be interesting to try and find a probability distribution which, if R is initialized, would result in a very small f_W for reasonable assumptions on the singular values of S .

See Subsection F.2 for more information.

3.8 Pruning Theorem for accuracy: how pruning affects accuracy

Theorem 3.17 (The Pruning Theorem for accuracy). Under the conditions of Theorem 3.15, we have:

$$\mathbb{P} \left(\text{acc}_{\alpha_{W'_2}, 0}(t) \geq \text{acc}_{\alpha_{W_2}, E_N(s)}(t) \right) \rightarrow 1 \text{ as } N \rightarrow \infty, \quad (76)$$

with $\text{acc}_{\alpha_{W_2}, E_N(s)}(t)$ and $\text{acc}_{\alpha_{W'_2}, 0}(t)$ given in (32) and (33).

Proof. By Theorem 2.7, after pruning the classification confidence of each object s is reduced by at most $E_N(s)$, with probability going to 1 as N goes to infinity. Theorem 2.9 then follows, given that the elements of the training or test sets are finite. $\square$

4 Numerical results for pruning Visual Transformers

Vision Transformers [19] rely on an attention-based mechanism [66] to classify images. On benchmarks such as Imagenet [15], they constitute a state-of-the-art method. As most of their trainable weights are expressed as part of a linear layer, either for the Multi-Layer-Perceptron (MLP) or Attention layers, transformers are uniquely suited to RMT-based approaches. This is because their weights are organized nicely into weight layer matrices and can be studied using RMT. For more on the structures of ViTs, see Appendix D.

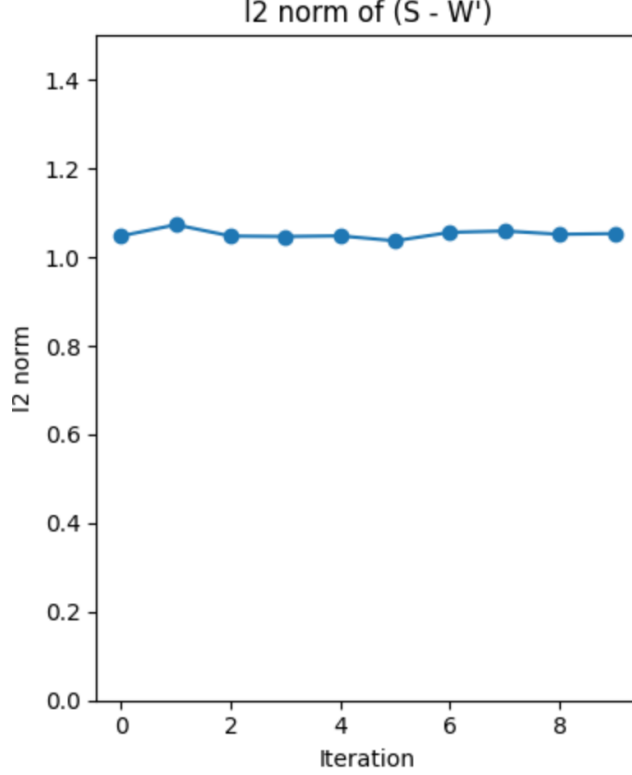


Figure 5: The norm $\|S - W'\|_2$ is shown for 10 random matrices with elements i.i.ds taken from $N(0, \frac{1}{N})$ and S a diagonal matrix with elements on the diagonal given by 30, 40, 50, 60, 70. We see that the norm is very close to $f_W \approx 1$.

4.1 Pruning strategy based on Random Matrix Theory

In this section, we detail our pruning strategy that leverages insights from RMT. The pruning process involves selectively removing weights from a DNN based on certain criteria derived from RMT. Our goal is to reduce the model size while preserving its accuracy.

Our pruning strategy is based on the assumptions made in Section 3, and extensive experiments on ViT models. Given our assumption that the weight matrices are perturbed low-rank structures, we can prune them by setting small values, which are likely noise, to zero. We define the pruning function for $x, \theta \in \mathbb{R}$:

$$\text{Prune}(x, \theta) = \begin{cases} x & \text{if } |x| > \theta \\ 0 & \text{if } |x| \leq \theta \end{cases} \quad (77)$$

This function can be vectorized for matrices $W \in \mathbb{R}^{N \times M}$ s.t. $\text{Prune}(W, \theta)_{i,j} = \text{Prune}(W_{ij}, \theta)$.

There are three types of pruning that can be done:

1. **Singular Vector Pruning:** Small values in the singular vectors may be interpreted as noise. This step helps protect the model during pruning by enhancing the low-rank structure and preserving accuracy.
2. **Singular Value Pruning:** Although small singular values could be considered noise according to our assumptions, experiments showed that pruning these values significantly degraded the network's performance. Therefore, this technique was excluded from our final pruning procedure.

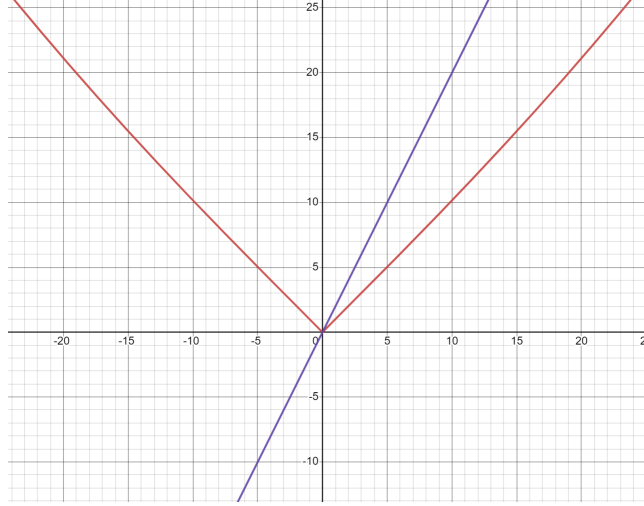


Figure 6: A graphical comparison between f_W and 2σ as functions of σ . The x-axis represents the variable σ ranging from -25 to 25. The y-axis provides the computed values for both f_W and 2σ . We see that $f_W \approx |\sigma|$, which is smaller than 2σ for positive σ . Note, to obtain this numerical result, we must ensure that $\sigma_r \geq \sigma$.

- 3. **Direct Coefficients Pruning:** We may also directly set to zero small coefficients in the weights layer, as they may be interpreted as noise.

After pruning, we observe that applying a combination of $L1$ and $L2$ regularization helps in refining the weight matrices, promoting sparsity while maintaining the network’s performance.

To guide the pruning and define what is interpreted as noise, we introduce two key metrics:

- **Spike Metric γ :** This metric quantifies the proportion of spikes in the weight matrices of the layers. It is defined as:

$$\gamma = \frac{1}{N} \cdot \text{Card} \left\{ \sigma \leq \sqrt{\lambda_+ N}, \text{ for } \sigma \text{ singular value of } W_\ell \right\}$$

where N is the size of the weight matrix W_ℓ , and λ_+ is the upper edge of the Marchenko-Pastur distribution fitted to the Empirical Spectral Density (ESD) of $X_\ell = \frac{1}{N} W_\ell^T W_\ell$. In the base ViT described below, its average value is 88.55%. This metric suggests how large the rank of the low-rank matrix S is compared to the total rank of W . See Appendix H for more on spikes.

- **MP Fit Metric μ :** This metric measures the error in fitting the Marchenko-Pastur (MP) distribution to the ESD of $X_\ell = \frac{1}{N} W_\ell^T W_\ell$. It is computed based on a fitting algorithm (see Subsection 3). In the base ViT described below, its average value is 0.3.

4.1.1 Singular vector pruning

For a triplet (v, w, σ) of respectively left, right singular vectors and singular value, we apply pruning such that:

$$v \leftarrow \text{Prune} \left(v, \theta \max \left\{ \frac{1}{750}, \left[1 - \frac{\sigma}{\sqrt{\lambda_+}} \right]^{30} \right\} \right) \quad (78)$$

A similar update is performed on w . The specifics of this pruning were found by extensive experiments. The pruning of the components of the singular vectors is based on their size, with smaller components

being pruned. The threshold of this pruning is based on $\theta = .00001125 \times r \times N \times M$, with N and M being the sizes of the matrix. This process reduces the complexity of the model by removing less significant components of the singular vectors. It seems to "protect" the DNN during the pruning phase and make the weight layers "more" low rank, see Algo. 5.

4.1.2 Direct coefficients pruning

To prune the weight matrices, we use the metric $\zeta_1(t)$, which depends on the randomness of the layer and the current pruning cycle t . It is defined as:

$$\zeta_1(t) = [(1 - \mu) \cdot \gamma]^{\frac{1.5}{t}} \cdot r \cdot \text{Card} \{|W_\ell|_{ij} > 0\} \quad (79)$$

where:

- μ is the MP fit metric of the layer.
- γ is the Spike Metric of the layer.
- r is the desired proportion of parameters to remove in each pruning cycle (set to $r = 6\%$).
- t is the current pruning cycle (with total cycles $N_{\text{cycle}} = 19$).
- $\text{Card} \{|W_\ell|_{ij} > 0\}$ is the number of non-zero parameters in the weight matrix W_ℓ .

The above equation for $\zeta_1(t)$ was derived through a process of trial and error, but it also embodies some common-sense principles. In particular, weight matrices that are determined to be more random—based on the randomness criteria provided by the MP Fit Metric (μ) and the spike metric (γ)—are pruned more aggressively in the initial pruning cycles. This approach leverages the idea that less structured (i.e., more random) weights are likely to be less critical to the network's performance.

The exponent $\frac{1.5}{t}$ ensures that as we progress through the pruning cycles (i.e., as t increases), the influence of the randomness metrics μ and γ diminishes. This design allows us to focus more on pruning layers that are identified as more random in the early cycles and shift towards a more uniform pruning strategy in later cycles. Using this, each matrix is updated using the following rule:

$$W_\ell \leftarrow \text{Prune}(W_\ell, f \max\{3, 5[(1 - \mu)\gamma]^{\frac{1.5}{t}}\}) \quad (80)$$

where f is the pruning factor. f is found to be the smallest number so that the total amount of weights pruned in the matrix is larger or equal to $\zeta_1(t)$. In practice, f is initialized at $f = 1e - 6$ and increased by steps of $5e - 6$ until the total amount of weights pruned is more than $\zeta_1(t)$.

4.1.3 Regularization

Regularization helps mitigate potential accuracy loss due to pruning. It also helps in refining the weight matrices after pruning, promoting sparsity while maintaining the network's performance.

After every pruning cycle, we minimize the loss function

$$\mathcal{L} = \mu_1 \|W\|_1 + \mu_2 \|W\|_2^2,$$

with $\mu_1 = 5 \times 10^{-6}$ and $\mu_2 = 2 \times 10^{-6}$ using SGD and a learning rate of 0.05×10^{-6} (without any training data), so that although an infinite number of minimization steps would drive all weights to zero, the finite number of epochs applied causes the weights to simply become smaller—with some small weights vanishing—thus allowing us to decay the weight magnitudes without directly pruning them. Again, note that no training data is used during this regularization, and there are no other constraints for this minimization.

Pruning Procedure Overview

Having defined the different steps used in our procedure, we may now describe the overall process. To progressively remove weights, we perform $N_{cycle} = 19$ times the following pruning cycle. First, we **prune the singular vectors** (only every other cycle). Then, coefficients of the weight matrix are pruned by **direct coefficient pruning**. Finally, we perform **regularization** for n_{reg} epochs. We start with $n_{reg} = 15$ and increase by 5 epochs each cycle, up to a maximum of 40 epochs.

Fine-tuning Finally, after completing all pruning and regularization cycles, we fine-tune the model to recover any potential loss in accuracy. This fine-tuning is performed while keeping the pruned weights frozen, thus keeping the sparsification we obtained through the procedure described above. We use SGD with a momentum of 0.9, a base learning rate of 3×10^{-5} with the cosine annealing scheduler, and a warmup phase of 2 epochs (learning rate divided by 100). The model is fine-tuned for fewer than 20 epochs over the entire training set using cross-entropy loss (the exact number of epochs is given in the results below). Finally, the gradient norm is clipped at 1 to prevent exploding gradients. Results are presented before and after fine-tuning.

Results for the visual transformer Vit_Base_Patch16_224 Model

Our numerical simulations explore the pruning of a pre-trained vit_base_patch16_224 model using RMT. This model has approximately 86.5 million parameters and achieves a 85.1% top-1 accuracy on the ImageNet validation set. We reduce the DNN’s size by 30-40% and evaluate its accuracy. We assess the model’s top-1 and top-5 accuracy on the ImageNet validation set, consisting of $\approx 200,000$ objects.

Remark 4.1. Some layers of this DNN model fit the MP distribution (based on Algorithm 3) with an error of 1% while others fit the MP distribution with an error of over 50%, see Subsection H.5 for more. The spike metric and MP fit metric depend on α , see Subsection H.5. When $\alpha = .25$, we have that the MP fit metric of this ViT model is .18.

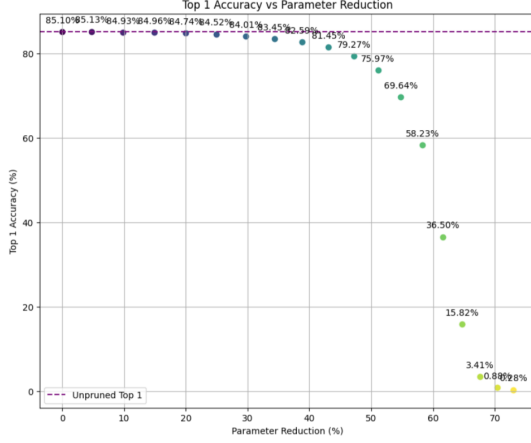
As the pruning progresses, the process yields a series of models with varying levels of sparsity and performance metrics. The final model selection can be based on a trade-off between accuracy and complexity. See Fig. 7 for the accuracies of the ViT model vs the number of parameters kept (there is no fine-tuning yet). We see that the DNN that was pruned by approximately 30% has a reduction in top-1 accuracy by only 1.1%. After 1 epoch of fine-tuning the ViT model on the ImageNet training set, the DNN accuracy improved to 84.5%, resulting in a total decrease in accuracy of only .6%.

Pruning the visual transformer Vit_large_Patch16_224 Model

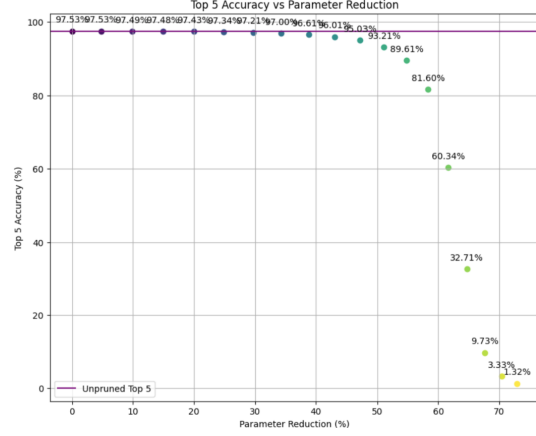
We performed the same pruning strategy on the Vit_large_Patch16_224 Model. This ViT model is bigger than the one in the previous subsection, having approximately 300 million parameters. It achieves a 85.85% accuracy on the ImageNet validation set. In Fig. 8, we see the accuracy vs percentage of parameters kept for this pruning. After pruning 30% of the DNN parameters, the accuracy only drops by .8%. Again, after fine-tuning for only one epoch, we recover that full reduction in accuracy (i.e. we have 85.85% accuracy). Finally, we also took the DNN for which 50% of the parameters were pruned and fine-tuned for 12 epochs, obtaining a 84.48% accuracy after the fine-tuning.

Pruning Results for ViT-B16/224 and ViT-L16/224 on ImageNet

In this section, we compare our pruning results with CP-ViT [55], using the same architecture but different pre-trained weights. Our initial accuracies are higher: 85.1% for ViT-B16/224 and 85.85% for ViT-L16/224, compared to 77.91% and 76.5% in CP-ViT. Figure 9 shows the pruning results with

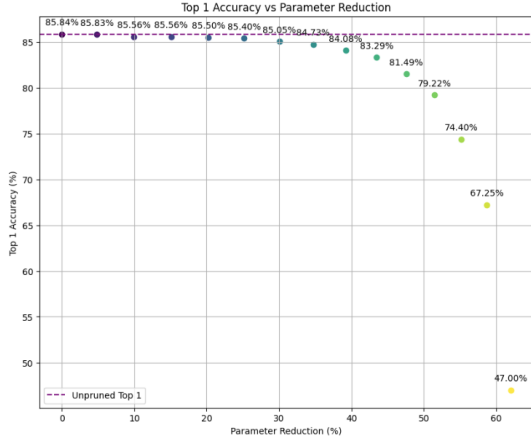


(a) Top 1 accuracy vs params. kept

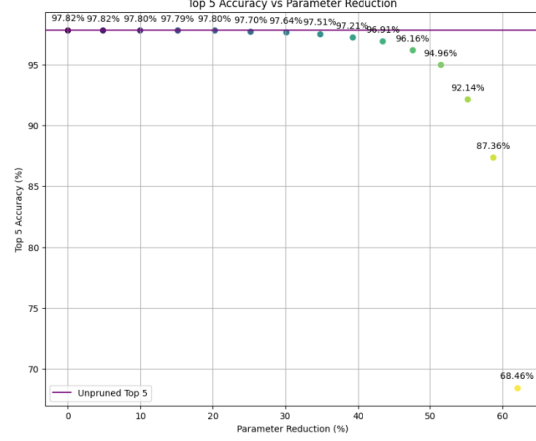


(b) Top 5 accuracy vs params. kept

Figure 7: The top 1 accuracy and top-5 accuracy of ViT-base vs. percentage of parameters kept for pruning through RMT-based sparsification and no fine-tuning. The accuracy of the DNN is given as a percentage above the data points.



(a) Top 1 accuracy vs params. kept



(b) Top 5 accuracy vs params. kept

Figure 8: The top 1 accuracy and top-5 accuracy of ViT-large model vs. percentage of parameters kept for pruning through RMT-based sparsification and no fine-tuning. The accuracy of the DNN is given as a percentage above the data points.

FLOP reduction and accuracy reduction. Our fine-tuning was done for less than 20 epochs, while the fine-tuning in [55] was done with 30 epochs.

RMT-based pruning of DeiT

We perform the same pruning strategy with the same hyperparameters from Subsection 4.1 on the three DeiT models, tiny, small, and base. Table 1 shows the result of our RMT-based pruning method and compares them with other pruning methods, both with and without fine-tuning. For these models, we fine-tune for only 7 epochs, which is much less than the fine-tuning done in the other works; for example, in [55], they fine-tune for 30 epochs. From Table 1 and Fig. 9, we see that we always outperform [55] (and the other works) when there is no fine-tuning. Furthermore, Fig. 9 shows that we outperform [55] for large DNNs even with fine-tuning, which makes sense from an RMT perspective,

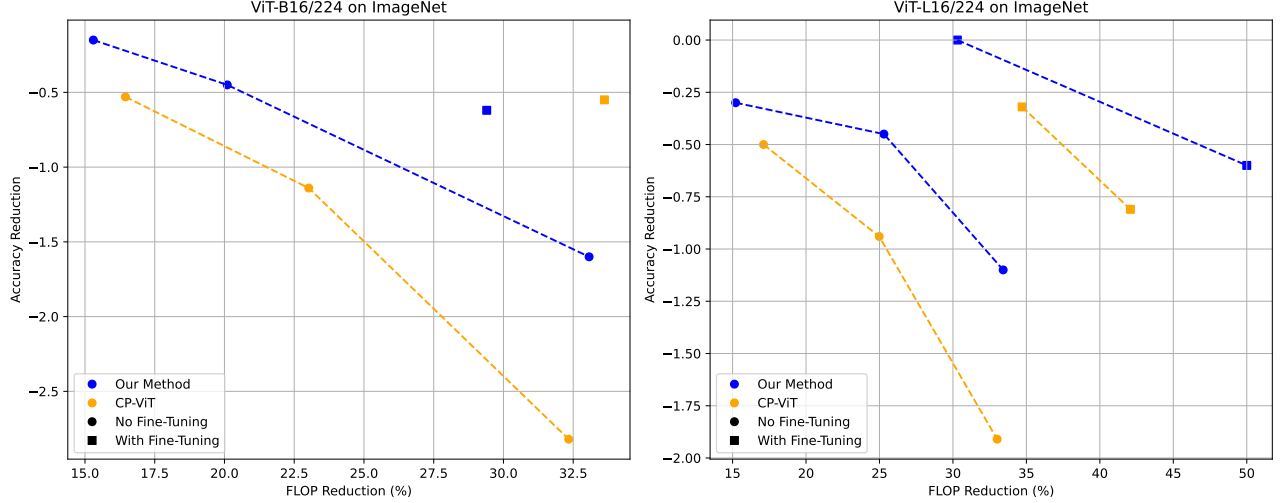


Figure 9: Comparison of pruning results between our method and CP-ViT [55]

given that the larger the DNN and the more parameters we have, the more RMT applies. Given the massive DNNs being trained these days, with over a trillion parameters, this RMT perspective becomes especially important. For the ViT_Base_Patch16_224 Model, after fine-tuning, we essentially achieve the same results as [55], but we only fine-tune for one epoch while they fine-tune for 30.

5 MP-based pruning of fully connected DNNs

5.1 Regularization and MP-based pruning for fully connected DNNs

This subsection provides some numerical results that show that MP-based pruning leads to a reduction in loss and an increase in accuracy, which is relevant to Theorem 2.5. We also confirm that adding (which in this context is the same as removing) a random matrix to the weight layer matrices of the DNN does not lead to a change in accuracy or cross-entropy loss but does lead to a change in the L_2 loss, numerically confirming Theorem 2.5 and Lemma 3.10.

Example 5.1. *In this example, we trained on Fashion MNIST a fully connected DNN with ReLU activation and the following topology:*

$$[784, 3000, 3000, 3000, 3000, 500, 10],$$

This means our network has 6 layers followed by activation functions, with layer k being a linear layer $L_k : \mathbb{R}^{T_k} \rightarrow \mathbb{R}^{T_{k+1}}$. The training was performed multiple times, with no regularization and no PM-based pruning, with L1 regularization, L2 regularization, L1 + L2 regularization, and MP pruning (see Algorithm 4), and so on, to compare these different forms of training. We also compare our approach with stable rank regularization, that is, adding the sum over l of:

$$\text{stable}(W_l) = \frac{\|W_l\|_F^2}{\|W_l\|_2^2} \quad (81)$$

to the loss function, see [67] for more. This regularization is supposed to help induce heavy tail in the weight layer matrices, making the weight layer matrices "less random".

In all cases, the training was done for 100 epochs. MP-pruning based training essentially involved pruning the small singular values in the weight layer matrices based on the MP distribution, and using

Table 1: Comparison with different ViT pruning methods on the ImageNet dataset. The accuracy without finetuning was not mentioned in VTP, PoWER, and HVT but were replicated in [55]. We fine-tune for only 7 epochs on the ImageNet training set. We see that the larger the DNN, the better our method performs, which makes sense from an RMT perspective. This is also reassuring, given that modern AI models are often much larger than these models.

Model	Not Finetune		Finetune	
	Top-1 Acc.(%)	FLOPs Saving	Top-1 Acc.(%)	FLOPs Saving
DeiT-Ti 16/224 [64]				
Baseline [64]	72.20	-	72.20	-
VTP [74]	69.37 (-2.83)	21.68%	70.55 (-1.65)	45.32%
PoWER [26]	69.56 (-2.64)	20.32%	70.05 (-2.15)	41.26%
HVT [48]	68.43 (-3.77)	21.17%	70.01 (-2.19)	47.32%
CP-ViT [55]	71.06 (-1.14)	23.02%	71.24 (-0.96)	43.34%
RMT-based (ours)	71.09 (-1.11)	24.83%	68.4 (-3.8)	47.23%
DeiT-S 16/224 [64]				
Baseline [64]	79.80	-	79.80	-
VTP [74]	77.35 (-2.45)	20.74%	78.24 (-1.56)	42.52%
PoWER [26]	77.02 (-2.78)	21.46%	78.30 (-1.50)	41.36%
HVT [48]	76.72 (-3.08)	20.52%	78.05 (-1.75)	47.80%
CP-ViT [55]	78.84 (-0.96)	20.96%	79.08 (-0.72)	42.24%
RMT-based (ours)	78.91 (-0.89)	23.72%	78.34 (-1.46)	41.92%
DeiT-B 16/224 [64]				
Baseline [64]	81.82	-	81.82	-
VTP [74]	79.46 (-2.36)	19.84%	80.70 (-1.12)	43.20%
PoWER [26]	79.09 (-2.73)	20.75%	80.17 (-1.65)	39.24%
HVT [48]	78.88 (-2.94)	20.14%	79.94 (-1.88)	44.78%
CP-ViT [55]	80.91 (-0.91)	22.16%	81.13 (-0.69)	41.62%
RMT-based (ours)	81.44 (-0.38)	21.38%	80.58 (-1.24)	44%

this reduction in rank to reduce the number of parameters in the DNN, see Appendix A, Algo 4 and [54, 9] for more information. The ReLU activation function was applied after every layer, including the final layer (which is not typical), and then the softmax activation function was applied to the output. While it might be easier to train DNNs without applying the activation function to the final layer, we found that we obtained the highest difference in accuracies (between normal training and training with MP-based pruning) when training with the affirmation structure while using MP-based pruning (see Table 2). Furthermore, without the activation function being applied to the final layer, the accuracy was less than 90.5% (without MP-pruning). For the same architecture, using L1 + L2 regularization and MP-pruning, we obtained a 91.37% accuracy on the Fashion MNIST test set, which is the highest accuracy we observed on the data set using a fully connected DNN (see Example 5.2). MP-based pruning also increases the accuracy of fully connected DNNs used for regularization tasks; for example, see Subsection B.

Every 4 epochs, we kept a portion of the smallest singular values (singular values less than $\sqrt{\lambda_+}$ given in Assumption 2.2 in Subsection 2) based on the formula:

$$f(epoch) = \max\left(0, -\frac{1}{200} \cdot epoch + 1\right). \quad (82)$$

By doing this, we are essentially removing "some" of the random matrices R_l during training. Once the training is finished, we sparsify the weight layer matrices of the DNN by removing the weights smaller than some threshold ξ , see Algorithm 6. The results are given in Fig. 10 and Table 2. In Subsection A.0.1 we show that one can reduce the number of parameters by a much larger amount (a reduction in parameters by up to 99.98%) if we remove

$$f(\text{epoch}) = \max\left(0, -\frac{1}{100} \cdot \text{epoch} + 1\right) \quad (83)$$

of the smallest singular values and sparsify the matrix once the training is complete. However, the accuracy that is obtained is also lower (see [9] for more on the relationship between MP-based pruning and sparsification). In Fig. 10, we show how accuracy drops as we sparsify the DNNs.

Training method	Final accuracy on training set	Final accuracy on testing set
MP-based pruning, $L1, L2$	78.03%	71.64%
MP-based pruning, $L1, L2$	88.12%	81.22%
MP-based pruning, $L1, L2$	99.82%	90.53%
MP-based pruning, $L1, L2$	87.57%	81.70%
MP-based pruning, $L1, L2$	87.42%	81.46%
MP-based pruning, $L1, L2$	87.37%	81.31%
MP-based pruning, $L1, L2$, stable rank	99.97%	90.16%
MP-based pruning, $L1, L2$, stable rank	79.01%	71.38%
MP-based pruning, $L2$	99.98%	90.16%

Table 2: Performance of fully connected DNN at final epoch for various training strategies.

Training method	Final loss on training set
MP-based pruning, $L1, L2$	0.629775
MP-based pruning, $L1, L2$	0.170905
MP-based pruning, $L1, L2$	0.003104
MP-based pruning, $L1, L2$	0.325156
MP-based pruning, $L1, L2$	0.399908
MP-based pruning, $L1, L2$	.431001
MP-based pruning, $L1, L2$, stable rank	.001134
MP-based pruning, $L1, L2$, stable rank	.476791
MP-based pruning, $L2$	0.001588

Table 3: Performance of fully connected DNN at final epoch for various training strategies.

The hyperparameters for the training are given in Table 4.

Example 5.2. In this example, we trained on Fashion MNIST a fully connected DNN with ReLU activation (on all but the final layer) and topology:

$$[784, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 10].$$

This means our network has 7 layers followed by activation functions (excluding the final layer), with layer k being a linear layer $L_k : \mathbb{R}^{T_k} \rightarrow \mathbb{R}^{T_{k+1}}$. The training was performed with $L1 + L2$ regularization and PM-based pruning (see Algorithm 4) and compared with using regularization alone.

hyperparameters	Value
Stable rank (SR)	0.000001
Every how many epochs the SR was used in the loss	6
$L1$	0.000001
$L2$	0.000001
Every how many epochs the MP pruning was done	4
Momentum	0.9
Learning rate	0.01 (decay by .96 every 4 epochs)

Table 4: Hyperparameter values

In all cases, the training was done for 2000 epochs. The ReLU activation function was applied after every layer, excluding the final layer (which is typical). Without MP-based pruning, the accuracy on the test set was lower than 90.5%. For the MP-based pruning, every 12 epochs we kept a portion of the smallest singular values (singular values less than $\sqrt{\lambda_+}$ given in Assumption 2.2 in Subsection 2) based on the formula:

$$f(\text{epoch}) = \max\left(0, -\frac{1}{2000} \cdot \text{epoch} + 1\right). \quad (84)$$

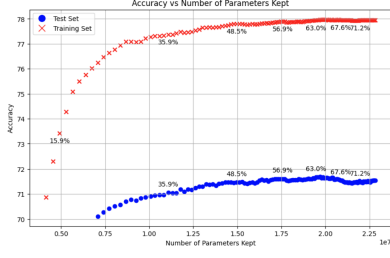
We used SGD together with a momentum of .97 and a learning rate of .001 with the CosineAnnealingLR scheduler. The L1 and L2 regularization hyperparameters were 0.0000005. Using MP-based pruning, the test set accuracy was 91.37.

Example 5.3. In this example, we trained on Fashion MNIST a fully connected DNN with ReLU activation and topology:

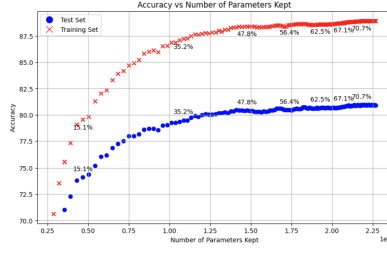
$$T = [784, 3000, 3000, 3000, 3000, 500, 10],$$

This means our network has 6 layers followed by activation functions, with layer k being a linear layer $L_k : \mathbb{R}^{T_k} \rightarrow \mathbb{R}^{T_{k+1}}$. We use the MP-pruning approach that can be found in Example 5.1, to achieve a 90.5% accuracy on the test set. Note that in Lemma B.5, the method for training the DNN does not matter; only the loss and accuracy of the DNN at time t are what matter. Furthermore, this lemma is equally valid if instead of replacing W_2 with S_2 , we replace the weight layer matrix W_2 with a new matrix $W_2 + R'_2$, with R'_2 a random matrix with components i.i.d. taken from $N(1, \epsilon)$ with $\epsilon \sim \frac{1}{N}$. Thus, to all but the final layer matrix of the DNN we add a random matrix R'_b with components i.i.d. from the distribution $N(0, \epsilon)$ and, in Fig. 11, we plot the loss (both with and without L2 regularization) and accuracy of the DNN as we vary ϵ . We see that for $\epsilon \sim \frac{1}{N}$ we have that the cross-entropy loss and accuracy don't change much, which numerically confirms Lemma B.5.

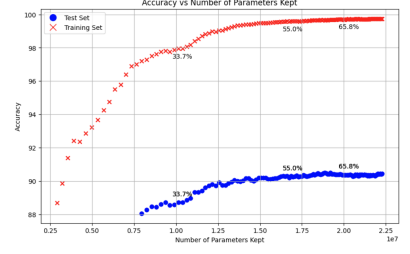
For more numerical examples of how MP-based pruning improves the accuracy of fully connected DNNs and helps reduce the parameters of DNNs and CNNs (via low rank approximation and sparsification) trained on MNIST, Fashion MNIST, and CIFAR-10, see Appendix A.



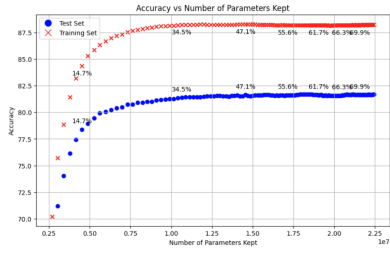
(a) No MP-based pruning, and no $L1$ or $L2$ regularization.



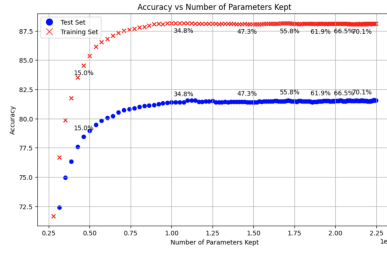
(b) MP-based pruning, no $L1$ or $L2$ regularization.



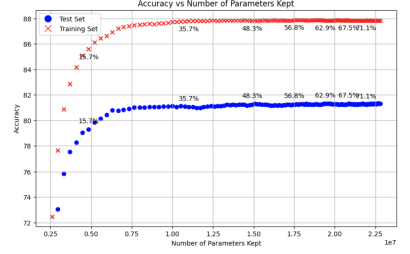
(c) MP-based training + $L1$ and $L2$ regularization.



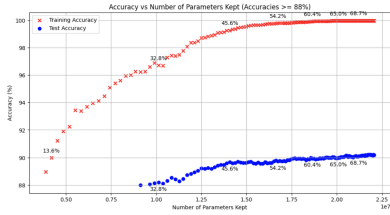
(d) No MP-based pruning, used $L1$ + $L2$ regularization.



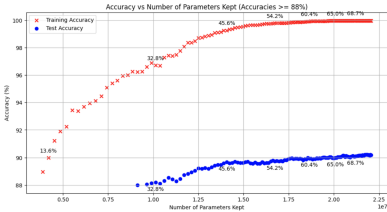
(e) Only used $L1$ regularization.



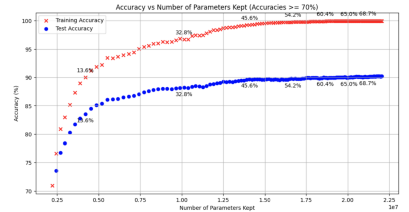
(f) Only used $L2$ regularization.



(g) MP-based pruning, $L1$, $L2$, stable rank.

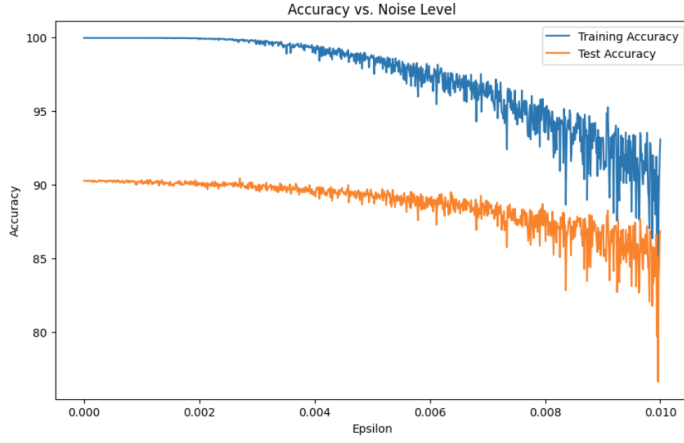


(h) $L1$, $L2$ and stable rank.

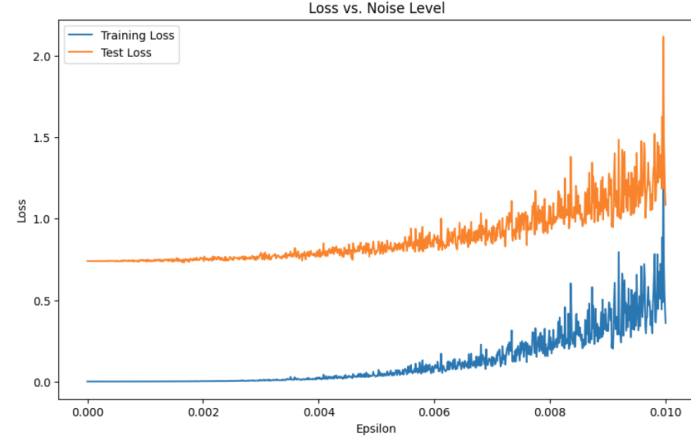


(i) MP-based pruning with $L2$.

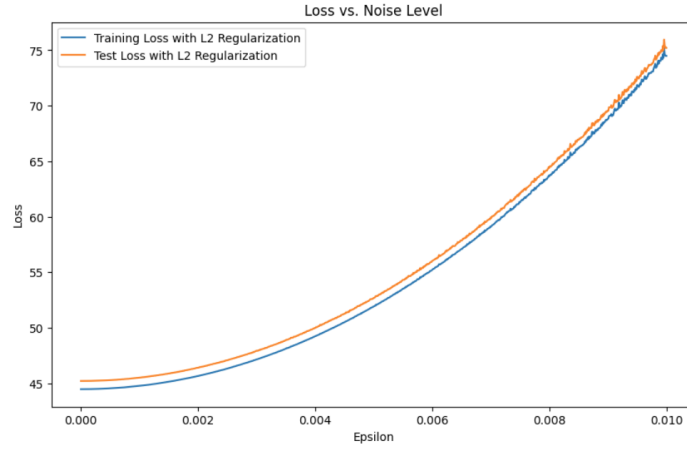
Figure 10: DNN performance vs. sparsification



(a) Accuracy of the DNN on the training and test sets, vs. ϵ .



(b) DNN loss (with no regularization) on training and test set vs. ϵ .



(c) DNN loss with L_2 regularization on training and test set vs. ϵ . The hyperparameter μ in the regularization is .01.

Figure 11: Accuracy and loss as we add noise to the weight layers of the DNN.

Chapter Three

Stability of Accuracy for the Training of DNNs

6 Stability of accuracy for the training of DNNs Via the Uniform Doubling Condition

6.1 Introduction to Stability of Accuracy

We study the stability of accuracy during the training of deep neural networks (DNNs). In this context, the training of a DNN is performed via the minimization of a cross-entropy loss function, and the performance metric is accuracy (the proportion of objects that are classified correctly). While training results in a decrease of loss, the accuracy does not necessarily increase during the process and may sometimes even decrease. The goal of achieving stability of accuracy is to ensure that if accuracy is high at some initial time, it remains high throughout training.

A recent result by Berlyand, Jabin, and Safsten introduces a doubling condition on the training data, which ensures the stability of accuracy during training for DNNs using the absolute value activation function. For training data in $\mathbb{R}^n$, this doubling condition is formulated using slabs in $\mathbb{R}^n$ and depends on the choice of the slabs. The goal of this paper is twofold. First, to make the doubling condition uniform, that is, independent of the choice of slabs. This leads to sufficient conditions for stability in terms of training data only. In other words, for a training set T that satisfies the uniform doubling condition, there exists a family of DNNs such that a DNN from this family with high accuracy on the training set at some training time t_0 will have high accuracy for all time $t > t_0$. Moreover, establishing uniformity is necessary for the numerical implementation of the doubling condition. We demonstrate how to numerically implement a simplified version of this uniform doubling condition on a dataset and apply it to achieve stability of accuracy using a few model examples. The second goal is to extend the original stability results from the absolute value activation function to a broader class of piecewise linear activation functions with finitely many critical points, such as the popular Leaky ReLU.

In stability of accuracy, RMT also plays a small role. In the Stability of Accuracy Theorem 8.6, we have a condition 101 on $d_{\min}$ (see Definition 8.5). When $d_{\min}$ is larger, it is easier to satisfy condition 101. $d_{\min}$, however, is bigger when the singular values of the DNNs weight layer matrices are bigger. Thus, by pruning singular values smaller than $\sqrt{\lambda_+}$ of the weight layers, the network not only benefits from reduced parameter complexity but also meets the prerequisites for stability of accuracy more easily.

The goal of this chapter is to provide a condition on the training set of a DNN that establishes the stability of the accuracy of the DNN during training. Specifically, DNNs trained on a training set satisfying the condition will have their accuracy bounded from below as loss decreases. Ideally, this means that the accuracy of the DNN should go up or remain constant as training progresses. This is not always the case during training, sometimes loss can decrease drastically while accuracy also decreases. Before we provide such an example, a quick introduction to DNNs is needed.

Recall, we start with a set of objects $S \subset \mathbb{R}^n$ which we wish to classify into K classes. A set $T \subset S$ exists for which we have an exact classifier $\phi^*(s)$ such that

$$\phi^* : s \in T \mapsto \phi^*(s) = (p_1(s), \dots, p_K(s)), \quad (85)$$

with $p_{i(s)} = 1$ and $p_j = 0$ when $j \neq i(s)$, and $i(s)$ denoting the correct class of s . This exact classifier is only known for T , and so we use T to train a classifier ϕ , thereby approximating the exact classifier ϕ^* via ϕ . Thus, we construct a ϕ such that

$$\phi : s \in T \mapsto \phi(s) = (p_1(s), \dots, p_K(s)), \quad p_1 + \dots + p_K = 1. \quad (86)$$

We can think of $p_i(s)$ as the probability of s belonging to the class i , and while we know the correct classes of the objects $s \in T$, the goal is to extend ϕ^* from T to all of S via ϕ . Thus, we wish to obtain a ϕ which maps s to the same class as ϕ^* when $s \in T$, with the hope that ϕ will also map elements $s \in S$ to their correct class. This is done by forming a family of parameterized functions $\phi(s, \alpha)$ and finding the correct parameters α so that $p_{i(s)} = 1$ and $p_j = 0$ when $j \neq i(s)$ for $s \in T$.

In this paper, only ϕ which are DNNs (with different activation functions) are studied. Thus, we consider DNNs which are a composition $\rho \circ X(\cdot, \alpha)$, with ρ the softmax (given in (88)) and $X(\cdot, \alpha)$ defined as follows. Take

$$X(\cdot, \alpha) = \lambda \circ M_L(\cdot, \alpha_L) \circ \dots \circ \lambda \circ M_1(\cdot, \alpha_1) \quad (87)$$

with:

- $M_k(\cdot, \alpha_k)$ an affine function $\mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ depending on an $\mathbb{N}_k \times \mathbb{N}_{k-1}$ parameter matrix W_k and bias vectors β_k (i.e. $M_k(x) = W_k(x) + \beta_k$).
- $\lambda : \mathbb{R}^m \mapsto \mathbb{R}^m$ is a nonlinear activation function. For now, we assume λ is the absolute value activation function. More precisely, for λ the absolute value activation function we have that $\lambda : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is defined as $\lambda(x_1, \dots, x_m) = (|x_1|, \dots, |x_m|)$. Later, we will also consider other activation functions with finite critical points.
- ρ : is the softmax function and is used to normalize the outputs of $X(\cdot, \alpha)$ to probabilities. The components of ρ are defined as:

$$\rho_i(s, \alpha) = \frac{\exp(X_i(s, \alpha))}{\sum_{j=1}^K \exp(X_j(s, \alpha))}. \quad (88)$$

Thus, the output of the DNN ϕ is a vector and provides the probability that an object $s \in T$ belongs to some class i , while $X(s, \alpha)$ is the final output before softmax. We train a classifier ϕ (in our case a DNN) to try and mimic the exact classifier. Thus, taking ϕ to be a DNN, we have that $\phi = \phi(s, \alpha)$ with $\alpha \in \mathbb{R}^\nu$ the parameters in the affine maps of ϕ . Here $\nu \gg 1$ is a very large number and is the dimension of the parameter space of our DNN. For $s \in T$, we know what $i(s)$ is and so we use a loss function to train ϕ . Specifically, in this paper, we will consider the cross-entropy loss function

$$loss(t) = \bar{L}(\alpha(t)) = - \sum_{s \in T} \mu(s) \log(p_{i(s)}(s, \alpha)). \quad (89)$$

Here μ is a measure given on the data set T which determines how important objects $s \in T$ are during training. The hope is that by minimizing the loss function (for example, through gradient descent or stochastic gradient descent), we can obtain a ϕ which classifies the elements of T accurately. The loss function is a smooth function while accuracy is discontinuous, and so training via gradient descent is done on the loss function while we wish to improve (increase) accuracy. However, as we will see, the accuracy of ϕ (the proportion of $s \in T$ that is mapped by ϕ to their correct class) and the value of the loss function is not completely correlated. Thus, there are situations where even as we decrease the loss via training, the accuracy of ϕ also decreases at the same time. The main goal of this paper is

to provide conditions on the training set T which ensure that this does not happen, so that accuracy is bounded from below as the loss decreases. The results in this paper build on similar stability of accuracy results for classifiers given in [8].

The relationship between the loss function and the accuracy of a model is foundational in the domain of neural networks. The loss function is one way to quantify the deviation of a model's predictions from the true labels in the dataset, with the fundamental goal of training being to adjust model parameters in order to minimize this loss [24]. Studies have shown relationships between minimizing loss and increasing accuracy on the training and test sets for cases with separable data [56]. Other studies provide estimates for the solution accuracy of support vector machines (SVMs) based on the number of steps taken during training [52]. Especially for separable data, this solution accuracy will be closely related to the accuracy of the SVM, thus providing a relationship between SVM training and the accuracy of the classifier. Furthermore, issues like overfitting, where a model excels on the training data but underperforms on unseen data, further complicate this relationship [71]. Regularization techniques have been developed to mitigate such overfitting [24]. Despite these advancements, a comprehensive mathematical study explicitly detailing the correlation between loss minimization and accuracy enhancement is notably absent in the literature, to the best of the author's knowledge. The current work endeavors to bridge this gap, seeking to establish a robust mathematical foundation elucidating the interplay between minimizing loss and maximizing accuracy on the training set. This remains a significant, albeit mostly underexplored, mathematical challenge warranting thorough investigation. For more on this mathematical question, see [8].

Understanding the intricate relationship between training loss and training accuracy is not just a matter of academic intrigue; it is a pivotal theoretical question with significant practical implications. Empirically, minimizing loss and improving accuracy have been shown to be strongly correlated, especially in scenarios involving extensive training datasets. This observed relationship demands a rigorous mathematical explanation to solidify our understanding of why this correlation persists and under what conditions it is most pronounced. By delving into this theoretical framework, our work sheds light on specific structures inherent within the training data, particularly focusing on their "doubling property." This property, we posit, can be leveraged to optimize DNNs to achieve superior accuracy on both training and testing datasets, see Example K.4, Example K.5, and Subsection K.1. This "doubling property" also correlates with the loss of DNNs on the training set, a phenomenon that our theory helps explain, see Examples K.1, K.2, K.3, Theorem 8.6, and Section L.3. Such an understanding does not merely serve theoretical purposes; it offers tangible guidance on how to better structure and utilize training data for improved model performance. Although the overarching theme of the interplay between loss functions and model accuracy has been touched upon in foundational machine learning literature, our focus on the "doubling property" and its influence on this relationship is an important contribution that bridges theoretical inquiry with practical application.

We now reintroduce a tool called classification confidence, which expresses how well an object s is classified. The classification confidence is a function that maps elements in T to $\mathbb{R}$, and provides information regarding how well $s \in T$ is classified by our DNN. Recall that we defined $X(s, \alpha)$ to be the final output of our DNN before the softmax function ρ . The softmax function preserves order in the sense that if $X_i(s, \alpha) > X_j(s, \alpha)$ then $\rho_i \circ X_i(s, \alpha) > \rho_j \circ X_j(s, \alpha)$. Thus, we recall that the classification confidence is defined via $X(s, \alpha)$ alone.

Definition 6.1. *The classification confidence is defined as follows:*

$$\delta X(s, \alpha) := X_{i(s)}(s, \alpha) - \max_{j \neq i(s)} X_j(s, \alpha). \quad (90)$$

In other words,

- $\delta X(s, \alpha(t)) > 0 \Rightarrow s$ is well-classified by ϕ .

- $\delta X(s, \alpha(t)) < 0 \Rightarrow s$ is misclassified by ϕ .

Intuitively, δX is a measure of how much the DNN prefers the correct class compared to all other classes. If δX is positive, it means that the DNN is more confident in its classification of the object s with respect to the correct class. Conversely, if δX is negative, it means that the DNN is more confident in classifying s with respect to another class, and thus, it is misclassifying s .

We can now define accuracy using the classification confidence,

$$\text{acc}(t) = \mu(\{s \in T : \delta X(s, \alpha(t)) > 0\}). \quad (91)$$

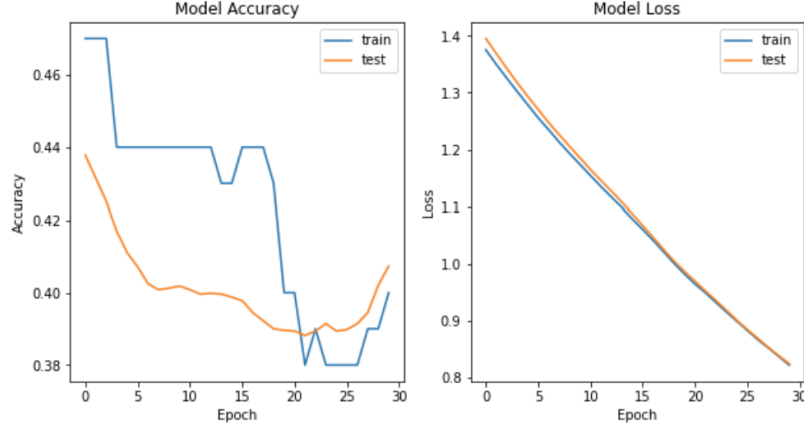


Figure 12: Despite the progress in training and reduction in loss, the accuracy has experienced a significant decline, falling from 48% at epoch 0 to 38% at epoch 25.

An example of when accuracy decreases even though loss decreases is given in Fig. 12. Here a DNN had been trained to find a decision boundary between two data sets in $\mathbb{R}^2$, see Example 7.1 for more details. Although training has progressed and loss has decreased, the training accuracy has dropped drastically (from 48% at epoch 0 to 38% at epoch 25). We would like to avoid the type of situations like in Fig. 12, at least for the cases when the DNN initially has high accuracy. In Section 7, we will provide more numerical examples of this behavior as well as a more detailed explanation for why it happens and how such behavior might be avoided.

One can show that instability happens because of small clusters in the training set T (see Def. 8.3 and Theorem 8.6). In Fig. 13a, these small clusters in T correspond with the small clusters in the histogram of $\delta X(T, \alpha_1)$ contained near 0, with α_k the parameters of the DNN at epoch k . As training progresses objects s corresponding to these small clusters might become misclassified (see Fig. 13b) while the objects s' which were badly misclassified, i.e. $\delta X(s', \alpha_1) < -2$, obtain larger δX but still remain misclassified (see Fig. 13). Because of this, loss decreases given that many objects have larger δX . However accuracy decreases, given that more elements have negative δX . Thus, preventing small clusters in the data leads to stability during training.

As mentioned, we would like to show that high accuracy at some initial time implies high accuracy for future time. That is, for t_0 , some time during training we would like that

$$\text{high acc}(t_0) \rightarrow \text{high acc}(t), \forall t > t_0. \quad (92)$$

In this paper, this will be achieved with the following steps:

$$\text{high acc}(t_0) \rightarrow \text{small loss}(t_0) \rightarrow \text{small loss}(t) \rightarrow \text{high acc}(t). \quad (93)$$

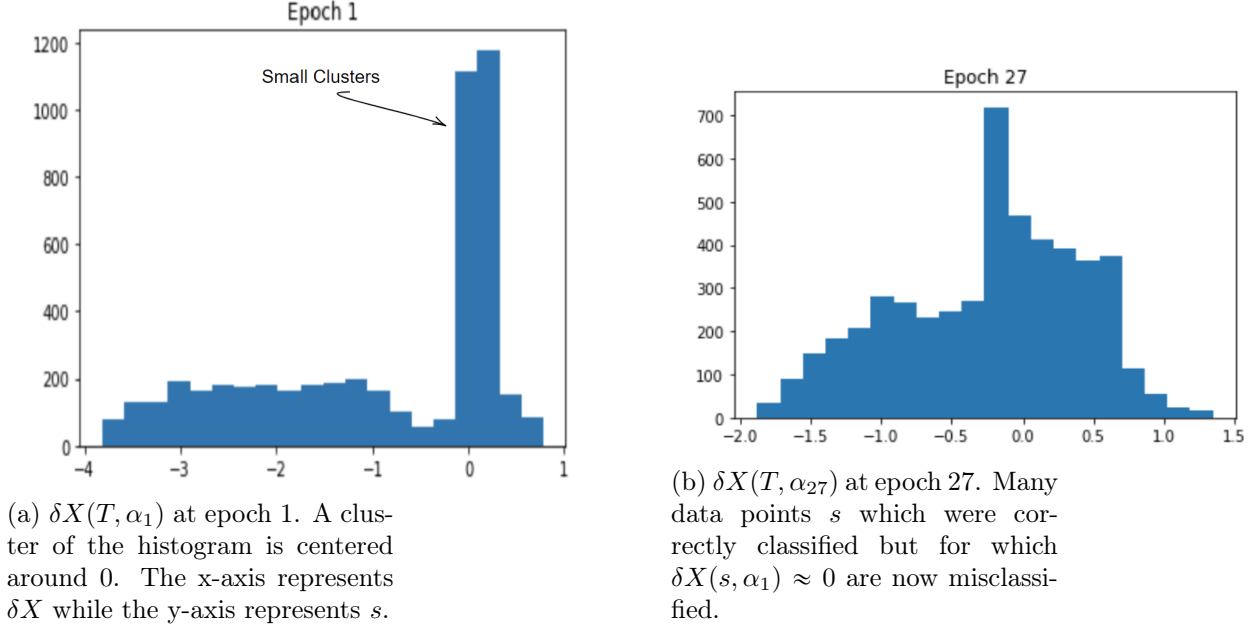


Figure 13: Histogram of δX at epoch 1 and 27

The step $\text{high acc}(t_0) \rightarrow \text{small loss}(t_0)$ is the most difficult part and is achieved by introducing the *uniform doubling condition* on the training set. The step $\text{small loss}(t_0) \rightarrow \text{small loss}(t)$ is fundamental to training because training is essentially the minimization of loss, and the step $\text{small loss}(t) \rightarrow \text{high acc}(t)$ is achieved using inequality (37) from [8]. Throughout the paper, we assume that the loss always decreases as training progresses. This, however, is not always the case. Nevertheless, we can reformulate the notion of stability of accuracy in the following way for the situations in which loss does not always decrease. **Stability of accuracy:** if accuracy is high at t_0 and for some $t > t_0$ we have that $\text{loss}(t) \leq \text{loss}(t_0)$ then accuracy at t will be high.

To obtain stability of accuracy, it is essential to demonstrate two key points. First, a low loss should result in high accuracy, which can be derived from the following inequality:

$$1 - \text{acc}(t) \leq \frac{\text{loss}(t)}{\log(2)}, \quad (94)$$

as presented in Lemma N.5. However, this bound is only useful if the loss is sufficiently small. So we wish to obtain conditions on the training set T which ensure that the loss not only decreases but also becomes sufficiently small during training. In general, it might not be possible to find such a condition on the training set. However, stability of accuracy, which is the main result of [8] and this paper (under different conditions), means that high accuracy at the initial time guarantees high accuracy for future time. So we already assume that accuracy is high at t_0 . Assuming that the uniform doubling condition holds on the training set T and that the accuracy of the DNN is high at t_0 , we will show that the loss at t_0 is small, see (245). Then, because loss decreases during training, we can use (94) to obtain that accuracy stays high for the rest of the training (i.e., for $t > t_0$).

So, it is important to understand why high accuracy does not always imply a low loss. Sometimes we can have high accuracy and high loss simultaneously. This occurs when some objects are significantly misclassified, and there is a small group of elements near zero that are correctly classified, yet still have a high loss. Therefore, to achieve stability of accuracy, it is crucial to ensure that elements are not grossly misclassified and that no small clusters of δX exist near zero. Refer to Def. L.3 and Prop L.4 for further information.

In previous work [8], a doubling condition was introduced on the training set T , which ensured no small clusters of objects in the training set. This doubling condition on T also ensures no small cluster in $\delta X(T, \alpha)$ and therefore guarantees the stability of accuracy. That is, if a DNN trained on T achieves sufficiently high accuracy at t_0 it will have that its accuracy will remain high for $t \geq t_0$, see Theorems L.2 and L.7. While leading to novel theoretical results, this doubling condition has three drawbacks. First, for $T \subset \mathbb{R}^n$ the doubling condition defined on slabs in $\mathbb{R}^n$ depends on the choice of the slabs. This means that the resulting stability of accuracy theorem depends on the choice of slabs used to verify the doubling condition and not only on the training set T . Second, this doubling condition cannot be implemented numerically as it requires checking it for infinitely many slabs with arbitrarily small widths. Finally, the constants which are given by the stability theorem in [8] are also very large and not practically useful, see Theorem L.7 and (223).

In this work, we present a new uniform doubling condition that resolves these three difficulties. First, this doubling condition does not depend on the choice of slabs and so depends only on the training set T . This leads to stability of accuracy in terms of the training set T only. That is, assuming the training set T satisfies the uniform doubling condition, there is a subset of parameters $A \subset \mathbb{R}^\nu$ such that for a DNN with parameters $\alpha \in A$ and certain architecture if it has high accuracy on the training set at some time t_0 then it will have high accuracy for all time $t > t_0$. Second, our uniform doubling condition can be verified numerically. In practice, for high-dimensional data (i.e. $T \subset \mathbb{R}^n$ for n very large), verification is subject to the curse of dimensionality and becomes harder to check as dimension n grows. Even in small dimensions, this uniform doubling condition probably cannot be fully verified; however, we show numerically that a simplified version of the uniform doubling condition can be checked for a data set and is an indication of whether loss becomes small during training. That is, we construct simple training sets for a decision boundary problem and check the simplified uniform doubling condition (SUDC) on them. We show that DNNs with similar accuracy trained on these training sets will have lower loss based on the SUDC. Using (94), we conclude that the SUDC is a good indication of whether the DNN has stability of accuracy in these model examples. For more, see Examples K.1, K.2 and K.3. In Subsection L.3, we explain why it is reasonable to assume that SUDC provides important information for the stability of accuracy of a DNN. Finally, we also show a correlation between the SUDC and the accuracy and loss of the DNN on the test set, see Example K.4, Example K.5, and Subsection K.1. That is, we show numerically that the training sets that better satisfy the SUDC have that:

1. DNNs trained on them have higher stability of accuracy.
2. DNNs trained on them have higher accuracy on the test set.

7 Motivation and some numerical examples

In this section, we motivate our results using some numerical examples on a toy problem as well as a brief explanation of overfitting and how the accuracy of the training set is related to the accuracy of the test set.

Example 7.1. *In this example, we are provided with a collection of two-dimensional points, and our goal is to determine the boundary separating the two regions. The dataset is generated by constructing a random polynomial function of a specific degree and then sampling points uniformly across a range of x -values. For each x -value, we calculate the polynomial function to obtain the corresponding y -value and then shift the points up or down to create points above and below the decision boundary. This results in two distinct regions of points, which we label as red or blue. The region above the polynomial function is labeled red, while the region below it is labeled blue. We also introduce noise to displace the y -values*

up or down, causing some red points to fall below the decision boundary and some blue points to rise above it. Refer to Fig. 14 for an illustration.

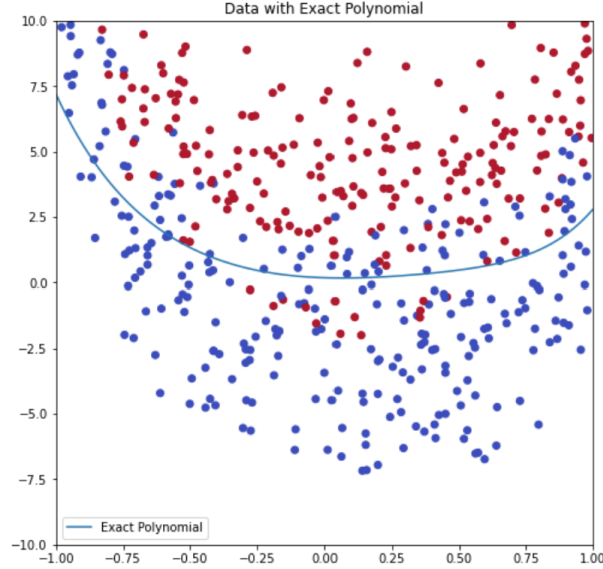


Figure 14: Decision boundary plot for a two-class classification problem generated by a random polynomial function of degree 6, with added noise. The blue and red points represent the two classes, and the blue line shows the original polynomial.

The objective is to train a DNN to learn the decision boundary between these regions. Specifically, the DNN should learn a function that takes a two-dimensional point as an input and outputs a binary label indicating whether the point belongs to the region above or below the polynomial. Since the dataset is synthetically generated, we know the true decision boundary and can use it to assess our DNN’s performance.

We create a neural network model for this classification task on the two-dimensional dataset. The model has two hidden layers: the first is a fully connected layer with five neurons and a ReLU activation function, while the second is a fully connected layer with two neurons and a softmax activation function. The model is trained using the cross-entropy loss function and the Adam optimizer. Although the main results in this work focus on the absolute value activation function or activation functions with only finitely many critical points, the ReLU activation function is used in this example for convenience. The relationship between δX and the DNN’s training accuracy does not depend on the activation function.

After training the DNN, we plot its accuracy, as shown in Fig. 15. In this example, the loss decreases throughout training, but for the first ten epochs, the accuracy on both the training and test set decreases before beginning to increase. The goal of this work is to explain why the DNN’s training accuracy sometimes decreases as the loss decreases and to provide conditions that ensure that this does not happen when accuracy is high. First, we examine this phenomenon in the simple case by analyzing δX , as defined in (29), for this DNN during training.

In Fig. 74, we display the classification confidence ($\delta X(T, \alpha)$) of the DNN at various stages of training for the training set T . During the initial ten epochs, the accuracy of the training set declines due to the existence of two small clusters of $\delta X(T, \alpha)$ near 0. One of these clusters, which we call the correctly classified cluster, is located to the right of 0, while the other, called the misclassified cluster, is to the left of 0. As the training progresses, the clusters merge into a single large cluster on the left side of 0. The loss decreases as the misclassified cluster moves closer to 0 during training; however, the accuracy drops because the correctly classified small cluster also shifts to the left of 0, resulting in incorrect classification of its elements.

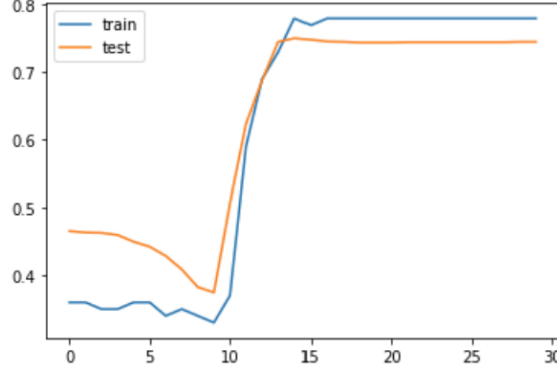


Figure 15: The graph of the training and test accuracy of our DNN. The x-axis represents the epochs of the DNN, and the y-axis represents its accuracy.

As mentioned, stability of accuracy occurs when the accuracy of a DNN is high, and the loss is sufficiently small. This relationship between high accuracy and low loss is essential for maintaining consistent performance during the training process. One of the primary motivations of this work is to achieve a sufficiently small loss, which in turn is expected to ensure high accuracy of the DNN. By focusing on finding conditions to obtain sufficiently small loss, we aim to create a direct connection between the decrease in loss and the increase in accuracy, thus promoting the stability of accuracy throughout the training process. This approach enables the development of DNNs that can maintain their performance as they train.

Example 7.2. It is important to note that avoiding small clusters in $\delta X(T, \alpha)$ is a sufficient but not necessary condition for stability of accuracy. In this example, we train the same model as in Example 7.1 and show the accuracy of the model over time and its δX at epoch 1, see Fig. 16. Even though there is a small cluster in δX , we still have stability of accuracy.

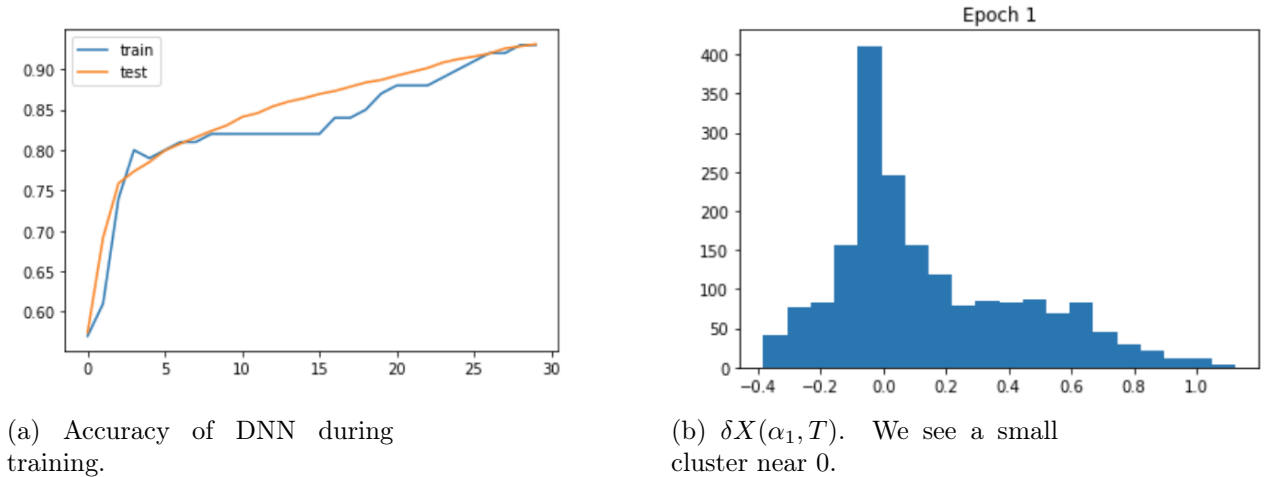


Figure 16: Avoiding small clusters is a sufficient but not necessary condition for stability of accuracy.

7.1 Overfitting: the connection between the accuracy on the training set and accuracy on the test set

Overfitting is a common issue in machine learning, particularly in deep neural networks. It occurs when a model learns to fit the training data too well, capturing not only the underlying patterns in the data but also the noise and other irrelevant details. As a result, even in cases when the model performed well on the training set, it might perform poorly on unseen or test data because it fails to generalize from the training data effectively. For more on overfitting of DNNs, see [39, 30, 13].

Monitoring the model's performance on both the training and test datasets during the training process can help identify overfitting. Overfitting is suspected when the model's performance on the training data continues to improve, while its performance on the test data plateaus or even worsens. This indicates that the model has become too specialized in recognizing features and patterns in the training data, making it unable to generalize to new, unseen data.

When training a DNN, it is crucial to ensure that the accuracy of the model on the training set increases while the loss decreases. If a model's accuracy on the training set increases and the DNN does not overfit, it indicates that the model is learning the underlying structure of the data and can generalize well to new, unseen data. Thus, to achieve a well-performing DNN, it is essential to strike a balance between increasing accuracy on the training set and avoiding overfitting. This can be done by using techniques such as regularization, dropout, and careful selection of optimization algorithms and their hyperparameters. By ensuring that the model's accuracy increases on the training set while avoiding overfitting, it is possible to obtain a DNN that can generalize well and perform effectively on new, unseen data.

In Examples K.4, K.5 and Subsection K.1, we see that data sets that satisfy the SUDC for more slabs have that DNNs training on them achieve higher accuracy and lower loss not only on the training set but also on the test set. This can be explained as follows: DNNs training on data that satisfy the UDC for more slabs (see Def. 8.3) achieve lower loss on the training set (see Theorem 8.6). The SUDC is a decent approximation of the UDC (see Subsection L.3). This leads to higher accuracy on the training set and greater stability of accuracy (see, e.g., (94)). If the DNN is not overfitting, this also leads to higher accuracy and lower loss on the test set.

8 Rigorous mathematical results for stability of accuracy and numerical examples

The following are some important definitions we must introduce before the main result of this paper is stated. As mentioned, our goal is to ensure the stability of accuracy during training (see Theorem 8.6). To this end, we introduce a condition on the training set T called the uniform doubling condition.

We define the uniform doubling condition on the data set $T \subset \mathbb{R}^n$ via truncated slabs in $\mathbb{R}^n$.

Definition 8.1 (Slab S' in $\mathbb{R}^n$). *A slab S' with width κ is defined by some unit vector u and center $s \in \mathbb{R}^n$ as,*

$$S'_\kappa := \{x : |(x - s) \cdot u| \leq \frac{\kappa}{2}\}. \quad (95)$$

Thus a **slab** S' in $\mathbb{R}^n$ is the domain between parallel $(n - 1)$ -dim hyperplanes, see Fig. 17. The **width of a slab** is given by the (shortest) distance between the bounding hyperplanes. Thus, a slab can be defined by a unit vector, a center, and a width, see Fig. 17.

Definition 8.2 (Truncated slab S in $\mathbb{R}^n$). *A truncated slab S is the intersection of a slab defined by some unit vector u and center at $s \in \mathbb{R}^n$ with a finite number k of linear constraints*

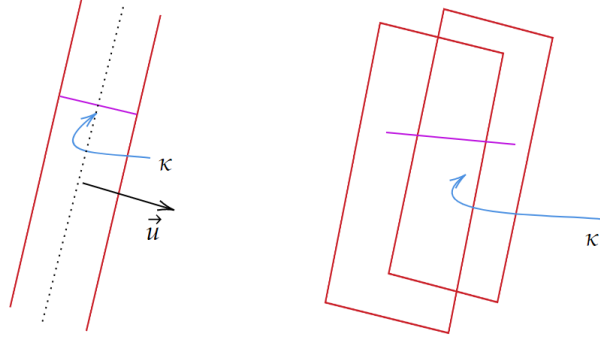


Figure 17: left: slab in $\mathbb{R}^2$. Width is κ , the center is a dashed line, and normal vector $\vec{u}$ is shown. right: slab in $\mathbb{R}^3$.

$$S_\kappa = S_\kappa(u, s, v_1, t_1, \dots, v_k, t_k) := \left\{ x : |(x - s) \cdot u| \leq \frac{\kappa}{2} \text{ and } v_i \cdot x \leq t_i \ \forall i = 1, \dots, k \right\}, \quad (96)$$

where the $v_i \in \mathbb{R}^n$ are the normal vectors of hyperplanes which form the truncation set. We call k the truncation number (the number of elements in the truncation set). See Fig. 18 for an example.

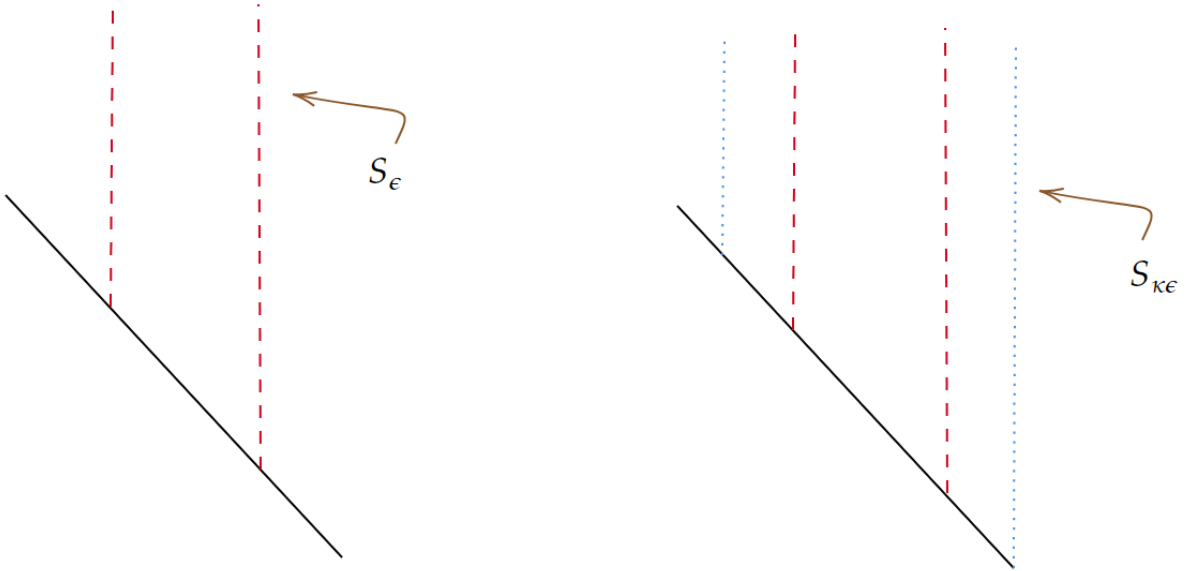


Figure 18: left: truncated slab S_ϵ . The truncation is a solid line (in black) and the slab is a dashed line (in red). right: S_ϵ is shown in dashed lines (red) and its scale $S_{\kappa\epsilon}$ is shown as the dotted lines (in blue).

A truncation set of size k is a set of k hyperplanes. We now consider a set L which is a family of size k of truncation sets. For a family L of truncation sets with k elements in each truncation set, we define the uniform doubling condition on the training set T :

Definition 8.3. Let $\bar{\mu}$ be the extension of the measure μ from T to $\mathbb{R}^n$ by $\bar{\mu}(A) = \mu(A \cap T)$ for all $A \subset \mathbb{R}^n$. Let L be a family of size k of truncation sets. The uniform doubling condition holds on T if there exists $\kappa > 1$ and $\delta, \sigma, \beta, \ell > 0$ so that $\forall$ truncated slabs S s.t. S is generated by a truncated set $h \in L$ there exists ℓ , such that for all $\ell\kappa^i < \beta$ with $i \in \mathbb{N}$, we have

$$\bar{\mu}(S_{\ell\kappa^{i+1}}) \geq \min\{\delta, (1 + \sigma) \bar{\mu}(S_{\ell\kappa^i})\}. \quad (97)$$

Essentially, this condition requires that as we double the width of a slab S by κ to obtain the slab S_κ we capture more data than was originally in the slabs S (i.e., S_κ contains more data than the slab S). See Fig. 19 as an example. This ensures that the data set T does not have small clusters of data, which in turn ensures the stability of accuracy of a DNN during training.

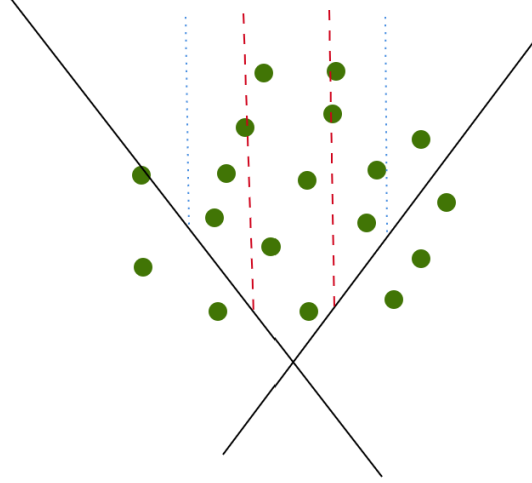


Figure 19: Truncated slab S_ϵ . The truncation is a solid line (black), and the original slab is a dashed line (in red). We also see the scaled slab $S_{\kappa\epsilon}$ as a dotted line (in blue).

For reasons that will be explained shortly, we call κ the doubling constant, ℓ the scaling factor, and β the scale parameter.

As mentioned, β is the scale parameter (of the doubling condition). Put simply, it is the largest width of the slab S for which the doubling condition is satisfied. The larger β , the more slabs must satisfy the doubling condition. If $(1 + \sigma)\mu(S) \geq \delta$, the doubling condition is automatically satisfied. Thus, we can think of δ as the largest mass for which the doubling condition must be satisfied. κ is the factor by which we "double" the slab S , and so it is referred to as the doubling constant. ℓ is the scaling factor, and for a slab S we have that ℓ is the smallest width for which the doubling condition for S must be satisfied. L is a family of size k truncations, and so the doubling condition holds for some set L if $\forall h \in L$ and slab S truncated by the k elements in h , we have that S (with the truncations) satisfies the doubling condition.

The truncation set L is closely associated with the non-linear absolute value activation function in the DNN. To clarify this connection, we first introduce the truncation set of a DNN.

We define the truncation set of a DNN as follows:

Definition 8.4. *We start with a DNN with only one layer:*

1. Take $f_1 = \lambda \circ M_1 : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (layer map), with $\lambda(x_1, \dots, x_m) = (|x_1|, \dots, |x_m|)$ the activation function and M_i a affine map, to be a DNN. Consider the set of coordinate hyperplanes of $\mathbb{R}^m$, $P_1 = (0, x_2, \dots, x_m), \dots, P_m = (x_1, x_2, \dots, 0)$. Define first layer truncation set of hyperplanes $B_1 = \{f_1^{-1}(P_1), f_1^{-1}(P_2), \dots, f_1^{-1}(P_m)\}$ for the $f_1^{-1}(P_w)$ which are $(n - 1)$ dimensional and $1 \geq w \geq m$. See Fig. 20.

For a DNN with more layers, we define i th layer of the truncation set of the DNN in a similar manner:

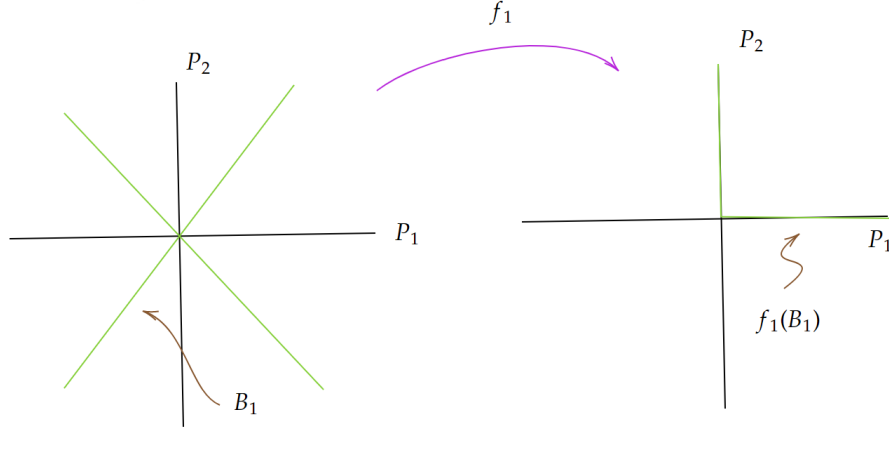


Figure 20: Mapping $f_1 : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. The set B_1 is shown in green.

2. Take $F_i = f_i \circ f_{i-1} \dots f_2 \circ f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{c(i)}$, with $c(i)$ the number of dimensions of the i th layer of the DNN. See Fig. 21. Take the coordinate hyperplanes $P_1 = (0, x_2, \dots, x_{c(i)})$, $P_2 = (x_1, 0, \dots, x_{c(i)})$, $\dots$, $P_{c(i)} = (x_1, x_2, \dots, 0)$. Define the i th layer truncation set

$$B_i = \{F_i^{-1}(P_1), F_i^{-1}(P_2), \dots, F_i^{-1}(P_{c(i)})\}, \quad (98)$$

for the $F_i^{-1}(P_w)$ which are $(n-1)$ dimensional and $1 \geq w \geq c(i)$.

3. We take $B = \bigcup_{i=1}^l B_i$ to be the DNN truncation set, with l the numbers of layers in the DNN.

To ensure stability for f_1 (as a 1 layer DNN), the data set T must satisfy the doubling condition only with truncations B_1 . Similarly, for DNNs with more than one layer. B is a family of sets of hyperplanes (or half hyperplanes, which for simplicity we will refer to as hyperplanes) that are mapped to the coordinate planes under layer maps of our DNN. As we will see, in order for a DNN to have stability of accuracy, we must ensure that the truncation set of the DNN is an element of the family of truncation sets L for which the training set T satisfies the doubling condition.

We now introduce two more definitions before the main result is stated.

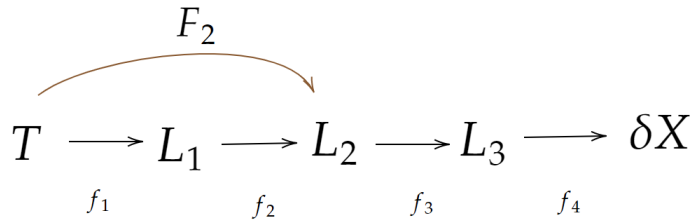


Figure 21: What are the F_i maps?

Definition 8.5. Take $d_{\min} = |\min_i (\sigma_i)^2|$ and $d_{\max} = |\max_i (\sigma_i)^2|$ with σ_i the nonzero singular values of the linear map $W_n \circ \dots \circ W_2 \circ W_1$ (in general the M_i are affine maps, but when obtaining $d_{\min}$ and $d_{\max}$ we ignore the bias vectors of M_i and only consider their linear part W_i).

Finally, define the good sets and bad sets of T as follows: for $\eta \geq 0$, the good set of margin η at time t is

$$G_\eta(t) = \{s \in T : \delta X(s, \alpha(t)) > \eta\} \quad (99)$$

and the bad set of margin η at time t is

$$B_{-\eta}(t) = \{s \in T : \delta X(s, \alpha(t)) < -\eta\}. \quad (100)$$

The main result of this paper is the following Theorem:

Theorem 8.6. *Assume that a data set $T \subset \mathbb{R}^n$ satisfies the doubling condition (8.3), with $\kappa > 1$ and $\delta, \sigma, \beta > 0$, $1 > \ell > 0$ and family of truncations L with truncation sets of size k . Then for a.e. DNN with truncation set $B \in L$ at time t_0 we have that there exists constants $C_2 = C_2(\eta, K, \kappa, \sigma, \beta, \delta_0, \ell, d_{\min}, d_{\max})$ and ξ' such that if we have:*

$$\frac{d_{\min}\beta}{2} \geq \eta, \quad 0 \leq \xi' \in [d_{\min}\ell, d_{\max}\ell], \quad \xi' < \eta, \quad \delta_0 < \delta, \quad (101)$$

and if the good and bad sets at $t = t_0$ satisfy

$$\mu(G_\eta(t_0)) > 1 - \delta_0 \quad \text{and} \quad B_{-\xi'}(t_0) = \emptyset, \quad (102)$$

then for all $t \geq t_0$,

$$\text{loss}(t) \leq \log(2)C_2, \quad \text{acc}(t) = \mu(G_0(t)) \geq 1 - C_2. \quad (103)$$

Here we have that

$$\begin{aligned} C_2(\eta, K, \kappa, \sigma, \beta, \delta_0, \ell, d_{\min}, d_{\max}) := & \\ & \max_{\xi \in [d_{\min}\ell, d_{\max}\ell]} \frac{1}{\log(2)} \left(\log(1 + (K-1)e^{-\eta}) + \log(1 + (K-1)e^{-(\frac{\eta}{\kappa})})\delta_0 \right. \\ & + \frac{\delta_0}{(\sigma+1)^{\frac{\log(\frac{\eta}{\xi})}{\log \kappa} - 1}} \log(1 + e^\xi(K-1)) \\ & \left. + \sum_{i=0}^{p-1} \frac{\delta_0}{(\sigma+1)^{\frac{\log(\frac{\eta}{\xi\kappa^{i+1}})}{\log \kappa} - 1}} \log(1 + (K-1)e^{-\xi\kappa^{i+1}}) \right), \end{aligned} \quad (104)$$

where $p = \lfloor \frac{\log(\frac{\eta}{\xi})}{\log \kappa} \rfloor$.

Here, the condition that the truncation set of the DNN, B , is an element of L , and condition (101) are only required at t_0 .

Remark 8.1. We would like the constant C_2 to be as small as possible so that accuracy is bounded by a large number (a number close to 1) and loss is bounded by a small number. It is important to note that the log functions which appear in C_2 come from the cross-entropy loss (89) and depend on the log in the cross-entropy loss. The exponential functions (such as $e^{-\eta}$), on the other hand, come from softmax (88). Because of this, if we use $\log_{10}$ for the cross-entropy while training (instead of $\ln$) we will have a smaller $\log(2)C_2$ constant, meaning that loss will be smaller. This, however, will not decrease the constant C_2 (or increase the bound on accuracy) given that we are dividing by $\log(2)$.

Changing the base in softmax (88) does, however, change the constant C_2 . By changing the base in softmax from e to some constant b , one can replace every e in C_2 with b . In Fig. 22 we show how C_2 changes based on b , though in this example we fix $\xi = .4$. We also take $K = 2$, $\eta = 18$, $\delta_0 = 0.2$, $\kappa = 2$ and $\sigma = 0.9$. We see that C_2 decreases from .06 when $b = 2$ to .03 when $b = 5$. This is a significant change in terms of stability of accuracy (i.e., bounding accuracy by .97 is much better than bounding it by .94). Throughout the paper, we use Desmos to compute C_2 because of its level of precision.

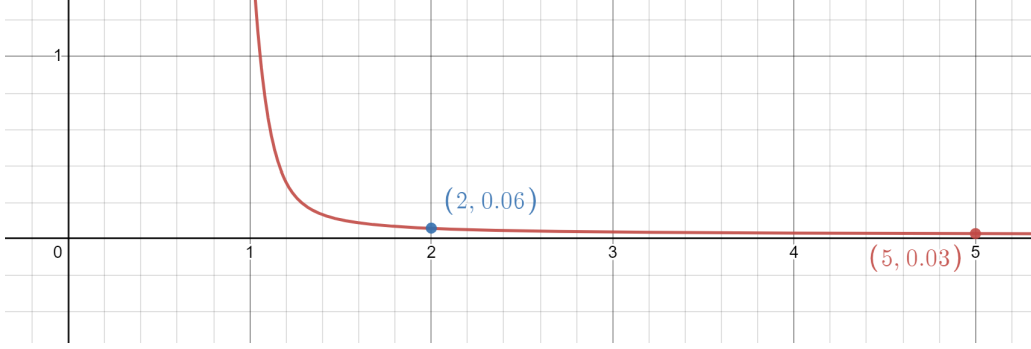


Figure 22: C_2 , represented on the y -axis, for different values of b , represented on the x -axis, with fixed $\xi = .4$. We see that C_2 decreases from .06 when $b = 2$ to .03 when $b = 5$. This is a significant change in terms of stability of accuracy.

Interpretation of the constant C_2

We would like the constant C_2 to be small so that accuracy is high. To make C_2 small, we need to understand how the different parameters in the equation affect its value. Let's analyze each part of the equation:

1. $\log(1 + (K - 1)e^{-\eta})$: To minimize this term, we want η to be large, because when η is large, $e^{-\eta}$ approaches 0, and the term inside the logarithm approaches 1, leading to a smaller value of the logarithm. The connection between η and the uniform doubling condition is given in the condition (101):

$$\frac{d_{\min}\beta}{2} \geq \eta. \quad (105)$$

This means that we want β , from the doubling condition, to be large. That is, we want the widths of the slabs that satisfy the doubling condition to be as large as possible.

2. $\log(1 + (K - 1)e^{-(\frac{\eta}{\kappa})})\delta_0$: We want δ_0 to be small to minimize this term since it is multiplied by the whole expression. Additionally, having a large η will also help in making this term small for the same reasons explained in point (1).

The connection between δ_0 and the uniform doubling condition is given by the condition (101):

$$\delta_0 < \delta, \quad (106)$$

with δ coming from the uniform doubling condition. In essence, the larger δ is, the bigger the measure of the training set that must satisfy the DC, see (97).

3. $\frac{\delta_0}{(\sigma+1)^{\frac{\log(\frac{\eta}{\xi})}{\log \kappa} - 1}} \log(1 + e^{\xi}(K - 1))$: To minimize this term, we want ξ to be small, because when ξ is small, e^{ξ} approaches 1, and the term inside the logarithm approaches $1 + (K - 1)$. Thus, we would want $d_{\max}\ell$ to also be small and we can ensure that $d_{\max}\ell$ is small by checking the uniform doubling condition on the training set for small ℓ . The size of the widths of the slabs that we start checking the UDC with should be small.

Minimizing this term also requires κ and $(1 + \sigma)$ to be roughly the same size, as this causes the term to decrease. Essentially, this means that "doubling" the width of the slab should "double" the number of points contained in the doubled slab. Ideally, $(1 + \sigma)$ should be much larger than κ , but satisfying the uniform doubling condition is unlikely in such cases. Also, a small δ_0 will help in minimizing this term.

4. $\sum_{i=0}^{p-1} \frac{\delta_0}{(\sigma+1) \frac{\log\left(\frac{\eta}{\xi\kappa^i+1}\right)}{\log \kappa} - 1} \log(1 + (K-1)e^{-\xi\kappa^i})$: To minimize this term, we want κ and $(1+\sigma)$ to be of about the same size, because when their sizes are similar, the term inside the summation becomes smaller. Additionally, as ξ decreases, the terms inside the logarithms increase, and as a result, the sum increases. However, increasing κ can help in reducing the size of the terms inside the logarithms, leading to a smaller sum overall. Finally, $p = \lfloor \frac{\log(\frac{\eta}{\xi})}{\log \kappa} \rfloor$. This value of p represents the number of terms in the sum and is determined by the value of ξ , which is also included in the denominator of the logarithmic term inside the sum. As ξ decreases, the value of p increases, meaning that the sum contains more terms, and thus, the overall value of the sum might increase. Again, a small δ_0 will help in minimizing this term.
5. The function depends on K through the terms inside the logarithms. As K increases, the terms inside the logarithms also increase, but since we are taking the logarithm of these terms, the effect of increasing K is dampened. Therefore, we can say that the function has a sub-linear dependence on K .

In summary, to make the equation small, we would like a large η , small ξ , small δ_0 , large β and large $d_{\min}$, so that (101) is satisfied, and to make κ and $(1+\sigma)$ of about the same size. The dependence on K is sub-linear.

A large η makes the good set $G_\eta(t_0)$ small, and so δ_0 would be large. In general, we try to balance having a large η and small δ_0 . Numerically it seems like having $\eta \approx 18$ and $\delta_0 \approx .2$ is reasonable for examples such as Example 7.1 when the DNN has high accuracy. To have a large η , we need a large β and $d_{\min}$ to not be too small, see Remark 8.3 for more on $d_{\min}$. We therefore also want δ to be large, so that δ_0 can be large, see (101). Finally, because C_2 is defined as the max over $\xi \in [d_{\min}\ell, d_{\max}\ell]$, we want $d_{\max}\ell$ to also be small. However, we don't want it to be too small otherwise finding a $\xi' \in [d_{\min}\ell, d_{\max}\ell]$ for which $B_{-\xi'}(t_0) = \emptyset$ might be impossible.

For a simple example, when $\delta_0 = 0.2$, $\eta = 18$, $K = 2$, $\kappa = 2$, $\sigma = 0.9$, $\ell = .001$, $d_{\min} = .01$ and $d_{\max} = 800$ we have that $C_2 = .05$, and so assuming the conditions of Theorem 8.6 are satisfied we have that the accuracy is bounded by .95 and loss is bounded by .015. More on this will be discussed in Section L.

Finally, the variable ξ is probably the most important parameter for the constant C_2 , and while we don't know it before training (given that we don't know $d_{\min}$ and $d_{\max}$), we can still study how stable training on some given data might be for different assumptions of what ξ will be after training. In Fig. 23 we show how C_2 depends on ξ , with the other parameters again being $\delta_0 = 0.2$, $\eta = 18$, $K = 2$, $\kappa = 2$, $\sigma = 0.9$. We see that for $\xi < 1$ one can obtain significant stability of accuracy results (i.e. accuracy will be bounded from below in a significant manner). We can obtain a small ξ by ensuring that $\ell d_{\max}$ is small, which means checking the uniform doubling condition on T for a small enough ℓ . We also need the bad set to be small, but we are assuming that accuracy on the training set is already high in which case the bad set would be small, see Example 3.12. Having a small bad set does not mean that training should stop, because it might still be worthwhile to increase the size of the good sets G_η and make the bad set even smaller. However, stability of accuracy will ensure a lower bound on the accuracy and loss of the DNN as training progresses. η is probably the second most important constant and we will talk more about this constant in Section L.

Remark 8.2. One can change condition (101) so that $\xi' \in [d_{\min}\ell\kappa^i, d_{\max}\ell\kappa^{i'}]$ for $i, i' \in \mathbb{N}$. Then C_2 is taken as the max over $\xi' \in [d_{\min}\ell\kappa^i, d_{\max}\ell\kappa^{i'}]$. For $i, i' = 0$, we obtain the previous result. This can allow better control when choosing ξ' and also means that we can obtain a smaller C_2 .

Now, we discuss the conditions on the DNN. First, the truncation set B of the DNN is a family of sets of hyperplanes in $\mathbb{R}^n$ and we require that no object in the training set be contained in one of

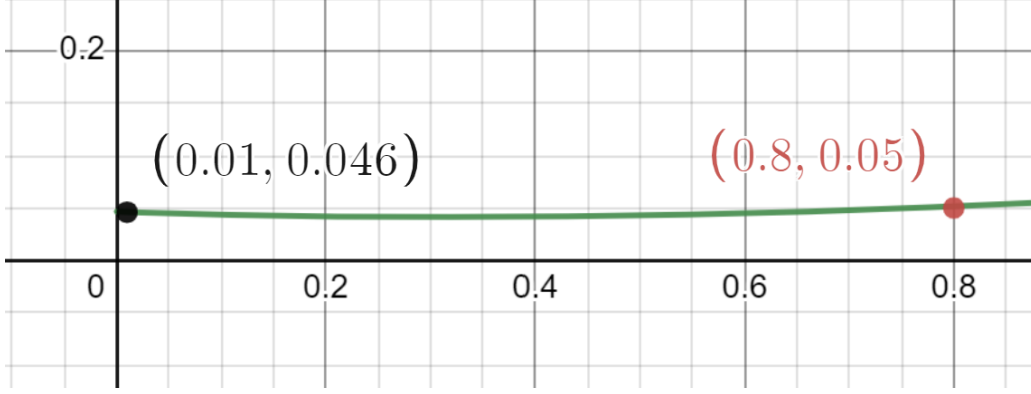


Figure 23: C_2 , represented on the y-axis, for different values of ξ , represented on the x-axis.

these hyperplanes. This is why we say that for a.e. DNN for which $B \in L$ do we have the stability of accuracy. The a.e. is to ensure that no element in T is contained in a hyperplane of one of the sets of B . Further, we require that $B \in L$ at t_0 , which in practice would mean that for each element in L the number of truncations would be very large (i.e. exponential in the layers l of the DNN) and so checking the uniform doubling condition with even one truncation set might not be feasible.

This is why we introduce the simplified version of the uniform doubling condition and numerically show that, on a model example, this SUDC is correlated with the stability of accuracy, see Subsection K. In this sense, Theorem 8.6 is still useful because we can check the SUDC on a training set for constants that would make C_2 small (assuming the good set is large and the bad set is small). Thus, even though the truncations are being ignored, we can still study numerically how the SUDC provides information on the stability of accuracy of the DNN using C_2 as a guide, see Example K.1. In Subsection L.3, we explain why truncations might not be important when verifying the uniform doubling condition, and why it might be reasonable to assume that the SUDC provides important information for the stability of DNNs.

In conclusion, in this theorem, a training set T which satisfies the uniform doubling condition and a set of DNNs are given, with the DNNs defined by their architecture and parameters. If at t_0 a DNN from this set trained on the training set T has high accuracy, in the sense that its good set is big and its bad set is small, then the DNN will have high accuracy for all time. This is because the DNN's loss will be small at t_0 . In Subsection L.1 we show how a doubling condition on δX leads to a similar result, and this is because (245) gives a bound on the loss while (94) bounds accuracy in terms of loss. The doubling condition on δX can be checked numerically, but it is much simpler to just look at the loss at t_0 and see if it is small, in the sense that equation (94) is the reason accuracy is high.

Remark 8.3. For most DNNs $d_{\min}$ can be very small (almost zero) and so satisfying (101) might be difficult. However, it has been shown that one can perform an SVD on the layer matrices of a DNN and remove the small singular values without affecting the accuracy of the DNNs, see [68, 70, 69, 11, 2, 9]. In this thesis, we also introduced a method for pruning small singular values of matrices, using the MP threshold $\sqrt{\lambda_+}$. This SVD pruning method is a valuable technique when training DNNs. During the training process of a DNN, every few epochs, one can apply the SVD on the weight layer matrices, denoted as W_i . Singular values that are smaller than a specified threshold can be pruned (set to zero) to reduce the size of the model. By judiciously selecting an appropriate threshold, one can ensure that the accuracy of the DNN remains largely unaffected, thus achieving a balance between model compression and performance. This pruning technique works well with standard training algorithms such as SGD and Adam; see [9].

For example, it was been shown that for DNNs trained on MNIST with a single 1000×1000 weight layer matrix W , one can generally remove the small singular values to make $d_{\min} > \sqrt{700}$ without

changing the accuracy of the DNN, see [54]. In general, many of the small singular values of a DNN can be removed without changing its accuracy, see [70, 54, 9, 58]. After the small singular values are removed, the new DNN will have the stability result given in this theorem, though with much bigger $d_{\min}$.

Further, for a DNN with a single $n \times n$ weight layer W , under the assumption that by removing small singular values of a weight matrix W one is only removing the noise in W while preserving the information contained in W , it should be the case that one can remove all singular values of W smaller than $2.858\sigma_{med}$ where σ_{med} is the median empirical singular value of W , without decreasing accuracy, see [21]. More on this will be discussed in a different paper. For more on applying singular value decomposition for DNNs, see [70, 69, 11, 2]. Finally, for numerical simulation of how the SUDC leads to stability of accuracy, see Appendix K.

References

- [1] Joshua Agterberg, Zachary Lubberts, and Carey E Priebe. Entrywise estimation of singular vectors of low-rank matrices with heteroskedasticity and dependence. IEEE Transactions on Information Theory, 68(7):4618–4650, 2022.
- [2] Xing Anhao, Zhang Pengyuan, Pan Jieli, and Yan Yonghong. Svd-based dnn pruning and re-training. Journal of Tsinghua University (Science and Technology), 56(7):772–776, 2016.
- [3] Jinho Baik, Gérard Ben Arous, and Sandrine Péché. Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices. 2005.
- [4] Zhigang Bao and Dong Wang. Eigenvector distribution in the critical regime of bbp transition. Probability Theory and Related Fields, 182(1-2):399–479, 2022.
- [5] Zhigang Bao, Xiucan Ding, Wang, and Ke. Singular vector and singular subspace distribution for the matrix denoising model. 2021.
- [6] Florent Benaych-Georges and Raj Rao Nadakuditi. The eigenvalues and eigenvectors of finite, low rank perturbations of large random matrices. Advances in Mathematics, 227(1):494–521, 2011.
- [7] Florent Benaych-Georges and Raj Rao Nadakuditi. The singular values and vectors of low rank perturbations of large rectangular random matrices. Journal of Multivariate Analysis, 111:120–135, 2012.
- [8] Leonid Berlyand, Pierre-Emmanuel Jabin, and Alex Safsten. Stability for the training of deep neural networks and other classifiers. Mathematical Models and Methods in Applied Sciences, 31(11):2345–2390, 2021.
- [9] Leonid Berlyand, Etienne Sandier, Yitzchak Shmalo, and Lei Zhang. Enhancing accuracy in deep learning using random matrix theory. arXiv preprint arXiv:2310.03165, 2023.
- [10] Leonid Berlyand, Theo Bourdais, Houman Owhadi, and Yitzchak Shmalo. Pruning deep neural networks via a combination of the marchenko-pastur distribution and regularization. arXiv preprint arXiv:2503.01922, 2025.
- [11] Chenghao Cai, Dengfeng Ke, Yanyan Xu, and Kaile Su. Fast learning of deep neural networks via singular value decomposition. In Pacific Rim International Conference on Artificial Intelligence, pages 820–826. Springer, 2014.

- [12] Yuxin Chen, Chen Cheng, and Jianqing Fan. Asymmetry helps: Eigenvalue and eigenvector analyses of asymmetrically perturbed low-rank matrices. Annals of statistics, 49(1):435, 2021.
- [13] Omry Cohen, Or Malka, and Zohar Ringel. Learning curves for overparametrized deep neural networks: A field theory perspective. Physical Review Research, 3(2):023034, 2021.
- [14] Romain Couillet and Zhenyu Liao. Random Matrix Methods for Machine Learning. Cambridge University Press, 2022.
- [15] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In 2009 IEEE Conference on Computer Vision and Pattern Recognition, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- [16] Prathapasinghe Dharmawansa, Pasan Dissanayake, and Yang Chen. The eigenvectors of single-spiked complex wishart matrices: Finite and asymptotic analyses. IEEE Transactions on Information Theory, 68(12):8092–8120, 2022.
- [17] Xiucui Ding. High dimensional deformed rectangular matrices with applications in matrix denoising. 2020.
- [18] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations, 2021.
- [19] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, June 2021. URL <http://arxiv.org/abs/2010.11929>. arXiv:2010.11929 [cs].
- [20] Mahyar Fazlyab, Alexander Robey, Hamed Hassani, Manfred Morari, and George Pappas. Efficient and accurate estimation of lipschitz constants for deep neural networks. Advances in Neural Information Processing Systems, 32, 2019.
- [21] Matan Gavish and David L. Donoho. The optimal hard threshold for singular values is $4/\sqrt{3}$. IEEE Transactions on Information Theory, 60(8):5040–5053, 2014. doi: 10.1109/TIT.2014.2323359.
- [22] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Proceedings of the thirteenth international conference on artificial intelligence and statistics, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [23] Gene H Golub and Charles F van Loan. Matrix computations. SIAM Review, 28(2):252–255, 1986.
- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. MIT press, 2016.
- [25] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. CoRR, abs/1412.6572, 2014. URL <https://api.semanticscholar.org/CorpusID:6706414>.
- [26] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. Power-bert: Accelerating bert inference via progressive word-vector elimination. In International Conference on Machine Learning, pages 3690–3699. PMLR, 2020.

- [27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. pages 1026–1034, 2015.
- [28] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. IEEE Signal Processing Magazine, 29(6):82–97, 2012.
- [29] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.
- [30] Kenji Kawaguchi, Leslie Pack Kaelbling, and Yoshua Bengio. Generalization in deep learning. arXiv preprint arXiv:1710.05468, 2017.
- [31] Zheng Tracy Ke, Yucong Ma, and Xihong Lin. Estimation of the number of spiked eigenvalues in a covariance matrix by bulk eigenvalue matching analysis. Journal of the American Statistical Association, pages 1–19, 2021.
- [32] Holger Kösters and Alexander Tikhomirov. Limiting spectral distributions of sums of products of non-hermitian random matrices. arXiv preprint arXiv:1506.04436, 2015.
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. Communications of the ACM, 60(6):84–90, 2017.
- [34] Anders Krogh and John A. Hertz. A simple weight decay can improve generalization. In Advances in Neural Information Processing Systems, pages 950–957, 1992.
- [35] Itamar D Landau, Gabriel C Mel, and Surya Ganguli. Singular vectors of sums of rectangular random matrices and optimal estimation of high-rank signals: The extensive spike model. Physical Review E, 108(5):054129, 2023.
- [36] Yann LeCun, Bernhard Boser, John Denker, Donnie Henderson, Richard Howard, Wayne Hubbard, and Lawrence Jackel. Handwritten digit recognition with a back-propagation network. Advances in Neural Information Processing Systems, 2, 1989.
- [37] William E Leeb. Matrix denoising for weighted loss functions and heterogeneous signals. SIAM Journal on Mathematics of Data Science, 3(3):987–1012, 2021.
- [38] Aitor Lewkowycz, Yasaman Bahri, Ethan Dyer, Jascha Sohl-Dickstein, and Guy Gur-Ari. The large learning rate phase of deep learning: the catapult mechanism. arXiv preprint arXiv:2003.02218, 2020.
- [39] Siyuan Ma, Raef Bassily, and Mikhail Belkin. The power of interpolation: Understanding the effectiveness of sgd in modern over-parametrized learning. In International Conference on Machine Learning, pages 3325–3334. PMLR, 2018.
- [40] Michael W Mahoney and Charles H Martin. Traditional and heavy tailed self regularization in neural network models. In International Conference on Machine Learning, pages 4284–4293. PMLR, 2019.
- [41] Vladimir Alexandrovich Marchenko and Leonid Andreevich Pastur. Distribution of eigenvalues for some sets of random matrices. Matematicheskii Sbornik, 114(4):507–536, 1967.

- [42] Charles H Martin and Michael W Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In Proceedings of the 2020 SIAM International Conference on Data Mining, pages 505–513. SIAM, 2020.
- [43] Charles H Martin and Michael W Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. The Journal of Machine Learning Research, 22(1):7479–7551, 2021.
- [44] Charles H Martin, Tongsu Peng, and Michael W Mahoney. Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data. Nature Communications, 12(1):4122, 2021.
- [45] Xuran Meng and Jianfeng Yao. Impact of classification difficulty on the weight matrices spectra in deep learning and application to early-stopping. Journal of Machine Learning Research, 24:1–40, 2023.
- [46] Sean O’rourke, Vu Van, and Ke Wang. Matrices with gaussian noise: optimal estimates for singular subspace perturbation. arXiv e-prints, pages arXiv–1803, 2018.
- [47] Sean O’Rourke, Van Vu, and Ke Wang. Random perturbation of low rank matrices: Improving classical bounds. Linear Algebra and its Applications, 540:26–59, 2018.
- [48] Zizheng Pan, Bohan Zhuang, Jing Liu, Haoyu He, and Jianfei Cai. Scalable vision transformers with hierarchical pooling. In Proceedings of the IEEE/cvf international conference on computer vision, pages 377–386, 2021.
- [49] Leonid Pastur. On random matrices arising in deep neural networks. gaussian case. arXiv preprint arXiv:2001.06188, 2020.
- [50] Leonid Pastur and Victor Slavin. On random matrices arising in deep neural networks: General iid case. Random Matrices: Theory and Applications, 12(01):2250046, 2023.
- [51] Thiziri Nait Saada and Jared Tanner. On the initialisation of wide low-rank feedforward neural networks. arXiv preprint arXiv:2301.13710, 2023.
- [52] Shai Shalev-Shwartz, Yoram Singer, and Nathan Srebro. Pegasos: Primal estimated sub-gradient solver for svm. In Proceedings of the 24th International Conference on Machine Learning, pages 807–814, 2007.
- [53] Yitzchak Shmalo. Stability of accuracy for the training of dnns via the uniform doubling condition. Annals of Mathematics and Artificial Intelligence, 92(2):439–483, 2024.
- [54] Yitzchak Shmalo, Jonathan Jenkins, and Oleksii Krupchytskyi. Deep learning weight pruning with rmt-svd: Increasing accuracy and reducing overfitting. arXiv preprint arXiv:2303.08986, 2023.
- [55] Zhuoran Song, Yihong Xu, Zhezhi He, Li Jiang, Naifeng Jing, and Xiaoyao Liang. Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction. arXiv preprint arXiv:2203.04570, 2022.
- [56] Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. The Journal of Machine Learning Research, 19(1):2822–2878, 2018.
- [57] Max Staats, Matthias Thamm, and Bernd Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. arXiv preprint arXiv:2206.03927, 2022.

- [58] Max Staats, Matthias Thamm, and Bernd Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. *Phys. Rev. E*, 108:L022302, Aug 2023. doi: 10.1103/PhysRevE.108.L022302. URL <https://link.aps.org/doi/10.1103/PhysRevE.108.L022302>.
- [59] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning*, pages 1139–1147. PMLR, 2013.
- [60] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27, 2014.
- [61] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.
- [62] Matthias Thamm, Max Staats, and Bernd Rosenow. Random matrix analysis of deep neural network weight matrices. *Physical Review E*, 106(5):054124, 2022.
- [63] Sunil Thulasidasan, Gopinath Chennupati, Jeff A Bilmes, Tanmoy Bhattacharya, and Sarah Michalak. On mixup training: Improved calibration and predictive uncertainty for deep neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.
- [64] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021.
- [65] Sunil Vadera and Salem Ameen. Methods for pruning deep neural networks. *IEEE Access*, 10: 63280–63300, 2022.
- [66] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [67] Xuanzhe Xiao, Zeng Li, Chuanlong Xie, and Fengwei Zhou. Heavy-tailed regularization of weight matrices in deep neural networks. *arXiv preprint arXiv:2304.02911*, 2023.
- [68] Yuhui Xu, Yuxi Li, Shuai Zhang, Wei Wen, Botao Wang, Wenrui Dai, Yingyong Qi, Yiran Chen, Weiyao Lin, and Hongkai Xiong. Trained rank pruning for efficient deep neural networks. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pages 14–17. IEEE, 2019.
- [69] Jian Xue, Jinyu Li, and Yifan Gong. Restructuring of deep neural network acoustic models with singular value decomposition. In *Interspeech*, pages 2365–2369, 2013.
- [70] Huanrui Yang, Minxue Tang, Wei Wen, Feng Yan, Daniel Hu, Ang Li, Hai Li, and Yiran Chen. Learning low-rank deep neural networks via singular vector orthogonality regularization and singular value sparsification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 678–679, 2020.
- [71] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3):107–115, 2021.

- [72] Zhixiang Zhang and Guangming Pan. Tracy-widom law for the extreme eigenvalues of large signal-plus-noise matrices. arXiv preprint arXiv:2009.12031, 2020.
- [73] Stephan Zheng, Yang Song, Thomas Leung, and Ian Goodfellow. Improving the robustness of deep neural networks via stability training. In Proceedings of the Ieee Conference on Computer Vision and Pattern Recognition, pages 4480–4488, 2016.
- [74] Mingjian Zhu, Kai Han, Yehui Tang, and Yunhe Wang. Visual transformer pruning. arXiv preprint arXiv:2104.08500, 2(6):7, 2021.

Appendix

A More examples of MP-based pruning together with regularization and sparsification: fully connected DNNs and CNNs

Numerical simulations presented in this subsection show that MP-based pruning enhances the accuracy of DNNs while reducing the number of DNN weights (parameters). The first set of numerical simulations employs fully connected DNNs trained on the MNIST and Fashion MNIST datasets, revealing that MP-based pruning during training improves accuracy by 20-30% while reducing the parameter count by 30-50%. These findings are consistent across various architectures and weight initializations, underscoring the consistency of the MP-based pruning approach. Further, the combination of this approach and sparsification (eliminating parameters below a certain threshold, see [65]) leads to even more significant reductions in parameters (up to 99.8%) while increasing accuracy (by 20-30%). This reduction in parameters is greater than what is achievable through sparsification alone (99.5%); see Subsection A.0.1.

Unless stated otherwise, in all numerical simulations, the parameter matrices were initialized from $N(0, 1/N)$, with N the number of input features, while the bias vectors were initialized to 0. This initialization is closest to the theoretical work in this paper, which is why we use it. The ReLU activation function was applied after every layer, including the final layer, and the softmax activation function was applied afterwards. While it might be easier to train DNNs with other initializations and architectures, we found that we obtained the high accuracies, and high improvements in accuracy (over normal training), when training with the affirmation structure while using MP-based pruning. For example, using a fully connected DNN, we obtained a 91.27% accuracy on the Fashion MNIST test set, which is one of the highest accuracies we observed on the dataset using a fully connected DNN (see Subsection A.0.1). Despite this DNN architecture not being typical for fully connected layers, it is still useful to study because it shows that RMT can be used to drastically improve the accuracy of DNNs that don't have very good architectures (relative to the datasets). MP-based pruning also increases the accuracy of fully connected DNNs that do not have an activation function (other than softmax) on the final layer; for example, see Subsection I.3.

As mentioned, the weight layer matrices W_l have singular values of two types: those that contain information and the ones that don't and, therefore, can be removed (pruned). This separation is done via MP threshold $\sqrt{\lambda_+}$. For the case of fully connected layers, we show numerically that MP-based pruning simplifies the loss landscape, leading to a significant increase in DNN accuracy (by 20-30%). Finally, we show that a combination of MP-based pruning with sparsification preserves or even increases accuracy while reducing parameters by 99.8% vs. MP-based pruning alone, with a reduction of 30-50%, or sparsification alone, with a reduction of 99.5%. Our theoretical results also explain why sparsification does not reduce accuracy; see Lemma 3.13 and Remark 3.5.

Our numerics also explores the application of MP-based pruning on DNNs that already achieve relatively high accuracy on MNIST (Section A.0.1), Fashion MNIST (Section A.0.1), and CIFAR10 datasets (Section A.0.3), including those using Convolutional Neural Networks (CNNs) and sparsification techniques (Section A.0.1, A.0.2, A.0.2 A.0.3). Our results show a substantial reduction in parameters (over 95%) while preserving accuracy through a combination of MP-based pruning during training and post-training sparsification, surpassing the efficiency of using sparsification alone (80-90% reduction in parameters). These extensive simulations for various architectures and initializations demonstrate the consistency and wide applicability of MP-based pruning in optimizing DNN performance.

To see how MP-based pruning performed on a simple regression problem, see Subsection I.3.

Training and testing procedure

The training and testing procedure for each network consists of the following steps:

1. We follow the standard partition for MNIST and Fashion MNIST, with 60,000 images for training and 10,000 images for testing.
2. We train the network for a certain number of epochs.
3. We test the network after each epoch and store the accuracy for later comparison.
4. For the pruned DNN, we apply algorithm 4 after a set number of epochs (defined by the split frequency).

A.0.1 Training of fully connected DNNs on MNIST: simplifying the loss landscape

The results of the simulations are presented in the examples below. These examples show figures and tables that compare the average accuracy and variance of accuracy for both the normally trained and pruned DNNs, as well as the average loss and number of parameters in both DNNs. Detailed discussion and analysis of these results are presented in the examples.

Training hyperparameters:

- Split frequency (ℓ) (every how many epochs we split the pruned DNN and remove small singular values): 7
- goodness of fit (GoF or τ) = .7

See Subsection I.5 for the other hyperparameters in these simulations.

Remark A.1. The algorithm for finding the GoF parameter is given in Subsection H.4. It is used to determine if Assumption 2.1-2.3, that $W_l = R_l + S_l$, are reasonable and that the weight layers can reasonably be modeled as a spiked model (that is, W_l is a deformed matrix).

Example A.1. *We conducted several numerical simulations to compare the performance of the normal DNNs, trained using conventional methods, and pruned DNNs, trained using our RMT approach.*

In all simulations, the networks start with different initial topologies and are trained over a course of 40 epochs. The portion of singular values smaller than $\sqrt{\lambda_+}$ that we retain (see step 3 in algorithm 4) is given by the linear function:

$$f(\text{epoch}) = \max \left(0, -\frac{1}{30} \cdot \text{epoch} + 1 \right). \quad (107)$$

The topologies and the results of the simulations are summarized in Table 1 and Fig. 24. The results indicate a consistent trend across different topologies: the pruned DNNs outperform the normal DNNs in terms of accuracy on the test set while also displaying smaller variance across multiple runs. Furthermore, the pruned DNNs consistently achieve a significant reduction in parameters by the end of the training, see Remark A.3.

Initial Topology	Unpruned DNN Accuracy	Pruned DNN Accuracy
[784, 3000, 3000, 2000, 500, 10]	~85%	~98.5%
[784, 1000, 1000, 1000, 500, 10]	~70%	~98.5%
[784, 2000, 2000, 1000, 500, 10]	~70%	~98.5%
[784, 1500, 3000, 1500, 500, 10]	~70%	~98.5%
[784, 1000, 1000, 1000, 500, 10] (GoF 1)	~70%	>98.5%

Table 5: Performance of normal and pruned DNNs for different initial topologies (on test set).

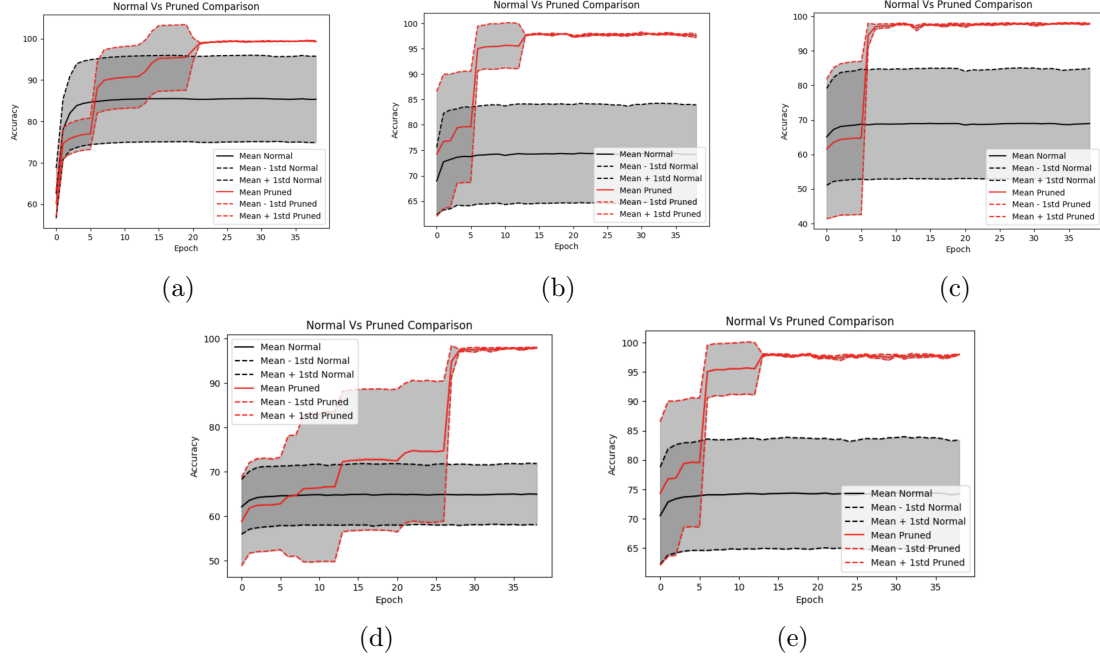


Figure 24: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test set.

Remark A.2. In these examples, the goodness of fit parameter can be very large (even 1) and does not change the accuracy of the DNN. This is not always the case, especially for state-of-the-art pre-trained DNNs, as we will show in future works. See also Subsection A.0.2 for an example of when GoF must be smaller.

Remark A.3. In Table 6, we observe the effect of our RMT training approach on the number of parameters in our DNN with different topologies. Each topology started with a fixed number of parameters, and by the end of training, we see a significant reduction in the number of parameters across all topologies for the pruned DNN. For each topology and across all seeds, the reduction in the number of parameters was consistent, indicating the robustness of our training process in pruning the network while maintaining performance.

Topology	Initial Parameters	Final Parameters	Percentage Reduction
[784, 3000, 3000, 2000, 500, 10]	18,365,510	10,471,510	42.98%
[784, 1000, 1000, 1000, 500, 10]	3,292,510	1,678,234	49.03%
[784, 2000, 2000, 1000, 500, 10]	8,078,510	4,619,376	42.82%
[784, 1500, 3000, 1500, 500, 10]	10,937,510	5,554,966	49.21%

Table 6: DNN topology, initial and final parameters for the pruned DNNs, with percentage reductions.

Simplification of loss landscape for more efficient training As mentioned, a common challenge with DNNs is the complex and high-dimensional loss landscape due to the large number of parameters. This complexity often leads to local minima or saddle points that hinder optimal training. However, by using this clever RMT pruning approach, we effectively eliminate redundant parameters, thereby simplifying the loss landscape. This simplification allows us to avoid suboptimal local minima and converge more readily to a global minimum.

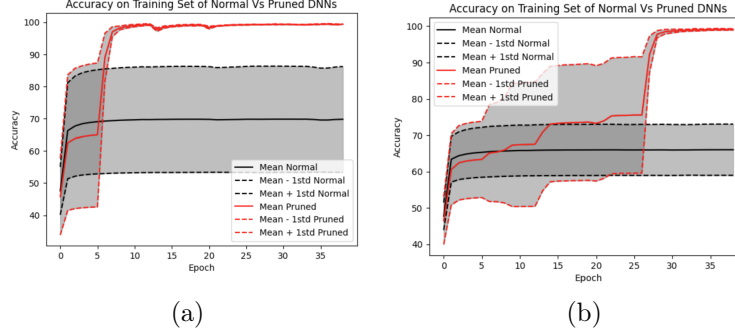


Figure 25: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the training set. The sub-figures correspond to the different initial topologies: (a) [784, 2000, 2000, 1000, 500, 10], (b) [784, 1500, 3000, 1500, 500, 10]. The other examples in Fig. 24 have similar-looking accuracies on their training set.

Topology	Type	Training Loss	Test Loss
[784, 3000, 3000, 2000, 500, 10]	Normal	1.324090	150.541977
	Pruned	0.015084	9.211179
[784, 1000, 1000, 1000, 500, 10]	Normal	0.938041	127.608980
	Pruned	0.001350	14.543617
[784, 2000, 2000, 1000, 500, 10]	Normal	0.567025	70.037265
	Pruned	0.019599	13.676276
[784, 1500, 3000, 1500, 500, 10]	Normal	1.037013	146.353504
	Pruned	0.009206	9.519642

Table 7: Comparison of training and test losses between normal and pruned DNNs for different topologies.

This improved optimization efficiency is evident when comparing the loss and accuracy of the original and pruned DNNs on both training and test sets; see Table 7, Fig. 24, and Fig. 25. In Fig. 24, the sub-figures correspond to the different initial topologies: (a) [784, 3000, 3000, 2000, 500, 10], (b) [784, 1000, 1000, 1000, 500, 10], (c) [784, 2000, 2000, 1000, 500, 10], (d) [784, 1500, 3000, 1500, 500, 10], and (e) [784, 1000, 1000, 1000, 500, 10] with a larger goodness-of-fit parameter of 1. We see that the pruned DNNs achieve lower loss and higher accuracy on the training set directly, indicating that they are finding deeper minima in the loss landscape and avoid suboptimal local minima.

Thus, the RMT pruning approach not only significantly reduces the complexity of DNNs but also enhances their performance by improving their optimization efficiency. Despite the reduction in parameters, the pruned DNNs still exhibit excellent performance on both training and test sets (even higher accuracy than the normally trained DNNs), demonstrating the effectiveness of this approach.

Simplifying the loss landscape for fully connected DNNs on Fashion MNIST In this section, we trained the normal and pruned DNNs on the Fashion MNIST dataset, and we look at the performance of both DNNs on the training and test sets. Again, the pruned DNN obtains higher accuracy and lower loss on both the training and test sets, evidence that pruning the DNN using RMT simplifies the loss landscape and allows the DNN to find a deeper global minimum.

Training hyperparameters:

- Split frequency (every how many epochs we split the modified DNN and remove small singular values): 7

- goodness of fit (GoF) = .7

The other hyperparameters for the simulations in this subsection can be found in Subsection I.6.

In all simulations, the networks start with different initial topologies, are trained over a course of 70 epochs, and the portion of singular values smaller than $\sqrt{\lambda_+}$ that we retain is given by the linear function:

$$f(\text{epoch}) = \max\left(0, -\frac{1}{60} \cdot \text{epoch} + 1\right) \quad (108)$$

Example A.2. The topologies and the results of the simulations are summarized in Table 8 and Fig. 26.

As with MNIST, in the case of training on Fashion MNIST the results indicate a consistent trend across different topologies: the pruned DNNs outperform the normal DNNs in terms of accuracy on the test set while also displaying smaller variance across multiple runs. Furthermore, the pruned DNNs consistently achieve a significant reduction in parameters by the end of the training, see Table 9.

Initial Topology	Unpruned DNN Accuracy	Pruned DNN Accuracy
[784, 2000, 4000, 2000, 500, 10]	~70%	~89%
[784, 2000, 2000, 2000, 2000, 1000, 500, 10]	~65%	~89%
[784, 3000, 4000, 3000, 500, 10]	~70%	~89%

Table 8: Performance of normal and pruned DNNs for different initial topologies.

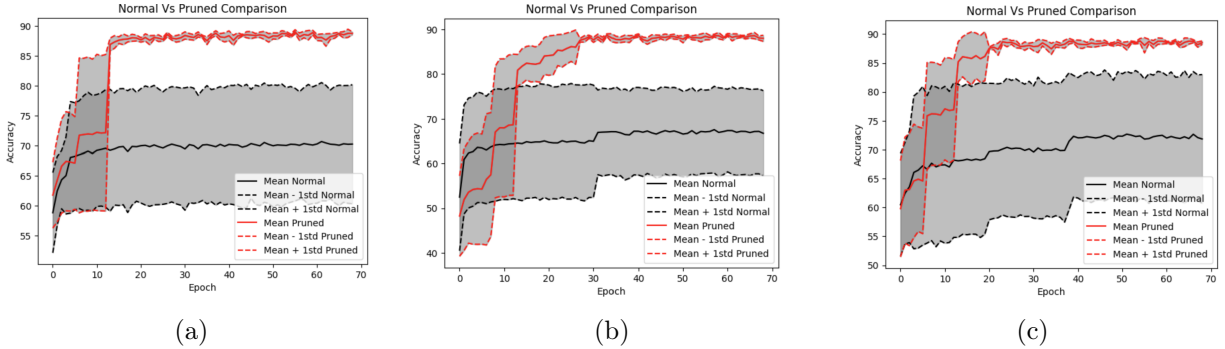


Figure 26: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test set. The sub-figures correspond to the different initial topologies: (a) [784, 2000, 4000, 2000, 500, 10], (b) [784, 2000, 2000, 2000, 2000, 1000, 500, 10], (c) [784, 3000, 4000, 3000, 500, 10].

In Table 9, we observe the effect of our RMT training approach on the number of parameters in our DNN with different topologies. Each topology started with a fixed number of parameters, and by the end of training, we see a significant reduction in the number of parameters across all topologies for the pruned DNN. For each topology and across all seeds, the reduction in the number of parameters was consistent, indicating the robustness of our training process in pruning the network while maintaining performance.

Remark A.4. Again, we see that the RMT approach helps simplify the loss landscape so that during gradient descent, the pruned DNN finds a deeper global minimum than the normal DNN. We can see this by looking at the accuracy of the DNNs on the training set; see Fig. 27.

Topology	Initial Parameters	Final Parameters	% Reduction
[784, 2000, 4000, 2000, 500, 10]	18,581,510	10,471,510	43.65%
[784, 2000, 2000, 2000, 2000, 1000, 500, 10]	16,082,510	8,950,860	44.34%
[784, 3000, 4000, 3000, 500, 10]	27,867,510	15,599,740	44.02%

Table 9: DNN topology, initial and final parameters for the pruned DNNs, with percentage reductions.

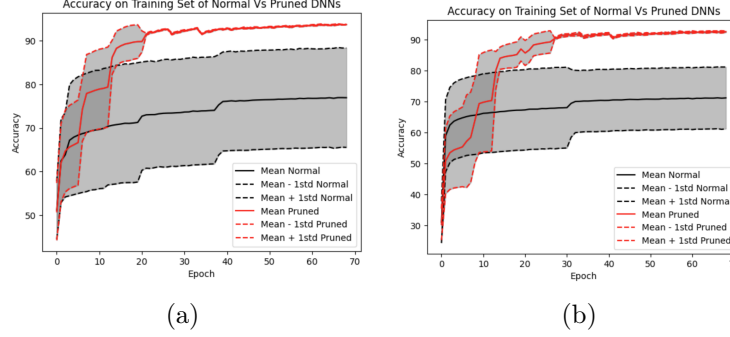


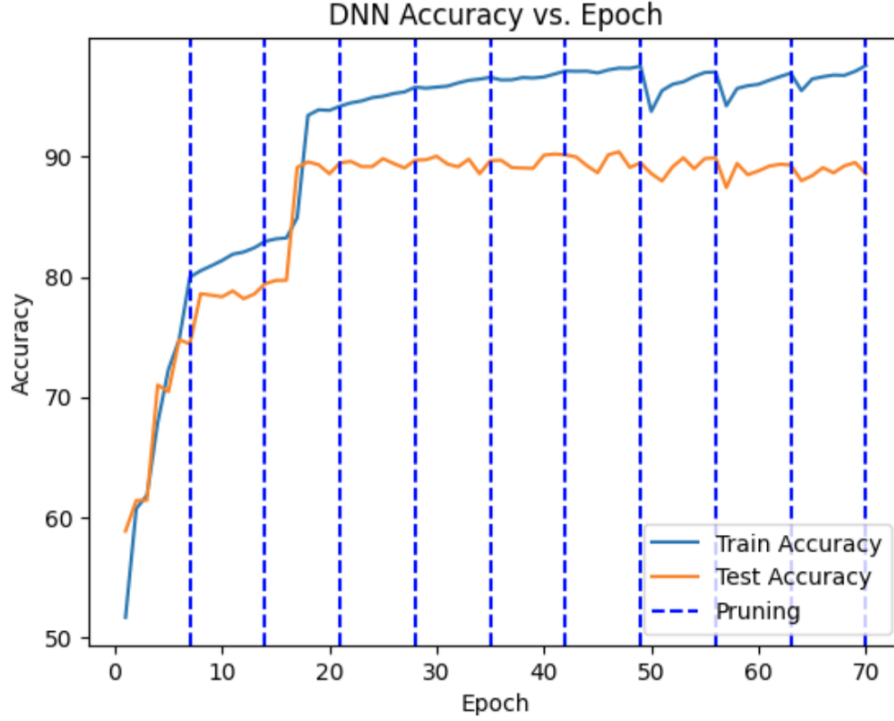
Figure 27: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the training set. The sub-figures correspond to the different initial topologies: (a) [784, 3000, 4000, 3000, 500, 10], (b) [784, 2000, 2000, 2000, 2000, 1000, 500, 10].

Topology	Type	Training Loss	Test Loss
[784, 2000, 4000, 2000, 500, 10]	Normal	0.460549	95.516107
	Pruned	0.144334	47.063285
[784, 3000, 4000, 3000, 500, 10]	Normal	0.880157	130.785274
	Pruned	0.191681	49.781472
[784, 2000, 2000, 2000, 2000, 1000, 500, 10]	Normal	0.790270	127.023410
	Pruned	0.270197	42.925396

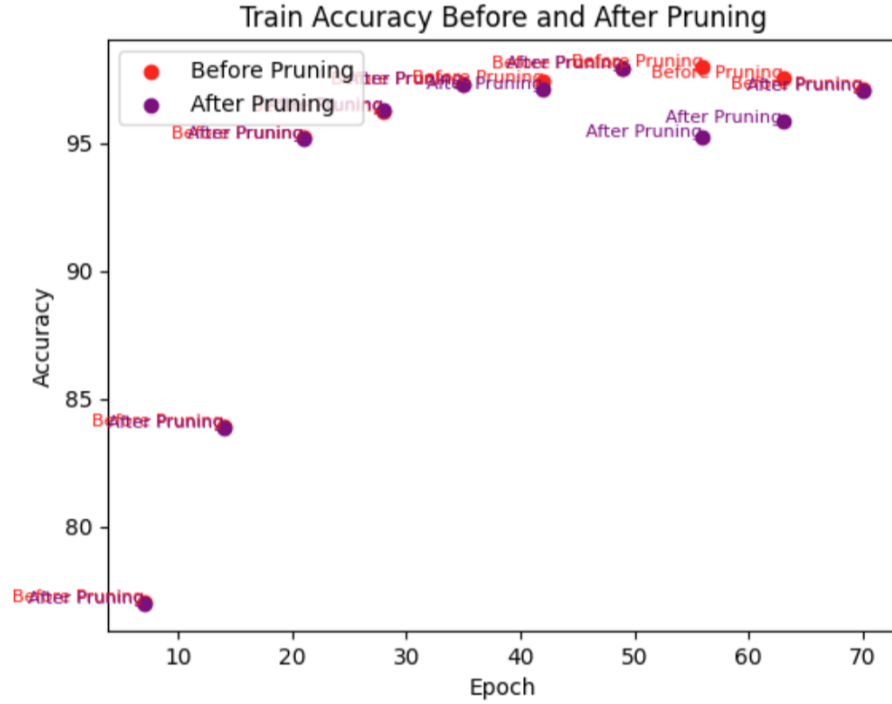
Table 10: Comparison of training and test losses between normal and modified DNNs for different topologies.

One can also see that the pruned DNN is obtaining a deeper global minimum by looking at Table 10.

We present two graphs to analyze the impact of pruning on the performance of the DNN with architecture [784, 2000, 4000, 2000, 500, 10], see Fig. 28. Figure 28a shows the training and testing accuracy of the DNN over epochs. The blue dashed lines indicate the points at which pruning was applied. It is observed that the accuracy does not change significantly after pruning, especially during the initial epochs when the DNN parameters are still random. Figure 28b specifically examines the impact of pruning on training accuracy. Red dots represent the accuracy before pruning, and purple dots represent the accuracy after pruning. The graph demonstrates that the training accuracy remains relatively stable before and after pruning, reinforcing the observation that MP-based pruning does not drastically affect performance, particularly in the early stages of training. After the pruning, the training seems to be easier, and the DNN accuracy improves as the training continues.



(a) Accuracy vs. Epoch



(b) Train Accuracy Before and After Pruning

Figure 28: Analysis of DNN Training and Pruning

We applied the MP-based pruning approach for DNNs with other initializations, such as the He and Xavier initializations, and obtained similar results in improvements of accuracy, see Subsection I.2. Also, for the DNN with architecture [784, 2000, 4000, 2000, 500, 10], we computed the cost of training

the DNN for 70 epochs on the Intel Xeon CPU with 2 vCPUs (virtual CPUs) and 13GB of RAM, with MP-based pruning and a split frequency of 7. The total cost was 12,215.13 seconds, and we achieved an accuracy of $\sim 89.7\%$. Finally, we trained the same DNN for the same amount of CPU time but without MP-based pruning, and the DNN accuracy plateaued at $\sim 84\%$ accuracy. This illustrates that the increase in accuracy provided by MP-based pruning is not obtained because of increases in computational costs alone.

We applied the MP-based pruning approach on a DNN without adding any regularization while training with GD alone and with a fixed $lr = .01$. The DNN architecture was [784, 3000, 3000, 3000, 3000, 500, 10] and we trained for 70 epochs with a split frequency of 7. Without pruning, the accuracy plateaued at 44%, while with pruning, it reached $\sim 80\%$. This indicates that other strategies, such as regularization or rate decay, are useful to take advantage of MP-based pruning. However, MP-based pruning improves the accuracy of GD alone.

We trained a DNN with architecture [784, 3000, 3000, 3000, 500, 10] on Fashion MNIST for 300 epoch, while adding both $L1$ and $L2$ regularization to the loss, see (214). The hyperparameters for the regularization were .0000005 and .0000001, respectively; the split frequency was 13 and the number of singular values kept was given by $f(\text{epoch}) = \max(0, -\frac{1}{1000} \cdot \text{epoch} + 1)$. All other hyperparameters were kept the same as in Example 26. The DNN achieved a 100% accuracy on the training set and a 91.27% accuracy on the test set, showing that the MP-based pruning algorithm attains higher accuracy when we combine it with regularization (see Corollary 3.9 for a possible explanation).

MP-based pruning with sparsification for fully connected DNNs on Fashion MNIST We train a fully connected DNN on Fashion MNIST to achieve $\sim 89\%$ accuracy (on the test set) with the same MP-based pruning approach as in Subsection A.0.1 for a DNN with the topology [784, 3000, 3000, 3000, 3000, 500, 10]. At the end of the training, we employ the sparsification method by setting to zero weights in the DNN smaller than the *sparsification threshold* ξ . Fig. 29 shows the accuracy of the DNN vs. the number of parameters kept (determined by varying ξ). This additional sparsification leads to a large reduction in parameters, by over 99.5%, without a significant drop in accuracy ($\sim .5\%$ drop). Assuming that the weights of the DNN which are smaller than the threshold ξ are i.i.d.s. from a distribution with zero mean and bounded variance, Lemma 3.13 provides an explanation for why removing the small weights (sparsification) does not affect accuracy.

Alternatively, one can prune the weight layers during training by combining the MP-based pruning approach together with sparsification (i.e., removing weights smaller than the threshold ξ every couple of epochs). We performed this training on the above DNN, with ξ depending on the epoch. In our case, we initially took $\xi = 0.001$ and set it to grow linearly so by the end of training $\xi = 0.02$. We achieved a 88% accuracy, while the final DNN had 71,331 parameters (keeping $\sim .2\%$ parameters).

Finally, we tried the sparsification pruning method during training without MP-based pruning. Again, we initially took $\xi = 0.001$ and set it to grow linearly so that by the end of training $\xi = 0.02$. Similar to our observations from subsection A.0.1, the DNN plateaued at $\sim 70\%$ accuracy while having 128,533 parameters at the end of training (keeping $\sim .4\%$ parameters). Thus, we see that a combination of MP-based pruning with sparsification is useful for pruning while also increasing DNN accuracy for fully connected DNNs trained on Fashion MNIST.

A.0.2 MP-based pruning of CNNs on MNIST and Fashion MNIST

In our further exploration, we perform numerical simulations on Convolutional Neural Networks (CNNs) using MNIST and Fashion MNIST. In this simulation, our primary objective is to investigate the effect of pruning the small singular values of the convolutional layers. The overall goal in this example is to reduce the number of parameters in the CNN while at the same time preserving its accuracy.

Given the multidimensional nature of convolutional layers, the direct application of singular value

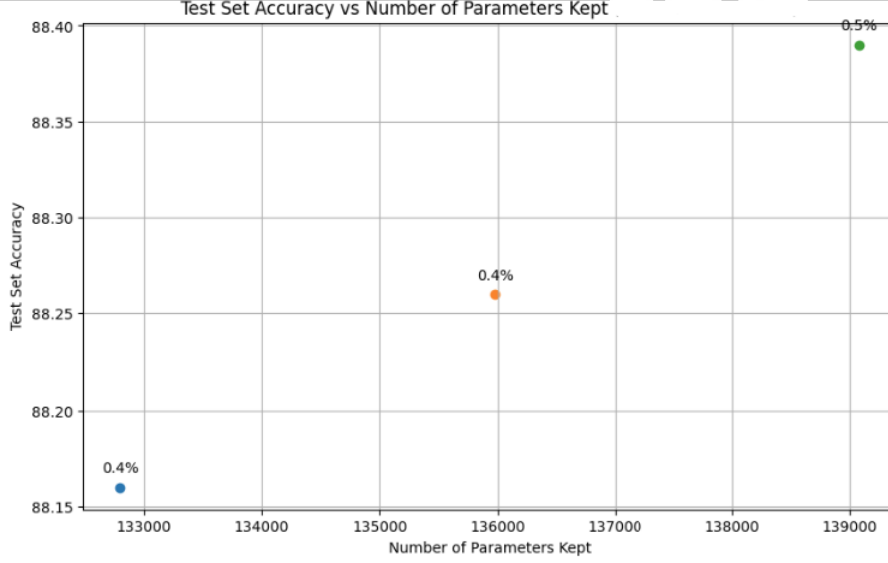


Figure 29: Accuracy vs. number of parameters kept. The percentage of parameters kept is also shown (above each point on the graph).

decomposition is not straightforward. To overcome this challenge, we first transform each convolutional layer into a 2-dimensional matrix. Specifically, for a convolutional layer with dimensions $m \times n \times p \times q$ (where m is the number of output channels, n is the number of input channels, and $p \times q$ is the kernel size), we reshape it into a matrix of size $m \times npq$.

After this flattening process, we proceed with the pruning as before, employing SVD to remove the smaller singular values. This step essentially compresses the convolutional layer, reducing its complexity while hopefully maintaining its representational capability.

The hyperparameters for the simulations in the next example can be found in Subsection I.7. The other parts of the CNN architecture (which is the same for all of the CNNs in this paper) can be found in Subsection I.8.

The learning rate (lr) is also modified every epoch to be:

$$lr_n = lr_{n-1} * .96 \quad (109)$$

where lr_k is the learning rate at epoch k . Thus it decays over the learning time, see [24] for more information.

Example A.3. *In this first example, we trained a CNN on MNIST for 30 epochs with a split frequency of 13. The convolutional layers are given by [1, 64, 128, 256, 512], and the model starts with one input channel, and then each subsequent number represents the number of filters in each subsequent convolutional layer. Therefore, the model has 4 convolutional layers with filter sizes of 64, 128, 256, and 512, respectively. We apply a kernel for each layer of size 3×3 .*

The fully connected layers are given by [41472, 20000, 10000, 5000, 3000, 1400, 10], we see that the model has 6 fully connected layers. The GoF parameter for the fully connected layers is .6 while the GoF parameter for the convolutional layers is .05.

In these numerical simulations, the portion of singular values smaller than $\sqrt{\lambda_+}$ that we retain is given by the linear function:

$$f(\text{epoch}) = \max \left(0, -\frac{1}{20} \cdot \text{epoch} + 1 \right) \quad (110)$$

The accuracy of this DNN on the training and test sets is given in Fig. 30.

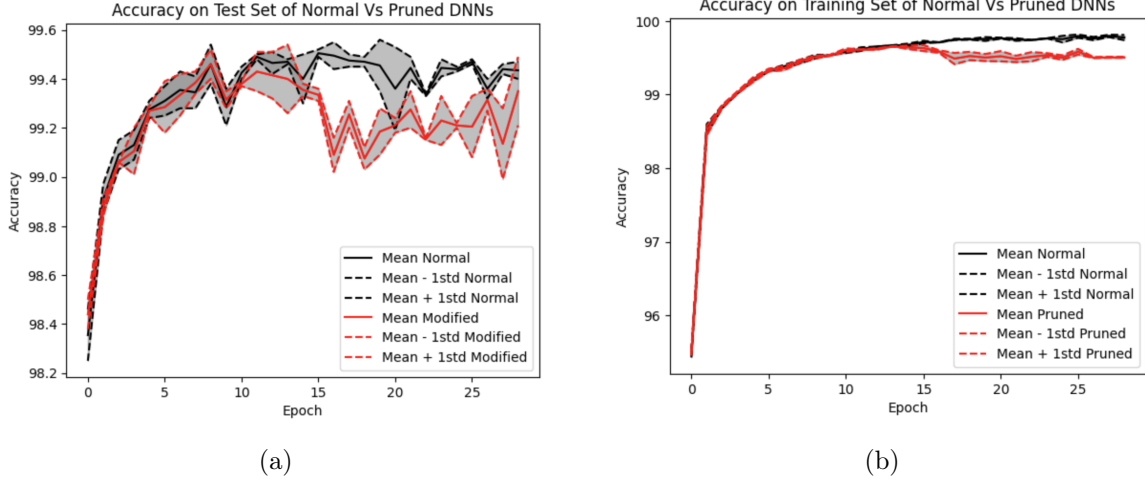


Figure 30: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach, on the test and training sets.

In Example A.3, it was observed that the pruned CNN exhibited a slightly lower accuracy in comparison to the normally trained CNN. Remarkably, despite this marginal drop in performance, the pruned CNN managed to maintain this level of accuracy with approximately half of the parameters used by the normally trained CNN. While the normally trained CNN has 1,100,323,974 parameters (on account of how large the fully connected layers are), the pruned CNN has 583,670,449 parameters.

In terms of performance on the training set, the normally trained CNN demonstrated an accuracy of 100%, an indicator of its potential overfitting to the training data. This is in contrast with the pruned CNN, which displayed a lower accuracy on the training set. The narrower gap between the training set and test set accuracies for the pruned CNN could be interpreted as a sign of reduced variance between the training and test set, suggesting less overfitting in the pruned model. At the same time, the fact that the normal CNN archives have high accuracy on the training set suggests that the loss function in this example is simple- i.e., finding the global max of the loss function is simple. This might be why the pruned CNN does not outperform the unpruned CNN.

In this example, the drop in test set accuracy for the pruned CNN is dependent on the Goodness-of-Fit (GoF) parameter. As the GoF parameter becomes more restrictive, the drop in accuracy becomes less pronounced. However, it is important to note that a more restrictive GoF parameter also leads to a smaller reduction in parameters. These observations suggest a delicate balance between the GoF parameter, model complexity (as indicated by the number of parameters), and model performance.

Example A.4. *In this example, we trained a CNN on the fashion MNIST dataset for 70 epochs with a split frequency of 17. The convolutional layers are given by [1, 64, 128, 256, 512]; we again apply a kernel for each layer of size 3×3 . The fully connected layers are given by [41472, 10000, 5000, 5000, 10]. The GoF parameter for the fully connected layers is .7, while the GoF parameter for the convolutional layers is .15.*

In this numerical simulation, the portion of singular values smaller than $\sqrt{\lambda_+}$ that we retain is given by the linear function:

$$f(\text{epoch}) = \max\left(0, -\frac{1}{60} \cdot \text{epoch} + 1\right) \quad (111)$$

The accuracy of this DNN on the training and test set is given in Fig. 31.

In Example A.4, the pruned CNN exhibited a slightly lower accuracy in comparison to the conventionally trained CNN. Despite this marginal drop in performance, the pruned CNN maintained

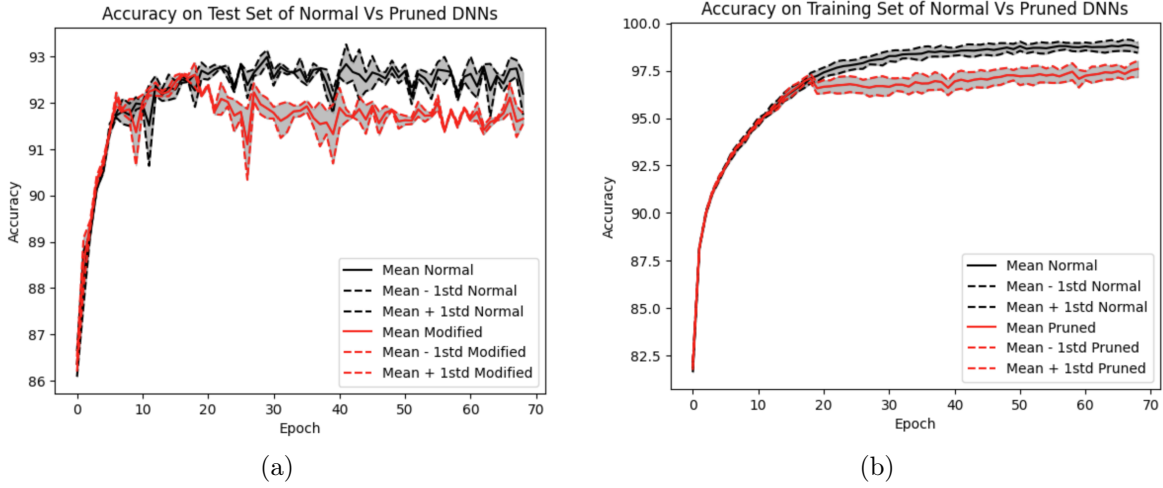
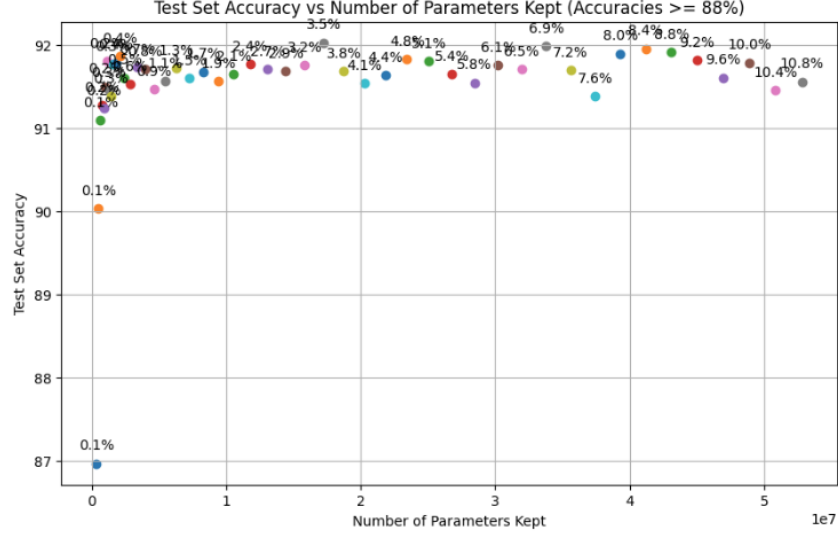


Figure 31: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the Fashion MNIST test and training sets.

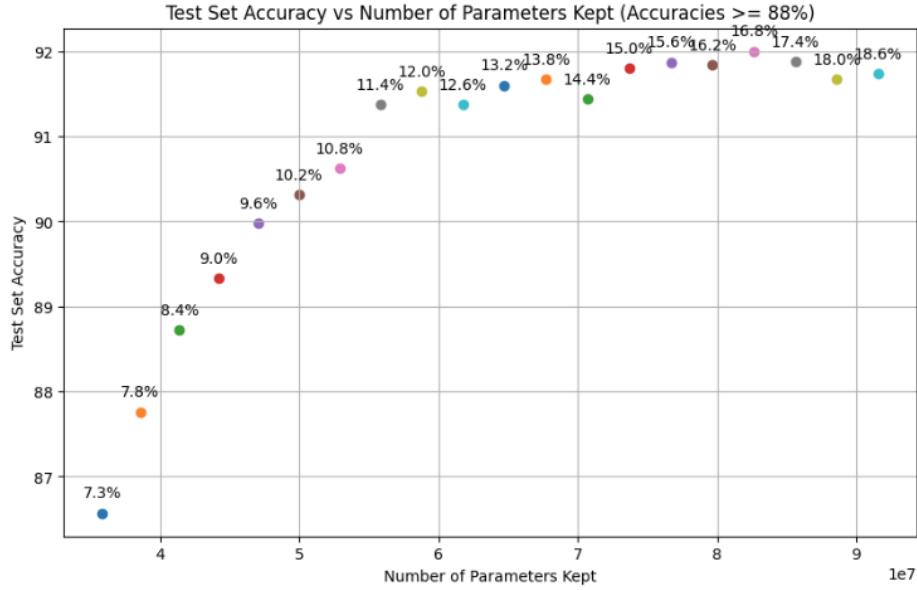
this level of accuracy with approximately half of the parameters used by the conventionally trained CNN. While the normally trained CNN had 491,381,774 parameters, the pruned CNN utilized only 261,891,332 parameters.

In terms of performance on the training set, the conventionally trained DNN demonstrated an accuracy of approximately 99%, suggesting potential overfitting to the training data. On the other hand, the pruned DNN displayed a lower accuracy on the training set. The smaller variance between the training and test set accuracies for the pruned DNN could again be interpreted as a sign of less overfitting.

MP-based pruning with sparsification for CNN trained on Fashion MNIST We train the CNN found in Example A.4 to achieve $\sim 92\%$ accuracy (on the Fashion MNIST test set) with the same MP-based pruning approach as in Subsection A.0.2. At the end of the training, we employ sparsification by setting to zero all weights in the DNN smaller than some threshold ξ .



(a) Sparsification with MP-based pruning. The percentage of parameters kept is shown above each point on the graph.



(b) Sparsification without MP-based pruning. The percentage of parameters kept is shown above each point on the graph.

Figure 32: Accuracy vs. number of parameters kept for CNNs trained on Fashion MNIST with and without MP-based pruning.

As shown in Fig. 32a, this additional sparsification leads to a large reduction in parameters, by over 99.5%, without a significant drop in accuracy ($\sim .1\%$ drop). As mentioned, Lemma 3.7 provides an explanation for why removing the small weights does not affect accuracy, as these weights appear to correspond to the noise in the weight layers, and removing them should not change accuracy.

Finally, we applied the sparsification pruning method after training without MP-based pruning (during training). Fig. 32b shows that the pruning threshold seems to affect the accuracy of the DNN in a much more significant manner. That is, even when pruning 95% of the parameters, the accuracy drops by multiple percentage points. We see that a combination of MP-based pruning with sparsification is useful for pruning while also ensuring the DNN accuracy does not decrease much for CNNs trained on Fashion MNIST.

A.0.3 Numerics for training DNNs on CIFAR-10: reducing parameters via MP-based pruning

In this numerical simulation, we applied the RMT algorithm to prune a DNN trained on the CIFAR-10 dataset. The CIFAR-10 dataset consists of 60,000 color images spanning 10 different classes. The dataset is split into a training set and a test set. The training set contains 50,000 images, while the test set comprises 10,000 images, which is standard.

Throughout the training process, we tracked the performance metrics of both the pruned and normally trained DNNs on both the test and training sets. Our analysis showed that the pruned network, despite having a reduced number of parameters, managed to achieve performance metrics comparable to those of the normally trained network. Additionally, the pruning process significantly reduced the number of parameters in the pruned DNN, resulting in a more efficient network with a lower computational footprint. The hyperparameters for the simulations in this subsection can be found in Subsection I.9.

The lr is also modified every epoch to be:

$$lr_n = lr_{n-1} * .96 \quad (112)$$

where lr_k is the learning rate at epoch k .

Example A.5. *In this example, we trained a CNN on CIFAR10 for 350 epochs with a split frequency of 40. The convolutional layers are given by $[3, 32, 64, 128, 256, 512]$. We again apply a kernel for each layer of size 3×3 .*

The fully connected layers are given by $[8192, 500, 10]$. The GoF parameter for the fully connected layers is .08, while the GoF parameter for the convolutional layers is .06.

In this simulation, the portion of singular values smaller than λ_+ that we retain is given by the linear function:

$$f(epoch) = \max\left(0, -\frac{1}{200} \cdot epoch + 1\right) \quad (113)$$

The accuracy of this DNN on the training and test set is given in Fig. 33.

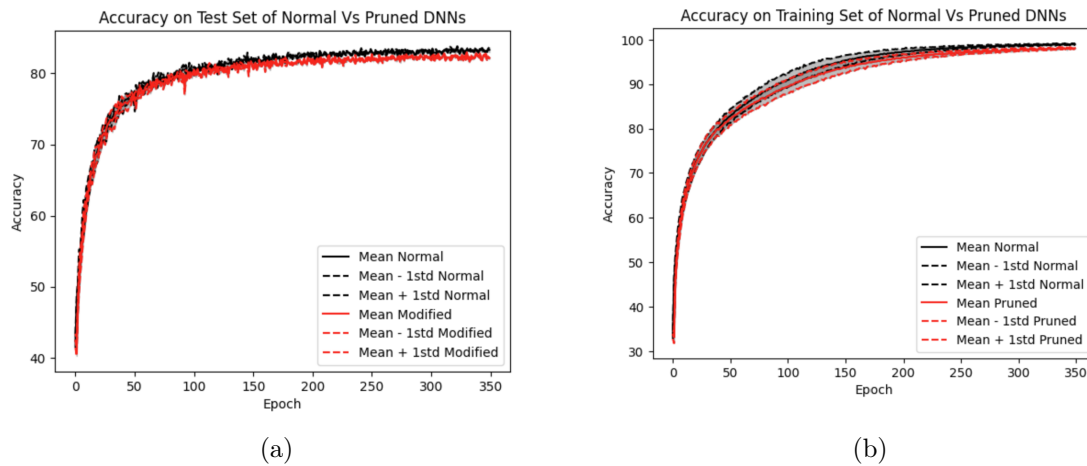


Figure 33: Comparison of Normal DNN, trained normally, and pruned DNN, trained using the RMT approach on the test and training sets.

In Example A.0.3, it was observed that the pruned DNN exhibited a slightly lower accuracy in comparison to the normally trained DNN. Again, despite this marginal drop in performance, the pruned DNN managed to maintain this level of accuracy with much fewer parameters than was used by the normally trained DNN. While the normally trained DNN has 5,673,090 parameters, the pruned DNN has 3,949,078 parameters.

MP-based pruning with sparsification for CNN trained on CIFAR10 We train the CNN found in Example A.5 on CIFAR10 to achieve $\sim 82\%$ accuracy (on the test set) with the same MP-based pruning approach as in Subsection A.0.3. At the end of training, we sparsify the DNN by setting to zero weights in the DNN smaller than some threshold ξ .

As shown in Fig. 34a, this additional sparsification leads to a large reduction in parameters, by over 97%, without a significant drop in accuracy (~ 0 drop). Finally, we tried the sparsification pruning method after training without MP-based pruning (during training). Fig. 34b shows that the threshold pruning seems to affect the accuracy of the DNN in a much more significant manner. That is, even when pruning 80% of the parameters, the accuracy drops by $\sim 2\%$. We see that a combination of MP-based pruning with sparsification is useful for pruning while also ensuring the DNN accuracy does not decrease much for CNNs trained on CIFAR-10.

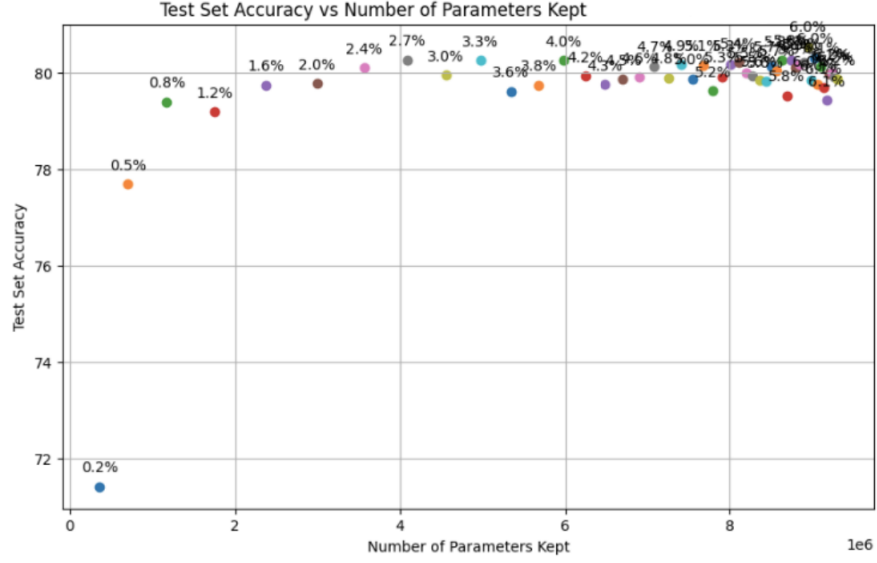
B A simple illustration of the relationship between RMT-based pruning and regularization

The core concept underlying the developed pruning method relies on the observation that Neural Network weight matrices frequently exhibit characteristics akin to those of a low-rank matrix addition to a random matrix component. The idea of the pruning method developed relies on the observation that oftentimes, the weight matrices of Neural Networks behave like a very low-rank matrix with an added random matrix. The origin of the randomness is two-fold. Firstly, Neural Networks undergo training through various iterations of stochastic gradient descent. Consequently, the resultant trained network serves as an approximate minimizer of the associated loss function, with its weight distribution potentially reflecting the inherent stochastic nature of the optimization algorithm employed. Secondly, the sample data used to train the network introduces randomness to the training process, particularly when the dataset incorporates random noise. The act of pruning can be conceptualized as a regularization technique operating across the spectrum aimed at promoting the recovery of this low-rank matrix structure.

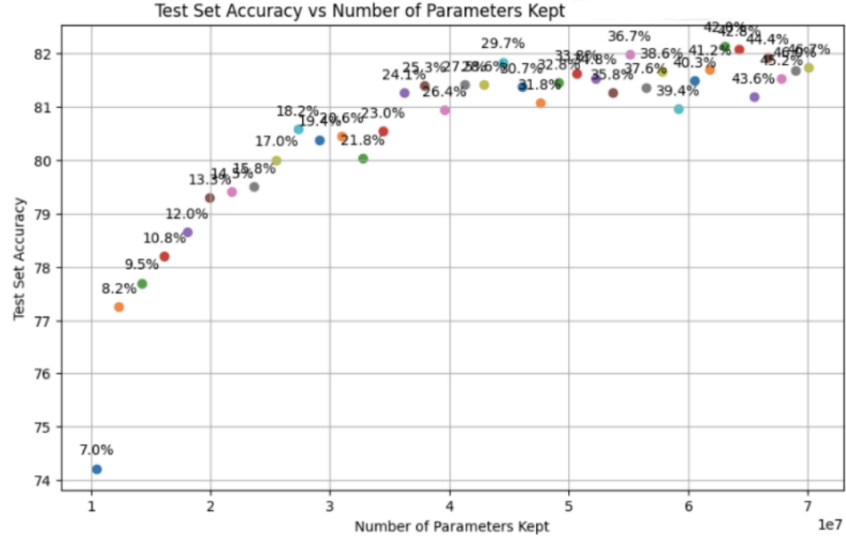
In order to better understand the implications of pruning, we present a simple problem designed to illuminate the effect outcomes of pruning in comparison to alternative forms of regularization.

B.1 Regularization in a simple regression setting

Having a low-rank weight matrix implies that the DNN has found a feature map representation of the task that is lower dimensional than the set dimension of the weight matrix. In order to emulate this situation, we introduce a simple regression model where an optimal representation is known. We sample M random function $y^{(k)} \in \text{Span}\{\cos(\pi f \cdot), \sin(\pi f \cdot), \text{ for } f \in \{0, 0.2, \dots, 2\}\}$, and take random, noisy samples $(X_j, Y_j) \in \mathbb{R} \times \mathbb{R}^M$ s.t. $(Y_j)_k = y^{(k)}(X_j) + 7\epsilon_j$, with $\epsilon_j \sim \mathcal{N}(0, 1)$. Our objective entails the recovery of the $y^{(k)}$ functions from this dataset. An illustrative example of such data along one coordinate is provided in Figure 35. Because of the structure of this problem, an efficient feature map representation of the data is through the Fourier transform, as y belongs to a vector space of low frequency. It has been argued that Neural Networks can be interpreted as a kernel with a learned feature map based on the data. For the sake of simplicity in this context, we assume a fixed feature



(a) Sparsification with MP-based pruning. The percentage of parameters kept is shown above each point on the graph.



(b) Sparsification without MP-based pruning. The percentage of parameters kept is shown above each point on the graph.

Figure 34: Accuracy vs. number of parameters kept for CNNs trained on CIFAR-10 with and without MP-based pruning.

map, the Fourier feature map. Accordingly, the Neural Network materializes as a linear layer on top of the selected feature map, equivalent to the kernel regression defined by the said feature map.

In the following paragraphs, we will study the impact of different regularizations, including pruning, on the singular values of the linear layer's weight matrix. Given the significant variations in these singular values on a logarithmic scale, we visualize the cumulative distribution of singular values rather than conventional histograms.

Ideal case In the absence of noise, the optimal weight matrix has a low rank, as y belongs to a functional space with a finite number of distinct frequencies. Using kernel regression theory, we

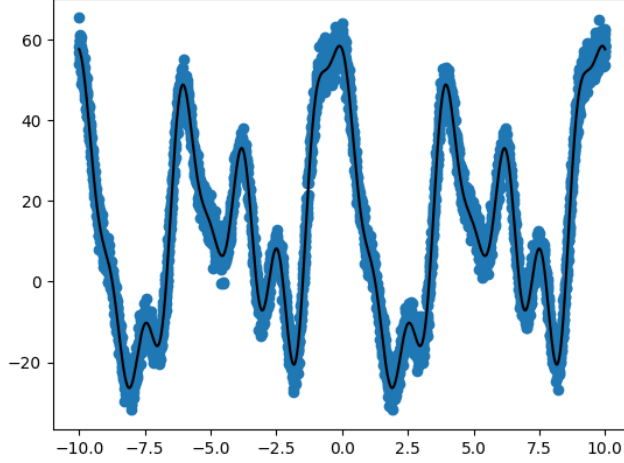


Figure 35: Example of data generated from the problem statement on one axis. The blue line is the function y_0 , and the blue dots are the data points.

find the optimal weight matrix and visualize its singular values in figure 36. Most singular values, barring 16, approach zero (numerically below 10^{-15}). This confirms that the optimal weight matrix is inherently low-rank in the absence of noise. Subsequently, our regression task pertains to recovering this low-dimensional structure when confronted with noisy data.

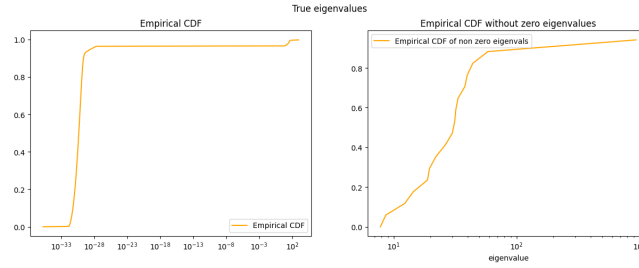


Figure 36: Cumulative distribution of singular values of the optimal weight matrix

We will now try the pruning regularisation and compare it to two classical regularisations, L_2 and L_1 regularisation.

L_2 regularisation in the Presence of Noise Employing Kernel Ridge Regression, we obtain a L_2 regularised weight matrix. We plot the singular values of this matrix in figure 37. We can see a large number of minimal singular values. By running the BEMA algorithm on this matrix, we identify a spectrum resembling the Marchenko-Pastur distribution. Moreover, the algorithm finds an upper bound for random singular values consistent with the ideal spectrum we observed above. This alignment with the Marchenko-Pastur distribution implies that the predominant source of randomness in our weight matrices is data noise rather than the stochastic nature of the optimization algorithm. Indeed, the weight matrix obtained here is deterministic when conditioned on the data, as the loss admits a minimizer with a closed-form solution.

L_1 regularisation in the Presence of Noise Turning to L_1 regularization, using kernelized Lasso Regression, we plot the singular values of the resulting matrix in Figure 38. Notably, 40% of the singular values have been zeroed out compared to the L_2 case. However, the remaining spectrum approximates

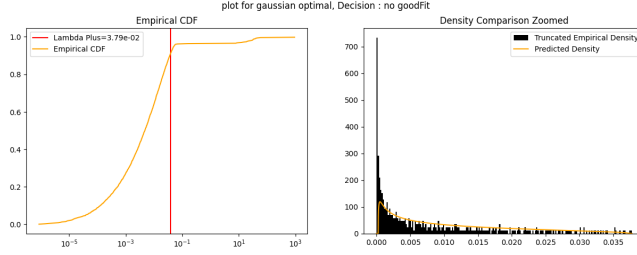


Figure 37: Cumulative distribution of singular values of the L_2 regularised weight matrix

an MP distribution, underscoring that the noise-induced randomness persists to some degree. It must be pointed out that L_1 regularisation tends to nullify coefficients of the weight matrix instead of singular values. In this scenario, our selected ideal feature map endows the optimal weight matrix with sparsity, thus rendering the recovery of a sparse weight matrix equivalent to the recovery of a low-rank one. It warrants emphasis, however, that with an alternate feature map, L_1 regularization might prove less effective in revealing the weight matrix’s low-rank structure and could even significantly diminish performance if weighed too heavily.

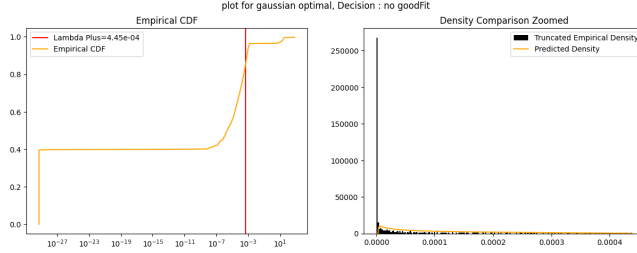


Figure 38: Cumulative distribution of singular values of the L_1 regularised weight matrix

Pruning in the Presence of Noise We visualize the singular values of the unregularised matrix in Figure 39. We see that this spectrum is very similar to the spectrum of the L_2 regularised weight matrix. This confirms the intuition that L_2 is not an efficient spectrum regularisation. Nonetheless, the BEMA algorithm successfully identifies a precise upper bound for the matrix’s random singular values. This shows the capacity of pruning regularization to eliminate the noise-driven randomness from the weight matrix.

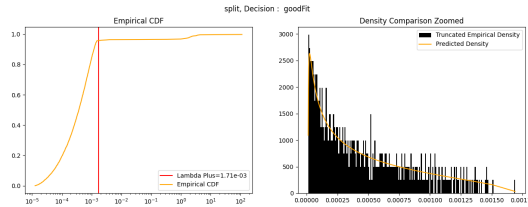


Figure 39: Cumulative distribution of singular values of the weight matrix prior to pruning

In this simple illustrative instance, pruning regularization emerges as an effective tool for sparsifying the weight matrix’s spectrum, thereby facilitating the recuperation of the underlying low-rank structure intrinsic to the problem at hand. Table 11 reports the effects of the different regularizations on the mean squared error. We see that successfully removing the random part of the weight matrix spectrum allows better accuracy.

Model	Mean Squared Error
No Regularization	2.5735
L2 Regularization	2.5627
L1 Regularization	0.4157
Pruning	0.1491

Table 11: Mean Squared Errors of Different Models for the simple regression task

B.2 Proof of Lemmas 3.10 and 3.13.

In this proof, $t \in \mathbb{R}$ does not refer to training time. We use this variable t because classically the Borell-TIS is given in terms of t .

We want to bound the probability of a single component of the output of the DNN when a random matrix R is included. Specifically, we aim to bound the probability that the i -th component of the following expression exceeds t :

Proof. Step 1: Apply the Triangle Inequality

By the triangle inequality, we can remove the activation functions (absolute value) and simplify the terms inside the norm:

$$Z \leq \mathbb{P}(|W_3 \circ \lambda \circ (R + S) \circ \lambda \circ W_1 s - W_3 \circ \lambda \circ S \circ \lambda \circ W_1 s|_i > t) \quad (114)$$

Step 2: Factor Out W_3

We can now factor out W_3 , using the inequality $|(Av)_i| \leq \|A\|_\infty \|v\|_\infty$, to move W_3 outside of the norm:

$$Z \leq \mathbb{P}(\|W_3\|_\infty \cdot \|(R + S) \circ \lambda \circ W_1 s - S \circ \lambda \circ W_1 s\|_\infty > t) \quad (115)$$

Step 3: Focus on R

We now separate the terms involving R and S , leaving us with:

$$Z \leq \mathbb{P}(\|W_3\|_\infty \cdot \|(R \circ \lambda \circ W_1 s)\|_\infty > t) \quad (116)$$

Step 4: Focus on the Random Matrix R Let $Y = R \circ \lambda \circ W_1 s$. Each element of Y is a weighted sum of independent Gaussian random variables:

$$Y_i = \sum_j R_{ij}(\lambda \circ W_1 s)_j$$

The variance of each Y_i is:

$$\text{Var}(Y_i) = \frac{1}{N} \sum_j (\lambda \circ W_1 s)_j^2 = \frac{\|W_1 s\|_2^2}{N}$$

Step 5: Apply Borell-TIS Inequality We can now apply the Borell-TIS inequality to Y . Note that each Y_i is a Gaussian random variable with mean 0 and variance $\frac{\|W_1 s\|_2^2}{N}$. Let $s_Y^2 = \frac{\|W_1 s\|_2^2}{N}$.

From the Borell-TIS inequality, for any $t > 0$:

$$\mathbb{P}\left(\|Y\|_\infty > \sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 + k\right) \leq 2 \exp\left(-\frac{Nk^2}{2\|W_1 s\|_2^2}\right)$$

Step 6: Incorporate Arbitrary Matrix W_3

Next, incorporate the effect of the arbitrary matrix W_3 . Using the properties of matrix norms, we have:

$$\|W_3 Y\|_i \leq \|W_3\|_\infty \|Y\|_\infty$$

Therefore, the probability bound becomes:

$$Z \leq \mathbb{P} \left(\|W_3\|_\infty \|Y\|_\infty > \left(\sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 + k \right) \|W_3\|_\infty \right) \quad (117)$$

$$\leq 2 \exp \left(-\frac{N k^2}{2 \|W_1 s\|_2^2} \right), \quad (118)$$

with $t = \left(\sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 + k \right) \|W_3\|_\infty$.

Step 7: Final Bound

Substituting $k = \frac{\|W_1 s\|_2}{N^{1.5/4}}$ results in:

$$Z \leq 2 \exp \left(-\frac{N^{1-1.5/2}}{2} \right) = 2 \exp \left(-\frac{N^{1/4}}{2} \right)$$

Conclusion:

$$Z = \mathbb{P} \left(|\lambda \circ W_3 \circ \lambda \circ (R + S) \circ \lambda \circ W_1 s - \lambda \circ W_3 \circ \lambda \circ S \circ \lambda \circ W_1 s|_i > \right. \quad (119)$$

$$\left. \sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 \|W_3\|_\infty + \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{1.5/4}} \right) \leq 2 \exp \left(-\frac{N^{1/4}}{2} \right) \quad (120)$$

This bounds the change in one component of the output of the DNN, completing the proof of Lemma 3.10. Then, to prove Lemma 3.13 one need to compute the total possible change of the classification. The classification confidence $\delta X(s, \alpha_{W_2})$ is given in terms of two components of the output of the DNN, the components $i(s)$ and the max of all other components, which we denote k^* . Thus for

$$\Delta(\delta X) := |\delta X(s, \alpha_{S_2}) - \delta X(s, \alpha_{W_2})|, \quad (121)$$

we have that $\Delta(\delta X)$ is the absolute change of the components $i(s)$ and k^* , and so:

$$\mathbb{P} \left(\Delta(\delta X) > 2 \left(\sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 \|W_3\|_\infty + \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{1.5/4}} \right) \right) \leq 2 \exp \left(-\frac{N^{1/4}}{2} \right). \quad (122)$$

For a DNN with more than three-layer matrices, the proof would be the same. For example, in the case of four-layer matrices, we have:

Step 1: Express the Problem We start with the probability we want to bound, now with four weight layers:

$$Z = \mathbb{P} (|\lambda \circ W_4 \circ \lambda \circ W_3 \circ \lambda \circ (S + R) \circ \lambda \circ W_1 s - \lambda \circ W_4 \circ \lambda \circ W_3 \circ \lambda \circ S \circ \lambda \circ W_1 s|_i > t) \quad (123)$$

Step 2: Use Matrix Norm Properties We can bound this using the ℓ_∞ norm and properties of matrix norms:

$$Z \leq \mathbb{P} (\|W_4\|_\infty \cdot \|W_3\|_\infty \cdot \|R \circ \lambda \circ W_1 s\|_\infty > t) \quad (124)$$

The rest of the proof then proceeds as above. $\square$

For a more general DNN, we first restate the relevant assumptions:

Assumption 4:

Consider a DNN denoted by ϕ , and assume that it can be written as:

$$\phi = \rho \circ \psi_2 \circ (R + S) \circ \psi_1, \quad (125)$$

where ψ_1 and ψ_2 are arbitrary functions and ρ is softmax. Take $W = R + S$ and assume that the matrices R and S satisfying the following assumptions:

Assumptions on the matrices R and S

We considered a class of admissible matrices W , where $W = R + S$ and W , R and S satisfy the following three assumptions. The first assumption is a condition on R :

Assumption 5: Assume R is a $N \times M$ matrix such that for any vector v we have:

$$\mathbb{P}(\|Rv\|_\infty > d_1(N)J(v)) \leq d_2(N), \quad (126)$$

with $d_1(N), d_2(N) \rightarrow 0$ as $N \rightarrow \infty$ and $J(v)$ some function that depends on v alone. Further, as $N \rightarrow \infty$, we have that $\sigma_{\max}(R) \rightarrow \sqrt{\lambda_+}$ a.s.

Example B.1. As mentioned, when R is i.i.d Gaussian one can show, using the Borell-TIS inequality (see Theorem B.2), that:

$$\mathbb{P}\left(\|Rv\|_\infty > \left(\sqrt{\frac{2 \log N}{N}} + \frac{1}{N^{\frac{3}{8}}}\right)\|v\|_2\right) \leq 2 \exp\left(-\frac{N^{\frac{1}{4}}}{2}\right)$$

We now prove Lemma 3.10.

Proof. Step 1: Apply Assumption 4

Take

$$Z := \mathbb{P}(|\psi_2 \circ (R + S) \circ \psi_1 s - \psi_2 \circ (S) \circ \psi_1 s|_i > t) \quad (127)$$

We can now factor out ψ_2 , using the inequality $\|(\psi_2(w - v))\|_{\max} \leq \|\psi_2\|_{\max \rightarrow \max} \|w - v\|_{\max}$, to move ψ_2 outside of the norm:

$$Z \leq \mathbb{P}(\|\psi_2\|_{\max \rightarrow \max} \cdot \|(R + S) \circ \psi_1 s - S \circ \psi_1 s\|_{\max} > t) \quad (128)$$

Step 3: Focus on R

We now separate the terms involving R and S , leaving us with:

$$Z \leq \mathbb{P}(\|\psi_2\|_{\max \rightarrow \max} \cdot \|(R \circ \psi_1 s)\|_{\max} > t) \quad (129)$$

Step 4: Focus on the Random Matrix R From Assumption 5, for any $t > 0$:

$$\mathbb{P}(\|R \circ \psi_1 s\|_{\max} > d_1(N)J(\psi_1 s)) \leq d_2(N)$$

Step 6: Incorporate ψ_2

Next, incorporate the effect of ψ_2 . Using the properties of matrix norms, we have:

$$Z \leq \mathbb{P}(\|\psi_2\|_{\max \rightarrow \max} \|R \circ \psi_1 s\|_{\max} > \|\psi_2\|_{\max \rightarrow \max} d_1(N)J(\psi_1 s)) \leq d_2(N), \quad (130)$$

with $t = \|\psi_2\|_{\max \rightarrow \max} d_1(N)J(\psi_1 s)$.

□

Borell-TIS Inequality for i.i.d. Gaussian Variables

In this proof, $t \in \mathbb{R}$ does not refer to training time. We use this variable t because classically the Borell-TIS is given in terms of t .

Theorem B.2 (Borell-TIS Inequality). *Let $X_1, X_2, \dots, X_n$ be i.i.d. centered Gaussian random variables with $X_i \sim N(0, \sigma^2)$. Set $s_X^2 := \max_{i=1, \dots, n} \mathbb{E}(X_i^2) = \sigma^2$. Then for each $t > 0$:*

$$P\left(\max_{i=1, \dots, n} |X_i| - \mathbb{E}\left[\max_{i=1, \dots, n} X_i\right] > t\right) \leq \exp\left(-\frac{t^2}{2s_X^2}\right).$$

For the absolute value bound:

$$P\left(\left|\max_{i=1, \dots, n} |X_i| - \mathbb{E}\left[\max_{i=1, \dots, n} X_i\right]\right| > t\right) \leq 2 \exp\left(-\frac{t^2}{2s_X^2}\right).$$

In particular, if $X_i \sim N(0, \frac{1}{N})$:

$$s_X^2 = \frac{1}{N},$$

and for any $t > 0$:

$$P\left(\max_{i=1, \dots, n} |X_i| > \sqrt{\frac{2 \log n}{N}} + t\right) \leq 2 \exp\left(-\frac{Nt^2}{2}\right).$$

Generalized Chernoff Bound

In this proof, $t \in \mathbb{R}$ does not refer to training time. We use this variable t because classically the Generalized Chernoff Bound is given in terms of t .

Theorem B.3 (Chernoff Bound). *For any random variable X with moment-generating function $M_X(t) = \mathbb{E}[e^{tX}]$ and for any $t > 0$:*

$$P(X \geq a) = P(e^{tX} \geq e^{ta}) \leq M_X(t)e^{-ta}.$$

For the absolute value bound:

$$P(|X| \geq a) \leq M_X(t)e^{-ta} + M_{-X}(t)e^{-ta}.$$

If the distribution is symmetric, such that $M_X(t) = M_{-X}(t)$:

$$P(|X| \geq a) \leq 2M_X(t)e^{-ta}.$$

Chernoff Bound for Normal Distribution

In this proof, $t \in \mathbb{R}$ does not refer to training time. We use this variable t because classically the Generalized Chernoff Bound is given in terms of t .

Theorem B.4 (Chernoff Bound: normal distribution). *For a random variable X that is normally distributed with $X \sim N(0, 1/N)$, the moment-generating function is $M_X(t) = \mathbb{E}[e^{tX}] = e^{\frac{1}{2N}t^2}$. For any $t > 0$:*

$$P(X \geq a) \leq e^{\frac{1}{2N}t^2} e^{-ta}.$$

By choosing $t = Na$:

$$P(X \geq a) \leq e^{-\frac{Na^2}{2}}.$$

For the absolute value bound:

$$P(|X| \geq a) \leq 2e^{-\frac{Na^2}{2}}.$$

B.3 Training of a DNN with three weight layers using L1 and L2 regularization with varying layer widths

The goal of this subsection is to check how $a(N, s) = \sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 \|W_3\|_\infty + \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{1.5/4}}$ depends on N and the training process. From equation (119), we know that removing the random matrix R_2 from the DNN will change the components of the outputs X by less than $a(N)$. We want to see if this change in the output is small in practice. We train a DNN with three fully connected weight layers: $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$. The middle layer matrix $\mathbf{W}_2$ is of variable width N , which we vary across different experiments. We aim to analyze how varying N affects both the network’s accuracy and certain key norms of the weight matrices.

We use both L1 regularization during training, with the regularization strengths kept small ($5e-6$) so that they minimally affect the training process. Additionally, the learning rate is set to a small value of $1e-1$ to ensure that the weight matrices do not change significantly during the training period. We also decay the learning rate by .97 every epoch.

Normalization of the Fashion MNIST Dataset

The Fashion MNIST dataset consists of grayscale images of size 28×28 . To normalize the data, we ensure that the largest element across the entire dataset has an L2 norm of 0.05. This normalization is essential for controlling the magnitude of the input vector, which ensures that $a(N)$ stays small.

The normalization procedure involves the following steps:

1. Compute the L2 norm of each input vector (each image flattened to a 1D array).
2. Find the maximum norm among all vectors in the dataset.
3. Scale all input vectors so that the largest vector has norm 0.05 .

Training Procedure

The network is trained for 200 epochs. The weights are initialized from $N(0, 1/N)$, with N the number of column vectors.

Analysis and Metrics

For each value of N (the layer weight matrix W_2 has size $N \times N$), we compute two key quantities:

- $a(N)$: This is a metric based on the norms of the weight matrices, calculated as:

$$a(N) = \sqrt{\frac{2 \log N}{N}} \|W_1 s\|_2 \|W_3\|_\infty + \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{1.5/4}}$$

where $\|W_3\|_\infty$ is the L1 norm of the final weight layer and $\|W_1 s\|_2$ is the largest L2 norm of the transformed input data.

- Test Accuracy: The percentage of correctly classified test samples, which allows us to track how network performance evolves as the width N increases.

We analyze the behavior of both $a(N)$ and the test accuracy as functions of the layer weight matrix W_2 of size $N \times N$. The values of N that we test for are

$$[500, 1000, 1500, 2000, 3000, 4000, 5000, 7000, 10000, 20000, 30000].$$

Fig. 40 shows that as we increase the size W_2 , we have that $a(N)$ goes to zero quickly.

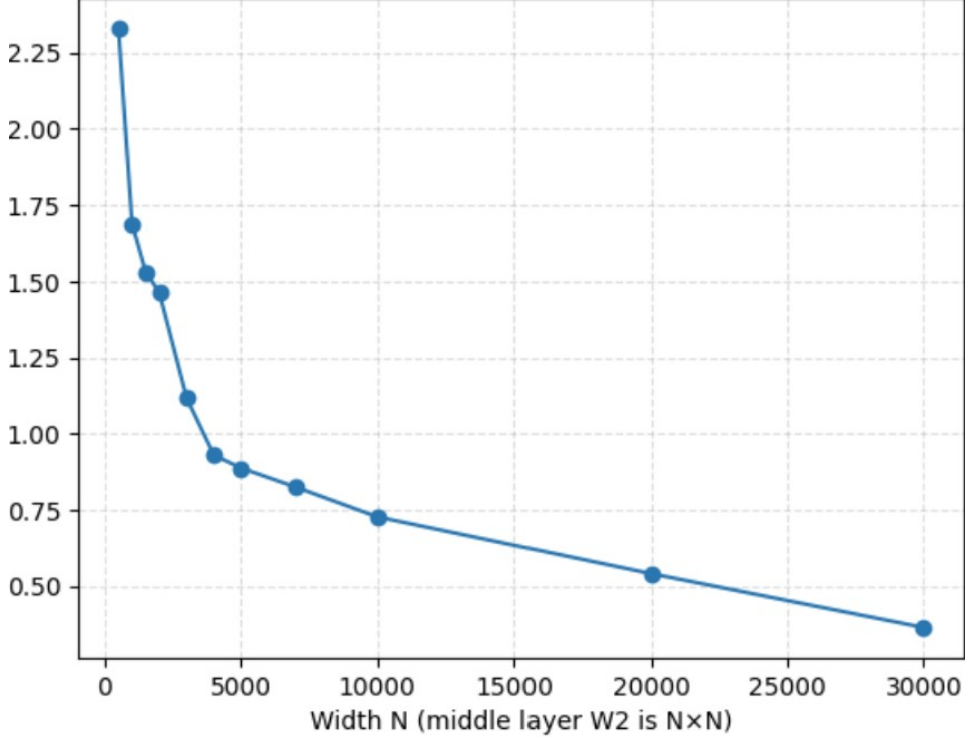


Figure 40: Plot of $a(N)$ vs. the width N of the middle layer $\mathbf{W}_2$. This plot shows how the complexity-related metric $a(N)$ behaves as the width N is increased, reflecting changes in the norms of the weight matrices and their dependence on N .

Normalization and Initialization of Weight Matrices

In this experiment, have the same hyperparameters as the previous experiment, the only difference is that the $L1$ regularization term is taken to be $1e - 6$.

Additionally, the weight matrices for each layer are initialized such that each entry is sampled from a normal distribution with zero mean and variance $1/N^2$, where N is the number of columns in the weight matrix.

B.3.1 Analysis of $b(N)$

Similar to the previous metric $a(N)$, we introduce a new metric $b(N)$, which is computed exactly like $a(N)$, but each occurrence of N is squared in the formula. Specifically, $b(N)$ is defined as:

$$b(N) = \sqrt{\frac{2 \log N^2}{N^2}} \|W_1 s\|_2 \|W_3\|_\infty + \frac{\|W_3\|_\infty \|W_1 s\|_2}{(N^2)^{1.5/4}}$$

where $\|W_3\|_\infty$ is the $L1$ norm of the final weight layer and $\|W_1 s\|_2$ is the largest $L2$ norm of the transformed input data.

Note, given the above initialization, equation (119) has $b(N)$ on the LHS rather than $a(N)$.

Results and Observations

We analyze the behavior of both $b(N)$ and the test accuracy as functions of the width N of the middle weight matrix $\mathbf{W}_2$. The values of N tested are [500, 1000, 1500, 2000, 3000, 4000, 5000, 7000, 10000, 20000, 30000]. Here, the DNN was trained for 200 epochs.

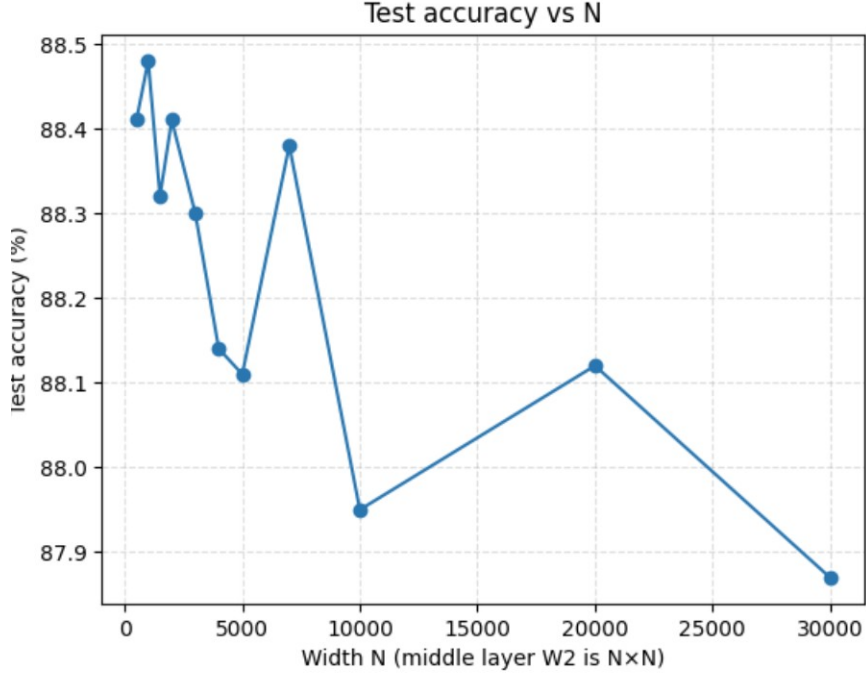


Figure 41: Plot of test accuracy vs. the width N of the middle layer $\mathbf{W}_2$. This plot demonstrates how the performance of the DNN changes with increasing N , showing the relationship between network width and generalization ability on the Fashion MNIST dataset.

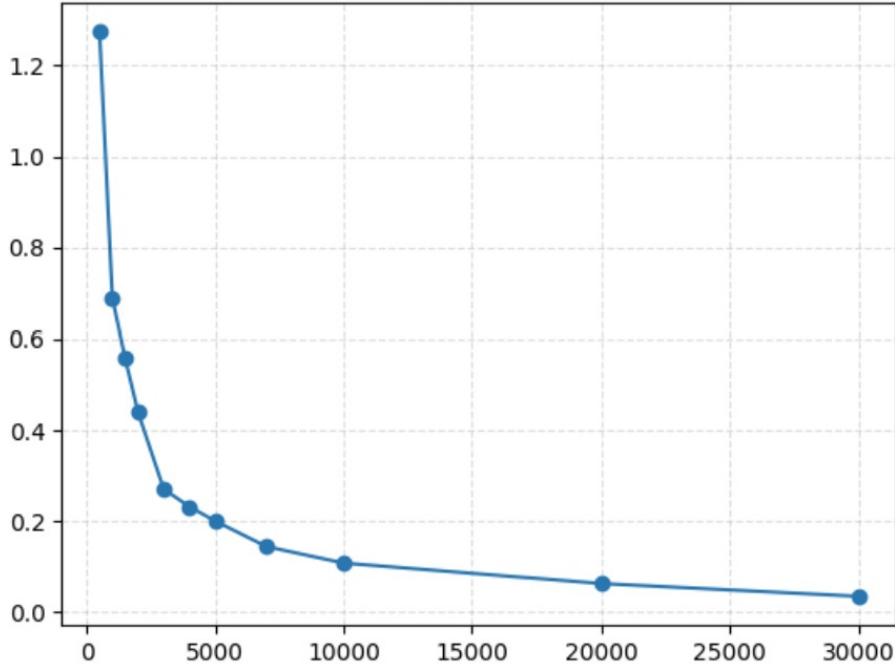


Figure 42: Plot of $b(N)$ vs. the width N of the middle layer $\mathbf{W}_2$. This plot shows how the metric $b(N)$ behaves as the size of the middle layer increases, incorporating a stronger dependence on N^2 .

From Fig. 42, we observe that as we increase the size of W_2 , the metric $b(N)$ converges toward zero at a faster rate compared to $a(N)$. This suggests that the dependence on N^2 significantly impacts the

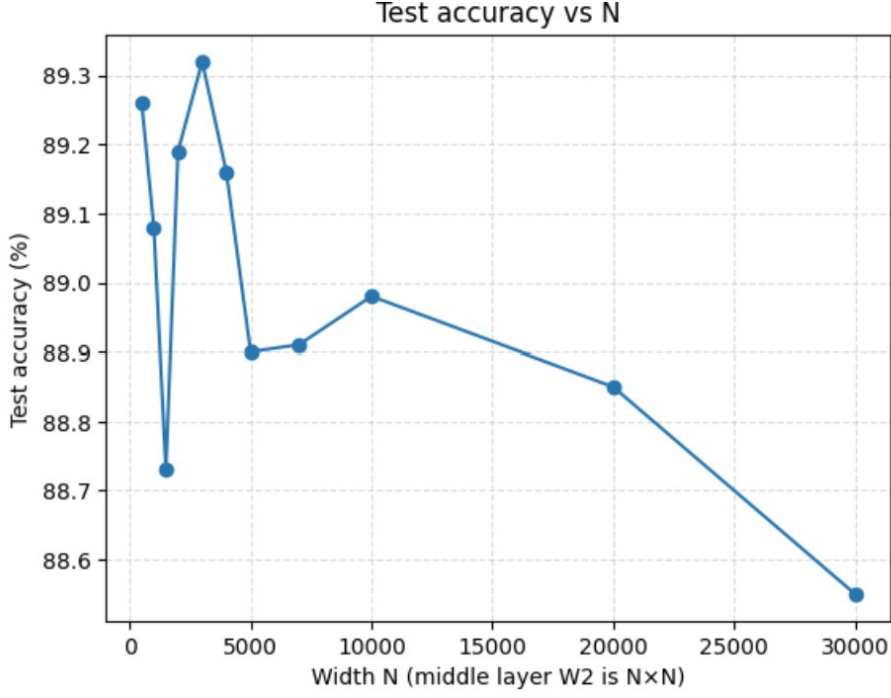


Figure 43: Plot of test accuracy vs. the width N of the middle layer $\mathbf{W}_2$. This plot demonstrates how the network's accuracy changes as N increases, with weight matrices initialized using $1/N^2$.

network's behavior.

Additionally, in Fig. 43, we see the test accuracy improving as the width N increases, consistent with the expectations from larger model capacity, though the gains diminish at higher values of N .

Lemma B.5. *Let ϕ be a DNN with weight layer matrices W_1, W_2, W_3 and λ the absolute value activation function:*

$$\phi(\alpha, s) = \rho \circ \lambda \circ W_3 \circ \lambda \circ W_2 \circ \lambda \circ W_1 s, \quad s \in \mathbb{R}^n. \quad (131)$$

Given (52), let W_2 be a $N \times N$ matrix such that $W_2(t) = R_2(t) + S_2(t)$, with R_2 and S_2 satisfying Assumptions 2.2-2.3.

Suppose we replace the weight layer matrix W_2 with the low-rank matrix S_2 . Then $\exists G_1(N)$ s.t.

$$\mathbb{P}\left(|L_{\text{reg}}(\alpha_{S_2}(t)) - L_{\text{reg}}(\alpha_{W_2}(t))| \leq G(N)H(\phi)\right) \geq 1 - G(N), \quad (132)$$

with $G_1(N) \rightarrow 0$ as $N \rightarrow \infty$ and $H(\phi)$ some function depending on ϕ with $H(\phi) \rightarrow 0$ as $N \rightarrow \infty$ if $a(N, s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{1.5/4}} \rightarrow 0$ as $N \rightarrow \infty$.

See Subsection B.4 for a proof of this lemma. See also Example 5.3 for a numerical example of this lemma.

B.4 Proof of Lemma B.5

Recall, we define:

$$L_{\text{reg}}(\alpha(t)) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(t))). \quad (133)$$

thus

$$L_{\text{reg}} = -\frac{1}{|T|} \sum_{s \in T} \log \left(\frac{e^{X_{i(s)}(\alpha(t), s)}}{\sum_{i=1}^K e^{X_i(\alpha(t), s)}} \right), \quad (134)$$

with K the number of classes in our classification problem.

From Lemma 3.10, we have that:

$$\mathbb{P} \left((|X_i(\alpha_W, s) - X_i(\alpha_S, s)| \leq a(N, s)) \geq 1 - C(N), \quad (135)$$

with $C(N) \rightarrow 0$ as $N \rightarrow \infty$.

Thus, for any fixed $s \in T'$, we have that, for big enough N , $\exists G(N) \rightarrow 0$ as $N \rightarrow \infty$ such that:

$$\mathbb{P} \left(\left| \log \left(\frac{e^{X_{i(s)}(\alpha_S(t), s)}}{\sum_{i=1}^K e^{X_i(\alpha_S(t), s)}} \right) - \log \left(\frac{e^{X_{i(s)}(\alpha_W(t), s)}}{\sum_{i=1}^K e^{X_i(\alpha_W(t), s)}} \right) \right| \leq G(N)H(s, \phi) \right) \geq 1 - G(N). \quad (136)$$

Furthermore, because the exponential and log functions are continuous, we have that if $a(N, s) \rightarrow 0$, then $G(N)C(s, \phi) \rightarrow 0$. Because that T' is a finite set, and because (136) holds for all $s \in T'$, we have that $\exists C(\phi)$ such that:

$$\mathbb{P} \left(\left| -\frac{1}{|T|} \sum_{s \in T} \log \left(\frac{e^{X_{i(s)}(\alpha_S(t), s)}}{\sum_{i=1}^K e^{X_i(\alpha_S(t), s)}} \right) + \frac{1}{|T|} \sum_{s \in T} \log \left(\frac{e^{X_{i(s)}(\alpha_W(t), s)}}{\sum_{i=1}^K e^{X_i(\alpha_W(t), s)}} \right) \right| \leq G(N)H(\phi) \right) \geq 1 - G(N). \quad (137)$$

This proves the first part of Lemma B.5 and gives us equation (132).

C Proof of Theorems 2.4, 2.5, 3.7 and 3.8.

In this section, we prove Theorems 2.4, 2.5, 3.7 and 3.8. Before proving this theorem, we state some useful RMT results for the type of model given in Assumptions 2.1-2.3 in Subsection 2.

C.1 Asymptotic behavior of singular values and vectors of the deformed matrix model given in Subsection 2

The following results can be found in [3, 7]. See [14] for other similar results.

Example C.1. Take $S = \sum_{i=1}^r \sigma_i u_i v_i^T$ to be a $N \times N$ low-rank matrix, with σ_i the singular values and v_i and u_i the singular vectors of S . Take R to be a $N \times N$ random matrix with real i.i.d. components taken from normal distribution $N(0, \frac{1}{N})$. For $W = R + S$ we have:

Theorem C.2. (Theorem for large singular values for Example C.1) The r largest singular values of W , denoted $\sigma'_i(W)$ for $1 \leq i \leq r$, exhibit the following behaviour as $N \rightarrow \infty$:

$$\sigma'_i(W) \xrightarrow{\text{a.s.}} \begin{cases} \frac{1+\sigma_i^2}{\sigma_i} & \sigma_i > 1 \\ 2 & \sigma_i < 1 \end{cases}$$

Remark C.1. We say that $X_n \rightarrow X$ in probability if $\forall \epsilon \mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$ as $n \rightarrow \infty$. One can show that $X_n \rightarrow X$ a.s. implies that $X_n \rightarrow X$ in law. Thus for Theorem C.2 $\forall \epsilon$ we have:

$$\mathbb{P}(|\sigma'_i(W) - \frac{1+\sigma_i^2}{\sigma_i}| > \epsilon) < B_N(\epsilon). \quad (138)$$

In our case, we want the variance of $R(t)$ to depend on t . That is, The matrix $W_b(t)$ is expressed as:

$$W_b(t) = R_b(t) + S_b(t), \quad (\text{deformed matrix}) \quad (139)$$

where $R_b(t)$ is an $N \times M$ matrix with i.i.d. entries drawn from $N(0, \frac{C(t)}{N})$, with $C(t) \leq 1$ representing the training time-dependent variance, and $S_b(t)$ is a low-rank matrix.

Lemma C.3 (Lemma for large singular values with training time-dependent variance). *Consider the matrix $W_b(t) = R_b(t) + S_b(t)$, where $R_b(t)$ is an $N \times M$ random matrix with i.i.d. entries drawn from $N(0, \frac{C(t)}{N})$ and $S_b(t)$ is a low-rank matrix. The r largest singular values of $W_b(t)$, denoted $\sigma'_i(W_b(t))$ for $1 \leq i \leq r$, exhibit the following behaviour as $N \rightarrow \infty$:*

$$\sigma'_i(W_b(t)) \xrightarrow{a.s.} \sqrt{C(t)} \times \begin{cases} \frac{1 + (\frac{\sigma_i}{\sqrt{C(t)}})^2}{\frac{\sigma_i}{\sqrt{C(t)}}} & \frac{\sigma_i}{\sqrt{C(t)}} > 1 \\ 2 & \frac{\sigma_i}{\sqrt{C(t)}} < 1 \end{cases}$$

Proof. Define $W' = R' + S'$ where $R' = \frac{1}{\sqrt{C(t)}} R_b(t)$ and $S' = \frac{1}{C(t)} S_b(t)$. The entries of R' are now drawn from $N(0, \frac{1}{N})$, matching the classical case.

For W' , the classical theorem gives the behavior of its largest singular values. Specifically, if σ'_i are the singular values of S' , then the singular values of W' , denoted as $\sigma'_i(W')$, behave as:

$$\sigma'_i(W') \xrightarrow{a.s.} \begin{cases} \frac{1 + \sigma'^2_i}{\sigma'_i} & \sigma'_i > 1 \\ 2 & \sigma'_i < 1 \end{cases}$$

Since $\sigma'_i = \frac{\sigma_i}{\sqrt{C(t)}}$, the theorem for W' becomes:

$$\sigma'_i(W') \xrightarrow{a.s.} \sqrt{C(t)} \times \begin{cases} \frac{1 + (\frac{\sigma_i}{\sqrt{C(t)}})^2}{\frac{\sigma_i}{\sqrt{C(t)}}} & \frac{\sigma_i}{\sqrt{C(t)}} > 1 \\ 2 & \frac{\sigma_i}{\sqrt{C(t)}} < 1 \end{cases}$$

To revert to the original matrix $W_b(t)$, we multiply the singular values of W' by $\sqrt{C(t)}$. This gives the asymptotic behavior of the singular values of $W_b(t)$ as stated in the lemma. $\square$

C.2 Proof of Theorems 2.4, 2.5, 3.7 and 3.8.

Proof. In this subsection, we prove Theorems 2.4 and 2.5. Using the more general Assumptions 3.1-3.5 for Theorems 3.7 and 3.8, we can use the proof for these results as well. We will prove the main result by showing that if the random component $R_2(t)$ of the weight matrix $W_2(t)$ is non-zero at a local minimum, it contradicts the assumption of a local minimum.

1. Notation and Assumptions:

- We assume that α^* is a local minimum of the loss function $L(\alpha(t))$ defined in Eq. (52).
- We assume that $\|R_2(\alpha^*)\|_2 \neq 0$, which implies the random component $R_2(\alpha^*)$ is not zero.

2. Constructing a DNN at the local minimum:

- We initialize a DNN with parameters α^* and a loss function $\bar{L}(\alpha(a))$ such that $\bar{L}(\alpha(0)) = L(\alpha^*)$. That is, we create a new training time, denoted with the variable a , such that when $a = 0$, the DNN parameters are those at the critical point α^* . This is achievable as the loss function is continuous.

3. Introducing a controlled random noise:

- We define a random matrix $R(a)$ with i.i.d. entries drawn from $N(0, \frac{g(a)}{N})$. We will choose $g(a)$ to vanish as a approaches infinity.

4. Key Inequality and its Consequence:

- We claim that there exists a constant $g(a)$ such that $g(a) \rightarrow 0$ as $a \rightarrow \infty$ and for all $a > 0$ and $N > N_0$ (for some N_0) we have that $\exists D(N)$ such that:

$$\mathbb{P}\left(L(\alpha^*) \geq \bar{L}(\alpha(a))\right) > 1 - D(N), \quad (140)$$

with $D(N) \rightarrow 0$ as $N \rightarrow \infty$.

5. Proof of the Key Inequality:

- We will utilize the following properties:
 - Lemma B.5: This lemma states that removing the random component R_2 from the weight matrix W_2 has a minimal impact on the loss function with high probability. That is based on Lemma B.5, we know that:

$$\mathbb{P}\left(|L_{\text{reg}}(\alpha_{S_2}(a)) - L_{\text{reg}}(\alpha_{W_2}(a))| \leq G(N)H(\phi)\right) \geq 1 - G(N), \quad (141)$$

where $G(N) \rightarrow 0$ as $N \rightarrow \infty$ and $\alpha_{S_2}(t)$ denotes the parameters of the DNN with the weight matrix $S_2(t)$.

- Frobenius norm:

$$\|A\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N A_{ij}^2} = \sqrt{\sigma_1^2 + \sigma_2^2 + \dots + \sigma_N^2}. \quad (142)$$

- The loss:

$$L(\alpha(a)) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(a))) + \sum_{i=1}^L \mu \|W_i(a)\|_F^2, \quad (143)$$

- Lemma C.3. We are assuming that for the non-zero singular values of S we have, $\sigma_i(a) > 1$. Because $0 \leq g(a) \leq 1$, we have, by Lemma C.3, that:

$$\sigma'_i(W') \xrightarrow{a.s.} g(a) \frac{1 + (\frac{\sigma_i}{\sqrt{g(a)}})^2}{\frac{\sigma_i}{\sqrt{g(a)}}}. \quad (144)$$

That is, $\exists D_2(N, \epsilon)$ such that $\forall \epsilon$:

$$\mathbb{P}\left(|\sigma'_i(W') - C(a) \frac{1 + (\frac{\sigma_i}{\sqrt{C(a)}})^2}{\frac{\sigma_i}{\sqrt{C(a)}}}| > \epsilon\right) \leq D_2(N, \epsilon), \quad (145)$$

with $D_2(N, \epsilon) \rightarrow 0$ as $N \rightarrow \infty$.

- Take α^* to be a local min of (52), and suppose $\|R(\alpha^*)\|_2 \neq 0$. Initialize the DNN

$$\phi(\alpha, s) = \rho \circ \lambda \circ W_3 \circ \lambda \circ W_2 \circ \lambda \circ W_1 s \quad (146)$$

with parameters α^* and loss $\bar{L}(\alpha(a))$ s.t. $\bar{L}(\alpha(0)) = L(\alpha^*)$. Assume that $R_2(0)$ is a random matrix with i.i.d. $N(0, \frac{g(a=0)}{N})$, such that at $a = 0$ we have that $W_2(0) = R_2(0) + S_2(0)$ satisfying the assumptions in Subsection 2. Then there $\exists g(a) \rightarrow 0$ monotonically as $a \rightarrow \infty$, and take $R_2(a)$ to be a random matrix with i.i.d. taken from $N(0, \frac{g(a)}{N})$ and $W_2(a) = R_2(a) + S_2(0)$. By Lemma B.5, we know that:

$$\mathbb{P}\left(|L_{\text{reg}}(\alpha_{S_2}(a)) - L_{\text{reg}}(\alpha_{W_2}(a))| \leq G(N)H(\phi)\right) \geq 1 - G(N). \quad (147)$$

Furthermore, one can apply the same proof that was used to prove Lemma B.5 to show that $\forall a$:

$$\mathbb{P}\left(|L_{\text{reg}}(\alpha_{S_2}(0)) - L_{\text{reg}}(\alpha_{W_2}(a))| \leq G(N)H(\phi)\right) \geq 1 - G(N), \quad (148)$$

given that $W_2(a) = R_2(a) + S_2(0)$ and the random part $R_2(a)$ has i.i.d. components with zero mean and variance $\frac{g(a)}{N}$. That is, as $a \rightarrow \infty$ if the only change in W_2 is that the random part R_2 goes to zero, then the loss L_{reg} won't change significantly with probability depending on N .

However, the loss:

$$L(\alpha(a)) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(a))) + \mu \sum_{i=1}^L \|W_i(a)\|_F^2, \quad (149)$$

will be decreasing because the term $\mu \sum_{i=1}^L \|W_i(a)\|_F^2$ will get smaller monotonically as $a \rightarrow \infty$. This can be seen from the equations:

$$\|A\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N A_{ij}^2} = \sqrt{\sigma_1^2 + \sigma_2^2 + \dots + \sigma_N^2}, \quad (150)$$

and (145). By (145), the r largest singular values of W_2 will become smaller as $a \rightarrow \infty$ (with some probability depending on N), making $\mu \sum_{i=1}^L \|W_i(a)\|_F^2$ smaller monotonically (again with probability depending on N). Furthermore, the $N - r$ smallest singular values of W_2 will also go to zero monotonically as $a \rightarrow \infty$, again ensuring that $\mu \sum_{i=1}^L \|W_i(a)\|_F^2$ decrease (with probability depending on N). Thus, because for (52) the term $-\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(a)))$ does not change as $a \rightarrow \infty$ (with probability depending on N) and $\mu \sum_{i=1}^L \|W_i(a)\|_F^2$ decreases as $a \rightarrow \infty$ (again with probability depending on N), we have that the loss given in (52) must be decreasing as $a \rightarrow \infty$ (with probability depending on N). That is, for $a > 0$, $\exists D(N)$ such that:

$$\mathbb{P}\left(L(\alpha^*) > \bar{L}(\alpha(a))\right) > 1 - D(N). \quad (151)$$

That is, α^* is not a local min (with probability depending on N).

Furthermore, for a fixed training time t as $N \rightarrow \infty$ we have that removing the random matrix R_2 will not affect the cross entropy term in the loss but will remove the singular values of W_2 smaller than $\sqrt{\lambda_+}$. That is, the singular values of W_2 corresponding to those of R_2 will be set to zero, which means that the loss will decrease by $\mu \|R_2\|_F^2$. This proves Theorem 2.5. In fact, if $\|R_2\|_F > \epsilon$, then one can remove it (by sending $g(t) \rightarrow 0$), and the loss will decrease monotonically (with some probability depending on N) by $\mu \|R_2\|_F^2$, showing that we were not in a local min.

□

D ViT model architecture

D.1 Image Representation and Embedding

Consider an input image I which is a matrix of pixels. This image is divided into a grid of N patches, $\{P_1, P_2, \dots, P_N\}$. Each patch P_i is then linearly embedded into a d -dimensional space. This process is done by flattening each patch and then applying a linear transformation, often followed by adding a positional encoding.

The mathematical process can be described as follows:

$$\text{vec}(P_i) = \text{Flatten}(P_i)$$

We now apply a fully connected layer with no activation function:

$$x_i = \text{vec}(P_i)W + b$$

Here, $\text{vec}(P_i)$ is the flattened vector of the patch P_i , W is a learnable weight matrix in $\mathbb{R}^{p \times d}$, where p is the number of pixels in a patch, and b is a learnable bias vector in $\mathbb{R}^d$. The result x_i is the embedded representation of the patch.

D.2 Positional Encoding

Due to the self-attention mechanism's permutation-invariant nature, it does not inherently account for the order or position of the input elements. In the context of images, where the relative position of patches is vital, positional encoding is added to the embedded representations to retain this spatial information.

Positional encodings are typically vectors of the same dimension d as the patch embeddings, which are added to the embeddings:

$$x'_i = x_i + \text{PositionalEncoding}(i)$$

These encodings can be either learned during the training process or predefined based on sinusoidal functions, capturing the relative or absolute position of the patches in the image grid. This way, the transformer can leverage the spatial structure of the image.

D.3 Self-Attention Mechanism

In the self-attention mechanism, each element in the sequence is updated by aggregating information from the entire sequence. This is done by computing a weighted sum of value vectors V , where the weights are determined by a compatibility function of query Q and key K vectors.

Each vector x_i is transformed into a set of query Q , key K , and value V vectors through linear transformations (rather than affine) using learnable weights W^Q , W^K , and W^V :

$$Q_i = x_i W^Q, \quad K_i = x_i W^K, \quad V_i = x_i W^V$$

where $W^Q, W^K, W^V \in \mathbb{R}^{d_{emb} \times d_k}$ and d_{emb} is the dimension space of the embeddings and $d_k < d_{emb}$. Similar to the previous weight matrix W , matrices W^Q, W^K , and W^V begin with random values that get learned throughout the training.

D.4 Calculation of Attention

The attention mechanism is a fundamental concept in various models for processing sequences of data (an example of such a sequence can be a sequence of patches of data). Fundamentally, it calculates the similarity of each part of the data sequence (patch) with respect to a different element of the sequence (a different patch).

Attention Score

The attention score is a measure of the relevance or importance between two elements in a sequence, typically represented as patches, tokens, or vectors. The attention score between the i -th query vector Q_i and the j -th key vector K_j is computed using the dot product of Q_i and K_j , which is then scaled by the inverse of the square root of the dimension of the key vectors. Mathematically, it is represented as:

$$\text{Attention}(Q_i, K_j) = \text{softmax} \left(\frac{Q_i K_j^T}{\sqrt{d_k}} \right)$$

Here, d_k represents the dimension of the key (and query) vectors. The dot product $Q_i K_j^T$ computes a measure of similarity or alignment between the query and key. The scaling factor $\frac{1}{\sqrt{d_k}}$ is used to avoid overly large dot products as the dimensionality increases, which can lead to gradients that are too small for effective learning.

D.4.1 Output of the Attention Layer

Once the attention scores are computed, the output for each query vector is obtained by taking a weighted sum of the value vectors, where the weights are the attention scores. The output for the i -th query vector is computed as:

$$\text{Output}_i = \sum_{j=1}^N \text{Attention}(Q_i, K_j) V_j$$

In this equation, V_j represents the value vector corresponding to the j -th element of the sequence. The weighted sum allows the model to aggregate information from the entire sequence, weighted by how relevant each element is to the query.

D.5 Position-Wise Feed-Forward Networks

Each layer of the Transformer also contains a feed-forward network, which consists of two linear transformations with an absolute value activation in between. These networks are applied to each position separately and identically.

$$\text{FFN}(x) = \text{Abs}(xW_1 + b_1)W_2 + b_2$$

E Some known results on perturbation of matrices

Matrix perturbation theory is concerned with understanding how small changes in a matrix can affect its properties, such as eigenvalues, eigenvectors, and singular values. In this section, we state a couple of known results from matrix perturbation theory.

E.1 Asymptotics of singular values and singular vectors of deformation matrix

The results in this subsection are taken from [7]. See [17, 35] for generalizations of many of these results. Given the assumptions 1'-2' on R and S described in Section 3.1, the authors were able to show that the largest eigenvalues and corresponding eigenvectors of $W = S + R$ are well approximated by the largest eigenvalues and eigenvectors of S .

We start by defining the following function:

$$D_{\mu_R}(z) = \left[\int \frac{z}{z^2 - t^2} d\mu_R(t) \right] \times \left[c \int \frac{z}{z^2 - t^2} d\mu_R(t) + \frac{1-c}{z} \right] \quad (152)$$

for $z > \sqrt{\lambda_+}$, with λ_+ given by the MP distribution of $R^T R$. Take $D_{\mu_R}^{-1}(\cdot)$ to be its functional inverse.

Set

$$\bar{\theta} = D_{\mu_R}(\sqrt{\lambda_+})^{-\frac{1}{2}} \quad (153)$$

Theorem E.1. *Theorem for large singular values [Benaych-Georges and Nadakuditi (2012)] Take $W = R + S$, with W, R and S all $N \times M$ matrices satisfying assumptions 1' – 2'. The r largest singular values of W , denoted as $\sigma'_i(W)$ for $1 \leq i \leq r$, exhibit the following behaviour as $N \rightarrow \infty$:*

$$\sigma'_i(W) \xrightarrow{a.s.} \begin{cases} D_{\mu_R}^{-1}(\frac{1}{(\sigma_i)^2}) & \sigma_i > \bar{\theta} \\ \sqrt{\lambda_+} & \sigma_i < \bar{\theta} \end{cases} \quad (154)$$

Theorem E.2. *Norm of projection of largest singular vectors [Benaych-Georges and Nadakuditi (2012)]*

Take indices $i_0 \in \{1, \dots, r\}$ such that $\sigma_{i_0} > \bar{\theta}$. Take $\sigma'_{i_0} = \sigma'_{i_0}(W)$ and let u', v' be left and right unit singular vectors of W associated with the singular value σ'_{i_0} and u, v be the corresponding singular vectors of S . Then we have, as $N \rightarrow \infty$:

$$| \langle u', \text{Span}\{u_i \text{ s.t. } \sigma_i = \sigma_{i_0}\} \rangle |^2 \xrightarrow{a.s.} \frac{-2\phi_{\mu_R}(\rho)}{\sigma_{i_0}^2 D'_{\mu_R}(\rho)} \quad (155)$$

and

$$| \langle v', \text{Span}\{v_i \text{ s.t. } \sigma_i = \sigma_{i_0}\} \rangle |^2 \xrightarrow{a.s.} \frac{-2\phi_{\mu_R}(\rho)}{\sigma_{i_0}^2 D'_{\mu_R}(\rho)} \quad (156)$$

Here $\rho = D_{\mu_R}^{-1}(\frac{1}{(\sigma_{i_0})^2})$ and $\tilde{\mu}_R = c\mu_R + (1+c)\delta_0$.

Further,

$$| \langle u', \text{Span}\{u_i \text{ s.t. } \sigma_i \neq \sigma_{i_0}\} \rangle |^2 \xrightarrow{a.s.} 0 \quad (157)$$

$$| \langle v', \text{Span}\{v_i \text{ s.t. } \sigma_i \neq \sigma_{i_0}\} \rangle |^2 \xrightarrow{a.s.} 0 \quad (158)$$

Example E.3. Take $S = \sum_{i=1}^r \sigma_i u_i v_i^T$ to be a $N \times N$ low-rank matrix, with σ_i the singular values and v_i and u_i the singular vectors of S . Take R to be a $N \times N$ random matrix with real i.i.d components taken from the normal distribution $N(0, \frac{1}{N})$. For $W = R + S$ we have:

Theorem E.4. (Theorem for large singular values for Example E.3) The r largest singular values of W , denoted $\sigma'_i(W)$ for $1 \leq i \leq r$, exhibit the following behaviour as $N \rightarrow \infty$:

$$\sigma'_i(W) \xrightarrow{a.s.} \begin{cases} \frac{1+\sigma_i^2}{\sigma_i} & \sigma_i > 1 \\ 2 & \sigma_i < 1 \end{cases}$$

Theorem E.5. (Theorem for large singular vectors for Example E.3) Assuming that the r largest singular values of W have multiplicity 1, then the right and left singular vectors u'_i, v'_i of W corresponding with the r largest singular values $\sigma'_i(W)$ exhibits the following behaviour as $N \rightarrow \infty$:

$$| \langle v_i, v'_i \rangle |^2, | \langle u_i, u'_i \rangle |^2 \xrightarrow{a.s.} \begin{cases} (1 - \frac{1}{\sigma_i^2}) & \sigma_i > 1 \\ 0 & \sigma_i < 1 \end{cases}$$

Remark E.1. We say that $X_n \rightarrow X$ in probability if $\forall \epsilon \mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$ as $n \rightarrow \infty$. One can show that $X_n \rightarrow X$ a.s. implies that $X_n \rightarrow X$ in law. Thus for Theorems E.1 and E.2 we have that if the r large singular values of S are bigger than 1 and have multiplicity 1 then there exists some constant $B_N(\epsilon)$ with $B_N(\epsilon) \rightarrow 0$ as $N \rightarrow \infty$ such that for the r largest singular values of W and their corresponding singular vectors and $\forall \epsilon$ we have:

$$\mathbb{P}(|\sigma'_i(W) - \frac{1 + \sigma_i^2}{\sigma_i}| > \epsilon) < B_N(\epsilon), \quad (159)$$

$$\mathbb{P}(|\langle v_i, v'_i \rangle|^2 - (1 - \frac{1}{\sigma_i^2})| > \epsilon) < B_N(\epsilon) \quad (160)$$

and

$$\mathbb{P}(|\langle u_i, u'_i \rangle|^2 - (1 - \frac{1}{\sigma_i^2})| > \epsilon) < B_N(\epsilon). \quad (161)$$

Finally, we also have that:

$$\mathbb{P}(|\langle u_i, u'_j \rangle| > \epsilon) < B_N(\epsilon). \quad (162)$$

and

$$\mathbb{P}(|\langle v_i, v'_j \rangle| > \epsilon) < B_N(\epsilon). \quad (163)$$

for $i \neq j$.

E.2 Gershgorin's Circle Theorem

Finally, we state Gershgorin's Circle Theorem:

Theorem E.6 (Gershgorin's Circle Theorem (GCT)). *Let $B = [b_{ij}]$ be an $n \times n$ complex matrix. Define the Gershgorin discs D_i for $1 \leq i \leq n$ as*

$$D_i = \left\{ z \in \mathbb{C} : |z - b_{ii}| \leq \sum_{j \neq i} |b_{ij}| \right\}. \quad (164)$$

Then, every eigenvalue λ of the matrix B lies within at least one of the Gershgorin discs D_i .

Remark E.2. We apply Theorem E.6 for almost diagonal matrices when (164) estimates how close the eigenvalues are to the diagonal elements. This closeness is estimated in terms of the magnitude of the non-diagonal elements.

F An Approximation Lemma – pruned matrix W' approximates the low-rank matrix S

Assume we are given a low-rank matrix S and we add to it a random matrix R , for the R and S given in Example E.3. Suppose we take $W = S + R$; it is well known that one can find a rank k approximation of W . This is done by taking the SVD of $W = U\Sigma V^T$ and setting all but the top k singular values in Σ to zero. The following is a known theorem of this result:

Theorem F.1. *Given the singular value decomposition (SVD) of $W = U\Sigma V^T$, where U and V are uni-*

tary matrices and Σ is a diagonal matrix containing the singular values of W , the rank k approximation of W is given by:

$$\tilde{W}_k = U_k \Sigma_k V_k^T \quad (165)$$

where U_k and V_k are the matrices obtained by retaining only the first k columns of U and V , respectively, and Σ_k is the matrix obtained by retaining only the first k diagonal entries of Σ . Here we use $\tilde{W}_k$ to distinguish it from the weight layer matrix W_k . This approximation represents the best rank k approximation to W in the following sense:

$$\tilde{W}_k = \min_{\text{rank}(X)=k} \|W - X\|_F \quad (166)$$

where X is an arbitrary matrix of rank k and $\|\cdot\|_F$ is the Frobenius norm.

See [23] for more on this result. In this section, we wish to obtain slightly different results in a similar direction. We wish to show that for the R and S given in Example E.3, $\tilde{W}_r$ is a good approximation of S (recall S has r non-zero singular values). That means that W_r is a good approximation of the low-rank part of W . We, therefore, state a lemma that shows that:

$$\|(\tilde{W}_r - S)z\| < f_W \|z\|. \quad (167)$$

Rather than state this lemma in terms of $\tilde{W}_r$, we state them in terms of W' , which is defined as follows:

Definition F.2. Take $W = R + S$, with R and S given in Assumptions 3.1-3.5 from Section 3.1. Take $W = U\Sigma V^T$ to be the SVD of W and take, for some arbitrarily small $1 > \epsilon_0(r) > 0$,

$$\Sigma'_{i,j} = \begin{cases} \Sigma_{i,j} & \Sigma_{i,j} > \sqrt{\lambda_+} + \epsilon_0(r) \\ 0 & \Sigma_{i,j} \leq \sqrt{\lambda_+} + \epsilon_0(r). \end{cases} \quad (168)$$

Then we obtain the truncated matrix W' by taking $W' = U\Sigma'V^T$. Here, $\epsilon_0(r)$ is an arbitrarily small number which depend on how much bigger $\sigma_r(W)$ is than $\sqrt{\lambda_+}$. For example, under Assumption 2.3 and when R is $N(0, \frac{1}{N})$, we have $\epsilon_0(r) = \frac{\epsilon_0^2}{2}$, see Lemma F.4.

Next, we define a padded unitary matrix, which is helpful when proving the approximation theorem.

Definition F.3 (Padded Unitary Matrix). A padded-unitary matrix Q of size $(n + m) \times (n + m)$ is defined as a matrix where the first $n \times n$ submatrix is a unitary matrix, and the remaining entries are zeros. Formally, if U is an $n \times n$ unitary matrix, then Q is constructed as

$$Q = \begin{pmatrix} U & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix},$$

where $\mathbf{0}$ represents the appropriate-sized matrix with all zero entries.

Next, we present an approximation lemma that gives a bound for $\|(W' - S)z\|_2$, where z is any vector. This lemma describes how well S is being approximated by W' .

Lemma F.4. Assume $\sqrt{\lambda_+} = 2$ (as is standard for the Marchenko-Pastur distribution with aspect ratio 1, where $\lambda_+ = 4$). Under Assumption 2.3, the singular values of $S_2(t)$, denoted $\sigma_i(t)$, satisfy $\sigma_i > 1 + \epsilon_0$ with $\epsilon_0 < 1$. Then, the limiting singular value from Theorem E.4, $\frac{1+\sigma_i^2}{\sigma_i}$, satisfies:

$$\frac{1 + \sigma_i^2}{\sigma_i} > 2 + \frac{\epsilon_0^2}{2}. \quad (169)$$

Proof. Let $x = \epsilon_0$, so $\sigma_i > 1 + x$. The expression $\frac{1+\sigma_i^2}{\sigma_i} = \sigma_i + \frac{1}{\sigma_i}$. The function $f(\sigma_i) = \sigma_i + \frac{1}{\sigma_i}$ is increasing for $\sigma_i > 1$ since $f'(\sigma_i) = 1 - \frac{1}{\sigma_i^2} > 0$.

Thus, the minimal value occurs as $\sigma_i \rightarrow (1+x)^+$. Evaluate at $\sigma_i = 1+x$:

$$\sigma_i + \frac{1}{\sigma_i} = (1+x) + \frac{1}{1+x} = 2 + \frac{x^2}{1+x}, \quad (170)$$

since

$$(1+x) + \frac{1}{1+x} = 2 + x + \frac{1}{1+x} - 1 = 2 + \frac{x(1+x) + 1 - (1+x)}{1+x} = 2 + \frac{x^2}{1+x}. \quad (171)$$

The pruning threshold is $2 + \frac{x^2}{2}$. We compare $2 + \frac{x^2}{1+x}$ and $2 + \frac{x^2}{2}$ because the former represents the boundary value of the limiting singular value, and showing it exceeds the threshold ensures the inequality holds even at the minimal separation, with the difference being smallest near the boundary.

Prove $2 + \frac{x^2}{1+x} > 2 + \frac{x^2}{2}$:

$$\frac{x^2}{1+x} > \frac{x^2}{2}. \quad (172)$$

For $x > 0$, divide by $x^2 > 0$:

$$\frac{1}{1+x} > \frac{1}{2} \iff 2 > 1+x \iff x < 1, \quad (173)$$

which holds since $\epsilon_0 < 1$. The inequality is strict for $x < 1$, and since $\sigma_i > 1+x$, $\frac{1+\sigma_i^2}{\sigma_i} > 2 + \frac{x^2}{1+x} > 2 + \frac{x^2}{2}$. $\square$

Theorem F.5 (Vanishing projection outside the spike subspace [7]). *Let $X_n = R_n + S_n$ be a spiked rectangular (or square) random matrix sequence with*

$$S_n = \sum_{i=1}^r \theta_i u_i v_i^\top,$$

where r is fixed, $\theta_i > 0$, and $\{u_i\}_{i=1}^r \subset \mathbb{R}^n$, $\{v_i\}_{i=1}^r \subset \mathbb{R}^m$ are orthonormal systems. Assume R_n satisfies the usual isotropic noise assumptions (e.g. i.i.d. entries with mean 0 and variance $1/n$, or any model for which the BBP/Benaych-Georges–Nadakuditi theory applies) so that there is a critical threshold θ_c and each “supercritical” spike $\theta_{i_0} > \theta_c$ creates an outlying singular value.

For a fixed index $i_0 \in \{1, \dots, r\}$ with $\theta_{i_0} > \theta_c$, let

$$\sigma_{i_0}^{(n)} = \sigma_{i_0}(X_n), \quad u_{i_0}^{(n)}, v_{i_0}^{(n)}$$

be the associated (unit) left and right singular vectors of X_n . Define the spike subspaces

$$\mathcal{U}_= := \text{Span}\{u_i : \theta_i = \theta_{i_0}\}, \quad \mathcal{U}_\neq := \text{Span}\{u_i : \theta_i \neq \theta_{i_0}\},$$

and similarly for the right side,

$$\mathcal{V}_= := \text{Span}\{v_i : \theta_i = \theta_{i_0}\}, \quad \mathcal{V}_\neq := \text{Span}\{v_i : \theta_i \neq \theta_{i_0}\}.$$

Then, almost surely as $n \rightarrow \infty$,

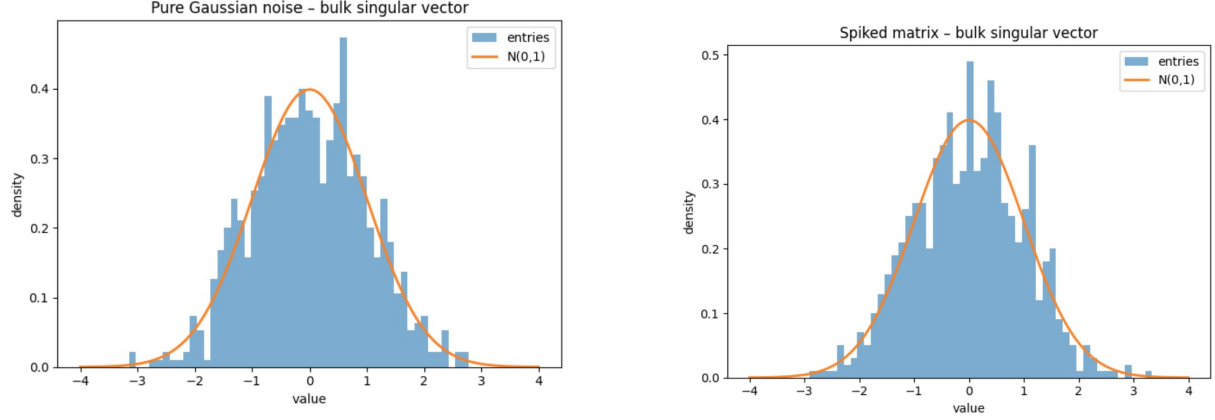
$$\|\text{Proj}_{\mathcal{U}_\neq} u_{i_0}^{(n)}\|_2^2 \xrightarrow{a.s.} 0, \quad (a)$$

$$\|\text{Proj}_{\mathcal{V}_\neq} v_{i_0}^{(n)}\|_2^2 \xrightarrow{a.s.} 0. \quad (b)$$

Equivalently,

$$\sum_{\{i: \theta_i \neq \theta_{i_0}\}} \langle u_{i_0}^{(n)}, u_i \rangle^2 \xrightarrow{a.s.} 0, \quad \sum_{\{i: \theta_i \neq \theta_{i_0}\}} \langle v_{i_0}^{(n)}, v_i \rangle^2 \xrightarrow{a.s.} 0. \quad (c)$$

Remark F.1. For other useful structures of the singular vectors of spiked matrices, see [17, 35].



(a) Pure Gaussian noise: histogram of the rescaled entries $\sqrt{N} u_i$ overlaid with the $\mathcal{N}(0, 1)$ pdf

(b) Spiked model ($r = 3$, $\theta = (3, 2.5, 2.0)$): histogram of the rescaled entries $\sqrt{N} u_i$

Figure 44: Entry-wise distribution of a bulk left singular vector after the normalisation $u_i \mapsto \sqrt{N} u_i$. In both panels the bars show the empirical histogram, while the solid curve is the standard normal density. Panel (a) confirms that for a pure Gaussian matrix R the bulk singular vectors are isotropic. Panel (b) shows that the same delocalisation persists when a rank- r super-critical spike S is added, i.e. for $X = R + S$.

Illustration of bulk singular-vector delocalisation

We simulate the spiked model

$$X = R + S, \quad R_{ij} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1/N), \quad S = U \text{diag}(\theta_1, \dots, \theta_r) V^\top,$$

with dimension $N = 800$, spike rank $r = 3$ and amplitudes $\sigma = (3, 2.5, 2)$. The factor matrices U, V are independent Haar draws on the Stiefel manifold $\text{St}(N, r)$. After computing the SVD of $X = R + S$ we pick a bulk left singular vector (index $r + 30$), rescale its entries by $\sqrt{N}$, and compare their empirical histogram with $\mathcal{N}(0, 1)$. For reference we repeat the same experiment on the pure-noise matrix R .

Approximation Lemma F.6. *Assume that $W = R + S$, with R a $N \times M$ random matrix and S a low-rank matrix satisfying assumptions 1 – 3, for the assumptions in Section 3.1. Assume that the singular values of S are bigger than $\bar{\theta}$, given in assumption 3, and all singular values have multiplicity one. Take z to be any vector in $\mathbb{R}^n$. Then for W' given in Def. F.2 we have that $\exists f_W > 0, \exists B_N^*(\epsilon)$ such that $\forall \epsilon > 0$:*

$$\mathbb{P}\left(\|(W' - S)z\|_2 \geq (1 + \epsilon)f_W\|z\|_2\right) < B_N^*(\epsilon), \quad (174)$$

with $B_N^*(\epsilon) \rightarrow 0$ as $N \rightarrow \infty$.

Moreover, assuming that σ_r is the smallest singular values of S , we have:

$$f_W = \max_{1 \leq i \leq r} \sqrt{(g_{\sigma_i}^2 + \sigma_i^2) - 2g_{\sigma_i}\sigma_i\sqrt{g_{u_i}}}, \quad (175)$$

with g_{σ_i} , g_{u_i} and g_{v_i} given in assumption 3.

Remark F.2. Here f_W depends on the distribution of the eigenvalues of R and on the eigenvalues of S .

Example F.7. *Assume that W is the matrix given in Example E.3. Then*

$$f_W = \max_{1 \leq i \leq r} \sqrt{\left(\sigma_i^2 + \left(\frac{1 + \sigma_i^2}{\sigma_i}\right)^2\right) - 2(1 + \sigma_i^2)\sqrt{\left(1 - \frac{1}{\sigma_i^2}\right)}}. \quad (176)$$

Example F.8. Assume that $W = R + S$, with R a $N \times M$ random matrix satisfying assumptions 1' and S a low-rank matrix satisfying assumption 2', for the assumptions in Section 3.1. Assume that the singular values of S are bigger than $\bar{\theta}$, given in (153), and all singular values have multiplicity one. Then for W' given in Def. F.2 we have that:

$$f_W = \max_{1 \leq i \leq r} \sqrt{D_{\mu_R}^{-1} \left(\frac{1}{(\sigma_i)^2} \right)^2 + \sigma_i^2} - 2\sigma_i D_{\mu_R}^{-1} \left(\frac{1}{(\sigma_i)^2} \right) \sqrt{\frac{-2\phi_{\mu_R}(\rho)}{\sigma_r^2 D'_{\mu_R}(\rho)}}.$$

Proof. We prove this for the simple case when $S = \sum_{i=1}^r \sigma_i u_i v_i^T$ and for the example given in E.3, when W , R and S are $N \times N$ matrices. The proof for the more general case is the same.

Take $W' = U_W \Sigma'_W V_W^T$ and $S = U_S \Sigma'_S V_S^T$ to be the SVD of W' and S , with Σ'_S a $N \times N$ matrix with r non-zero singular values σ_i on its diagonal and all other elements zero, and assume the smallest singular value of S , which is σ_r , is bigger than 1. Furthermore, take U_W, V_W to be $N \times N$ unitary matrices, and U_S, V_S to be $N \times N$ padded unitary matrices, as defined in Def F.3, with the "singular vectors" corresponding to the zero singular values σ_i being the zero vector.

If we look at the SVD in terms of its summation representation, changing the singular vectors corresponding to the zero singular values to zero vectors (which are not unit vectors) will not change the reconstructed components of the matrix S . Let's clarify this:

The SVD of a matrix S can be written as:

$$S = \sum_{i=1}^r \sigma_i u_i v_i^T$$

where σ_i are the singular values, u_i are the left singular vectors, and v_i are the right singular vectors, with r being the rank of S . This sum runs over all singular values, including the zero singular values.

In this sum, each term $\sigma_i u_i v_i^T$ contributes to the matrix S . However, for terms where $\sigma_i = 0$, the entire term $\sigma_i u_i v_i^T$ becomes a zero matrix, regardless of what u_i and v_i are. This is because multiplying by zero annihilates any contribution from these vectors.

Therefore, if you change the singular vectors corresponding to the zero singular values (let's say, replacing them with zero vectors), these terms in the sum still contribute nothing to S because they are multiplied by zero. The non-zero singular values and their corresponding vectors still determine the matrix S , and the contributions from the terms with zero singular values remain zero.

In the adapted form of the SVD of S being considered, the matrices U and V do not strictly adhere to the traditional definition of unitary matrices. Instead, they can be described as padded unitary matrices. For the purposes of the proof in question, this "almost" unitary nature of U and V is sufficient.

Then we have

$$\begin{aligned} \|(W' - S)\|_2 &= \|(U_W \Sigma'_W V_W^T - U_S \Sigma'_S V_S^T)\| \\ &= \sqrt{\lambda_{\max}((U_W \Sigma'_W V_W^T - U_S \Sigma'_S V_S^T)^T (U_W \Sigma'_W V_W^T - U_S \Sigma'_S V_S^T))}, \end{aligned} \quad (177)$$

with $\lambda_{\max}(A)$ the largest eigenvalue of A .

We can multiply the right and left of the matrix $(V_W \Sigma_W'^2 V_W^T + V_S \Sigma_S'^2 V_S^T - V_W \Sigma_W' U_W^T U_S \Sigma_S' V_S^T - V_S \Sigma_S' U_S^T U_W \Sigma_W' V_W^T)$ by the unitary matrices V_W and V_W^T respectively, without increasing the absolute value of the max eigenvalue, to obtain:

$$\|(W' - S)\|_2 = \sqrt{\lambda_{\max}(\Sigma_W'^2 + V_W^T V_S \Sigma_S'^2 V_S^T V_W - \Sigma_W' U_W^T U_S \Sigma_S' V_S^T V_W - V_W^T V_S \Sigma_S' U_S^T U_W \Sigma_W').} \quad (178)$$

Structure of $V_W^T V_S \Sigma_S'^2 V_S^T V_W$ and its operator norm

Let $V_W \in \mathbb{R}^{N \times N}$ be a unitary matrix (i.e., $V_W^T V_W = I$), and $V_S \in \mathbb{R}^{N \times N}$ be the right singular vector matrix of the low-rank spike S , with only the first r columns nonzero. Let $\Sigma_S'^2 \in \mathbb{R}^{N \times N}$ be diagonal with entries $\sigma_1^2, \dots, \sigma_r^2$ on the first r diagonal positions and zeros elsewhere.

The matrix V_S has the structure:

$$V_S = \left[\begin{array}{ccc|ccc} | & | & & | & 0 & \cdots & 0 \\ v_1 & v_2 & \cdots & v_r & 0 & \cdots & 0 \\ | & | & & | & 0 & \cdots & 0 \end{array} \right] \in \mathbb{R}^{N \times N}$$

so that only the first r columns are nonzero (orthonormal), and the rest are zero vectors.

Then,

$$\Sigma_S'^2 = \left[\begin{array}{cccc|ccc} \sigma_1^2 & 0 & \cdots & 0 & & & \\ 0 & \sigma_2^2 & \cdots & 0 & & & \\ \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & \cdots & \sigma_r^2 & & & \\ & & & & 0 & \cdots & 0 \\ & & & & \vdots & \ddots & \vdots \\ & & & & 0 & \cdots & 0 \end{array} \right] \in \mathbb{R}^{N \times N}$$

Now consider the matrix:

$$G := V_W^T V_S \Sigma_S'^2 V_S^T V_W \in \mathbb{R}^{N \times N}$$

We proceed step by step.

Structure of $A := V_W^T V_S$.

Since V_W is unitary and V_S has only the first r columns nonzero, A has the following structure:

$$A = \left[\begin{array}{ccc|ccc} a_{11} & \cdots & a_{1r} & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ a_{N1} & \cdots & a_{Nr} & 0 & \cdots & 0 \end{array} \right]$$

Here each entry $a_{ij} = \langle v_i^{(W)}, v_j^{(S)} \rangle$, i.e., the inner product between the i -th right singular vector of W and the j -th right singular vector of S . For fixed r , we have:

Structure of $B := A \Sigma_S'^2$.

We now multiply A on the right by the diagonal matrix $\Sigma_S'^2$. This scales only the first r columns, leading to:

$$B = A \Sigma_S'^2 = \left[\begin{array}{ccc|ccc} a_{11}\sigma_1^2 & \cdots & a_{1r}\sigma_r^2 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ a_{N1}\sigma_1^2 & \cdots & a_{Nr}\sigma_r^2 & 0 & \cdots & 0 \end{array} \right]$$

Structure of $G := A \Sigma_S'^2 A^T$.

$$BA^T = \underbrace{\begin{bmatrix} a_{11}\sigma_1'^2 & \cdots & a_{1r}\sigma_r'^2 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ a_{N1}\sigma_1'^2 & \cdots & a_{Nr}\sigma_r'^2 & 0 & \cdots & 0 \end{bmatrix}}_{N \times N} \underbrace{\begin{bmatrix} a_{11} & \cdots & a_{N1} \\ \vdots & \ddots & \vdots \\ a_{1r} & \cdots & a_{Nr} \\ 0 & \cdots & 0 \\ \vdots & & \vdots \\ 0 & \cdots & 0 \end{bmatrix}}_{N \times N} = \underbrace{\begin{bmatrix} a_{11}\sigma_1'^2 & \cdots & a_{1r}\sigma_r'^2 \\ \vdots & \ddots & \vdots \\ a_{N1}\sigma_1'^2 & \cdots & a_{Nr}\sigma_r'^2 \end{bmatrix}}_{N \times r} \underbrace{\begin{bmatrix} a_{11} & \cdots & a_{N1} \\ \vdots & \ddots & \vdots \\ a_{1r} & \cdots & a_{Nr} \end{bmatrix}}_{r \times N}$$

1. **A is a contraction (not necessarily unitary).** Since V_W is orthogonal and V_S may be padded, we have

$$A^\top A = (V_S^\top V_W)(V_W^\top V_S) = V_S^\top (V_W V_W^\top) V_S = V_S^\top V_S \leq I_N, \quad (179)$$

hence $\|A\|_2 \leq 1$ and $AA^\top \leq I_N$.

2. **Direct spectral-norm bound for $BA^\top$.** Using $\|M\|_2 = \sqrt{\lambda_{\max}(M^\top M)}$ and $B = A\Sigma_S'^2$,

$$\|BA^\top\|_2 = \sqrt{\lambda_{\max}((A\Sigma_S'^2 A^\top)^\top (A\Sigma_S'^2 A^\top))} = \sqrt{\lambda_{\max}(A\Sigma_S'^2 (A^\top A) \Sigma_S'^2 A^\top)}. \quad (180)$$

By (179), $A^\top A \leq I_N$, so

$$\|BA^\top\|_2 \leq \sqrt{\lambda_{\max}(A\Sigma_S'^2 I_N \Sigma_S'^2 A^\top)} = \sqrt{\lambda_{\max}(A\Sigma_S'^4 A^\top)}. \quad (181)$$

Finally, $\lambda_{\max}(AXA^\top) \leq \|A\|_2^2 \|X\|_2$ gives

$$\|BA^\top\|_2 \leq \sqrt{\|A\|_2^2 \|\Sigma_S'^4\|_2} = \|A\|_2 \|\Sigma_S'^2\|_2 \leq \|\Sigma_S'^2\|_2. \quad (182)$$

3. **Explicit diagonal bound.** Since $\Sigma_S'^2 = \text{diag}(\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0)$,

$$\boxed{\|BA^\top\|_2 \leq \|\Sigma_S'^2\|_2 = \max_{1 \leq i \leq r} \sigma_i^2 = \sigma_1^2}. \quad (183)$$

Structure of $\Sigma_W' U_W^\top U_S \Sigma_S' V_S^\top V_W$ and operator norm

1. **Drop the V 's by unitary invariance.** Let

$$T := \Sigma_W' U_W^\top U_S \Sigma_S' V_S^\top V_W. \quad (184)$$

Since $\|V_S^\top V_W\|_2 \leq 1$, the spectral norm is invariant under right multiplication by it:

$$\|T\|_2 \leq \|\Sigma_W' U_W^\top U_S \Sigma_S'\|_2. \quad (185)$$

2. **Block the factors and show the explicit matrix.** Write the diagonal spikes as

$$\Sigma_W' = \begin{bmatrix} D_\rho & 0 \\ 0 & 0 \end{bmatrix}, \quad \Sigma_S' = \begin{bmatrix} D_\sigma & 0 \\ 0 & 0 \end{bmatrix}, \quad D_\rho = \text{diag}(\rho_1, \dots, \rho_r), \quad D_\sigma = \text{diag}(\sigma_1, \dots, \sigma_r), \quad (186)$$

and block

$$H := U_W^\top U_S = \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix}, \quad H_{11} \in \mathbb{R}^{r \times r}, \quad H_{12} \in \mathbb{R}^{r \times (N-r)}, \quad H_{21} \in \mathbb{R}^{(N-r) \times r}, \quad H_{22} \in \mathbb{R}^{(N-r) \times (N-r)}. \quad (187)$$

Then the product in (185) is

$$\Sigma_W' H \Sigma_S' = \begin{bmatrix} D_\rho & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix} \begin{bmatrix} D_\sigma & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} D_\rho H_{11} D_\sigma & 0 \\ 0 & 0 \end{bmatrix}. \quad (188)$$

3. **Reduce the $N \times N$ norm to an $r \times r$ norm.** By (188), the only nonzero block is the top-left $r \times r$ matrix. Therefore

$$\|\Sigma_W' U_W^\top U_S \Sigma_S'\|_2 = \|D_\rho H_{11} D_\sigma\|_2. \quad (189)$$

4. **Structure of H_{11} from singular-vector overlaps.** The entries of H_{11} are the left-singular-vector overlaps

$$(H_{11})_{ij} = \langle u_{W,i}, u_{S,j} \rangle, \quad 1 \leq i, j \leq r. \quad (190)$$

In the spiked regime with fixed r and $N \rightarrow \infty$, we have the diagonal-dominant structure

$$H_{11} = \text{diag}(\alpha_1, \dots, \alpha_r) + E_N, \quad \alpha_i = \langle u_{W,i}, u_{S,i} \rangle, \quad \|E_N\|_2 \rightarrow 0, \quad (191)$$

i.e. the $r(r-1)$ off-diagonal entries vanish (each row/column has a fixed number $r-1$ of vanishing components).

5. **Spectral-norm evaluation/bounds in terms of diagonal entries.** From (189)–(191),

$$\|D_\rho H_{11} D_\sigma\|_2 = \|D_\rho (\text{diag}(\alpha_1, \dots, \alpha_r) + E_N) D_\sigma\|_2 \leq \max_{1 \leq i \leq r} \rho_i \sigma_i |\alpha_i| + \|D_\rho\|_2 \|E_N\|_2 \|D_\sigma\|_2. \quad (192)$$

Consequently,

$$\boxed{\|\Sigma'_W U_W^\top U_S \Sigma'_S V_S^\top V_W\|_2 = \|D_\rho H_{11} D_\sigma\|_2 \xrightarrow{N \rightarrow \infty} \max_{1 \leq i \leq r} \rho_i \sigma_i |\alpha_i|,} \quad \text{by (185), (189), (191).} \quad (193)$$

In particular, under strong alignment ($\alpha_i \rightarrow 1$) and outlier matching ($\rho_i \rightarrow \sigma_i$),

$$\|\Sigma'_W U_W^\top U_S \Sigma'_S V_S^\top V_W\|_2 \longrightarrow \max_{1 \leq i \leq r} \sigma_i^2. \quad (194)$$

Extension to G^* . Each term in

$$G^* = \Sigma_W'^2 + V_W^T V_S \Sigma_S'^2 V_S^T V_W - \Sigma_W' U_W^T U_S \Sigma_S' V_S^T V_W - V_W^T V_S \Sigma_S' U_S^T U_W \Sigma_W'$$

goes to the diagonal matrices $\Sigma_W'^2, \Sigma_S'^2, \Sigma_W' \Sigma_S' D_1, \Sigma_W' \Sigma_S' D_1$ in the operator norm, respectively. Here, D_1 contains the elements on its diagonal. Thus,

$$\|G^* - (\Sigma_W'^2 + \Sigma_S'^2 - 2\Sigma_W' \Sigma_S' D_1)\|_2 \xrightarrow{\text{a.s.}} 0,$$

so, for any $\epsilon > 0$, there exists $B_N^*(\epsilon) \rightarrow 0$ with

$$\mathbb{P}(|\lambda_{\max}(G^*) - \lambda_{\max}(\Sigma_W'^2 + \Sigma_S'^2 - \Sigma_W' \Sigma_S' D_1 - \Sigma_W' \Sigma_S' D_2)| > \epsilon) < B_N^*(\epsilon).$$

Final bound on $\|W' - S\|_2$. Because

$$\|W' - S\|_2 = \sqrt{\lambda_{\max}(G^*)},$$

we obtain (using your explicit closed form for the diagonal expression)

$$\mathbb{P}\left(\|W' - S\|_2 \geq (1 + \epsilon) \max_{1 \leq i \leq r} \sqrt{\left(\sigma_i^2 + \left(\frac{1 + \sigma_i^2}{\sigma_i}\right)^2\right) - 2(1 + \sigma_i^2) \sqrt{\left(1 - \frac{1}{\sigma_i^2}\right)}} \leq B_N^*(\epsilon)\right).$$

This is because as $N \rightarrow \infty$, we have that W' only has r non-zero singular values remaining, and, by Lemma F.4, they are all the singular values corresponding to the singular values of S . Given that all the singular values of R go to λ_+ , none of the singular values of R remain.

In fact, using the above argument, we can show that as $N \rightarrow \infty$:

$$\|(W' - S)\| \leq^{a.s.} \max_{1 \leq i \leq r} \sqrt{\left(\sigma_i^2 + \left(\frac{1 + \sigma_i^2}{\sigma_i}\right)^2\right) - 2(1 + \sigma_i^2) \sqrt{\left(1 - \frac{1}{\sigma_i^2}\right)}}. \quad (195)$$

This completes the proof. $\square$

F.1 Numerics for Example 3.15

In the following subsection, we provide a figure of the dot products of the 5 left and right singular values for the matrices $W = R + S$ and S described in Example 3.15, see Fig 45. As mentioned earlier, the 5 singular values of S were 30, 40, 50, 60, 70. We see that the dot product of the left and right singular vectors of W and S can be approximated almost perfectly by the equation $\sqrt{1 - \frac{1}{\sigma_i^2}}$, see (159). That is, for $\sigma_5 = 30$, we have $\sqrt{1 - \frac{1}{\sigma_i^2}} \approx 0.99944$ and indeed $\langle u_5, u'_5 \rangle \approx 0.99943$ and similarly for the other dot products.

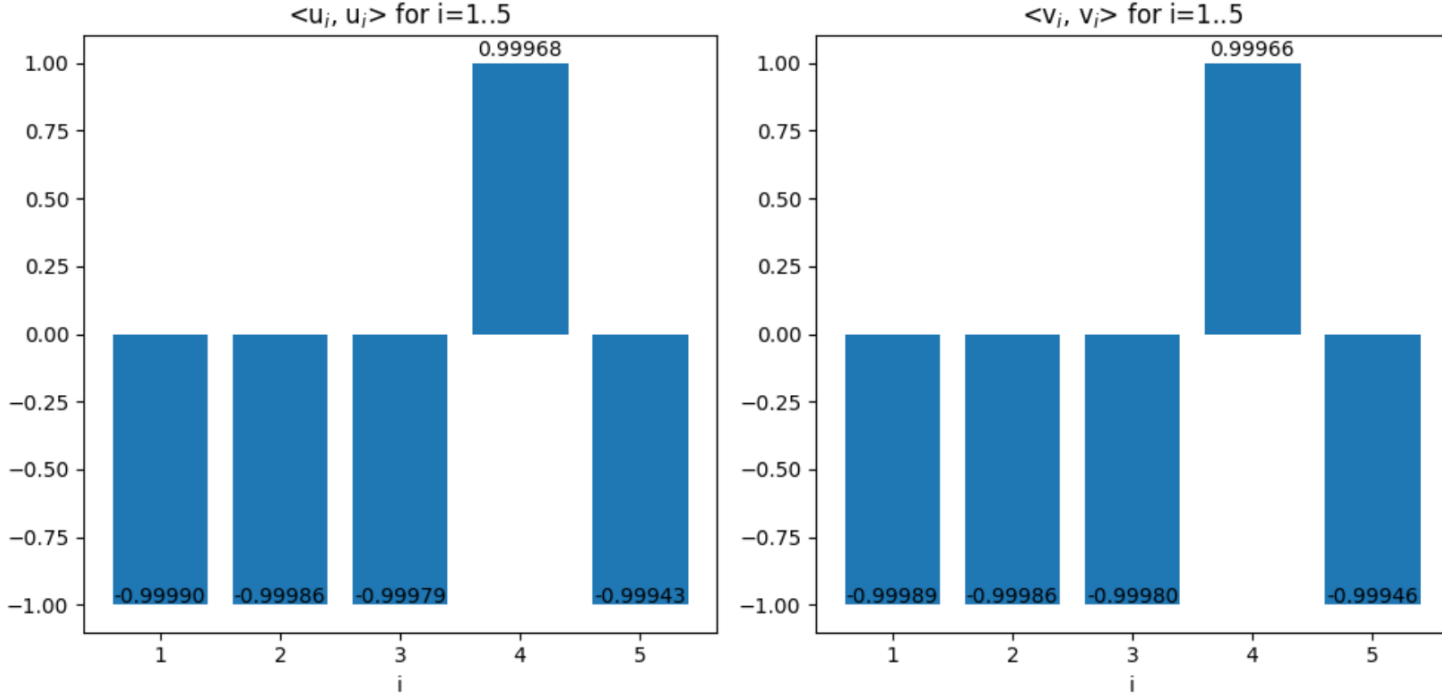


Figure 45: Left: Dot product of the left singular vectors. Right: Dot product of the right singular vectors.

F.2 Details for Example 3.16

The details provided in the subsection were taken from [6]. Under the setting given in Example 3.16, let u'_i be a unit-norm eigenvector of $R + S$ associated with its r largest eigenvalues. We have for $1 \leq i \leq r$,

$$\lambda_i(R + S) \xrightarrow{\text{a.s.}} \begin{cases} \sigma_i + \frac{\sigma^2}{\sigma_i} & \text{if } \sigma_i > \sigma, \\ 2\sigma & \text{otherwise,} \end{cases}$$

as $n \rightarrow \infty$.

We also have

$$|\langle u, u' \rangle|^2 \xrightarrow{\text{a.s.}} \begin{cases} 1 - \frac{\sigma^2}{\sigma_i^2} & \text{if } \sigma_i > \sigma, \\ 0 & \text{otherwise.} \end{cases}$$

G Proof for Pruning Theorem 2.7

The proof of the Pruning Theorem 2.7 consists of two steps. First, we show how classification confidence changes when the weight matrix W_b is replaced with its low-rank part S_b . Second, we approximate

the low-rank matrix S_b with the pruned matrix W'_b , keeping track of the corresponding change in the classification confidence. The key idea in the second step is to use asymptotics of the spectrum of deformed matrices $W = R + S$ with a finite number of singular values of S , see [7], in combination with GCT.

Proof for the Pruning Theorem 2.7:

Proof. We first provide a proof for the case when our DNN has only one layer matrix W .

By Lemma 3.13 we have that there $\exists D(N)$ such that for

$$E' := a(N)h_\phi(s, b) \quad (196)$$

we have the conditional probability

$$\mathbb{P}(\delta X(s, \alpha_S) \geq 0 \mid \delta X(s, \alpha_W) \geq E') \geq 1 - D(N), \quad (197)$$

with $D(N), a(N) \rightarrow 0$ as $N \rightarrow \infty$ and $h_\phi(s, b)$ coming from (47). Again, α_S are the parameters of the DNN with the weight matrix S , and α_W are the parameters of the DNN with the weight layer matrix W .

We then use Approximation Lemma F.6 to obtain that $\exists f_W, \exists B_N^*(\epsilon)$ such that $\forall \epsilon$:

$$\mathbb{P}(\|(W' - S)z\|_2 > (1 + \epsilon)f_W\|z\|_2 < B_N^*(\epsilon), \quad (198)$$

with $B_N^*(\epsilon) \rightarrow 0$ as $N \rightarrow \infty$.

We then follow an argument similar to that given for Lemma I.1. That is, the change in classification confidence due to pruning is:

$$\Delta(\delta X) = |\delta X(s, \alpha_S) - \delta X(s, \alpha_{W'})|$$

For a particular component i , the change ΔX_i due to pruning is given by:

$$\Delta X_i = |X_i(s, \alpha_S) - X_i(s, \alpha_{W'})|$$

We have that $X(s, \alpha_S) = \lambda \circ (S)s$ and $X(s, \alpha_{W'}) = \lambda \circ (W')s$.

Given that λ is the absolute value activation function, for any scalar values x and y , we have:

$$|\lambda(x) - \lambda(y)| \leq |x - y|. \quad (199)$$

Thus, from (198) we have $\|X(s, \alpha_S) - X(s, \alpha_{W'})\| \leq \|(S - W')s\| \leq (1 + \epsilon)f_W\|s\|_2$, with probability $1 - B_N^*(\epsilon)$. Furthermore,

$$\Delta X_{\max} + \Delta X_{\max-1} \leq \sqrt{2}(1 + \epsilon)f_W\|s\|_2, \quad (200)$$

with probability $1 - B_N^*(\epsilon)$. Here $\Delta X_{\max-1}$ is the change in the component of X which has the second to biggest change, and $\Delta X_{\max}$ is the change in the component of X which had the biggest change.

Then, using the same steps given in Lemma I.1, we obtain:

$$\Delta(\delta X) \leq \Delta X_{\max} + \Delta X_{\max-1} \leq \sqrt{2}(1 + \epsilon)f_W\|s\|_2, \quad (201)$$

with probability $1 - B_N^*(\epsilon)$.

Thus, given that $\exists D(N) \rightarrow 0$ as $N \rightarrow \infty$ such that:

$$\mathbb{P}(\delta X(s, \alpha_S) \geq 0 \mid \delta X(s, \alpha_W) \geq E') \geq 1 - D(N), \quad (202)$$

and

$$\mathbb{P}(\delta X(s, \alpha_{W'}) \geq 0 \mid \delta X(s, \alpha_S) \geq \sqrt{2}(1 + \epsilon)f_W \|s\|_2) \geq 1 - D(N), \quad (203)$$

we have that there exists an explicit function $f_W > 0$ such that there $\exists C(N)$ such that for

$$E := (\sqrt{2} \min\{f_W, \sqrt{\lambda_+}\} g_\phi(s, b) + a(N)h_\phi(s, b) \quad (204)$$

we have the conditional probability

$$\mathbb{P}(\delta X(s, \alpha_{W'}) \geq 0 \mid \delta X(s, \alpha_W) > E) \geq 1 - C(N), \quad (205)$$

with $C(N), a(N) \rightarrow 0$ as $N \rightarrow \infty$ and $g_\phi(s, b)$ coming from (45). f_W is given in Lemma F.6.

When the DNN has more than one layer, we continue this proof with the same steps found in Subsection I.1. $\square$

H Other algorithms required for implementing RMT-SVD based pruning of DNN

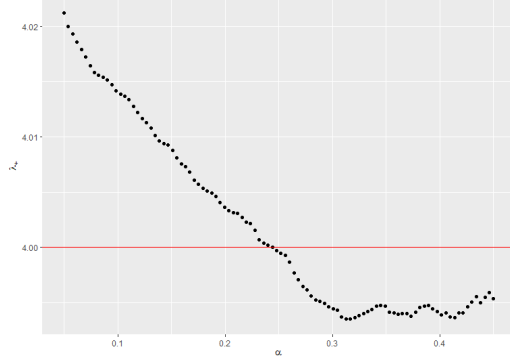
H.1 BEMA algorithm for finding λ_+

The following is the BEMA algorithm for finding the best fit λ_+ of $\frac{1}{N}R^T R$ based on the ESD of $X = R + S$. It is used in the analysis of matrices with the information plus noise structure (i.e., deformed matrices), where one wants to determine the rightmost edge of the compact support of the MP distribution. In this context, the Tracy-Widom distribution provides the limiting distribution of the largest eigenvalue λ_+ of large random matrices, allowing us to compute a confidence interval for λ_+ in the presence of the Marčenko-Pastur distribution, see [31]. The BEMA algorithm is computationally efficient and has been shown to provide accurate results for matrices with the information plus noise structure. More details on the algorithm and its relationship with the MP and Tracy-Widom distributions can be found in [31]. Here, we present a simplified version of the algorithm for R a $N \times N$ matrix:

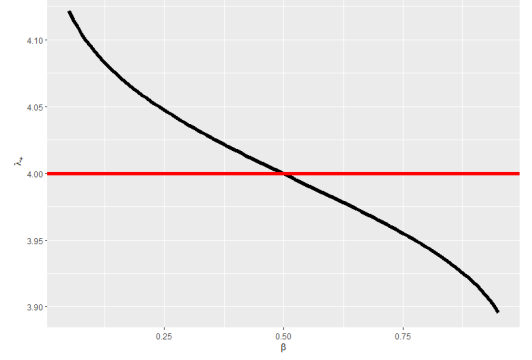
Algorithm 1 Computation of λ_+ using MP and Tracy-Widom Distributions

- 1: Choose parameters $\alpha \in (0, 1/2), \beta \in (0, 1)$.
 - 2: **for** each $\alpha N \leq k \leq (1 - \alpha)N$ **do**
 - 3: Obtain q_k , the (k/N) upper-quantile of the MP distribution with $\sigma^2 = 1$ and $c = 1$. $\triangleright$ Each q_k is a solution to $\int_0^{q_k} \frac{1}{2\pi} \frac{\sqrt{(4-\lambda)\lambda}}{\lambda} = k/N$
 - 4: **end for**
 - 5: Compute $\hat{\sigma}^2 = \frac{\sum_{\alpha N \leq k \leq (1-\alpha)N} q_k \lambda_k}{\sum_{\alpha N \leq k \leq (1-\alpha)N} q_k^2}$. $\triangleright$ where λ_k is the k^{th} smallest eigenvalue of X
 - 6: Obtain $t_{1-\beta}$, the $(1 - \beta)$ quantile of Tracy-Widom distribution.
 - 7: Return $\lambda_+ = \hat{\sigma}^2[4 + 2^{4/3}t_{1-\beta} \cdot N^{-2/3}]$.
-

Remark H.1. The algorithm depends on parameters $\alpha \in (0, 1/2), \beta \in (0, 1)$. We show this by varying α and β for the case found in Example H.1. See Fig. 46a and 46b. The red line is $\lambda_+ = 4$, which is the correct λ_+ of $\frac{1}{N}R^T R$. In this example, while dependence on α is insignificant for sufficiently large values, dependence on β allows us to control the confidence that the eigenvalues of the random matrix R will be smaller than the estimator for λ_+ of the MP distribution. In all the numerical simulations given in Section A, we took $\alpha = .25$ and $\beta = .1$. It would be interesting to try and see what happens when we take a larger β , as it would prevent the algorithm from pruning too many parameters but might lead to even higher accuracy.



(a) Dependence of algorithm the choice of α , $\beta = 0.5$. In this example, the rank of the low-rank matrix S is fairly low.



(b) Dependence of algorithm on the choice of β , $\alpha = 0.25$.

In the numeric portion of the paper, we always divide $R^T R$ by $\frac{1}{N}$ when obtaining the ESD of X regardless of the original distribution of the initial random matrix $R(0)$. If $R(o)$ is distributed using $N(0, \frac{1}{N})$, dividing $R^T R$ by N does not seem to change the fact that the ESD of X is given by the MP distribution.

Example H.1. In this example, we create a random $N \times N$ matrix R with components taken from i.i.ds using the normal distribution of zero mean and unit variance ($\sigma^2 = 1$). We take S to be a $N \times N$ low-rank matrix with components given by

$$S[i, j] = \tan\left(\frac{\pi}{2} + \frac{1}{j+1}\right) + \cos(i) \cdot \log(i+j+1) + \sin(j) \cdot \cos\left(\frac{i}{j}\right), \quad (206)$$

$W = R + S$ and $X = \frac{1}{N} W^T W$. The BEMA algorithm is used to find the λ_+ of the ESD of X , as described in Subsection H.1. R is a random matrix satisfying the conditions of Theorem 1.2, and so the ESD of $\frac{1}{N} R^T R$ converges to the Marchenko-Pastur distribution as $N \rightarrow \infty$ and has a λ_+ that determines the rightmost edge of its compact support. We can imagine a situation in which R is not directly known, and the goal is to find an estimator of λ_+ from the ESD of X . See Fig. 47 for the result of the ESD of X with the Marchenko-Pastur distribution that best fits the ESD shown in red.

The bulk of the eigenvalues are well-fit by the MP distribution, but some eigenvalues bleed out to the right of λ_+ . These eigenvalues correspond to the singular values of S . The direct calculation of the λ_+ of the MP distribution corresponding to $\frac{1}{N} R^T R$ gives $\lambda_+ = \sigma^2 \cdot (1+1)^2 = 4$, and the λ_+ obtained to fit the bulk of the ESD of X and the λ_+ of $\frac{1}{N} R^T R$ are approximately the same.

The BEMA algorithm will be employed to estimate λ_+ from the ESD of $X_l(t)$. As the DNN training progresses, it is expected that the majority of the eigenvalues of $X_l(t)$ will conform to the MP distribution. Nonetheless, some eigenvalues may extend beyond the bulk of the MP distribution and be associated with the singular values of $S_l(t)$. The purpose of the BEMA algorithm is to identify the furthest edge of the MP distribution, which helps determine the value of λ_+ . Understanding λ_+ is crucial as it offers insights into the DNN's behavior during training and its capacity to generalize to new data.

In combination with the SVD, the BEMA algorithm can be applied to decide which singular values of the DNN's weight matrices W_l should be eliminated during training. The SVD breaks down the weight matrix into its singular values and singular vectors, allowing for an RMT-based analysis of their distribution. Utilizing the BEMA algorithm, one can pinpoint the eigenvalues associated with the singular values of S_l and differentiate them from the eigenvalues related to the singular values of R_l . By removing the eigenvalues corresponding to R_l , the DNN's training process can be made more effective and efficient.

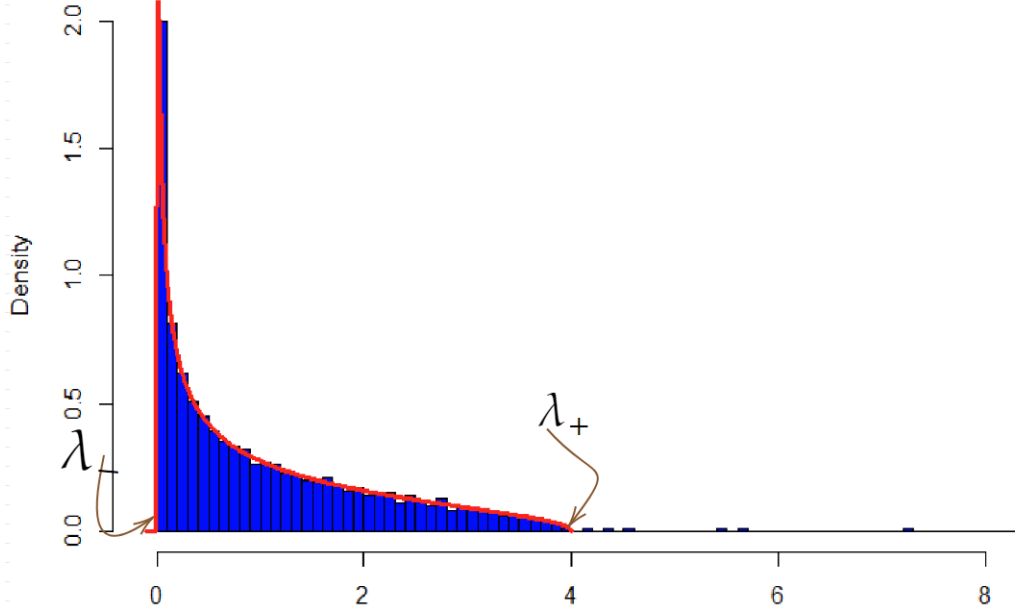


Figure 47: In blue, we have the ESD of X ; in red, the Marchenko-Pastur distribution, which best fits the ESD based on the BEMA algorithm.

H.2 The role of singular value decomposition in deep learning

Consider a $N \times M$ matrix A . A singular value decomposition (SVD) of A consists of a factorization $A = U\Sigma V^T$, where:

- U is an $N \times N$ orthogonal matrix.
- V is an $M \times M$ orthogonal matrix.
- Σ is an $N \times M$ matrix with the i th diagonal entry equal to the i th singular value σ_i and all other entries of Σ being zero.

For λ_i , the eigenvalues of a matrix $X = W^T W$, the singular values of W are given by $\sigma_i = \sqrt{\lambda_i}$. Consequently, singular values are connected to eigenvalues of the symmetrization of a matrix W .

For a DNN's W_l , it has been demonstrated that discarding small singular values of W_l through its SVD during the DNN's training can decrease the number of parameters while improving accuracy, as shown in [70, 69, 11, 2]. In the remainder of this work, we illustrate how RMT can aid in identifying the singular values to be removed from a DNN layer without compromising the DNN's accuracy.

In particular, the BEMA algorithm can be combined with the SVD of W_l to ascertain which singular values should be removed during the DNN's training. To achieve this, one first computes the SVD of W_l and then calculates the eigenvalues of the symmetrized matrix $X_l = \frac{1}{N} W_l^T W_l$. The eigenvalues derived from the symmetrization can be linked to the singular values of W_l through $N\lambda_i = \sigma_i^2$. Employing the BEMA algorithm to estimate the value of λ_+ allows for the determination of a threshold for the singular values of W_l . Singular values below the threshold can be removed without impacting the DNN's accuracy, as they are likely less crucial for the DNN's performance. This process can be carried out iteratively during the DNN's training since the threshold can be updated as training advances.

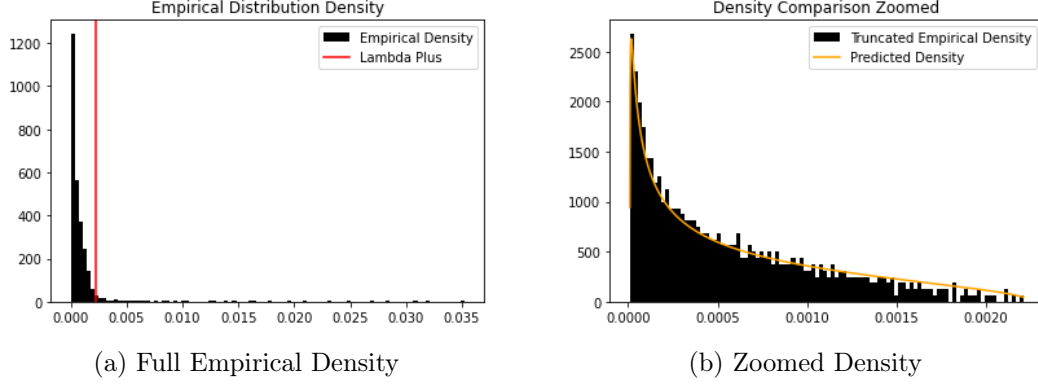


Figure 48: The ESD of X_l and its best fit MP distribution

H.3 Eliminating singular values while preserving accuracy

In this subsection, we demonstrate how SVD can be employed to remove the random components of W_l without compromising accuracy. This could potentially lead to a significant reduction in the number of parameters in the DNN, resulting in faster training.

Algorithm 2 Pruning a Weight Matrix from a Trained DNN

- 1: Acquire a weight matrix W_l from a trained DNN.
 - 2: Perform SVD on W_l : $W_l = U\Sigma V^T$.
 - 3: Calculate the eigenvalues λ_i of the square matrix $\frac{1}{N}W_l^T W_l$.
 - 4: Apply the BEMA algorithm from Subsection H.1 to find the best fit MP distribution for the ESD of $X = \frac{1}{N}W_l^T W_l$ and its corresponding λ_+ . ▷ See Fig. 48
 - 5: Determine whether the ESD of X fits the MP distribution using the algorithm in Subsection H.4. ▷ Ensures $W_l = R + S$ assumption is valid
 - 6: Replace a portion, e.g., 0.1, of the singular values less than $\sqrt{\lambda_+ N}$ with zeros to form a new diagonal matrix Σ' and the truncated matrix W'_l .
 - 7: Use Σ' to obtain $W'_{1,l} = U\sqrt{\Sigma'}$ and $W'_{2,l} = \sqrt{\Sigma'}V^T$.
-

Example H.2. Consider an original DNN with two hidden layers, each consisting of 10 nodes. The total number of parameters in this case would be 100. By employing SVD and removing 8 small singular values in the weight layer matrix of this DNN, we can split the hidden layer into two, resulting in a new DNN with three hidden layers. The first layer will have 10 nodes, the second layer will have 2 nodes, and the third layer will have 10 nodes. By keeping only two singular values in the SVD, we now have only 20 parameters; see Figure 49. In practice, we don't actually split the layer.

Example H.3. We used the above approach for a DNN trained on MNIST. In this example, the DNN has two layers, the first with a 784×1000 matrix W_1 and the second with a 1000×10 matrix W_2 . The activation function was ReLU. We trained the DNN for 10 epochs and achieved a 98% accuracy on the test set.

We perform an SVD on W_1 , in this case Σ is a 784×1000 matrix. Even if we only keep the biggest 60 σ_i of W_1 and transform the first layer into two layers $W_{1,1}$ and $W_{2,1}$ the accuracy is still 98%. W_1 had 784,000 parameters, while $W_{1,1}$ and $W_{2,1}$ have $784(60) + 1,000(60) = 107,040$ parameters (not including the bias vector parameters). This is a reduction of over 85%. In Fig. 50 we show how the accuracy of the DNN depends on the number of singular values that we keep. The red line corresponds to the threshold given by the MP distribution (via λ_+) for how many of the large singular values should

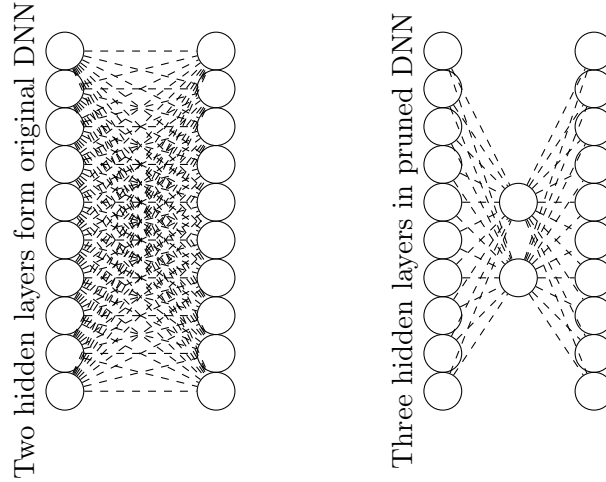


Figure 49: Two hidden layers from the original, in the right figure, have 10 nodes (total 100 parameters). Layers are transformed into three hidden layers, in the left figure, in pruned DNN. The first layer has 10 nodes, the second layer has 2 nodes (keeping only two singular values in the SVD), and the third layer has 10 nodes, resulting in a total of 20 parameters.

be kept. As the figure shows, this threshold is highly accurate. This example also numerically confirms Theorem 2.7 and shows that the threshold given in the theorems (for which singular values to keep) is highly useful and accurate.

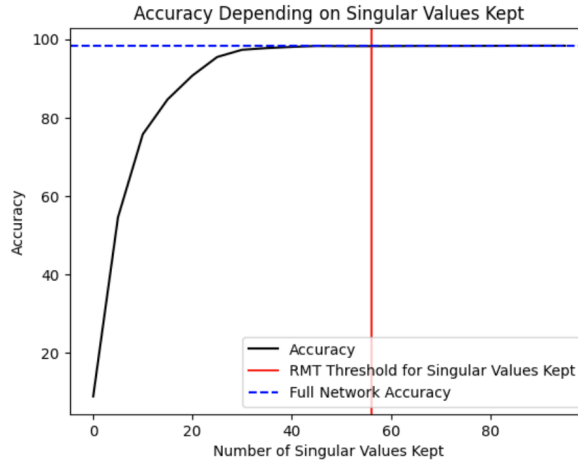


Figure 50: Number of eigenvalues kept is shown on the x -axis while the accuracy is shown on the y -axis.

H.4 MP fit Criteria: Checking if the ESD of X fits a MP distribution

This subsection details a procedure to evaluate whether the ESD of a square matrix X is possibly drawn from a specific MP distribution (with a possibility of spiked eigenvalues). The initial step of this procedure relies on the BEMA method to identify the most fitting MP distribution. This optimal fitting distribution provides a theoretical cumulative distribution function (CDF), while the empirical cumulative spectral distribution related to X can be computed. A comparison of these two distributions allows us to dismiss the hypothesis that X follows the suggested MP distribution if the difference between the distributions is substantial. Let's formalize these concepts, starting with the

concept of an empirical cumulative spectral distribution.

Definition H.4. Assume G is an $N \times M$ matrix and its ESD μ_{G_M} is defined as in Definition 1.1. The empirical cumulative spectral distribution of G , symbolized as $F_G : \mathbb{R} \rightarrow \mathbb{R}$, is defined as:

$$F_G(a) = \mu_{G_m}((-\infty, a]) \quad (207)$$

Interestingly, the cumulative distribution functions for the MP distribution are known and can be expressed in a closed form. With these equations, we can comprehensively describe our procedure. We set a tuning parameter $\gamma \in (0, 1)$ corresponding to the sensitivity of our test.

Algorithm 3 How well does the ESD of X fit the MP Distribution with Spikes

- 1: Accept $X = \frac{1}{N}W^TW$ as input, where W is an $N \times M$ matrix.
 - 2: Calculate the spectrum of $X = \{\sigma_1, \dots, \sigma_M\}$.
 - 3: Compute the empirical cumulative spectral distribution of X , denoted F_X .
 - 4: Execute the BEMA method with parameters α and β to determine $\hat{\sigma}^2$, the anticipated variance of each coordinate of W .
 - 5: Calculate $0 \leq i_{\text{low}} < i_{\text{high}} \leq M$ such that i_{low} is the smallest integer with $\frac{i_{\text{low}}}{M} \geq \alpha$ and i_{high} is the largest integer with $\frac{i_{\text{high}}}{M} \leq 1 - \alpha$.
 - 6: Define F'_X as the theoretical cumulative distribution function for the MP distribution with parameters $\hat{\sigma}^2$ and $\lambda = N/M$.
 - 7: Evaluate $s = \max_{i \in [i_{\text{low}}, i_{\text{high}}]} |F_X(i) - F'_X(i)|$.
 - 8: **if** $s > \gamma$ **then**
 - 9: Dismiss the hypothesis that X follows the proposed distribution.
 - 10: **else**
 - 11: Do not reject this hypothesis.
 - 12: **end if**
-

This procedure computes the maximum difference between the expected and empirical cumulative distribution functions by sampling at each point in the empirical distribution. Since this is intended for the unique case of testing for spiked MP distributions, we can utilize this information to enhance our test over simply calculating the L^∞ difference between the expected and empirical distributions.

This enhancement is reflected in the step which calculates i_{low} and i_{high} . As BEMA only uses data in the quantile between $(\alpha, 1 - \alpha)$ to find the best fit, it is logical to only examine for fit within the same range. In this context, we would expect a spiked MP distribution to be poorly approximated by its generative MP distribution around the highest eigenvalues (i.e., the spiked values), and hence it makes sense to only test the bulk values for goodness of fit.

H.5 The MP and spike metrics of a DNN model

First, we present two figures of the ESD of different layers of the base ViT model studied in Subsection 4, see Figs. 51, 52. We see that some layers fit the MP distribution "very" well, while some layers don't. When pruning layers of the DNN, we remove larger parameters in the layers that are more random because of the larger amount of randomness in those layers.

Second, we show how the hyperparameter α discussed in Remark H.1 affects the MP fit and spike metrics; see Fig. 53.

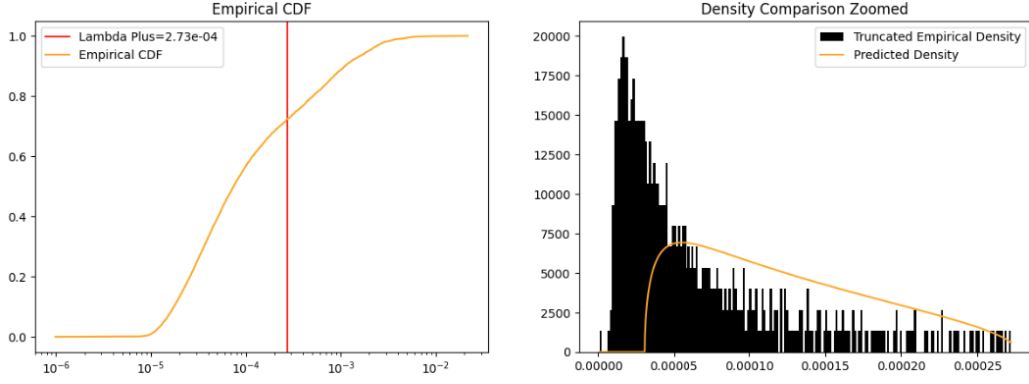


Figure 51: Best fit MP distribution (on the left) for $\alpha = .25$ and $\beta = .8$ for a layer of the base ViT model. Here the MP fit error is .74 while the percentage of singular values smaller than $\sqrt{\lambda_+}$ is 73%. For more on the right figure, see Section B.

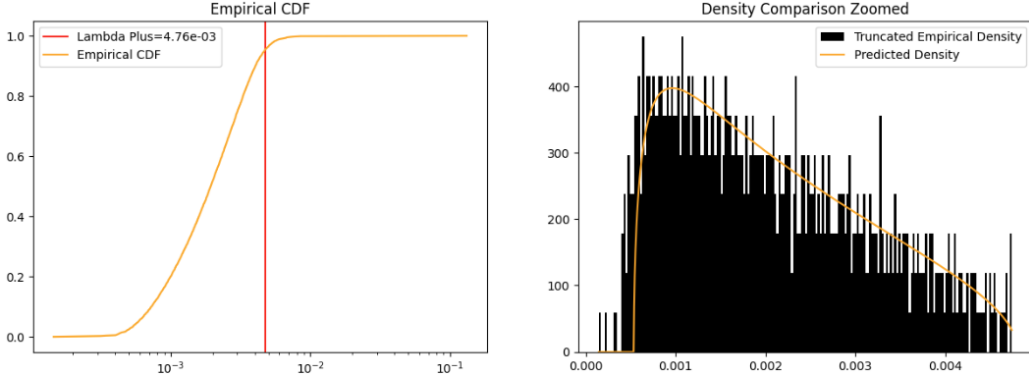


Figure 52: Best fit MP distribution (on the left) for $\alpha = .25$ and $\beta = .8$ for a layer of the base ViT model. Here the MP fit error is .01 while the percentage of singular values smaller than $\sqrt{\lambda_+}$ is 99.73%. For more on the right figure, see Section B.

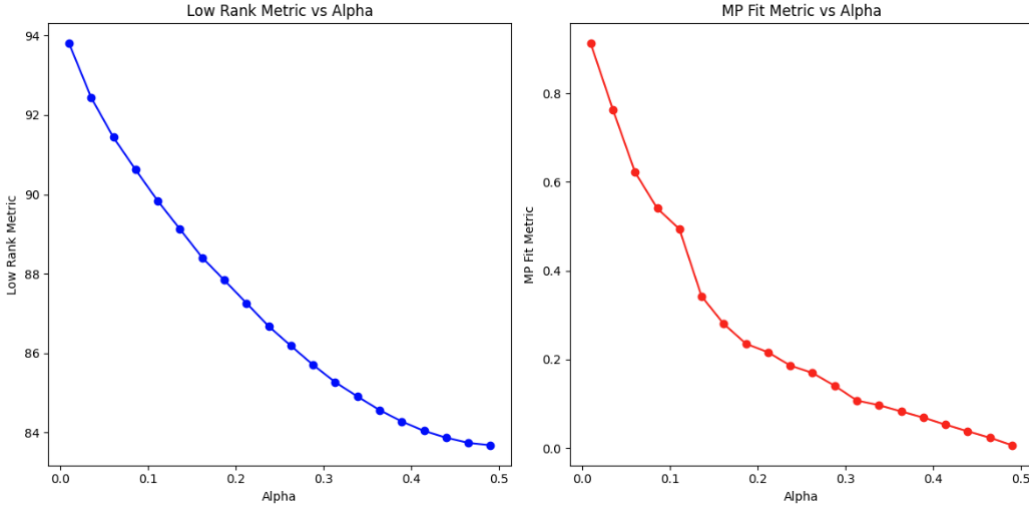


Figure 53: In the left figure, we have α vs LRM and on the right we have α vs MP fit metric. This is for the base ViT model studied in 7.

I Some of the proofs and numerics

I.1 Proof of Lemma 3.11

Proof. The classification confidence before pruning is:

$$\delta X(s, \alpha_{W_b}) = X_{i(s)}(s, \alpha_{W_b}) - \max_{j \neq i(s)} X_j(s, \alpha_{W_b})$$

After pruning, it is:

$$\delta X(s, \alpha_{W'_b}) = X_{i(s)}(s, \alpha_{W'_b}) - \max_{j \neq i(s)} X_j(s, \alpha_{W'_b})$$

For simplicity, we will start by proving the theorem for the case of a DNN with only one layer matrix W and a bias vector β . Thus, we take $X(s, \alpha_{W_b}) = X(s, \alpha_W)$ and $X(s, \alpha_{W'_b}) = X(s, \alpha_{W'})$.

Then, the change in classification confidence due to pruning is:

$$\Delta(\delta X) = |\delta X(s, \alpha_W) - \delta X(s, \alpha_{W'})|$$

For a particular component i , the change ΔX_i due to pruning is given by:

$$\Delta X_i = |X_i(s, \alpha_W) - X_i(s, \alpha_{W'})|$$

We have that $X(s, \alpha_W) = \lambda \circ (W + \beta)s$ and $X(s, \alpha_{W'}) = \lambda \circ (W' + \beta)s$.

Given that λ is the absolute value activation function, for any scalar values x and y , we have:

$$|\lambda(x) - \lambda(y)| \leq |x - y|. \quad (208)$$

Thus $\|X(s, \alpha_W) - X(s, \alpha_{W'})\| \leq \|(W - W')s\| \leq \sqrt{\lambda_+} \|s\|_2$. If the change in the norm of the entire output vector due to pruning is at most $\sqrt{\lambda_+} \|s\|_2$, then the maximum change in any individual component must also be bounded by that amount. That is:

$$\Delta X_{\max} \leq \sqrt{\lambda_+} \|s\|_2.$$

Furthermore,

$$\Delta X_{\max} + \Delta X_{\max-1} \leq \sqrt{2\lambda_+} \|s\|_2, \quad (209)$$

with $\Delta X_{\max-1}$ the change in the component of X which has the second to biggest change and $\Delta X_{\max}$ the change component of X which had the biggest change.

Now assume $\max_{j \neq i(s)} X_j(s, \alpha_{W'}) \leq \max_{j \neq i(s)} X_j(s, \alpha_W)$, then given that $|X_{i(s)}(s, \alpha_W) - X_{i(s)}(s, \alpha_{W'})| \leq \sqrt{\lambda_+} \|s\|_2$, we must have $\delta X(s, \alpha_{W'}) = X_{i(s)}(s, \alpha_{W'}) - \max_{j \neq i(s)} X_j(s, \alpha_{W'}) \leq \sqrt{\lambda_+} \|s\|_2$.

Next, assume $\max_{j \neq i(s)} X_j(s, \alpha_{W'}) \geq \max_{j \neq i(s)} X_j(s, \alpha_W)$. Let's expand the change in δX due to pruning:

$$\begin{aligned} \Delta(\delta X) &= |(X_{i(s)}(s, \alpha_W) - \max_{j \neq i(s)} X_j(s, \alpha_W)) - (X_{i(s)}(s, \alpha_{W'}) - \max_{j \neq i(s)} X_j(s, \alpha_{W'}))| \\ &\leq |X_{i(s)}(s, \alpha_W) - X_{i(s)}(s, \alpha_{W'})| + |\max_{j \neq i(s)} X_j(s, \alpha_W) - \max_{j \neq i(s)} X_j(s, \alpha_{W'})|. \end{aligned}$$

Take k^* to be an integer such that $\max_{j \neq i(s)} X_j(s, \alpha_{W'}) = X_{k^*}(s, \alpha_{W'})$, we have, given that $\max_{j \neq i(s)} X_j(s, \alpha_{W'}) \geq \max_{j \neq i(s)} X_j(s, \alpha_W)$, that:

$$\begin{aligned} &|X_{i(s)}(s, \alpha_W) - X_{i(s)}(s, \alpha_{W'})| + |\max_{j \neq i(s)} X_j(s, \alpha_W) - \max_{j \neq i(s)} X_j(s, \alpha_{W'})| \\ &\leq |X_{i(s)}(s, \alpha_W) - X_{i(s)}(s, \alpha_{W'})| + |X_{k^*}(s, \alpha_{W'}) - X_{k^*}(s, \alpha_W)|. \end{aligned} \quad (210)$$

Consider the worst-case scenario where the change is maximally concentrated in the components $i(s)$ and k^* . Thus, given (209) we have:

$$\Delta(\delta X) \leq \Delta X_{\max} + \Delta X_{\max-1} \leq \sqrt{2\lambda_+} \|s\|_2. \quad (211)$$

Next, we consider the case where the DNN has multiple layers given by the matrices $W_1, \dots, W_L$, and bias vectors $\beta_1, \dots, \beta_L$ and we prune the last matrix layer of the DNN, i.e. we prune layer matrix W_L to obtain W'_L . In this case, we can reduce this problem to that of a DNN with a single layer matrix W_L and a single bias vector β_L and an input vector $z = \lambda \circ W_{L-1} \circ \dots \circ \lambda \circ W_1 s$. Then, by the above argument, we have:

$$\Delta(\delta X) \leq \sqrt{2\lambda_+} \|\lambda \circ (W_{L-1} + \beta_{L-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1) s\|_2. \quad (212)$$

Finally, we assume that we prune a layer W_b to obtain W'_b . By a similar argument to what we have above, we see that the max change of a component of the output vector $X(s, \alpha_{W_b})$ after pruning is:

$$\Delta X_{\max} \leq \sqrt{\lambda_+} \|\lambda \circ (W_{b-1} + \beta_{b-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1) s\|_2 \sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L),$$

given that $\|\lambda \circ (Az + \beta) - \lambda \circ (Az' + \beta)\|_2 \leq \sigma_{\max}(A) \|z' - z\|_2$.

Again, we obtain,

$$\Delta X_{\max} + \Delta X_{\max-1} \leq \sqrt{2\lambda_+} \|\lambda \circ (W_{b-1} + \beta_{b-1}) \circ \dots \circ \lambda \circ (W_1 + \beta_1) s\|_2 \sigma_{\max}(W_{b+1}) \dots \sigma_{\max}(W_L). \quad (213)$$

Thus, this completes the proof of the lemma. $\square$

I.2 Effectiveness of MP-based pruning for different initialization methods

In this subsection, we investigated the performance of various initialization methods, including He (see [27]) and Xavier (see [22]) initialization. It is important to note that both the He and Xavier initializations align with the principles of the MP theorem given in Theorem 2.9. That is, the weight layer components are i.i.ds with mean zero and bounded variance. In practice, for the weight layer matrices W initialized based on those distributions, we have that the ESD of $W^T W$ fits the MP distribution well (with an error, see Subsection H.4, of $\sim .001$ for both). Thus, the Pruning Theorem 2.9 would hold for both of these initializations.

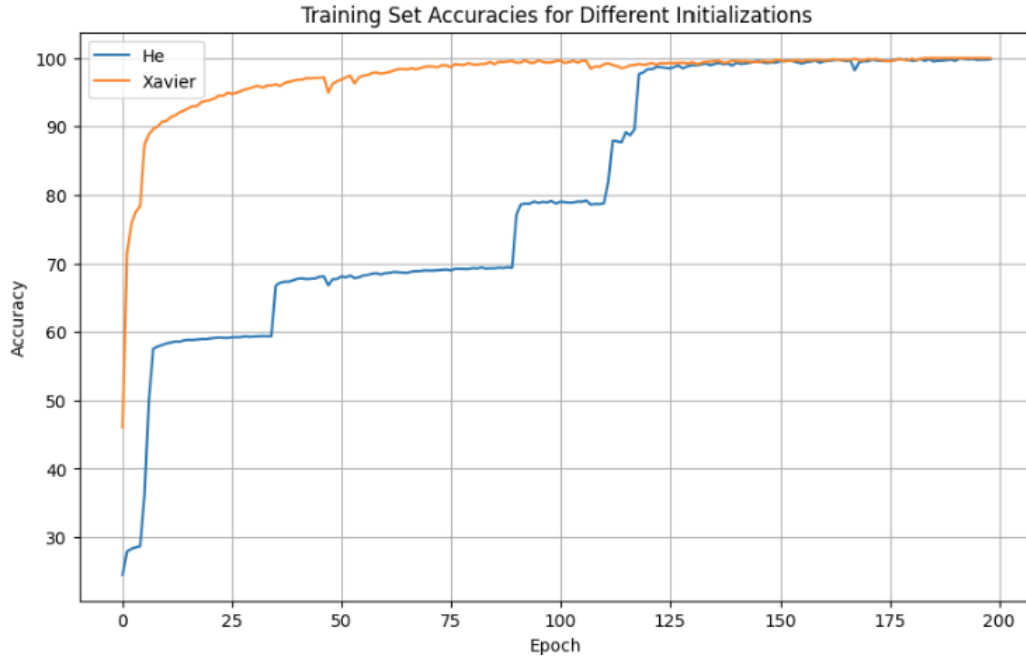
The DNNs were fully connected, and their architecture was given by [784, 3000, 3000, 3000, 3000, 500, 10]. When pruning with MP-based pruning, both initializations achieved test accuracies above 90%, which was comparable to the performance of networks initialized with a normal distribution (which was 90.74%), see Fig. 54. Without MP-based pruning, the DNNs achieved accuracy on the test set of $\sim 89\%$. All the DNNs were trained using a combination of $L2$ and $L1$ regularization (see (214)), which is why the DNNs achieved an accuracy above 90%. The using of both $L1$ and $L2$ regularization together with MP-pruning is a critical factor in achieving these high accuracies, consistently above 90%. We used the hyperparameters $\mu_1 = .0000005$ and $\mu_2 = .0000001$. The other hyperparameters are the same as those given in the numerical simulations from Subsection A.0.1.

$$L(\alpha(t)) = -\frac{1}{|T|} \sum_{s \in T} \log(\phi_{i(s)}(s, \alpha(t))) + \mu_1 \sum_{i=1}^L \|W_i(t)\|_1 + \mu_2 \sum_{i=1}^L \|W_i(t)\|_F^2 \quad (214)$$

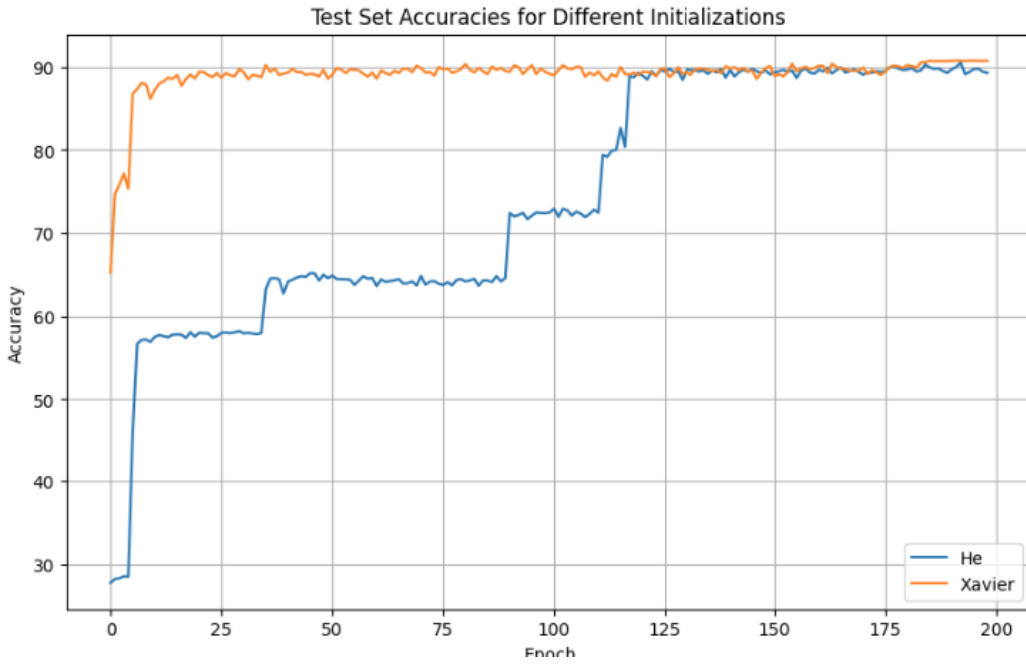
Additionally, we conducted a similar simulation where the initial weights were drawn from a normal distribution $N(0, \frac{1}{N})$, but 90% of the parameters were randomly initially sparsified afterward and set to zero (though we still used them during training). The resulting DNN's test accuracy plateaued at

$\sim 80\%$ and failed to improve beyond this point, even when using MP-based pruning. **It is important to note that the MP theorem does not hold for weight layers initialized in such a manner.** Similar results were found when we used He and Xavier’s initializations but initially sparsified them so that 90% of the parameters started out as zero (but were used during training).

Finally, we also ran the numerical simulations for the above DNN architecture and hyperparameters, but setting $\mu_1 = 0$ (that is only using $L2$ regularization, which is what was also done in Subsection A.0.1). For the He and Xavier initialization, the DNNs, with MP-based pruning, obtained an accuracy of $\sim 90\%$, comparable to the simulations found in Subsection A.0.1, while for the sparse initialization, their accuracy never exceeded 80% even when using MP-based pruning. Without MP-based pruning, the He and Xavier initialization accuracies were $\sim 88\%$.



(a) Training Set Accuracy vs Epoch for He and Xavier Initializations



(b) Test Set Accuracy vs Epoch for He and Xavier Initializations

Figure 54: Accuracy vs Epoch for He and Xavier Initializations

I.3 A Regression Problem: MP-based pruning in regression

We consider the task of finding a DNN that approximates a function best fitting the given data in terms of Mean Squared Error (MSE); see Fig. 55.

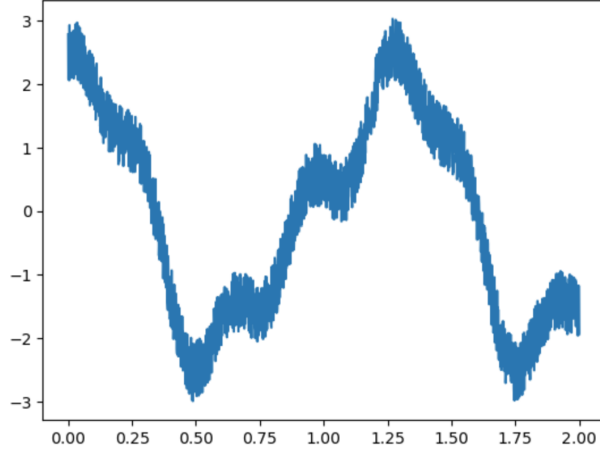


Figure 55: Illustration of the regression problem.

Training Data The training data consists of 2000 equally spaced points between 0 and 2:

$$x_{\text{train}} = \left\{ x_i \mid x_i = \frac{2i}{1999}, i = 0, 1, 2, \dots, 1999 \right\}.$$

Noise is added to the training labels. The noise is uniformly distributed between -1 and 1 with a scaling factor of $\sigma = 0.5$:

$$\text{noise}_{\text{train}} = \{\epsilon_i \mid \epsilon_i \sim \mathcal{U}(-1, 1)\}.$$

The training labels are generated using the following function:

$$y_{\text{train}} = 0.5 \cos(20x_{\text{train}}) + 2 \cos(5x_{\text{train}}) + 0.5 \sin(10x_{\text{train}}) + \sigma \cdot \text{noise}_{\text{train}}.$$

Testing Data The testing data consists of 500 equally spaced points between 0 and 2:

$$x_{\text{test}} = \left\{ x_i \mid x_i = \frac{2i}{499}, i = 0, 1, 2, \dots, 499 \right\}.$$

The testing labels are generated using the following function without noise:

$$y_{\text{test}} = 0.5 \cos(20x_{\text{test}}) + 2 \cos(5x_{\text{test}}) + 0.5 \sin(10x_{\text{test}}).$$

Defining Output and Target The DNN used for this 2D regression has a simple structure:

- The first and last layers each consist of a single neuron.
- The network outputs a single value and is trained using MSE together with $L2$ regularization as the loss function.

DNN Topology We use a fully connected DNN for the regression problem given in Fig. 55, that is, to find the curve that best fits the data. The fully connected DNN used in this regression problem has the following topology: $[1, 1000, 1000, 1000, 1000, 1]$, which represents the number of nodes in each layer. We don't use an activation function after the final layer of the DNN.

Training Process Both an unpruned DNN and a DNN using RMT pruning were trained to minimize MSE loss. The hyperparameters for training can be found in Table 12. We decreased the lr by .997 every epoch.

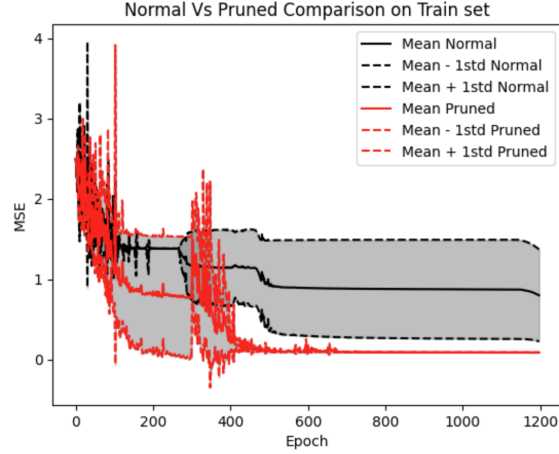


Figure 56: Training loss comparison between normal and pruned DNN.

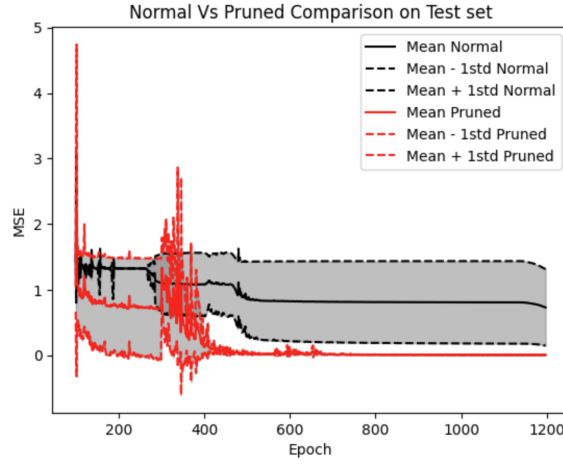


Figure 57: Test loss comparison between normal and pruned DNN.

Comparison of Unpruned vs Pruned DNN The pruned DNN shows a significantly smaller loss on both training and test sets, with a 50% reduction in the number of parameters, see Figs. 56 and 57. It is important to note that a smaller DNN might perform better on this task when MP-based pruning is not employed. However, for larger fully connected DNNs, it is clear that MP-based pruning helps the DNN achieve much lower loss for this task.

Hyperparameter	Value
Number of epochs	1198
Number of seeds	5
Learning rate (lr)	0.025
Momentum	0.95
Batch size	128
$L2$ regularization on loss	0.001

Table 12: Training hyperparameters for the DNN.

I.4 Numerical example used to calculate δX .

In this demonstration, we have a set of two-dimensional data points, and our objective is to identify the boundary that divides them into two categories. The dataset originates from a randomly constructed polynomial function of a designated degree. We then sample data points uniformly across a spectrum of x-values. For every x-value, the polynomial function gives a y-value. These points are then slightly offset either upwards or downwards, forming two distinguishable point clusters labeled as red and blue. Points positioned above the polynomial curve get a blue label, and those below are tagged red. We also add Gaussian noise to slightly modify the y-values, causing a few red data points to appear below the boundary and some blue ones above. This scenario is depicted in Fig. 14.

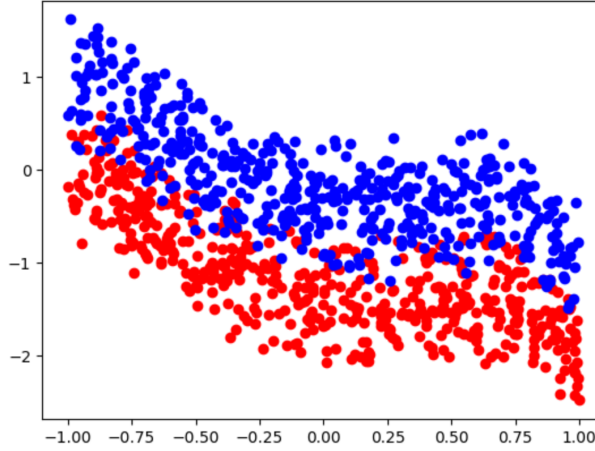


Figure 58: Illustration of a decision boundary for a binary classification challenge created using a random polynomial function, supplemented with noise. The distinct blue and red points signify the two classes.

The goal is to harness a DNN to capture the decision boundary demarcating the two clusters. This DNN is designed to process a two-dimensional data point, producing a binary output indicating whether the point is red or blue. Given that the dataset is artificially curated, the actual decision boundary is known, allowing us to gauge the efficacy of our DNN.

For this task, our neural network model had one hidden layer with 500 neurons. We also used the ReLU activation function. Training is executed using the cross-entropy loss complemented by the SGD optimizer and momentum.

I.5 Hyperparameters for Subsection A.0.1

- Number of epochs: 40
- Number of seeds: 10
- Learning rate (lr): 0.02
- Momentum: 0.9
- Batch size: 128
- $L2$ regularization on loss: 0.0005. (The regularization term is applied for both DNNs, the normal and pruned versions).

See [38], [59], [59] and [29] for information on learning rate, momentum, batch size, and regularization, respectively.

I.6 Hyperparameters for Subsection A.0.1

- Number of epochs: 70
- Number of seeds: 10
- Learning rate (lr): 0.02.
- Momentum: 0.9
- Batch size: 128
- $L2$ regularization on loss: 0.0005.

I.7 Hyperparameters for Subsection A.0.2

Training hyperparameters:

- Number of epochs: Depends on the example.
- Number of seeds: 5
- Learning rate (lr): 0.02.
- Momentum: 0.9
- Batch size: 128
- Split frequency: Depends on the example.
- $L2$ regularization on loss: .001.
- goodness of fit (GoF): changes for every simulation

I.8 CNN architecture description

The CNN model in this study consists of multiple convolutional and fully connected layers, incorporating batch normalization and dropout for regularization. The architecture is designed as follows:

Convolutional Layers:

- The network contains several convolutional layers with kernels of size 3. Each convolutional layer is followed by batch normalization.
- Activation functions (ReLU) are applied conditionally based on predefined configurations.
- Max-pooling layers are introduced after every second convolutional layer, except the first one. Specifically, max-pooling with a pool size of 2 is used to reduce the spatial dimensions of the feature maps.
- Dropout is applied after each convolutional layer to prevent overfitting, with a dropout rate of 0.35.

Fully Connected Layers:

- After the convolutional layers, the output is flattened and passed through a series of fully connected layers.
- Each fully connected layer is followed by batch normalization and, conditionally, by a ReLU activation function based on the configuration.
- Dropout is also applied to the output of the first fully connected layer to further prevent overfitting.

Output Layer:

- The final layer applies a log-softmax function to produce the output probabilities for classification.

I.8.1 Pooling and regularization details

Pooling Layers: Max-pooling layers are strategically placed to downsample the feature maps, specifically after every second convolutional layer. This pooling strategy helps in reducing the computational complexity and in extracting invariant features.

Batch Normalization: Batch normalization is applied after each convolutional and fully connected layer to stabilize and accelerate the training process by normalizing the input to each layer.

Dropout: Dropout is utilized after each convolutional layer and the first fully connected layer with rates of 0.35 and variable values, respectively, to reduce overfitting by randomly setting a fraction of activations to zero during training.

I.9 The hyperparameters for Subsection A.0.3

Training hyperparameters:

- Number of epochs: 300.
- Number of seeds: 5
- Learning rate (lr): 0.001.
- Momentum: 0.9
- Batch size: 128
- Split frequency (every how many epochs we split the pruned DNN and remove small singular values): 40.
- $L2$ regularization on loss: .001.
- goodness of fit (GoF): 0.08 for fully connected layers, 0.06 for convolutional layers.

J Some algorithms

The following algorithms are used to compute λ_+ using MP and Tracy-Widom Distribution (Algorithm 1), compute how well the MP distribution fits the ESD of a matrix (Algo 3), the MP-based pruning algorithm (Algo 4) and how we sparsify singular vectors (Algo 5).

Algorithm 4 MP-based pruning algorithm

Require: ℓ , a predetermined number of epochs; τ , a threshold for the MP fit criteria in Subsection 3; $f(\text{epoch})$, a monotonically decreasing function from 1 to 0 (i.e. (84)) and for each $1 \leq l \leq L$ and weight layer matrix W_l **state** $\text{split}_l = \text{false}$.

- 1: Initialize: Train the DNN for ℓ epochs. Take $\text{epoch} := \ell$.
- 2: **while** a predefined training condition is met (i.e. $\text{epoch} \leq 100$) **do**
- 3: **for** each l , if $\text{split}_l = \text{false}$ then for weight matrix W_l in the DNN ϕ **do**
- 4: Perform SVD on W_l to obtain $W_l = U_l \Sigma_l V_l^T$.
- 5: Calculate eigenvalues of $W_l^T W_l$.
- 6: Apply BEMA algorithm (see Subsection H.1) to find the best fit MP distribution for ESD of $X = W_l^T W_l$ and corresponding λ_+ .
- 7: Check if ESD of X fits the MP distribution using MP fit criteria from Subsection H.4 and threshold τ .
- 8: **if** ESD fits the MP distribution **then**
- 9: Eliminate the portion $(1 - f(\text{epoch}))$ of singular values smaller than $\sqrt{\lambda_+}$ to obtain Σ' and form $W'_l = U_l \Sigma'_l V_l^T$.
- 10: Use Σ' to create $W'_{1,l} = U_l \sqrt{\Sigma'_l}$ and $W'_{2,l} = \sqrt{\Sigma'_l} V_l^T$.
- 11: **if** $W'_{1,l}$ and $W'_{2,l}$ together have fewer parameters than W'_l **then**
- 12: Replace W_l in the DNN ϕ with $W'_{1,l} W'_{2,l}$, change $\text{split}_l = \text{true}$.
- 13: **else**
- 14: Replace W_l in the DNN ϕ with W'_l .
- 15: **end if**
- 16: **else**
- 17: Don't replace W_l .
- 18: **end if**
- 19: **end for**
- 20: Train the DNN for ℓ epochs. Take $\text{epoch} := \text{epoch} + \ell$.
- 21: **for** each l , if $\text{split}_l = \text{true}$ **do**
- 22: **if** for $W_l := W'_{1,l} W'_{2,l}$ the ESD of X_l fits the MP distribution with thresholds τ and λ_+ **and** if, we (hypothetically) applied steps 4-12 to W_l , the number of parameters in the DNN ϕ would decrease **then**
- 23: replace $W'_{1,l} W'_{2,l}$ with W_l and $\text{split}_l = \text{false}$.
- 24: **else**
- 25: Don't change anything.
- 26: **end if**
- 27: **end for**
- 28: **end while**

Algorithm 5 Sparsify Singular Vectors

```
1: Let  $W_l$  denote the  $l$ -th layer matrix of a DNN of size  $N \times N$  and an initial threshold  $\theta$ .
2: Perform SVD on  $W_l$ :  $W_l = U\Sigma V^T$ .
3: Given threshold  $\sqrt{\lambda_+}$ .
4: for  $i = 1, \dots, N$  do
5:   if  $\Sigma_{ii} < \sqrt{\lambda_+}$  then
6:      $M_i = 1$ 
7:   else
8:      $M_i = 0$ 
9:   end if
10:   $T_i(\sigma) = \theta \cdot \left(1 - \frac{\sigma}{\sqrt{\lambda_+}}\right)^{30}$ 
11:  if  $M_i = 1$  then
12:     $U_{:,i} = \text{apply\_sparsity}(U_{:,i}, T_i(\Sigma_{ii}))$ 
13:     $V_{i,:} = \text{apply\_sparsity}(V_{i,:}, T_i(\Sigma_{ii}))$ 
14:  end if
15: end for
16: Extra universal sparsification step:
17: Given a threshold,  $\theta$ .
18: for  $i = 1, \dots, N$  do
19:   $U_{:,i} = \text{apply\_sparsity}(U_{:,i}, \frac{\theta}{750})$ 
20:   $V_{i,:} = \text{apply\_sparsity}(V_{i,:}, \frac{\theta}{750})$ 
21: end for
22: Recompose matrix:  $W'_l = U\Sigma V^T$ .
```

Algorithm 7 Element-wise Sparsification

```
1: Input: Matrix  $W_l$ , threshold  $\xi$ 
2: Output: Sparsified matrix  $W'_l$ 
3: for each element  $w_{ij}$  in  $W_l$  do
4:   if  $w_{ij} < \xi$  then
5:      $w'_{ij} = 0$ 
6:   else
7:      $w'_{ij} = w_{ij}$ 
8:   end if
9: end for
10: return  $W'_l$ 
```

K Numerical results for stability of accuracy

We verify these results numerically for examples such as Example 7.1. The given numerical simulations aim to investigate the relationship between the uniform doubling condition of a slab and the loss and accuracy of deep neural networks (DNNs) on various data sets. In these examples, the absolute value and Leaky ReLU activation functions are used and cross-entropy loss function is what we train with. We only check the uniform doubling condition for slabs and don't involve the truncation sets in these numerics. We call this the simplified uniform doubling condition (SUDC).

Here, the SUCD involves the expansion of the slab's thickness by a factor κ , followed by counting the number of data points inside the slab. This process is repeated until the number of new data points inside the slab is less than $\sigma + 1$ times the number of data points in the previous iteration, see Fig. 59. The code calculates the average number of doubling steps, the average number of points in the final slab, and the average width of the final slab for multiple slabs. The average width of the final slab is the most important of these quantities and gives us an estimate of what β might be for the training data set. From here on, we will call this quantity $\bar{\beta}$ and assume that it is a good estimator for β . For simplicity, we also sometimes refer to $\bar{\beta}$ as the average width of DC (i.e. the average of the largest widths for which SUDC is satisfied).

The data sets are generated using the same method as in Example 7.1. To create variability across the data sets, the number of samples is chosen randomly within a predefined range. In Example K.1 each data set has between 200 and 15,000 objects and the number of objects is chosen uniformly from this range. Each data set comprises data points and corresponding labels, where the data points lie in a two-dimensional space and the labels represent the target class of each point.

After generating a data set, it is split into training and testing subsets. The number of training samples is also chosen randomly, within a range of 100 to the total number of samples in the data set. This random selection process ensures that the trained DNNs are exposed to varying amounts of training data, enabling the analysis of the relationship between the doubling condition, loss, and accuracy of the DNNs under different training scenarios.

For each of the generated data sets, the code trains a DNN and evaluates its performance in terms of loss and accuracy. The SUCD is then checked for the training data sets, and $\bar{\beta}$ is calculated for the training data. The results are collected and sorted by the size of $\bar{\beta}$. For an example of the SUCD applied to a data set, see Fig. 59.

The code proceeds to plot various graphs to visualize the relationships between the doubling condition, loss, and accuracy of the DNNs.

Example K.1. *In this example, we take $\delta = 1$, $\kappa = 2$, $\sigma = .9$, and $\ell = .001$. We train 100 different DNNs on 100 different data sets and check the SUCD for all data sets using 50,000 initial slabs and doubling them until the SUDC is not satisfied. We train the DNNs for 5 epochs, and each DNN has the same number of layers (4 hidden layers with 1000 nodes in each layer). In Fig. 60b we show the accuracy of the DNNs on the training set vs $\bar{\beta}$. Then, we find the average accuracy of the DNNs on the training set and the 50 DNNs with accuracy closest to this average accuracy. In Fig. 60a we graph the loss on the training set vs $\bar{\beta}$ (for the training set), for the 50 chosen DNNs. Fig. 60a shows that even DNNs with similar accuracy will have lower loss if the data they trained on had more slabs that satisfied this simplified uniform doubling condition (i.e. the $\bar{\beta}$ is bigger). This means that those DNNs, with lower loss, will have greater stability of accuracy. As we see from both graphs, training sets that satisfy the SUDC for more slabs have that the DNNs trained on them obtain lower loss and, in this example, even higher accuracy after the same epochs of training.*

One can use C_2 to analyze the behavior in Fig. 60a. The main reduction in loss happens when $\bar{\beta}$ changes from 0 to .1. By increasing the size of $\bar{\beta}$, we increase the possible size of η . For these DNNs, $d_{\max} \approx 1000$ and $d_{\text{med}} \approx 700$ (though there is variation), meaning that taking $\eta < 700 \frac{\bar{\beta}}{2}$ is probably

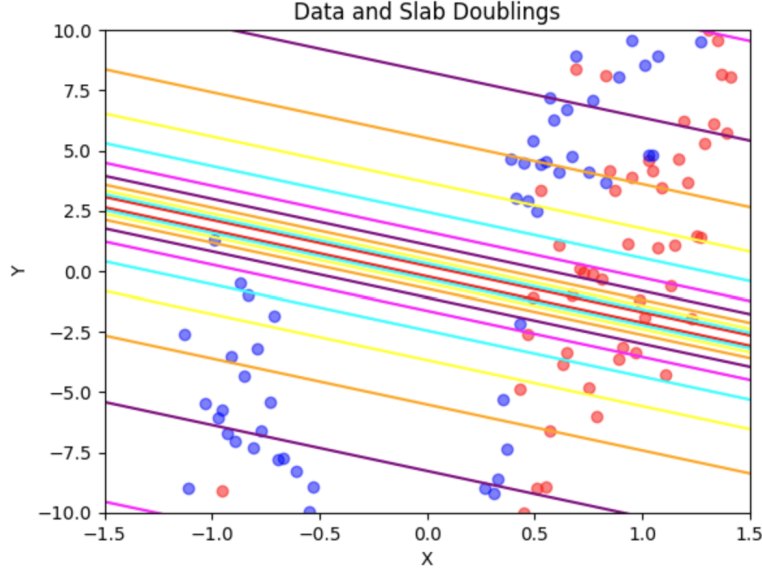
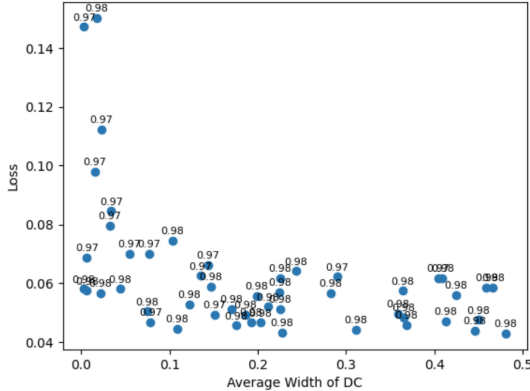


Figure 59: SUDC for data set with two classes, red and blue. We start with a single slab of small width and continue to double it (shown in different colors) until the doubling condition is not satisfied.

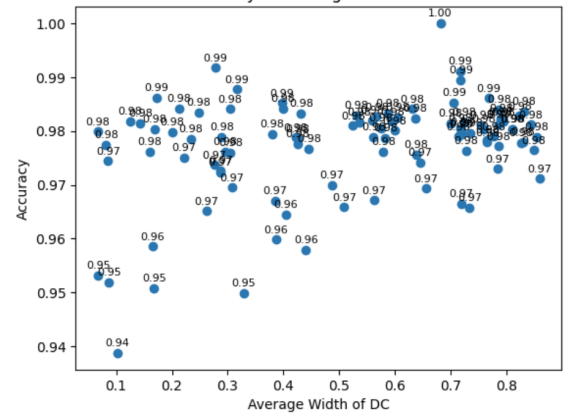
reasonable, see (101). That means $0 < \eta < 35$. We assume that $\delta_0 \approx 1 - 1.03^{-\eta}$, which means that the size of the good set decreases exponentially with η , see Fig. 70. Finally, we can assume that $\xi \in [.7, 1.5]$ and the bad set can be of size 1.5. Fig. 61 shows the constant C_2 under these assumptions, when η varies from 2 to 16. We see a similar drop in C_2 from .12 to .06. In this example, we see that the loss itself does seem to depend on β though the loss does not lead to tight bounds on the accuracy. Nevertheless, we still obtain important information on the size of the loss. We also see that C_2 might provide an explanation for why loss drops so quickly when $\bar{\beta}$ goes from 0 to .1 and then plateaus. This is because C_2 itself plateaus when η gets large, see Fig. 61.

Loss vs Average Width of DC for 50 DNNs closest to the average training accuracy



(a) Training loss, represented on the y-axis, vs $\bar{\beta}$ (or average width of DC), represented on the x-axis.

Accuracy vs Average Width of DC



(b) Training accuracy, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.

Figure 60: DNN training loss and training accuracy vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.

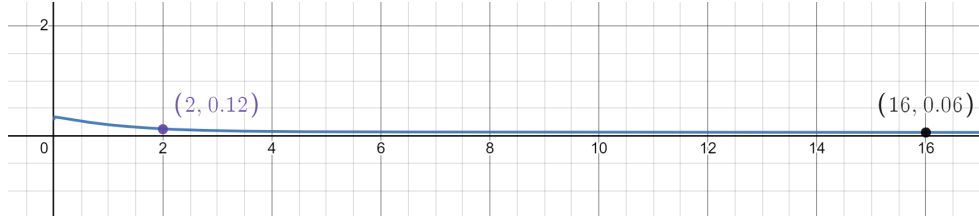
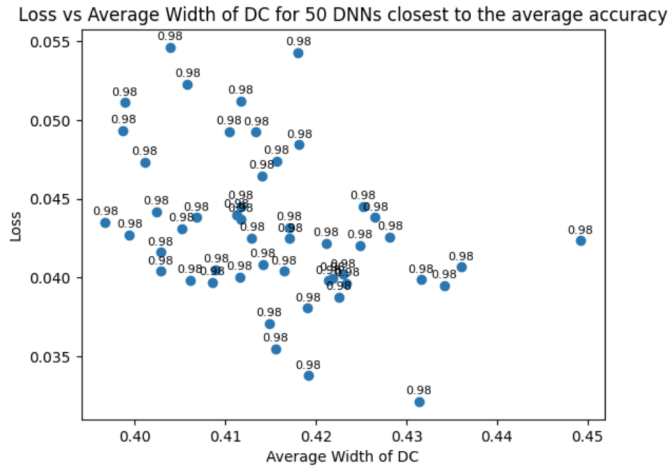
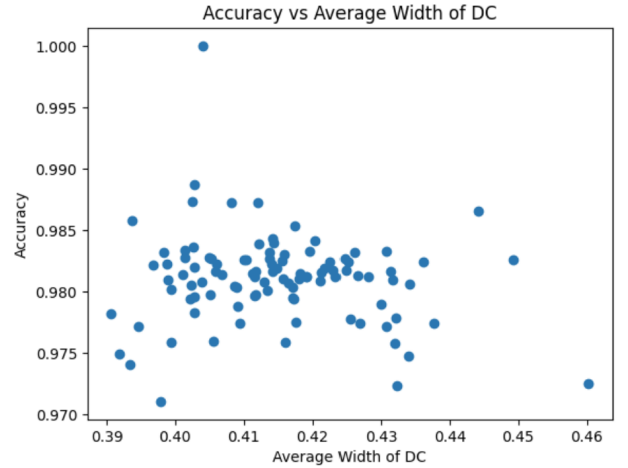


Figure 61: C_2 , represented on the y-axis, vs η , represented on the x-axis.

Example K.2. In this example, we take $\delta = 1$, $\kappa = 2$, $\sigma = .9$ and $\ell = .003$. We train 100 different DNNs on 100 different training sets, each having between 10,000 to 20,000 objects, and check the SUDC for all training sets using 50,000 initial slabs and doubling them until the simplified uniform doubling condition is not satisfied. We train the DNNs for 5 epochs, and each DNN has the same number of layers (4 hidden layers with 1000 nodes in each layer). In Fig. 62b, we show the accuracy of the DNNs on the training set vs $\bar{\beta}$. Then, we find the average accuracy of the DNNs on the training set and the 50 DNNs with accuracy closest to this average accuracy, and in Fig. 62a we graph the loss on the training set vs $\bar{\beta}$. Fig. 62a shows that even DNNs with similar accuracy will have lower loss if the data they trained on had more slabs that satisfied this simplified uniform doubling condition. Again, this means that those DNNs, with lower loss, will have greater stability of accuracy. In this example, accuracy does not seem to depend on the simplified uniform doubling condition, though more rigorous analysis does show a correlation on similar problems, see Example K.4 and Subsection K.1.



(a) Training loss, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



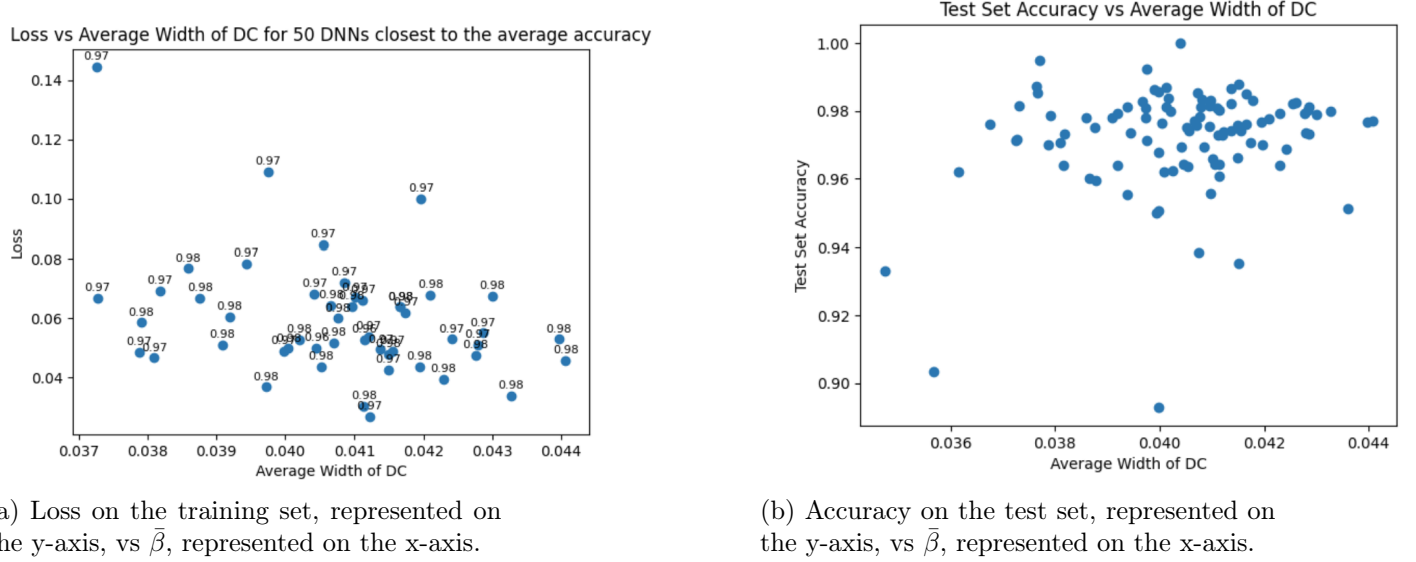
(b) Training accuracy, represented on the y-axis vs $\bar{\beta}$, represented on the x-axis.

Figure 62: DNN training loss and training accuracy vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.

In Fig. 64a we show a graph of the number of points in the training set vs $\bar{\beta}$, to illustrate that there does not seem to be an immediate connection between the number of points in the training set and the SUDC.

Example K.3. In this example, we take $\delta = 1$, $\kappa = 2$, $\sigma = .9$ and $\ell = .003$. We train 100 different DNNs on 100 different data sets and check the simplified doubling condition for all data sets using 100,000 initial slabs and doubling them until the SUDC is not satisfied. We train the DNNs for 5 epochs, and each DNN has the same number of layers (4 hidden layers with 1000 nodes in each layer).

In Fig. 63b, we show the accuracy of the DNN on the test vs $\bar{\beta}$. Then, we find the average accuracy of the DNNs on the training set and the 50 DNNs with accuracy (on the training set) closest to this average accuracy, and in Fig. 63a we graph the loss on the training set vs $\bar{\beta}$. Fig. 63a shows that even DNNs with similar accuracy will have lower loss if the data they trained on had more slabs that satisfied this simplified doubling condition. So again, we have greater stability of accuracy. There does not seem to be a good correlation between $\bar{\beta}$ and accuracy on the test set in this example, though more rigorous analysis does show a correlation on similar problems, see Example K.4 and Subsection K.1.



(a) Loss on the training set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.

(b) Accuracy on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.

Figure 63: DNN loss on training set and accuracy on test set vs $\bar{\beta}$. The numbers above the blue points represent the accuracy of the DNNs.

Fig. 64b shows a graph of the number of points in the training set vs $\bar{\beta}$, to illustrate that there does not seem to be an immediate connection between the number of points in the training set and the simplified uniform doubling condition.

Example K.4. In the following example, we perform a similar analysis as the previous examples, but we fit the data to a simple linear regression model to see the underlying correlation between $\bar{\beta}$, loss, and accuracy on the test sets. Here, we use the Leaky ReLU activation function:

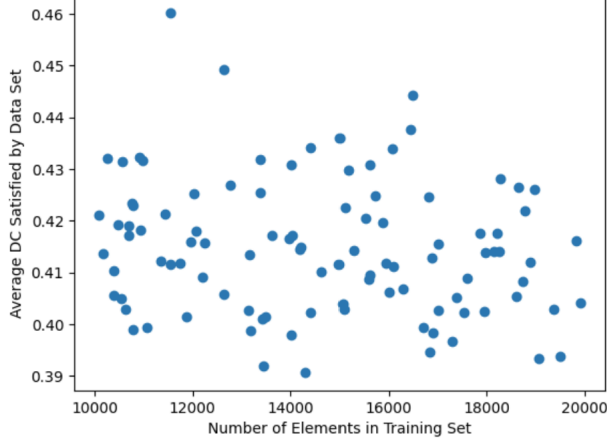
$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0.1x & \text{if } x < 0. \end{cases} \quad (215)$$

Note that while Theorem 8.6 is given for DNNs with the absolute values activation function, we have that a similar theorem can be obtained for DNNs with the Leaky ReLU activation function, see Section O.

In this example, we configure the parameters as follows:

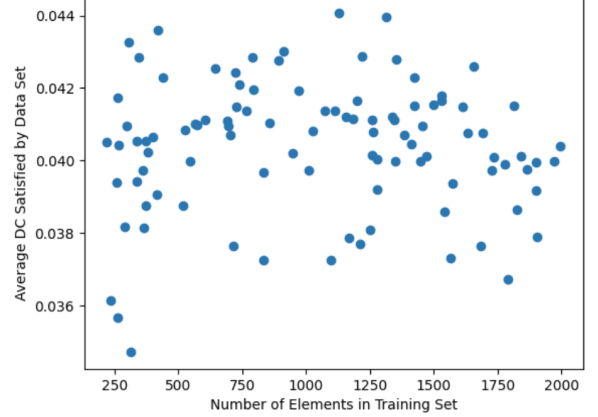
- Number of data sets we train on: 200
- Epochs trained: 10
- κ : 2.1
- σ : 0.9

Average DC Satisfied by Data Set vs Number of Elements in Training Set



(a) $\bar{\beta}$, represented on the y-axis, vs the number of elements in the training set for Example K.2, represented on the x-axis.

Average DC Satisfied by Data Set vs Number of Elements in Training Set



(b) $\bar{\beta}$, represented on the y-axis, vs the number of elements in the training set for Example K.3, represented on the x-axis.

Figure 64: Number of objects in data set vs $\bar{\beta}$.

- Width of the original slab: 0.001
- Number of original slabs: 700,000

See Fig. 65 for the results of this numerical simulation. We see a positive correlation between $\bar{\beta}$ and accuracy on the test set.

Example K.5. In this example, we perform a similar analysis as the previous examples, again fitting the data to a simple linear regression model to see the underlying correlation between $\bar{\beta}$, loss, and accuracy on the test sets. In this example, we configure the parameters as follows:

- Number of data sets we train on: 400
- Epochs trained: 10
- κ : 2.1
- σ : 1
- Width of the original slab: 0.0001
- Number of original slabs: 2,000,000

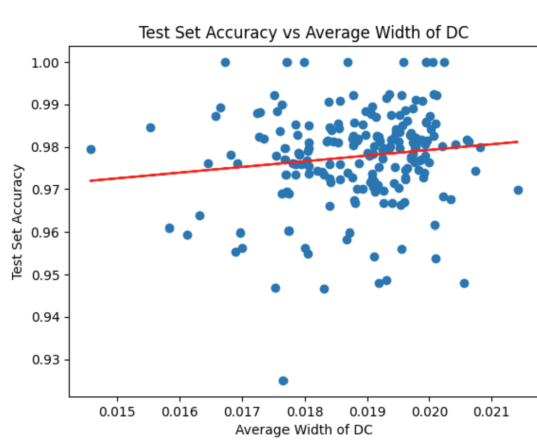
See Fig. 66 for the results of this numerical simulation. Again, we see a strong correlation between the SUDC and the accuracy and loss on the test set.

K.1 Numerics for a different classification problem

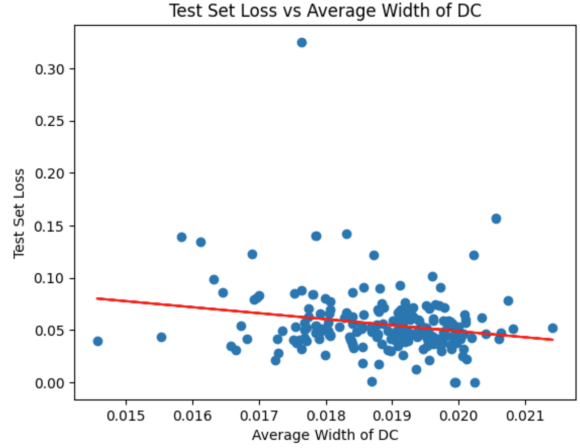
Given an integer M , let's first define a sequence of random functions. Each function is represented as:

$$f_i(x) = \cos(\pi a_i x + \theta_1^i) \times \sin(\pi b_i x + \theta_2^i).$$

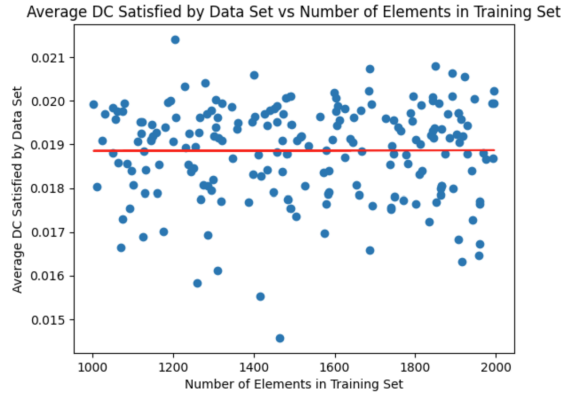
Where the frequencies a_i and b_i are randomly sampled from the discrete set $\{\frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}\}$ and the



(a) Accuracy on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



(b) Loss on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



(c) $\bar{\beta}$ vs number of points in training set.

Figure 65: DNN loss on test set and accuracy on test set vs $\bar{\beta}$.

phases θ_1^i and θ_2^i are sampled from a uniform distribution over the interval $[0, 2\pi]$. The vector function $f(x)$ has a total of M components $f(x)_i$.

Furthermore, we define a random matrix $B \in \mathbb{R}^{M \times M}$ where each entry of B is sampled uniformly from the interval $[0, 5]$. This matrix interacts with our random functions as follows:

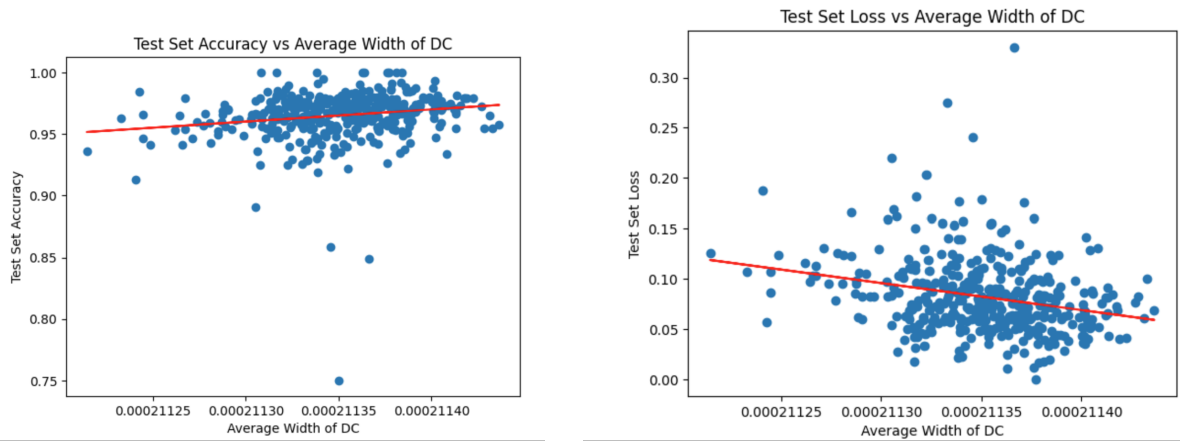
$$g(x) = Bf(x),$$

where the product represents a matrix-vector multiplication, and $f(x)$ is a vector of the evaluated random functions at a given x .

We take a series of N points x_j which are sampled uniformly from the interval $[-10, 10]$. The classification problem arises when we generate two classes of data points around the curve defined by g_1 , the first component of g . For each j , we obtain a point in $\mathbb{R}^2$ from one of the two classes, with both classes having an equal number of elements. The first class, represented in red, is generated by taking $g_1(x_j)$ and subtracting a random positive number from it to obtain z_j , whereas the second class, in blue, is obtained by adding a random positive number to the curve.

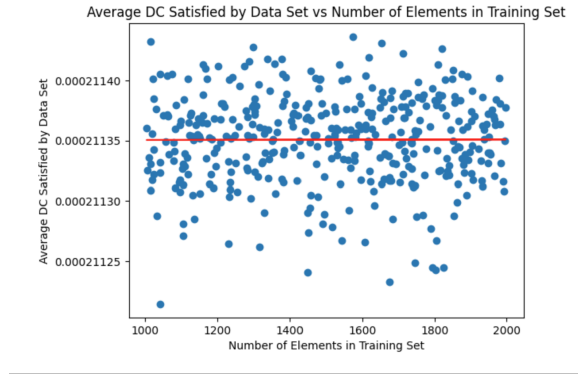
Our dataset D is then composed of these points:

$$D = \{(x_j, z_j)\}_{j=1}^N.$$



(a) Accuracy on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.

(b) Loss on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



(c) $\bar{\beta}$ vs number of points in training set.

Figure 66: DNN loss on test set and accuracy on test set vs $\bar{\beta}$.

We use this to generate both a training and test data set with N elements in total. Additionally, Gaussian noise is added to both classes of the training set to scatter the points, see Fig. 67.

The task is to discriminate between these two classes, recognizing the underlying structure provided by the function g_1 and the added perturbations. We use a DNN with the same architecture as in the previous examples (see Example K.4) to train on the training set in order to classify the red and blue points. In this example, we randomly create a training set of between 40,000 to 60,000 data points and a test set composed of a similar size of data points.

In this example, we configure the parameters as follows:

- Number of data sets we train on: 200
- Epochs trained: 10
- κ : 2.1
- σ : .9
- Width of the original slab: 0.001
- Number of original slabs: 700,000

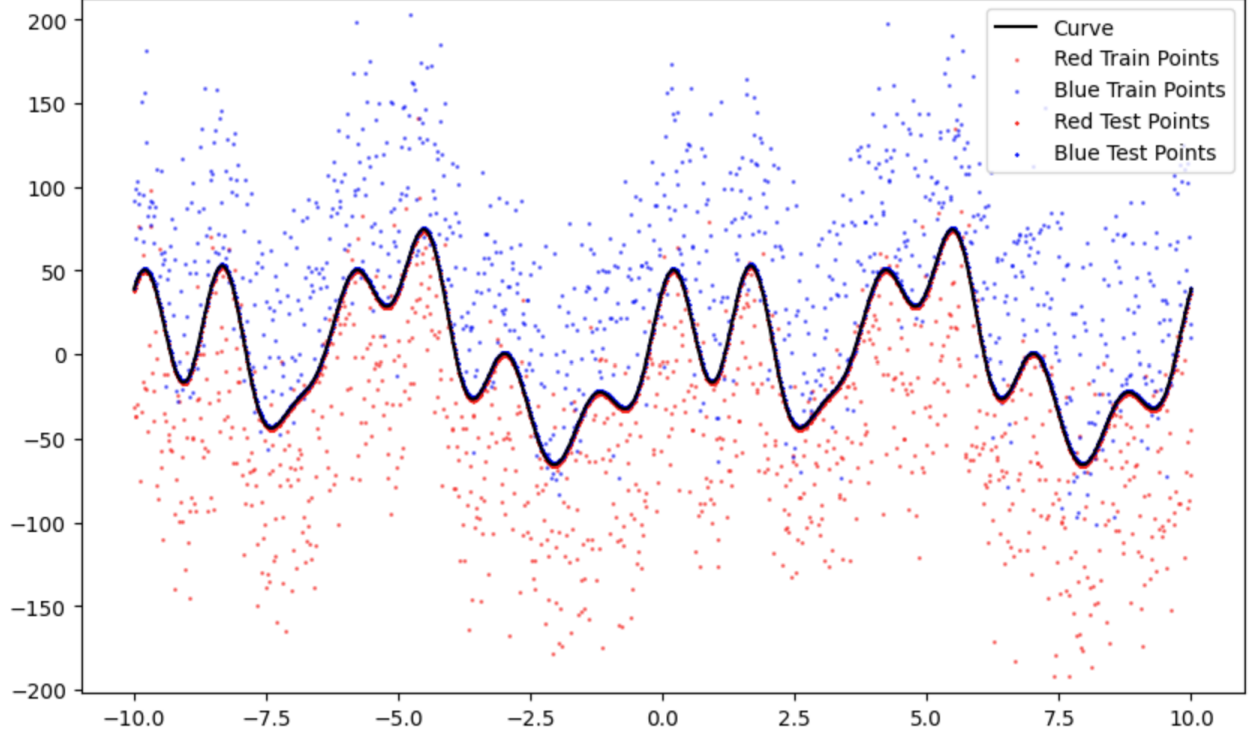


Figure 67: Illustrative example of the generated data. The black curve showcases the function $g_1(x)$, the red points indicate the first class, and the blue points signify the second class.

For the numerical results, see Fig. 68. The classification problem here is more difficult than the previous problem, and we only train for 10 epochs, leading to low accuracy (a little above 50% for most of the DNNs trained, which is slightly better than pure probability given that this is a classification problem with two classes). Nevertheless, we still see a distinct linear correlation between the loss and accuracy on the test set and $\bar{\beta}$. Again, we confirmed that no linear correlation existed between the number of data points being trained on and $\bar{\beta}$.

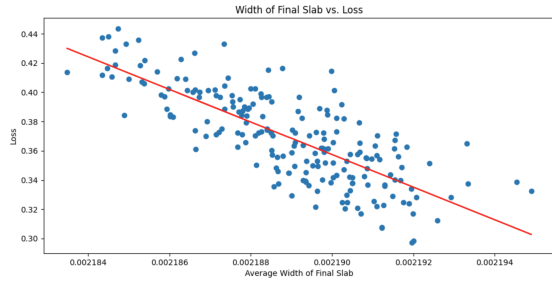
In future work, we will use a new random matrix theory pruning approach (see [54, 9]) to achieve better accuracy for this simple classification problem, and we will perform this simple SUDC analysis to see if the correlations between $\bar{\beta}$ and accuracy on the test set still persist in this more complex situation.

K.2 An application of the SUDC

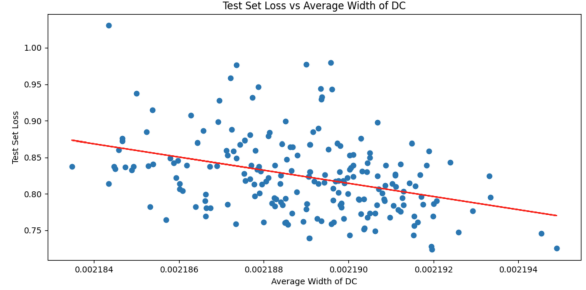
In the field of machine learning, data augmentation has emerged as a powerful technique to improve the generalization and robustness of deep neural networks. By introducing variations to the original training data, models are encouraged to recognize patterns that are invariant to these modifications, see [25, 61, 73, 63].

The essence of data augmentation is the creation of modified versions of the original dataset. Formally, given a dataset $T = \{(x_i, y_i)\}$, data augmentation yields a new dataset $T' = \{(x'_i, y'_i)\}$ where the transformations applied to produce x'_i and y'_i are determined by the chosen augmentation strategy. Training typically proceeds either on T' or on an extended dataset, often the union of the original and augmented data, $T \cup T'$.

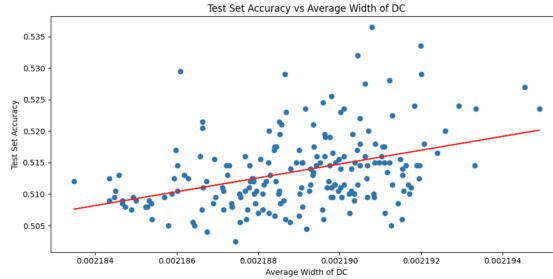
While numerous studies have underscored the efficacy of various data augmentation strategies in bolstering model robustness the challenge remains in choosing the right augmentation technique. Fac-



(a) Loss on the training set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



(b) Loss on the test set, represented on the y-axis, vs $\bar{\beta}$, represented on the x-axis.



(c) Accuracy on test set vs $\bar{\beta}$.

Figure 68: DNN loss on test set and accuracy on test set vs $\bar{\beta}$.

tors such as the type and extent of augmentation can significantly influence model performance.

A potential approach to this challenge lies in the utilization of the SUDC. For different augmented training sets derived from T , the SUDC can provide a measure, $\bar{\beta}$, which may correlate with the training and generalization capabilities of a model trained on such data. By comparing $\bar{\beta}$ for different augmentation strategies, one could potentially infer which method might be most beneficial. For instance, a higher $\bar{\beta}$ might indicate a more favorable training set for a given problem. Harnessing this insight, one could optimize the augmentation process to yield training datasets that promise better test-time performance. A more exhaustive exploration of this application will be the focus of future work.

L Previous results for stability of accuracy.

It is useful to review the first results given in this direction. In [8] it was shown that small clusters in δX lead to instability of accuracy, and so a doubling condition on δX was introduced which prevents the existence of small clusters.

The following is the doubling condition on δX :

Definition L.1. *The set $\{\delta X(s, \alpha(t_0)) : s \in T\}$ satisfies the doubling condition at time t_0 if there exists a mass $m_0 > 0$ and constants $\kappa > 1$, and $\delta, \sigma, \beta > 0$ such that for $x = 0$, and for all intervals $I \subset \mathbb{R}$ centered at $x = 0$ with $|I| \leq \beta$,*

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa I\}) \geq \min \left\{ \delta, \max \{m_0, (1 + \sigma) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\})\} - m_0 \right\}, \quad (216)$$

where $\kappa I = I_\kappa$ is the interval with the same center as I but whose width is multiplied by κ .

Essentially, this doubling condition requires that

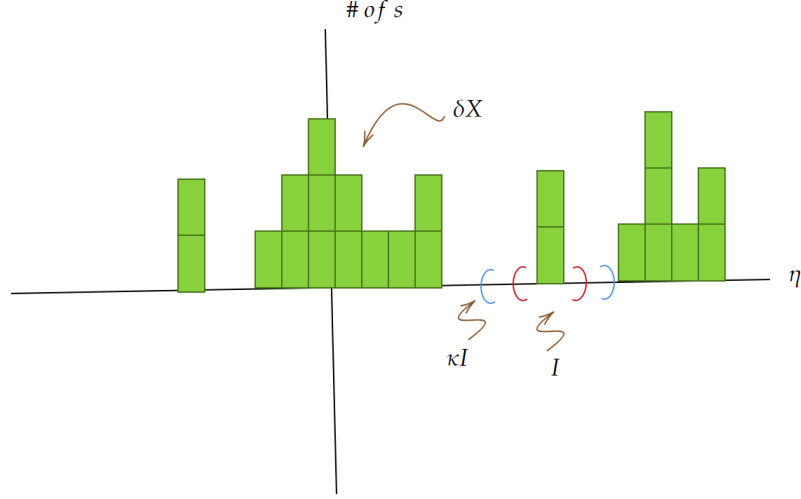


Figure 69: An example of a histogram of $\delta X(T)$ with slabs (in this case intervals) κI and I not satisfying the doubling condition.

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa I\}) \geq (1 + \sigma) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\}), \quad (217)$$

for all intervals I centered at $x = 0$. This means that as we double the interval I by the constant κ (which is therefore referred to as the doubling constant), we must capture more data than what was originally in I .

In Fig. 69 we see that δX satisfies the doubling condition at $x = 0$, however, it does not satisfy the doubling condition for the intervals I and κI shown in the figure. This is because as we double I by the constant κ to obtain κI we don't capture any more of the data.

We call m_0 the residual mass. If $(1 + \sigma)\mu(I) \leq m_0$ the doubling condition is automatically satisfied, and so we can think of m_0 as the smallest mass for which the doubling condition must be satisfied.

This doubling condition on δX provides for the stability result given in Theorem L.2, however, one can only verify the doubling condition after training has commenced (in order to obtain δX). Thus, one must know the architecture of the DNN and its parameters to obtain δX and verify whether the DNN has the stability of accuracy during training. As we will see, a doubling condition on the training set directly can also be used to ensure stability.

We now state the stability of accuracy result with the doubling condition on δX .

Theorem L.2 (Berlyand, Jabin, Safsten, 2021). *Assume that the set $\{\delta X(s, \alpha(t_0)) : s \in T\} \subset \mathbb{R}^1$ satisfies the doubling condition given in Def. L.1 with constants $\beta, \sigma, \kappa, m_0, \delta$ at point $x = 0$ at some time t_0 . Then for every $\varepsilon > 0$ there exists a constant $C_1 = C_1(\gamma, \eta, K, \kappa, \sigma)$ such that if*

$$m_0 \leq \frac{\varepsilon}{2C_1}, \quad \kappa(\log \frac{1}{\varepsilon} + C_1) \leq \eta \leq \frac{\beta}{2}, \quad \delta_0 \leq \frac{\varepsilon}{C_1} \eta^\gamma, \quad (218)$$

and if the good and bad sets at $t = t_0$ satisfy

$$\mu(G_\eta(t_0)) > 1 - \delta_0 \quad \text{and} \quad B_{-1}(t_0) = \emptyset, \quad (219)$$

then for all $t \geq t_0$,

$$\text{acc}(t) = \mu(G_0(t)) \geq 1 - \varepsilon. \quad (220)$$

and

$$\mu(G_{\eta^*}(t)) \geq 1 - 2 \log 2 \varepsilon e^{\eta^*}, \quad (221)$$

for all $\eta^* > 0$.

We take any γ such that

$$0 < \gamma \leq \min\{1, \log(1 + \sigma)/\log \kappa\}. \quad (222)$$

Recall, K is the number of classes in the DNN.

Remark L.1. Here

$$C_1 = \max\{6\eta^\gamma, \frac{1}{\log(2)}\kappa 6(K-1)(\sigma+1)a_p(e+2\kappa^\gamma)\}. \quad (223)$$

with $a_j = \sum_{k=1}^j (1+\sigma)^k$ and $p = \lfloor \log(\eta)/\log \kappa \rfloor$. Already for $K = 10$, $\sigma = 1$ and $\kappa = 2$ we have that C_1 is at least 800 (and so η is large and $\mu(G_\eta(t_0))$ is probably small). In Section N, we will address this difficulty by finding a smaller constant C_3 and changing the three conditions given in (218).

Note that $\eta > C_1$ and so when $\gamma = 1$ (which depends on the doubling condition via the constants σ and κ) we have that $\delta_0 > \epsilon$. This makes it easier to satisfy condition 219. However only if η can remain small.

In this work, a new version of Theorem L.2 will be proved, one which will be used to prove the main result given in Theorem 8.6. This theorem uses a uniform doubling condition on δX (see Def. L.3) instead of the doubling condition. Changing the doubling condition to the uniform doubling condition allows us to find a smaller constant than C_1 and the uniform doubling condition can, at least in theory, be checked numerically giving a more practical stability of accuracy result. The new version of Theorem L.2 is given in Proposition L.4.

L.1 Stability of accuracy via uniform doubling condtion on $\delta X(T, \alpha)$

We now present another stability of accuracy result which complements Theorem 8.6. First, we define a uniform doubling condition on δX :

Definition L.3. *The set $\{\delta X(s, \alpha(t_0)) : s \in T\}$ satisfies the uniform doubling condition at time t_0 if there exists constants $\kappa > 1$, and $\delta, \sigma, \ell, \beta > 0$ such that for $x = 0$, and for all intervals $I \subset \mathbb{R}$ around $x = 0$ with $|I| = \ell\kappa^i \leq \beta$ with $i \in \mathbb{N}$,*

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa I\}) \geq \min\{\delta, (1 + \sigma) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\})\}, \quad (224)$$

where κI is the interval with the same center as I but whose width is multiplied by κ .

Essentially, this doubling condition requires that

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa I\}) \geq (1 + \sigma) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\}), \quad (225)$$

for some intervals I centered at $x = 0$ with length $\ell\kappa^i$, $i \geq 0$ and smaller than β . This means that as we double the interval I by the constant κ , we must capture more data than what was originally in I .

Based on this uniform doubling condition on δX , we prove the following stability of accuracy result.

Proposition L.4. *Assume that the set $\{\delta X(s, \alpha(t_0)) : s \in T\} \subset \mathbb{R}^1$ satisfies the uniform doubling condition given in Def. L.3, with $\kappa > 1$ and $\delta, \sigma, \beta > 0$, $1 > \ell > 0$ at point $x = 0$ at some time t_0 . Then for the constant C_3 given in (247) we have:*

$$\frac{\beta}{2} \geq \eta, \quad \xi := \ell\kappa^i \quad i \in \mathbb{N}, \quad \delta_0 < \delta. \quad (226)$$

and if good and bad sets at $t = t_0$ satisfy

$$\mu(G_\eta(t_0)) > 1 - \delta_0 \quad \text{and} \quad B_{-\xi}(t_0) = \emptyset, \quad (227)$$

then for all $t \geq t_0$,

$$\text{loss}(t) \leq \log(2)C_3, \quad \text{acc}(t) = \mu(G_0(t)) \geq 1 - C_3. \quad (228)$$

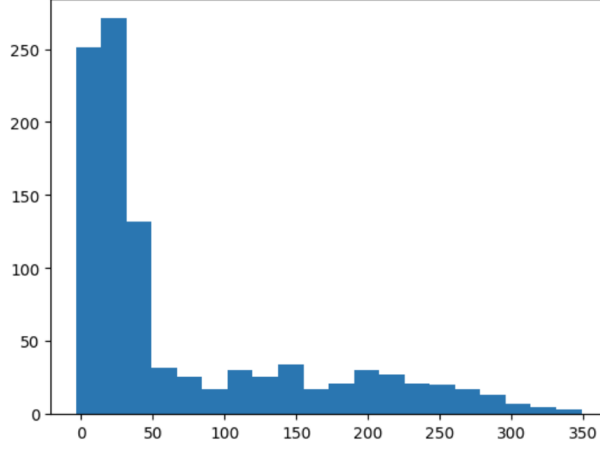


Figure 70: δX for final epoch of DNN.

Example L.5. We trained a DNN for the problem given in Example 7.1 and obtained a 98% accuracy on the training set, which had 1000 objects. δX for the 600th epoch of training is given in Fig. 70.

In this example, $\xi = .7$ and we checked the uniform doubling condition on δX for the parameters: $\sigma = .9$, $\kappa = 2$, $\delta = .2$, and $\ell = .7$. The uniform doubling condition was satisfied in this example, and taking $\eta = 18$ we have that $\delta_0 = .2$ and $C_3 = .047$. This means that loss is bounded by .015, which indeed it was (loss was .015) and accuracy was bounded by .95 for the rest of the training.

Of course, once training has started we need not check the uniform doubling condition on δX as the loss will give us all the information we need regarding the stability of accuracy of the DNN. Still, to obtain stability of accuracy from the training data T only, we first need to prove Proposition L.4.

L.2 Doubling condition on the data set

In [8] a doubling condition was also introduced on the training set directly and it was shown that this doubling condition propagates through the DNN, such that if the training set satisfies the doubling condition then δX also satisfies the doubling condition. One can then ensure that δX satisfies the doubling condition by checking that the training set satisfies the doubling condition. Thus, verifying that the training set satisfies the doubling condition will, in turn, ensure that we have the stability of accuracy.

We can now state the doubling condition for the training set T :

Definition L.6. Let $\bar{\mu}$ be the extension of the measure μ from T to $\mathbb{R}^n$ by $\bar{\mu}(A) = \mu(A \cap T)$ for all $A \subset \mathbb{R}^n$. The doubling condition holds on T if there exists $\kappa > 1$, $k \in \mathbb{N}$ and $m_0, \delta, \sigma > 0$ so that $\forall$ truncated slabs $S \exists \beta_S$ (depending on the slab) such that for any $\varepsilon \leq \beta_S$ we have

$$\bar{\mu}(S_{\kappa\varepsilon}) \geq \min \left\{ \delta, \max \{m_0, (1 + \sigma) \bar{\mu}(S_\varepsilon)\} - m_0 \right\}. \quad (229)$$

Here, β depends on the truncated slab, and so we denote β_S as the β for which the truncated slab S satisfies the doubling condition. See Fig. 19 for an example of the doubling condition. $m_0 > 0$ is needed in the doubling condition on a discrete data set T because we can always find a slab with a small enough width such that it and its double contains only one data point of T . Thus, no discrete data set T will satisfy the doubling condition with $m_0 = 0$. The residual mass m_0 is the smallest amount of data for which the doubling condition must be satisfied and we want it to at least be bigger than the measure of a single data point in the training set T .

Using this doubling condition, the following theorem was essentially proved in [8]:

Theorem L.7 (Berlyand, Jabin, Safsten, 2021). *Suppose the training set T satisfies the doubling condition with constants m_0, κ, σ , and β_S then for a DNN with the absolute value activation function, δX will also satisfy the doubling condition though with different constants $2^{\sum_{i=1}^l c(i)} K m_0, \frac{\sigma}{2^{\sum_{i=1}^l c(i)} K m_0}$ and κ . Here $c(i)$ is the number of nodes in the i th layer of the DNN and K is the number of classes.*

There are a couple of difficulties with using Theorem L.7 and Theorem L.2. First, the constant $2^{\sum_{i=1}^l c(i)} K m_0$ can be large if the number of nodes in the DNN is large, and so $2^{\sum_{i=1}^l c(i)} K m_0 \leq \frac{\varepsilon}{2C_1}$ in condition (218) will be hard to satisfy. Further, Theorem L.7 does not provide a method for obtaining β on δX from the β_S on the training set T , and so one cannot verify condition (218) from the training set alone. This will be described in greater detail in the next section which motivates our new results. Using the uniform doubling condition, one can obtain the following similar result:

Proposition L.8. *Suppose the training set T satisfies the uniform doubling condition with constants ℓ, κ, σ , and β and with the family of truncation sets L , then for a.e. DNN with the absolute value activation function and truncation set $B \in L$, δX will also satisfy the doubling condition though with different constants $\ell d_{\min} \leq \ell' \leq \ell d_{\max}, \beta d_{\min}, \sigma$ and κ .*

L.3 Why truncations in the uniform doubling condition might not be important

The essence of the proof for Theorem 8.6 is as follows. First, we prove Prop. L.4 for stability of accuracy based on the uniform doubling condition on δX . Then we use Prop. L.8 to say that the uniform doubling condition propagates from the training set T to δX . What we want is that the interval I_ℓ of size ℓ around 0 in δX satisfies the uniform doubling condition given in Def. L.3. The preimage of this interval, under the DNN, is a union of truncated slabs in $\mathbb{R}^n$ which we denote U_ℓ . For large DNNs, we might have millions if not more truncated slabs in this union. If by doubling the widths of all truncated slabs in this union by κ we obtain a set, which we call $U_{\kappa\ell}$, that contains a total of $(1 + \sigma)$ more elements (counting all elements in two or more truncated slabs only once) than were in the original union, we would know that

$$\mu(I_{\kappa\ell}) > (1 + \sigma)\mu(I_\ell). \quad (230)$$

In general, if

$$\mu(U_{\kappa^{i+1}\ell}) > (1 + \sigma)\mu(U_{\kappa^i\ell}) \quad (231)$$

then

$$\mu(I_{\kappa^{i+1}\ell}) > (1 + \sigma)\mu(I_{\kappa^i\ell}). \quad (232)$$

So if (231) holds for all $\ell\kappa^i < \beta$, we would have the stability of accuracy result given in Theorem L.4 and Theorem 8.6. Equation (231) can be checked directly for different unions of truncated slabs U' , though the computation might not practically be feasible. However, the essence of (231) is that for some large set U of possibly millions of truncated slabs we have some "doubling" property. Meaning that if we "double" U we obtain $(1 + \sigma)$ more elements. Because U is large, it might be practically useful to consider other methods for finding the global "doubling" properties of the data set. This can be done by either ignoring the truncations in the uniform doubling condition (i.e. SUDC) or possibly considering checking the doubling condition for arbitrary sets (possibly even very complicated ones such as a subset of what U might look like). The more different subsets of $\mathbb{R}^n$ satisfy an equation of the form given in (231), the more likely it is that (231) itself is satisfied and that we have stability of accuracy.

M More motivation for introducing the uniform doubling condition

As mentioned, we want to have a doubling condition on the training set that can be propagated through the DNN to δX so that verifying the doubling condition on the training set ensures the stability of accuracy during training. This is why Definition L.6 and Theorem L.7 are introduced. Note, however, that in Definition L.1 we have a single constant β in the doubling condition on δX , while in Definition L.6 the constant β_S depends on the slab S . In [8] no method is given for obtaining the β for the doubling condition on δX from the β_S on T . Thus, we say that there is no way of quantifying the relationship between β_S on T and β on δX . Even if we show that the doubling condition holds on T , we cannot use Theorem L.7 and Theorem L.2 to obtain a complete stability of accuracy result. This is because, we must know β on δX to know whether the condition (218) in Theorem L.2 is satisfied, and there is no way of knowing β from the β_S given in the doubling condition on T . Specifically, in condition (218) β is used to obtain an upper bound on η (which is important for quantifying the good and bad sets), yet there is no way of knowing what β on δX is from the β_S in the doubling condition on T .

Thus the doubling condition on T does not give a fully quantifiable stability result. That is, the doubling condition on the training set provides some constants which are then used in the stability of accuracy theorem, however in [8] the full relationship between the constants of the doubling condition and the stability theorem had not been established. The first goal of the new results is to introduce a new uniform doubling condition (given in Def. 8.3) on the training set which ensures the stability of accuracy during training in a quantifiable manner. Thus, by verifying the doubling condition on the data set, one can ensure the stability of accuracy without having to know δX directly.

This uniform doubling condition has some benefits over the doubling condition given in Definition L.6. First, β is fixed and does not depend on the slab S . This allows us to propagate this β through the DNN to a doubling condition on δX and thus obtain a quantifiable stability of accuracy result. Thus, all constants in the stability theorem come directly from the given data T , though only DNNs with specific architecture and parameters and reasonable $d_{\min}$ and $d_{\max}$ will have that δX satisfies the doubling condition. Knowing what $d_{\min}$ and $d_{\max}$ are and if the DNN truncation set $B \in L$ requires knowledge of the DNN, but no knowledge of $X(T)$ is needed. Second, we replace the constant m_0 with the constant ℓ , and while the constant m_0 blows up in Theorem L.7 the constant ℓ does not blow up during the propagation through the DNN. Recall that the constant m_0 ensured that a data set with discrete data points might still satisfy the doubling condition, however, because in the uniform doubling condition we don't need to check the doubling condition for all $\epsilon < \beta$ but only for $\ell\kappa^i < \beta$, m_0 is not needed. Thus, in the doubling condition, if we did not have m_0 then no data set would satisfy the doubling condition because we can take slabs small enough that only one data point is contained in S_ϵ and $S_{\kappa\epsilon}$. However, in the uniform doubling condition, we only consider slabs bigger than ℓ and so we can remove the constant m_0 from the condition. Finally, for any given truncated slab S , we need only check the condition for a finite number of slabs $S_{\ell\kappa^i}$ with $\ell\kappa^i < \beta$ rather than an infinite number of slabs S_ϵ with $\epsilon < \beta$.

Remark M.1. Take a manifold H to satisfy the uniform doubling condition with $k > 0$. For many such H , with H a smooth manifold with measure μ distributed uniformly, we must have a restriction on the truncation sets in the family of truncation sets L . For simplicity, it suffices to explain why this is true for the case when H is a bounded curve in $\mathbb{R}^2$ with a boundary. Suppose L includes all possible truncation sets of size k , then we can truncate a slab with center x which is β distance from the edges of the curve in such a manner that the uniform doubling condition will be violated for every H . This is because we are β close to the boundary of the curve H and so $S_{\kappa\epsilon}$ will not contain more of the curve if the truncation is normal to the curve and passes through x . See Fig. 71. Thus, H would only satisfy the uniform doubling condition if no truncation sets in L has a truncation β close to the boundary of H . This is why the family of truncation sets L must be introduced in the case of the uniform doubling

condition (in which β does not depend on the slabs S).

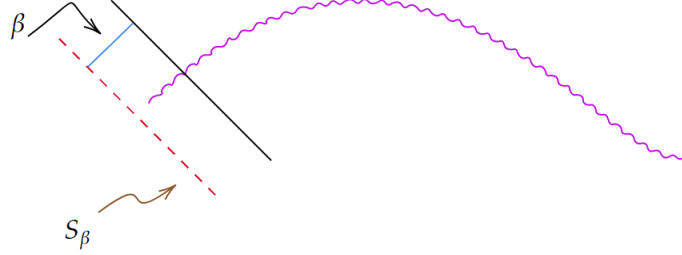


Figure 71: The curve H is shown as a squiggly curve (in purple). Here $\mu(S_\beta) = \mu(S_{\kappa\beta})$, and so would not satisfy the doubling condition.

N Proof of main result: stability theorem via uniform doubling condition on T

The main result in this section is given in Theorem 8.6. But first, we provide a couple of lemmas that show how one can quantitatively propagate the uniform doubling condition from the training set T to δX .

First we introduce a slight variation of the uniform doubling condition which is only needed for a technical reason:

Definition N.1. Let $\bar{\mu}$ be the extension of the measure μ from T to $\mathbb{R}^n$ by $\bar{\mu}(A) = \mu(A \cap T)$ for all $A \subset \mathbb{R}^n$. Let L be a family of size k truncation sets. The uniform doubling condition holds on T if there exists $\kappa > 1$ and δ, σ, β so that $\forall$ truncated slabs S s.t. S is generated by a truncated set $h \in L$ there exists $\ell_S > 0$, such that for all $\ell_S \kappa^i < \beta$ with $i \in \mathbb{N}$, we have

$$\bar{\mu}(S_{\ell_S \kappa^{i+1}}) \geq \min\{\delta, (1 + \sigma) \bar{\mu}(S_{\ell_S \kappa^i})\}. \quad (233)$$

Remark N.1. The only difference between the uniform doubling condition and this definition is that now ℓ depends on the slab S .

We can now state the following lemma which shows how the doubling condition on T can propagate through the absolute value activation function:

Lemma N.2. Suppose the data set T satisfies the uniform doubling condition with constants $\kappa > 1$, $\delta, \beta, \ell, \sigma > 0$, family L of size k truncation sets consisting of a single set of coordinate planes $\{P_1, \dots, P_n\}$ and truncation number $k = n$. Define $\lambda : \mathbb{R}^n \rightarrow \mathbb{R}^n$:

$$\lambda(x_1, x_2, \dots, x_n) = (|x_1|, |x_2|, \dots, |x_n|).$$

Then $\lambda(T)$ also satisfies the doubling condition for scale parameter β , scale factor ℓ , truncation number 0, δ and $\frac{\sigma}{2n}$. Furthermore, if no elements of T are contained in the coordinate planes $P_1, \dots, P_n$ then σ stays constant. λ also propagates the variation uniform doubling condition given in Def. N.1 with ℓ_S and all other constants staying the same.

Proof. Note that $\lambda = \lambda_1 \circ \lambda_2 \circ \dots \circ \lambda_n$, with $\lambda_k(x_1, x_2, \dots, x_n) = (x_1, x_2, \dots, |x_k|, \dots, x_n)$. Thus we prove this lemma for the simpler case, that $\lambda_1(x_1, \dots, x_n) \rightarrow (|x_1|, \dots, x_n)$ preserves the doubling condition on T with the same scale parameter and $\frac{\sigma}{2}$ instead of σ . Then one can use the same proof to show that $\lambda_k(x_1, x_2, \dots, x_n)$ also preserves the doubling condition with $\frac{\sigma}{2}$ instead of σ and the proof for λ follows from $\lambda = \lambda_1 \circ \lambda_2 \circ \dots \circ \lambda_n$. For all truncated slabs S , we want to show that $S_{\ell_S \kappa^i}$ satisfies the doubling condition for $\ell_S \kappa^i < \beta$. Take $\bar{S}_\epsilon$ to be the preimage of S_ϵ with $x_1 \geq 0$ and $\bar{S}'_\epsilon$ to be the preimage with $x_1 \leq 0$. It then makes sense to separate

$$\begin{aligned} \bar{S}_\epsilon &= S_\epsilon^2 \cup S_\epsilon^0, & \bar{S}'_\epsilon &= S_\epsilon^1 \cup S_\epsilon^0, \\ S_\epsilon^2 &= \bar{S}_\epsilon \cap \{0 > x_1\}, & S_\epsilon^0 &= \bar{S}_\epsilon \cap \{x_1 = 0\} = \bar{S}'_\epsilon \cap \{x_1 = 0\}, & S_\epsilon^1 &= \bar{S}'_\epsilon \cap \{x_1 < 0\}. \end{aligned}$$

By the doubling condition on $\bar{S}_\epsilon$, we have for $\epsilon = \ell_S \kappa^i < \beta$

$$\mu(\bar{S}_{\kappa\epsilon}) = \mu(S_{\kappa\epsilon}^2) + \mu(S_{\kappa\epsilon}^0) \geq \min\{\delta, (1 + \sigma) \mu(\epsilon \bar{S})\}. \quad (234)$$

Similarly with regard to $\bar{S}'_\epsilon$, for $\epsilon = \ell_S \kappa^i < \beta$

$$\mu(\bar{S}'_{\kappa\epsilon}) = \mu(S_{\kappa\epsilon}^1) + \mu(S_{\kappa\epsilon}^0) \geq \min\{\delta, (1 + \sigma) \mu(\epsilon \bar{S}')\}. \quad (235)$$

Notice, σ remains fixed when $\mu(P_1) = \emptyset$, due to the fact that the truncated slab $\bar{S}_\epsilon$ and $\bar{S}'_\epsilon$ both satisfy the doubling condition independently. Thus, if $\mu(P_1)$ is empty then simply applying (234) and (235) we get

$$\mu(S_{\kappa\epsilon}) > \min\{\delta, (1 + \sigma) \mu(S_\epsilon)\} \quad (236)$$

For the more complicated case, when $\mu(P_1) \neq \emptyset$, see Section P.

Remark N.2. $\lambda_1(T)$ satisfies doubling condition with truncation number $k - 1$ because one truncation is required at $(0, x_2, \dots, x_n)$. Thus, λ will satisfy the doubling condition with truncation number 0. $\square$

We now show how the doubling condition propagates through a linear map. First, we collect some useful facts from linear algebra in the following lemma. This lemma will help determine how β and ℓ change under a linear map.

Lemma N.3. *Let M be an $m \times n$ matrix. Then the minimum value of $\|Mx\|$, where x ranges over unit vectors in $\mathbb{R}^n$ orthogonal to $\text{Ker}(M)$, is the smallest nonzero singular value $\sigma_{\min}$ of M . Similarly, the maximum values of $\|Mx\|$ is the largest singular value $\sigma_{\max}$ of M .*

For a linear map M and a family of truncation sets L , take $M(L)$ to be the mapping of the truncation sets in L by M . Thus, $M(L)$ is also a family of truncation sets.

Lemma N.4. *If T satisfies the uniform doubling condition (Definition N.1) for some constants and some family of truncation sets L , then $M(T)$ also satisfies the variation uniform doubling condition (Definition N.1) for any linear map M with the family of truncation sets $M(L)$. Take $d_{\min} = \min_i (\sigma_i)^2$ and $d_{\max} = \max_i (\sigma_i)^2$ with σ_i the nonzero singular values of M . Then all slabs $S \subset M(T)$ satisfy the doubling condition with $d_{\min}\beta$ instead of β , some ℓ'_S with $d_{\min}\ell \leq \ell'_S \leq d_{\max}\ell$ instead of ℓ . All other constants stay the same. Similarly with regard to Def. N.1. If T satisfies Def. N.1 then all slabs $S \subset M(T)$ satisfy the variation of the uniform doubling condition with $d_{\min}\beta$ instead of β , some ℓ'_S with $d_{\min}\ell_S \leq \ell'_S \leq d_{\max}\ell_S$ instead of ℓ_S .*

Proof. Suppose $v_i \cdot x \leq t_i$ then any y s.t. $My = x$ also satisfies that

$$(M^T v_i) \cdot y \leq t_i.$$

Take S to be a truncated slab in $M(T)$ and S' to be its preimage in T . By Lemma N.3 the width of every slab $M(S')$ is bigger or equal to $d_{\min}$, and thus the width of $M(S'_\beta)$ is bigger or equal to $d_{\min}\beta$. Thus, we can take $d_{\min}\beta$ to be the scale parameter in $M(T)$. We now need to find the scale factor for the doubling condition in $M(T)$

For the slab S , there exists $d_{\min}\ell \leq \ell'_S \leq d_{\max}\ell$ such that the preimage of $S_{\ell'_S}$ is the slab $S'_{\ell'_S}$. We also have that the preimage of $S_{\kappa^i\ell'_S}$ is $S'_{\kappa^i\ell'_S}$. Thus because $S_{\kappa^i\ell_S}$ satisfies the doubling condition for $\kappa^i\ell_S < \beta$ we must have that $S_{\kappa^i\ell'_S}$ also satisfies the doubling condition for $\kappa^i\ell'_S < d_{\min}\beta$ with $d_{\min}\ell \leq \ell'_S < d_{\max}\ell$ for all S . We arrive at the conclusion that $d_{\min}\beta$ is the scale parameter in $M(T)$ and each slab S in $M(T)$ now has a different scale factor ℓ'_S . $\square$

In both of these lemmas, we fully quantify the manner in which the constants of the doubling condition change, and so given that a layer of a DNN is a linear map composed with the activation function we have a quantifiable manner in which the doubling condition propagates through a DNN. If we find an β for which the doubling condition holds on T , and we know $d_{\min}$ and $d_{\max}$ of every linear map of the DNN then we know exactly how δX satisfies the doubling condition.

Remark N.3. There is difficulty with allowing the family of truncation sets L to consist of all possible truncation sets when $k > 1$ (the size of the truncation sets is bigger than 1). This is because two truncations β close to each other will ensure that the uniform doubling condition is violated, see Fig 72. Thus no data set will satisfy the current uniform doubling condition if L contains all possible truncation sets with $k > 1$. The truncation set does not move under the doubling of a slab S , and so we can bound any slab (regardless of the dimension we are in), by two truncations which are β close to each other.

However, we need the truncation number $k \geq$ the number of nodes in DNN. By Lemma N.2 each layer of the DNN reduces the truncation number k by the nodes in that layer. This is because each activation function $\lambda(x_1, x_2, \dots, x_n) \rightarrow (|x_1|, |x_2|, \dots, |x_n|)$ requires a reduction of the truncation number k by at least n . So the doubling condition on T implies stability of DNN only if $k \geq$ is the number of nodes in the DNN. But as mentioned, for $k \geq 2$, slabs will never satisfy the doubling condition for any data T . Example: if truncation number $k = 2$ the slab S_β^0 does not satisfy the uniform doubling condition (truncations don't move under doubling), see Fig. 72.

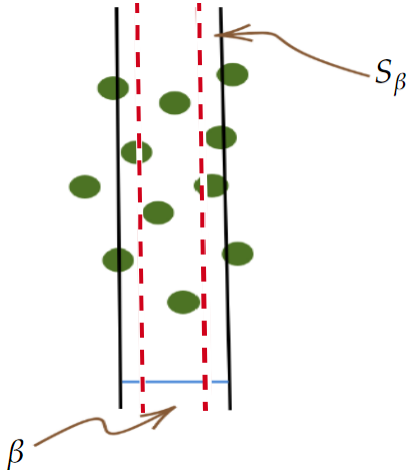


Figure 72: Slabs in $\mathbb{R}^2$. Red dashed lines are the original slab, and black solid lines are two truncations. $\mu(S_{\kappa\beta}) = \mu(S_\beta)$ (slab bounded by truncations), thus the doubling condition is not satisfied.

'Almost' parallel truncations also pose a problem. This is because not only parallel but also 'almost' parallel truncations ensure that the doubling condition is never satisfied. The data set T is bounded, so

'almost' parallel truncations can capture lots of data and ensure the doubling condition is not satisfied, see Fig. 73.

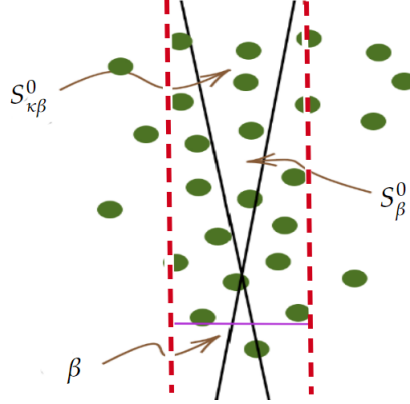


Figure 73: Slab in $\mathbb{R}^2$. Red dashed lines are the original slab (without the truncation), and black solid lines are two truncations. S^0 is the center slab after truncation. Even though truncations are not parallel, still $\mu(S^0_{\kappa\beta}) = \mu(S^0_{\beta})$, thus the doubling condition is not satisfied.

- So data T never satisfies the uniform doubling condition with single β because of 'almost' parallel truncations
- Thus, we introduce the set L of all admissible truncations and we use L to impose restrictions on the allowed truncation sets.

We now give a lemma providing an upper bound on the loss function.

Lemma N.5. *For a DNN with the cross entropy loss function we have*

$$\sum_{s \in G_{\eta^*}^c(t)} \mu(s) \log(1 + e^{-\eta^*}) \leq \bar{L}(t) \leq \bar{L}(t_0). \quad (237)$$

See Section P for proof of the lemma. Now we derive an upper bound on the loss function based on the uniform doubling condition on δX .

Lemma N.6. *Suppose that $T \subset \mathbb{R}^n$ with weights $\mu(s)$ is a training set for a softmax DNN such that $\delta X(T)$ satisfies the uniform doubling condition, at time t_0 at $x = 0$, for some constants σ, δ, β , $1 > \ell > 0$ and $\kappa > 1$. If for some $1 < \eta < \frac{\beta}{2}$, $\delta_0 > 0$, and $\delta_0 < \delta$,*

$$\mu(G_{\eta}(t_0)) > 1 - \delta_0 \quad (238)$$

and

$$B_{-\xi}(t_0) = \emptyset, \quad (239)$$

then the cross-entropy loss is bounded by:

$$\begin{aligned} \bar{L}(t_0) &\leq \log(1 + (K-1)e^{-\eta}) + \log(1 + (K-1)e^{-(\frac{\eta}{\kappa})})\delta_0 + \\ &\quad \frac{\delta_0}{(\sigma+1)^{\frac{\log(\frac{\eta}{\xi})}{\log \kappa} - 1}} \log(1 + e^{\xi}(K-1)) \\ &\quad + \sum_{i=0}^{p-1} \frac{\delta_0}{(\sigma+1)^{\frac{\log(\frac{\eta}{\ell\kappa^i+1})}{\log \kappa} - 1}} \log(1 + (K-1)e^{-\ell\kappa^{i+1}}). \end{aligned} \quad (240)$$

with $\xi = \ell\kappa^i$ for some $i \geq 0$ and with $p = \lfloor \log(\frac{\eta}{\xi}) / \log \kappa \rfloor$.

Proof. We have that $\eta \geq 1$. Consider $\kappa > 1$ and $I := [-\ell, \ell) \subset [-\infty, \eta)$. Since $\eta < \frac{\beta}{2}$, we may apply the doubling condition at point 0 and we have that

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa I\}) \geq \min\{\delta, (1 + \sigma) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\})\}.$$

Applying the doubling condition repeatedly j times in this last case, we conclude that

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa^j I\}) \geq \min\{\delta, (1 + \sigma)^j \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\})\}.$$

By (238), $\mu(\{s \in T : \delta X(s, \alpha(t_0)) < \eta\}) \leq \delta_0$. Therefore, provided $\kappa^j I \subset (-\infty, \eta)$,

$$\delta_0 \geq \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in \kappa^j I\}) \geq \min\{\delta, (1 + \sigma)^j \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\})\}.$$

We conclude that

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\}) \leq \frac{\delta_0}{(\sigma + 1)^j} \quad (241)$$

for all j so that $\kappa^j I \subset (-\infty, \eta)$, or equivalently, for all j so that $\ell \kappa^j \leq \eta$. Obviously, (241) is best for j as large as possible with the largest value given by

$$j_{\max} = \left\lfloor \frac{\log(\frac{\eta}{\ell})}{\log \kappa} \right\rfloor > \frac{\log(\frac{\eta}{\ell})}{\log \kappa} - 1.$$

Therefore we have thus proved the bound,

$$\begin{aligned} \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I\}) &\leq \frac{\delta_0}{(\sigma + 1)^{j_{\max}}} \\ &< \frac{\delta_0}{(\sigma + 1)^{\frac{\log(\frac{\eta}{\ell})}{\log \kappa} - 1}} \end{aligned}$$

Now we can apply the calculation which obtained (N) to the intervals $[0, \ell \kappa^i)$ with $i \in \mathbb{N}$. Let $p = \lfloor \log(\eta/\ell)/\log \kappa \rfloor$ and let $I_i = [\ell \kappa^i, \ell \kappa^{i+1})$ for all $i \in \mathbb{N}$ with $i < p$. We also define $I_{-1} = [-\xi, \ell)$ and $I_p = [\ell \kappa^p, \eta)$. For $i \geq 0$, $I_i \subset [-\ell \kappa^{i+1}, \ell \kappa^{i+1})$. Therefore by the above calculation which gave (N) we obtain,

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_i\}) < \frac{\delta_0}{(\sigma + 1)^{\frac{\log(\frac{\eta}{\ell \kappa^{i+1}})}{\log \kappa} - 1}} \quad (242)$$

The interval $I_{-1} \subset [-\xi, \xi]$ which is centered at 0 and has width smaller than ξ , so

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_{-1}\}) \leq \frac{\delta_0}{(\sigma + 1)^{\frac{\log(\frac{\eta}{\xi})}{\log \kappa} - 1}}. \quad (243)$$

Finally since $\kappa^p > \kappa^{\log \eta / \log \kappa - 1}$, we simply bound for I_p

$$\mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_p\}) \leq \delta_0. \quad (244)$$

For each $s \in T$, either $\delta X(s, \alpha(t_0)) \geq \eta$, or $\delta X(s, \alpha(t_0)) \in I_i$ for integer $i \geq -1$. Therefore,

$$\begin{aligned} \bar{L}(t_0) &\leq \sum_{s \in T} \mu(s) \log \left(1 + (K - 1) e^{-\delta X(s, \alpha(t_0))} \right) \\ &= \sum_{\substack{s \in T \\ \delta X(s, \alpha(t_0)) \geq \eta}} \mu(s) \log \left(1 + (K - 1) e^{-\delta X(s, \alpha(t_0))} \right) + \sum_{i=-1}^p \sum_{\substack{s \in T \\ \delta X(s, \alpha(t_0)) \in I_i}} \mu(s) \log \left(1 + (K - 1) e^{-\delta X(s, \alpha(t_0))} \right) \\ &\leq \log(1 + (K - 1) e^{-\eta}) \sum_{\substack{s \in T \\ \delta X(s, \alpha(t_0)) \geq \eta}} \mu(s) + \sum_{i=-1}^p \log \left(1 + (K - 1) e^{-\inf I_i} \right) \sum_{\substack{s \in T \\ \delta X(s, \alpha(t_0)) \in I_i}} \mu(s). \end{aligned}$$

By decomposing

$$\begin{aligned}
\bar{L}(t_0) &\leq \log(1 + (K-1)e^{-\eta})\mu(G_\eta(t_0)) + \sum_{i=-1}^p \log\left(1 + (K-1)e^{-\inf I_i}\right) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_i\}) \\
&\leq \log(1 + (K-1)e^{-\eta}) + \log\left(1 + (K-1)e^\xi\right) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_{-1}\}) \\
&\quad + \log(1 + (K-1)e^{-\left(\frac{\eta}{\kappa}\right)}) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_p\}) \\
&\quad + \sum_{i=0}^{p-1} \log(1 + (K-1)e^{-\kappa^{i+1}}) \mu(\{s \in T : \delta X(s, \alpha(t_0)) \in I_{i+1}\}).
\end{aligned}$$

We now use (242), (243) and (244) to derive

$$\begin{aligned}
\bar{L}(t_0) &\leq \log(1 + (K-1)e^{-\eta}) + \log(1 + (K-1)e^{-\left(\frac{\eta}{\kappa}\right)}) \delta_0 + \\
&\quad \frac{\delta_0}{(\sigma+1)^{\frac{\log\left(\frac{\eta}{\xi}\right)}{\log \kappa} - 1}} \log(1 + e^\xi (K-1)) \\
&\quad + \sum_{i=0}^{p-1} \frac{\delta_0}{(\sigma+1)^{\frac{\log\left(\frac{\eta}{\ell \kappa^{i+1}}\right)}{\log \kappa} - 1}} \log(1 + (K-1)e^{-\ell \kappa^{i+1}}).
\end{aligned} \tag{245}$$

N.1 Proof of Proposition L.4

We start with the following bound given in Lemma N.5

$$\sum_{s \in G_{\eta^*}^c(t)} \mu(s) \log(1 + e^{-\eta^*}) \leq \bar{L}(t) \leq \bar{L}(t_0). \tag{246}$$

As a consequence, we obtain from Lemma N.6 that provided $G_{-\xi}^c(t_0) = \emptyset$ and $\mu(G_\eta(t_0)) > 1 - \delta_0$,

$$\begin{aligned}
\mu(G_0^c(t)) &\leq C_3(\eta, K, \kappa, \sigma, \ell, \xi) := \frac{1}{\log(2)} (\log(1 + (K-1)e^{-\eta}) + \log(1 + (K-1)e^{-\left(\frac{\eta}{\kappa}\right)}) \delta_0 + \\
&\quad \frac{\delta_0}{(\sigma+1)^{\frac{\log\left(\frac{\eta}{\xi}\right)}{\log \kappa} - 1}} \log(1 + e^\xi (K-1)) + \sum_{i=0}^{p-1} \frac{\delta_0}{(\sigma+1)^{\frac{\log\left(\frac{\eta}{\ell \kappa^{i+1}}\right)}{\log \kappa} - 1}} \log(1 + (K-1)e^{-\ell \kappa^{i+1}})).
\end{aligned} \tag{247}$$

Here $p = \lfloor \log(\eta/\ell) / \log \kappa \rfloor$. This completes the proof. $\square$

We can now present the proof for the main result in this paper, Theorem 8.6

N.2 Proof of Theorem 8.6

Proof. Take I to be an interval in $\mathbb{R}^1$ centered at $x = 0$ with width $\ell' \kappa^i < d_{\min} \beta$ with $\ell_S < d_{\max} \ell$. We wish to prove that $\exists \ell'$ for which $\mu(I_\kappa) > \min\{\delta, (1 + \sigma)\mu(I)\}$ at t_0 . Take $\tilde{T}$ to be the image of the data T by the DNN. We note that if $f_i^{-1}(\tilde{T})$ (the preimage of $\tilde{T}$ with respect to the last layer in the DNN) satisfies the uniform doubling condition with truncation set $\{f_i^{-1}(P_1), f_i^{-1}(P_2), \dots, f_i^{-1}(P_{c(i)})\} \cup f_i^{-1}(P_k)$ with $1 \leq k \leq c(i)$ such that $f_i^{-1}(P_k)$ are strict subsets of $R^{c(i-1)}$, then $\tilde{T}$ satisfies the variation uniform doubling condition (Def. N.1) though with different constants given in lemma N.2. In fact, by lemmas N.2 and N.3, we get that if $f_i^{-1}(\tilde{T})$ satisfies the uniform doubling condition with β, σ, k, ℓ

and the mentioned truncation set, then $\tilde{T}$ satisfies the variation uniform doubling condition with $\frac{\sigma}{2^{c(i)}}$, $d'_{\min}\beta$ and some $\ell'd'_{\min} \leq \ell' \leq d'_{\max}\ell$. Here $d'_{\min}$ is the smallest singular value of the matrix part of the affine function M_i and $d'_{\max}$ is the largest singular value. However, for a.e. DNN the truncation set $\mu(\{f_i^{-1}(P_1), f_i^{-1}(P_2), \dots, f_i^{-1}(P_{c(i)})\})$ has measure zero (the measure with respect to the data), and so by lemma N.2 we have that σ remains the same.

Similarly, assume $f_{i-1}^{-1}(f_i^{-1}(\tilde{T}))$ satisfies the uniform doubling condition with truncation set

$$\{f_{i-1}^{-1}(P_1), f_{i-1}^{-1}(P_2), \dots, f_{i-1}^{-1}(P_{c(i-1)})\} \cup \{f_{i-1}^{-1}(f_i^{-1}(P_1)), \dots, f_{i-1}^{-1}(f_i^{-1}(P_{c(i)}))\}$$

(again only considering the preimages which are strict subsets of $R^{c(i-2)}$). Then we have that $f_i^{-1}(\tilde{T})$ satisfies the variation uniform doubling condition with truncation set

$$\{f_i^{-1}(P_1), f_i^{-1}(P_2), \dots, f_i^{-1}(P_{c(i)})\}$$

and so $\tilde{T}$ satisfies the doubling condition for $d_{\min}\ell \leq \ell_S \leq \ell d_{\max}$ with $d_{\min}$ and $d_{\max}$ coming from $W_2 \circ W_1$.

Thus, by induction if T satisfies the uniform doubling condition with truncation set $B \in L$ then, for a.e. DNN, $\tilde{T}$ satisfies the variation uniform doubling condition with $d_{\min}\beta$, and some $d_{\max}\ell \leq \ell_S \leq \ell d_{\max}\ell$. Finally, we must show that if $\tilde{T}$ satisfies the uniform doubling condition then so does $\delta X(\tilde{T})$, though possibly with different constants. Note that $\delta X(s)$ for s in the class $i(s)$ is a shift of $X_{i(s)}(s, \alpha)$ by $\max_{j \neq i(s)} X_j(s, \alpha)$ and then a projection of the other components of $X(s, \alpha)$ to zero. Thus, $\delta X(s, \alpha)$ restricted to every class $i(s)$ and further restricted to elements $s \in T$ such that $\max_{j \neq i(s)} X_j(s, \alpha)$ occurs for class i' is a linear map with $d_{\max} = d_{\min} = 1$. Thus, by using Proposition L.4 and the fact that δX satisfies the uniform doubling condition at $x = 0$ at time t_0 , with the constants $d_{\min}\beta$, some $\ell d_{\min} \leq \ell_I \leq \ell d_{\max}$ and σ , we obtain the result. Specifically, the constant C_2 is obtained from taking the max over all $\ell d_{\min} \leq \ell_I \leq \ell d_{\max}$ of the constant C_3 . All other constants stay the same. $\square$

O Other Activation Functions

The goal of this section is to prove that piecewise linear activation functions with only finitely many critical points also preserve the uniform doubling condition. As we will see, there is only one problem that arises when trying to show that a piecewise linear function g (with only finitely many critical points) preserves the doubling condition. This is dealing with the critical points of g .

Definition O.1. Take g to be a 1D function with only finitely many critical points in every bounded interval. Take v_g to be the infimum distance between the critical values of neighboring critical points of g . Each critical point u has at most two neighbors, which are the closest two critical points to u with one on its left and the other on its right.

Lemma O.2. Let $g_1 : \mathbb{R} \rightarrow \mathbb{R}$ be any piece-wise continuous linear functions with only a finite number of critical points in every bounded interval. Suppose for g_1 that v_{g_1} is bigger than $2\kappa\beta$. Take $\{a_1^1, \dots, a_1^i\}$ to be the set of critical points of g_1 , and suppose the data set T satisfies the uniform doubling condition with the set L consisting of the hyperplanes $\{P_{a_1^1}, \dots, P_{a_1^i}\}$ with $P_{a_1^k} \subset \mathbb{R}^n$ and $P_{a_1^k} = \{(x_1, x_2, \dots, x_n) : x_1 = a_1^k\}$. Then

$$g : (x_1, x_2, \dots, x_n) \rightarrow (g_1(x_1), x_2, \dots, x_n)$$

preserves the doubling condition for T with new constants, $\alpha_1\ell$ instead of ℓ , $\sigma/2$ instead of σ , $\alpha_2\beta$ instead of β and $k - \hat{l}$ instead of k .

Here, we take $l = \max_{x \in g(T)}(|x_1|)$ and $\hat{l}$ is the number of critical points of g_1 in the interval $[-l, l]$. We take α_1 to be the largest slope of the linear piecewise function g_1 and α_2 to be the smallest slope.

Proof.

Step 1: g_1 has one critical point.

Now let us assume that g has critical points. Note that such critical points are obtained from critical points of g_1 . In general, when dealing with such critical points we use the same strategy that was used with regard to absolute value activation functions (when g_1 is the absolute value). First, let us assume that there is only one critical point for g and that this critical point is at $x_1 = a_1^1$.

For all truncated slabs E , we want to show that E satisfies the doubling condition for $\beta' = \alpha_2\beta$ and $\ell_E < \alpha_2\ell$. Take $\bar{E}_\varepsilon$ to be the preimage of E_ε with $x_1 \geq a_1^1$ and $\bar{E}'_\varepsilon$ to be the preimage with $x_1 \leq a_1^1$. We can separate

$$\begin{aligned}\bar{E}_\varepsilon &= E_\varepsilon^2 \cup E_\varepsilon^0, & \bar{E}'_\varepsilon &= E_\varepsilon^1 \cup E_\varepsilon^0, \\ E_\varepsilon^2 &= \bar{E}_\varepsilon \cap \{x_1 > a_1^1\}, & E_\varepsilon^0 &= \bar{E}_\varepsilon \cap \{x_1 = a_1^1\} = \bar{E}'_\varepsilon \cap \{x_1 = a_1^1\}, & E_\varepsilon^1 &= \bar{E}'_\varepsilon \cap \{x_1 < a_1^1\}.\end{aligned}$$

Because $\ell' = \alpha_1\ell$ and $\beta' = \alpha_2\beta$ we have that $\bar{E}_\varepsilon$ and $\bar{E}'_\varepsilon$ satisfy the doubling condition for $\varepsilon = \ell'_E\kappa^i < \alpha_2\beta$ for some $\ell'_E < \alpha_1\ell$ depending on $\bar{E}$ and $\bar{E}'$. Thus, for $\bar{E}_\varepsilon$, we have $\forall \varepsilon = \ell'_E\kappa^i < \alpha_2\beta$

$$\mu(\bar{E}_{\kappa\varepsilon}) = \mu(E_{\kappa\varepsilon}^2) + \mu(E_{\kappa\varepsilon}^0) \geq \min\{\delta, \max\{m_0, (1+\sigma)\mu(\bar{E}_\varepsilon)\} - m_0\}. \quad (248)$$

Similarly with regard to $\bar{E}'_\varepsilon$, $\forall \varepsilon = \ell'_{\bar{E}'}\kappa^i < \alpha_2\beta$

$$\mu(\bar{E}'_{\kappa\varepsilon}) = \mu(E_{\kappa\varepsilon}^1) + \mu(E_{\kappa\varepsilon}^0) \geq \min\{\delta, \max\{m_0, (1+\sigma)\mu(\bar{E}'_\varepsilon)\} - m_0\}. \quad (249)$$

Now, if either $(1+\sigma)\mu(\bar{E}_\varepsilon) \geq \delta$ or $(1+\sigma)\mu(\bar{E}'_\varepsilon) \geq \delta$, we are done.

If both $(1+\sigma)\mu(\bar{E}_\varepsilon) < \delta$ and $(1+\sigma)\mu(\bar{E}'_\varepsilon) < \delta$. Then

$$\begin{aligned}\mu(g^{-1}(E_{\kappa\varepsilon})) &= \mu(E_{\kappa\varepsilon}^1) + \mu(E_{\kappa\varepsilon}^0) + \mu(E_{\kappa\varepsilon}^2) \\ &\geq \frac{\mu(E_{\kappa\varepsilon}^1) + \mu(E_\varepsilon^1)}{2} + \mu(E_{\kappa\varepsilon}^0) + \frac{\mu(E_{\kappa\varepsilon}^2) + \mu(E_\varepsilon^2)}{2} \\ &\geq \mu(E_\varepsilon^1) + \mu(E_\varepsilon^0) + \mu(E_\varepsilon^2) + \frac{\mu(E_{\kappa\varepsilon}^1) + 2\mu(E_{\kappa\varepsilon}^0) + \mu(E_{\kappa\varepsilon}^2) - \mu(E_\varepsilon^1) - 2\mu(E_\varepsilon^0) - \mu(E_\varepsilon^2)}{2}.\end{aligned} \quad (250)$$

Again, we are using the averaging trick to deal with the complication that we only have one E_ε^0 and $E_{\kappa\varepsilon}^0$. Applying now (248) and (249), we find that

$$\mu(g^{-1}(E_{\kappa\varepsilon})) \geq \mu(g^{-1}(E_\varepsilon)) + \frac{\sigma}{2}(\mu(\bar{E}_\varepsilon) + \mu(\bar{E}'_\varepsilon)) \geq (1+\sigma/2)\mu(g^{-1}(E_\varepsilon)) \quad (251)$$

proving the doubling condition with $\sigma' = \sigma/2$.

Note that this part of the proof shows that absolute value activation functions and Leaky ReLU activation functions also preserve the doubling condition. In all these cases we only obtain a single E_ε^0 while we need two E_ε^0 to use the doubling condition. Once again, we deal with this complication by introducing the averaging trick and exchanging σ with $\frac{\sigma}{2}$.

Step 2: g_1 has general critical points.

At this point, we wish to show that if g_1 is any piecewise linear function with only a finite number of critical points in every bounded interval then

$$g : (x_1, x_2, \dots, x_i) \rightarrow (g_1(x_1), x_2, \dots, x_i)$$

preserves the doubling condition for any bounded data T , though with different constants.

Take $\tilde{E}_\varepsilon$ to be the preimage of the slab E_ε and $a_1^1 < \dots < a_1^i$ to be the critical points of g_1 . Recall, the images of neighboring a_1^i (a critical point has at most two neighbors which are the two closest

critical points to it, from the left and right) are more than $2\kappa\beta$ distance away from one another. For $\epsilon < \beta'$ we separate $\bar{E}_{a_i\epsilon} = \tilde{E}_\epsilon \cap \{a_1^{i+1} \geq x_1 \geq a_1^i\}$ and $\bar{E}'_{a'_i\epsilon} = \tilde{E}_\epsilon \cap \{a_1^{i-1} \leq x_1 \leq a_1^i\}$. Note each disjoint connected component of $\tilde{E}_\epsilon$ intersects at most one critical point of g_1 . Furthermore, for each i , we have that $\bar{E}_{a_i\kappa\epsilon} \cup \bar{E}'_{a'_i\kappa\epsilon}$ are disjoint.

Now take, $E_{a_i\epsilon}^2 = \bar{E}_{a_i\epsilon} \cap \{x_1 > a_1^i\}$, $E_{a'_i\epsilon}^1 = \bar{E}'_{a'_i\epsilon} \cap \{x_1 < a_1^i\}$ and $E_{a_i\epsilon}^0$ is the complement of the two.

We must show that E_ϵ satisfies the doubling condition for $\epsilon = \ell'_E < \beta'$ with $\ell'_E \kappa^i < \ell\alpha_1$ and depending on the slab $E_{a_i\epsilon}^1$ and $E_{a'_i\epsilon}^2$. That is, take E_ϵ with $\epsilon < \beta'$.

If $(1 + \sigma)\mu(\bar{E}_{a_i\epsilon}) > \delta$ or $(1 + \sigma)\mu(\bar{E}'_{a'_i\epsilon}) > \delta$ for any i we are done. Dealing with the other cases, we look at each pair of slabs $\bar{E}_{a_i\epsilon}$ and $\bar{E}'_{a'_i\epsilon}$ together. If both $(1 + \sigma)\bar{E}_{a_i\epsilon} < \delta$ and $(1 + \sigma)\bar{E}'_{a'_i\epsilon} < \delta$ (we do this for every i individually) then the same averaging strategy from above works for these two. Specifically,

$$\begin{aligned} \mu(g^{-1}(E_{\kappa\epsilon}))|_{\bar{E}_{a_i\kappa\epsilon} \cup \bar{E}'_{a'_i\kappa\epsilon}} &> \mu(E_{a_i\kappa\epsilon}^1) + \mu(E_{a_i\kappa\epsilon}^0) + \mu(E_{a_i\kappa\epsilon}^2) \\ &\geq \frac{\mu(E_{a'_i\kappa\epsilon}^1) + \mu(E_{a'_i\epsilon}^1)}{2} + \mu(E_{a_i\kappa\epsilon}^0) + \frac{\mu(E_{a_i\kappa\epsilon}^2) + \mu(E_{a_i\epsilon}^2)}{2} \\ &\geq \mu(E_{a'_i\epsilon}^1) + \mu(E_{a_i\epsilon}^0) + \mu(E_{a_i\epsilon}^2) + \\ &\quad \frac{\mu(E_{a'_i\kappa\epsilon}^1) + 2\mu(E_{a_i\kappa\epsilon}^0) + \mu(E_{a_i\kappa\epsilon}^2) - \mu(E_{a'_i\epsilon}^1) - 2\mu(E_{a_i\epsilon}^0) - \mu(E_{a_i\epsilon}^2)}{2}. \end{aligned} \tag{252}$$

Then because $\bar{E}'_{a'_i\epsilon}$ and $\bar{E}_{a_i\epsilon}$ satisfy the doubling condition we get

$$\begin{aligned} \mu(g^{-1}(E_{\kappa\epsilon}))|_{\bar{E}_{a_i\kappa\epsilon} \cup \bar{E}'_{a'_i\kappa\epsilon}} &\geq (\mu(g^{-1}(E_\epsilon))|_{\bar{E}_{a_i\epsilon} \cup \bar{E}'_{a'_i\epsilon}} \\ &+ \frac{\sigma}{2} (\mu(\bar{E}_{\epsilon a_i}) + \mu(\bar{E}'_{\epsilon a'_i}))) \geq (1 + \sigma/2) \mu(g^{-1}(E_\epsilon))|_{\bar{E}_{a_i\epsilon} \cup \bar{E}'_{a'_i\epsilon}}, \end{aligned}$$

Putting both of these scenarios together and summing over i , we get

$$\mu(g^{-1}(E_{\kappa\epsilon})) > (1 + \sigma/2) \mu(g^{-1}(E_\epsilon))$$

given that the $\bar{E}_{a_i\kappa\epsilon} \cup \bar{E}'_{a'_i\kappa\epsilon}$ are disjoint for every i . Thus, for each pair of slabs, the doubling condition holds with $\frac{\sigma}{2}$ instead of σ so it holds for E_ϵ completing the proof. $\square$

P Proof of Lemma N.2

P.1 Proof of Lemma N.2.

Suppose $\mu(P_1) \neq \emptyset$. If either $(1 + \sigma) \mu(\varepsilon \bar{S}) \geq \delta$ or $(1 + \sigma) \mu(\varepsilon \bar{S}') \geq \delta$, we are done.

If both $(1 + \sigma) \mu(\varepsilon \bar{S}) < \delta$ and $(1 + \sigma) \mu(\varepsilon \bar{S}') < \delta$. Then

$$\begin{aligned} \mu(g^{-1}(\kappa \varepsilon S)) &= \mu(S_{\kappa \varepsilon}^1) + \mu(S_{\kappa \varepsilon}^0) + \mu(S_{\kappa \varepsilon}^2) \\ &\geq \frac{\mu(S_{\kappa \varepsilon}^1) + \mu(S_{\varepsilon}^1)}{2} + \mu(S_{\kappa \varepsilon}^0) + \frac{\mu(S_{\kappa \varepsilon}^2) + \mu(S_{\varepsilon}^2)}{2} \\ &\geq \mu(S_{\varepsilon}^1) + \mu(S_{\varepsilon}^0) + \mu(S_{\varepsilon}^2) + \frac{\mu(S_{\kappa \varepsilon}^1) + 2\mu(S_{\kappa \varepsilon}^0) + \mu(S_{\kappa \varepsilon}^2) - \mu(S_{\varepsilon}^1) - 2\mu(S_{\varepsilon}^0) - \mu(S_{\varepsilon}^2)}{2}. \end{aligned}$$

Here we are using an averaging trick to deal with the complication that we only have one S_{ε}^0 and $S_{\kappa \varepsilon}^0$. Applying now (234) and (235), we find that

$$\mu(g^{-1}(\kappa \varepsilon S)) \geq \mu(g^{-1}(\varepsilon S)) + \frac{\sigma}{2} (\mu(\varepsilon \bar{S}) + \mu(\varepsilon \bar{S}')) \geq (1 + \sigma/2) \mu(g^{-1}(\varepsilon S)) \quad (253)$$

proving the doubling condition with $\sigma' = \sigma/2$. Meaning, for $\varepsilon = \ell_S \kappa^i < \beta$ we have

$$\mu(S_{\kappa \varepsilon}) > \min\left\{ \delta, \left(1 + \frac{\sigma}{2}\right) \mu(S_{\varepsilon}) \right\} \quad (254)$$

The critical point of λ_1 at $x_1 = 0$ would then be the only complicated part of λ_1 .

P.2 Proof of Lemma N.5: Upper bound on the loss

Proof. □

We know the DNN ϕ is of the form.

$$\phi(s, \alpha) = \rho \circ X(s, \alpha) = \left(\frac{e^{X_1(s, \alpha)}}{\sum_{k=1}^K e^{X_k(s, \alpha)}}, \dots, \frac{e^{X_K}}{\sum_{k=1}^K e^{X_k(s, \alpha)}} \right). \quad (255)$$

Thus, (255) gives

$$p_{i(s)}(s, \alpha) = \frac{e^{X_{i(s)}(s, \alpha)}}{\sum_{k=1}^K e^{X_k(s, \alpha)}} = \frac{1}{\sum_{k=1}^K e^{X_k(s, \alpha) - X_{i(s)}(s, \alpha)}} = \frac{1}{1 + \sum_{k \neq i(s)} e^{X_k(s, \alpha) - X_{i(s)}(s, \alpha)}}. \quad (256)$$

For each $k \neq i(s)$, $X_k(s, \alpha) - X_{i(s)}(s, \alpha) \leq \max_{k \neq i(s)} X_k(s, \alpha) - X_{i(s)}(s, \alpha) = -\delta X(s, \alpha)$. Using (29) and (256), we obtain the following estimates on $p_{i(s)}(s, \alpha)$:

$$\frac{1}{1 + (K - 1)e^{-\delta X(s, \alpha)}} \leq p_{i(s)}(s, \alpha) \leq \frac{1}{1 + e^{-\delta X(s, \alpha)}}. \quad (257)$$

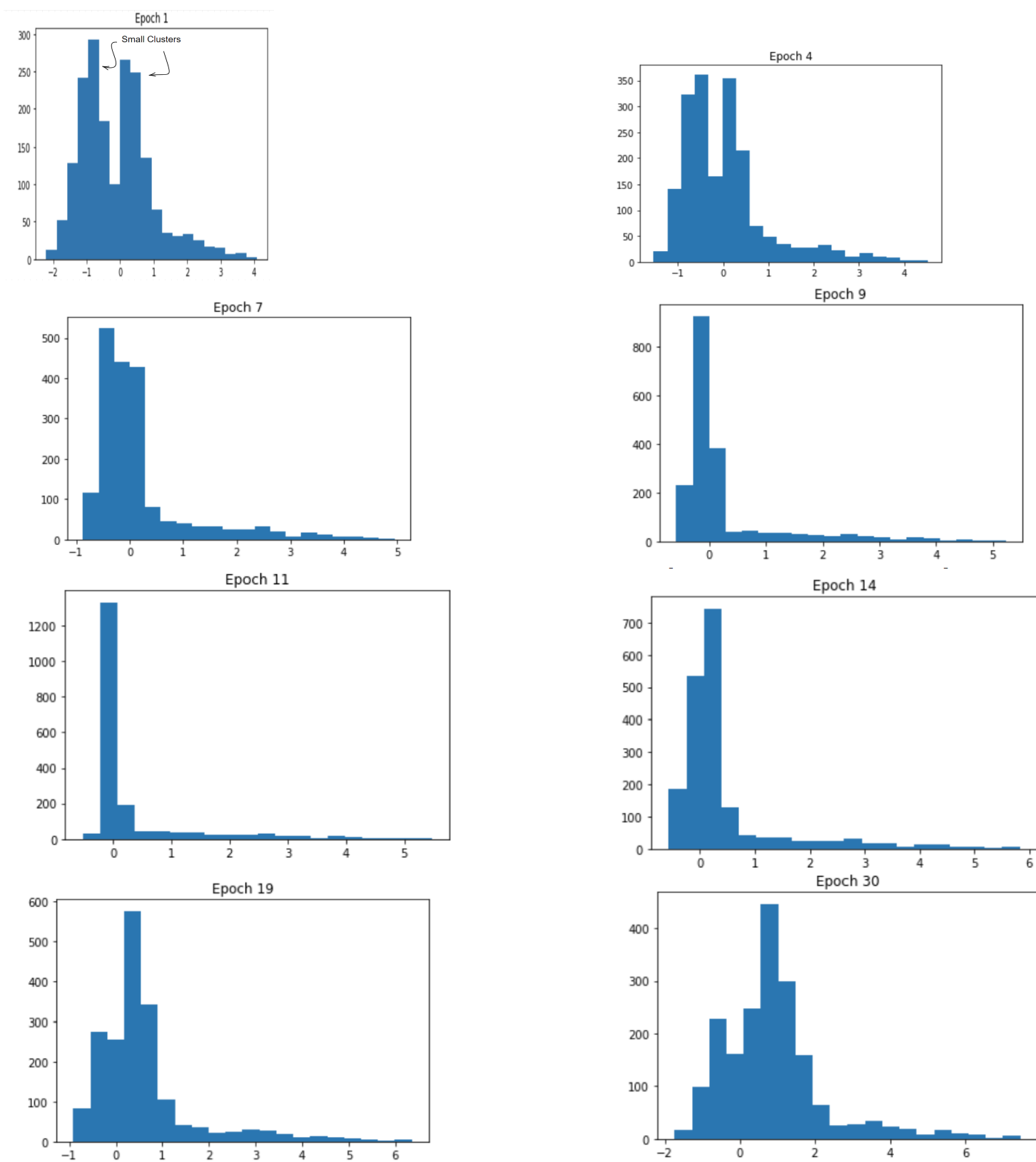
Finally, (257) gives estimates on loss:

$$\sum_{s \in T} \mu(s) \log \left(1 + e^{-\delta X(s, \alpha)} \right) \leq \bar{L}(\alpha) \leq \sum_{s \in T} \mu(s) \log \left(1 + (K - 1)e^{-\delta X(s, \alpha)} \right). \quad (258)$$

From (258) we get

$$\sum_{s \in G_{\eta^*}^c(t)} \mu(s) \log(1 + e^{-\eta^*}) \leq \bar{L}(t) \leq \bar{L}(t_0). \quad (259)$$

Figure 74: Classification confidence for the toy DNN for Example 7.1 at different stages of training.



VITA

Yitzchak Shmalo

Department of Mathematics, Penn State University
University Park, PA 16802, USA
Email: yitzchak.shmalo@gmail.com

Education

Ph.D. in Mathematics, Penn State University, Expected 2025; Thesis: “Applications of Random Matrix Theory to Deep Learning.”

M.A. in Mathematics, Yeshiva University, 2017; Thesis: “Combinatorial Approach to Detection of Fixed Points, Periodic Orbits, and Symbolic Dynamics.”

B.A. in Philosophy & Mathematics (Honors), Yeshiva University, 2016

Publications

- **Berlyand, L., Bourdais, T., Owhadi, H., and Shmalo, Y.** “Pruning Deep Neural Networks via a Combination of the Marchenko-Pastur Distribution and Regularization.” Submitted to *Journal of Machine Learning Research*, 2025; arXiv: 2503.01922.
- **Berlyand, L., Sandier, E., Shmalo, Y., and Zhang, L.** “Enhancing Accuracy in Deep Learning Using Random Matrix Theory.” *Journal of Machine Learning*, 2024, doi:10.4208/jml.231220.
- **Shmalo, Y.** “Stability of Accuracy for the Training of DNNs via the Uniform Doubling Condition.” *Annals of Mathematics and Artificial Intelligence*, Springer, 2024, pp. 1–45.
- **Prodan, E., and Shmalo, Y.** “The K-theoretic Bulk-Boundary Principle for Dynamically Patterned Resonators.” *Journal of Geometry and Physics*, 135 (2019): 135–171.
- **Gidea, M., and Shmalo, Y.** “Combinatorial Approach to Detection of Fixed Points, Periodic Orbits, and Symbolic Dynamics.” *Discrete and Continuous Dynamical Systems - A*, 38(1), 2018.

Talks

- **June 2024:** Research talk on RMT applications to Visual Transformers at Caltech.
- **August 2024:** Research talk on RMT applications in Deep Learning at JPL, NASA, Pasadena, CA.